

۱: مبانی جاوا (Java Fundamentals) - کامل و عمیق

عمیق

این بخش، سنگ بنای همه چیزه. اگر این مفاهیم رو عمیق نفهمی، توی مراحل بعدی (مخصوصاً OOP و کالکشن‌ها) به مشکل میخوری. توی مصاحبه‌های اینترپرایز هم از همین مبانی کلی سوال میپرسن.

1.1 انواع داده (Data Types)

جاوا یه زبان Strongly Typed و Statically Typed هست. یعنی چی؟

• Statically Typed: نوع هر متغیر باید در زمان کامپایل مشخص باشه.

• Strongly Typed: وقتی نوع متغیر مشخص شد، نمیتونی توش یه مقدار از نوع دیگه بریزی (مگر با تبدیل صریح).

انواع داده در جاوا به دو دسته بزرگ تقسیم میشن:

الف) انواع اولیه (Primitive Data Types)

این‌ها پایه‌ای‌ترین نوع‌ها هستن. توی Stack ذخیره میشن و مقدارشون مستقیماً توی متغیره. ۸ نوع هستن:

نوع	سایز	بازه	پیش فرض	مثال	نکته
byte	۱ بایت (۸ بیت)	128- تا 127	0	byte b = 100;	کم مصرف ترین نوع عددی. برای کار با فایل ها و Stream ها.
short	۲ بایت (۱۶ بیت)	32,768- تا 32,767	0	short s = 1000;	خیلی کم استفاده میشه.
int	۴ بایت (۳۲ بیت)	-2,147,483,648 تا 2,147,483,647 (حدود ۲ میلیارد)	0	int i = 50000;	پیش فرض و اصلی ترین نوع برای اعداد صحیح.
long	۸ بایت (۶۴ بیت)	خیلی بزرگ (۹ کوینتیلیون)	0L	long l = 100000L;	باید آخر عدد حرف L بذاری (کوچیک هم فرق نداره ولی بزرگش بهتره چون با ۱ اشتباه گرفته نشه).
float	۴ بایت (۳۲ بیت)	اعشاری با دقت حدود ۷ رقم	0.0f	float f = 5.6f;	باید آخر عدد حرف f بذاری. دقتش کمه، برای کارهای علمی دقیق خوب نیست.
double	۸ بایت (۶۴ بیت)	اعشاری با دقت حدود ۱۵ رقم	0.0d	double d = 5.6;	پیش فرض و اصلی ترین نوع برای اعداد اعشاری.
char	۲ بایت (۱۶ بیت)	یک کاراکتر یونیکد (Unicode)	'\u0000'	char c = 'A'; یا char c = 65;	تک کوت (') نه دابل کوت ("). میتونه عدد هم قبول کنه (کد ASCII).
boolean	۱~ بیت (وابسته به JVM)	true یا false	false	boolean flag = true;	فقط همین دو مقدار. برخلاف C، 0 و 1 نیست.

! سوال مصاحبه‌ای ۱: چرا $d = 0.1 + 0.2$ double برابر 0.3 نیست؟

جواب: به خاطر نحوه ذخیره‌سازی اعداد اعشاری بر اساس استاندارد IEEE 754. بعضی اعداد اعشاری دقیقاً در مبنای ۲ (باینری) قابل نمایش نیستن (مثل ۱/۳ در مبنای ۱۰). برای کارهای مالی (مثل پول) هرگز از float و double استفاده نکن. همیشه از کلاس BigDecimal استفاده کن.

```
1 import java.math.BigDecimal;
2
3 // غلط برای پول
4 double bad = 0.1 + 0.2; // 0.30000000000000004
5
6 // درست برای پول
7 BigDecimal price = new BigDecimal("10.99");
8 BigDecimal tax = new BigDecimal("0.20");
9 BigDecimal total = price.add(price.multiply(tax)); // دقیق
```

! سوال مصاحبه‌ای ۲: Boxing و Unboxing چیه؟

جواب:

• Autoboxing: تبدیل خودکار یک نوع Primitive به کلاس Wrapper معادلش. مثلاً int به Integer.

• Unboxing: تبدیل خودکار کلاس Wrapper به Primitive معادلش.

1 Integer wrapper = 5; // Autoboxing: int -> Integer

2 int primitive = wrapper; // Unboxing: Integer -> int

نکته مهم: این کار هزینه داره و توی حلقه‌های بزرگ میتونه Performance رو بیاره پایین.
همچنین Integer میتونه null باشه ولی int نه، پس ممکنه موقع Unboxing به
NullPointerException بخوری.

ب) انواع مرجع (Reference Data Types)

هر چیزی که Primitive نیست، Reference Type هست: کلاس‌ها، اینترفیس‌ها، آرایه‌ها، Enum‌ها، String و ...

• نحوه ذخیره‌سازی: خود متغیر (Reference) توی Stack ذخیره میشه و به آدرس شیء توی Heap اشاره میکنه.

خود شیء توی Heap هست.

• مقدار پیش‌فرض: همیشه null.

1 String str; // است null هست و در Stack مقدار str

2 String str2 = new String("Hello"); // اشاره میکنه که داخلش Heap تو String هست، به یه شیء Stack تو str2
"Hello" هست.

! سوال مصاحبه‌ای ۳: تفاوت String با Primitive‌ها چیه؟ (چرا همه جا هست)

جواب: String یه کلاس Immutable (تغییرناپذیر) و یه Reference Type هست.

• Immutable: یعنی وقتی یه شیء String ساخته شد، محتوای دیگه عوض نمیشه. هر متدی که روش صدا بزنی

(مثل toUpperCase()) یه String جدید برمیگردونه.

1-Java Interview Question

•String Pool: یه منطقه خاص توی Heap هست. وقتی یه String رو بدون new بسازی ("String s = "hello")، جاوا اول استخر رو میگرد. اگه "hello" اونجا بود، همون reference رو برمیگردونه. این کار برای صرفه‌جویی در حافظه است.

```
1 String s1 = "hello";
```

```
2 String s2 = "hello";
```

```
3 System.out.println(s1 == s2); // true! اشاره میکنن String Pool چون هر دو به یک شیء توی
```

```
4
```

```
5 String s3 = new String("hello");
```

```
1 System.out.println(s1 == s3); // false! چون s3 با new از جدید بیرون شده یه شیء جدید بیرون از new
```

```
2 // استفاده کن equals همیشه برای مقایسه محتوا از
```

```
3 System.out.println(s1.equals(s3)); // true
```

! سوال مصاحبه‌ای ۴: تفاوت String، StringBuffer و StringBuilder چیه؟ (سوال

طلایی)

جواب: هر سه برای کار با رشته‌ها هستن، اما تفاوت‌های کلیدی دارن:

ویژگی	String	StringBuilder	StringBuffer
تغییرپذیری	Immutable (غیرقابل تغییر)	Mutable (قابل تغییر)	Mutable (قابل تغییر)
امنیت ریس	✓ Thread-Safe	✗ ناامن	✓ Thread-Safe
سرعت	کندترین	سریع‌ترین	متوسط
استفاده	وقتی رشته تغییر نمی‌کنه	تکریمه‌ای	چندریمه‌ای

```
1 // String - غیرقابل تغییر
```

```
2 String s = "Hello";
```

1-Java Interview Question

```
3 s = s + " World"; // جدید ساخته میشه String به
4
5 // StringBuilder - سریع ولی ناامن
6 StringBuilder sb = new StringBuilder();
7 sb.append("Hello");
8 sb.append(" World"); // همان شیء تغییر میکنه
9
10 // StringBuffer - امن برای چند ریسه
11 StringBuffer sbf = new StringBuffer();
12 sbf.append("Hello");
13 sbf.append(" World");
```

1.2 عملگرها (Operators)

! سوال مصاحبه‌ای ۴: فرق << و <<< چیه؟ (شیفت بیتی)

جواب: اینا عملگرهای شیفت بیتی هستن و کمتر پیش میاد، ولی خوبه بدونی:

- << (شیفت به راست علامت‌دار): بیت علامت (چپ‌ترین بیت) رو حفظ میکنه. برای اعداد منفی، ۱ به چپ اضافه میشه.

- <<< (شیفت به راست بدون علامت): صفر به چپ اضافه میکنه. نتیجه همیشه مثبته.

```
16 ``java
```

```
17 int negative = -8; // 1000...1111 باینری:
```

```
18 int signedShift = negative >> 2; // 1111...1110 = -2
```

```
19 int unsignedShift = negative >>> 2; // 0011...1110 = 1073741822
```

! سوال مصاحبه‌ای ۵: & و && (و همینطور | و ||) چه فرقی دارن؟

جواب: این خیلی مهمه:

• & و | : همیشه هر دو طرف عبارت رو ارزیابی میکنن (هم برای boolean، هم bitwise).

• && و || : ارزیابی اتصال کوتاه (Short-circuit) دارن. یعنی اگر با ارزیابی سمت چپ، نتیجه کل عبارت مشخص بشه، سمت راست اصلاً اجرا نمیشه.

1 // a. یک متد سنگین یا دارای عوارض جانبی است `getValue()` فرض کنید

2 `if (false && getValue()) {` // b. هرگز فراخوانی نمی‌شود `getValue()` تابع

3 }

4 `if (false & getValue()) {` // c. `getValue()` حتی اگر نتیجه

5 }

1.3 ساختارهای کنترلی (Control Flow)

! سوال مصاحبه‌ای ۶: switch-case قدیم و جدید (Enhanced switch) چه فرقی دارن؟

(+Java 14)

جواب:

• قدیم (Statement): مثل ++C/C هست. نیاز به break داره، وگرنه میافته توی case بعدی (fall-through).

• جدید (Expression): میتونه یه مقدار برگردونه. از علامت -> استفاده میکنه. نیازی به break نداره. خیلی تمیزتر و کم‌خطاتر هست.

1 // a. (بهتر)

2 String dayType = switch (day) {

1 case MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY -> "Weekday";

2 case SATURDAY, SUNDAY -> "Weekend";

3 // b. لازم نیست default استفاده کنی و همه حالات رو پوشش بدی، دیگه enum اگه

4 };

! سوال مصاحبه‌ای ۷: فرق for، for-each، و while در جاوا؟ بهترین زمان استفاده از

هرکدام؟

جواب:

• for سنتی: وقتی به ایندکس (شماره خونه) نیاز داری. (مثلاً میخوای عنصر iام رو با i+1 مقایسه کنی).

• for-each (enhanced for): وقتی فقط میخوای همه عناصر یه مجموعه رو بخونی. خوندنی‌ترین و امن‌ترین حالت پیمایشه. نمیتونی توش عناصر رو حذف یا تغییر بدی (فقط خوندنیه).

• while: وقتی تعداد تکرار مشخص نیست و به یه شرط وابسته است (مثلاً تا وقتی که کاربر دستور exit نزده).

• do-while: مثل while، اما تضمین میکنه که بدنه حلقه حداقل یک بار اجرا بشه.

1.4 آرایه‌ها (Arrays)

سوال: آرایه توی جاوا چطوریه؟

• یه ساختمان داده با طول ثابت (Fixed Size) که تعدادی خونه پشت سر هم توی حافظه Heap داره.

•نوع همه خونه‌هاش باید یکی باشه.

•سایزش بعد از ساخته شدن دیگه عوض نمیشه. اگه سایز داینامیک میخوای، باید بری سراغ ArrayList.

1 // سه روش برای ساختن آرایه a:

1 `int[] arr1 = new int[5];` // b. همه خونه‌ها مقدار پیش‌فرض 0 دارن

2 `int[] arr2 = {1, 2, 3, 4, 5};` // c. سایز خودکار 5 میشه

3 `int[] arr3 = new int[] {1, 2, 3};` // d. یه جور دیگه

4

5 // آرایه دو بعدی (آرایه‌ای از آرایه‌ها) a:

6 `int[][] matrix = {`

7 `{1, 2, 3},`

8 `{4, 5, 6}`

9 `};`

10 `System.out.println(matrix[0][1]);` // خروجی: 2 b.

! سوال مصاحبه‌ای ۸: `ArrayIndexOutOfBoundsException` کی رخ میده؟

جواب: وقتی سعی کنی به یه ایندکس خارج از بازه آرایه دسترسی پیدا کنی (مثلاً آرایه ۵ تایی، ایندکس ۵). در جاوا ایندکس‌ها از صفر شروع میشن. این یه `Unchecked Exception` هست.

1.5 آرگومان‌های خط فرمان (args)

سوال: args [] String توی متد main چیه؟ جواب: هر چیزی که موقع اجرای برنامه جاوا از خط فرمان بعد از اسم برنامه بنویسی، توی این آرایه قرار میگیره.

1 java MyProgram arg1 arg2

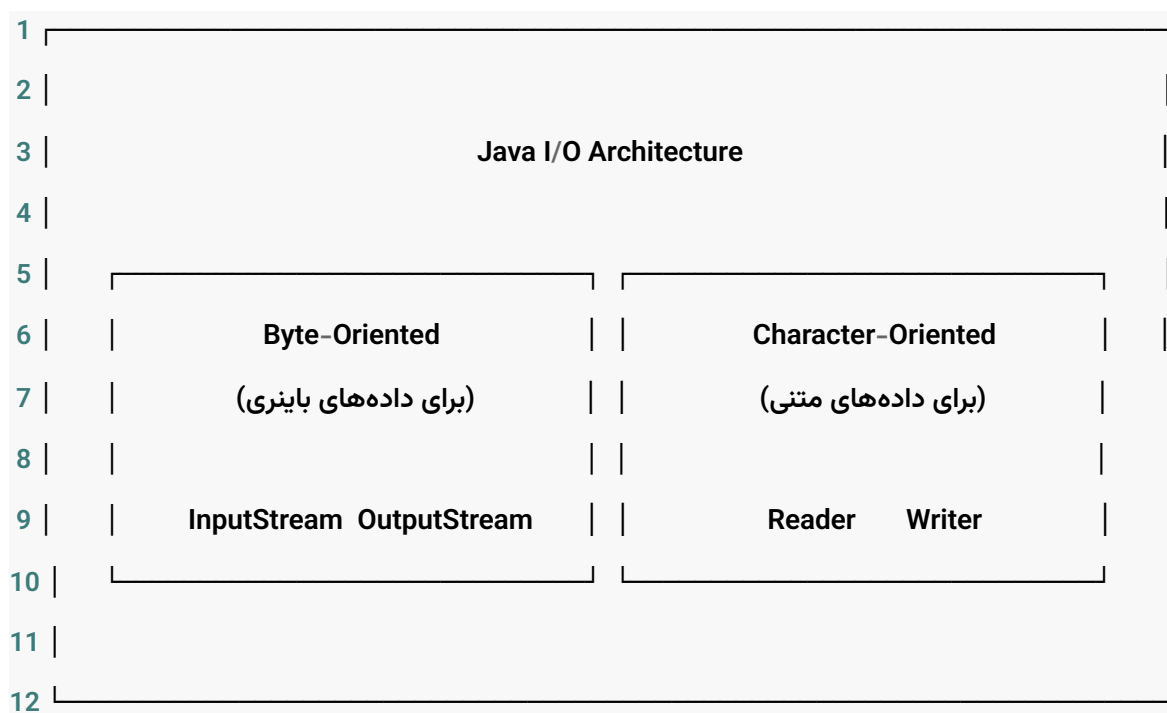
2

3 هست String همیشه از نوع "arg2" میشه [1] args و "arg1" میشه [0] args، توی برنامه

فاز ۱.۵: ورودی/خروجی (I/O) در جاوا – کامل و عمیق

۱.۵.۱ مقدمه: I/O در جاوا چیست؟

جاوا برای کار با ورودی/خروجی (Input/Output) دو دسته کلی داره:



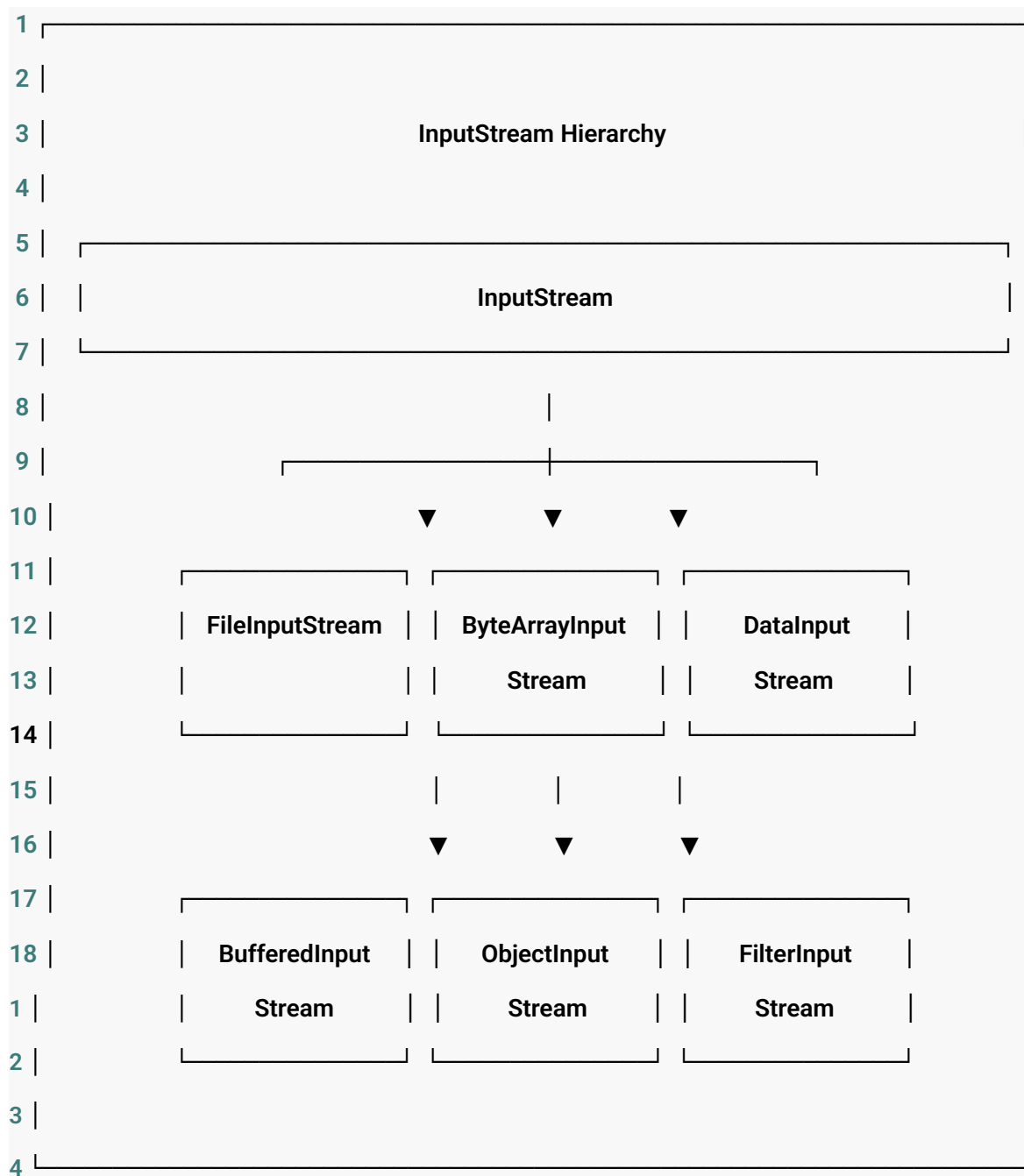
فرق اصلی:

• Stream (بایت): برای فایل‌های باینری (تصویر، ویدیو، صدا، PDF)

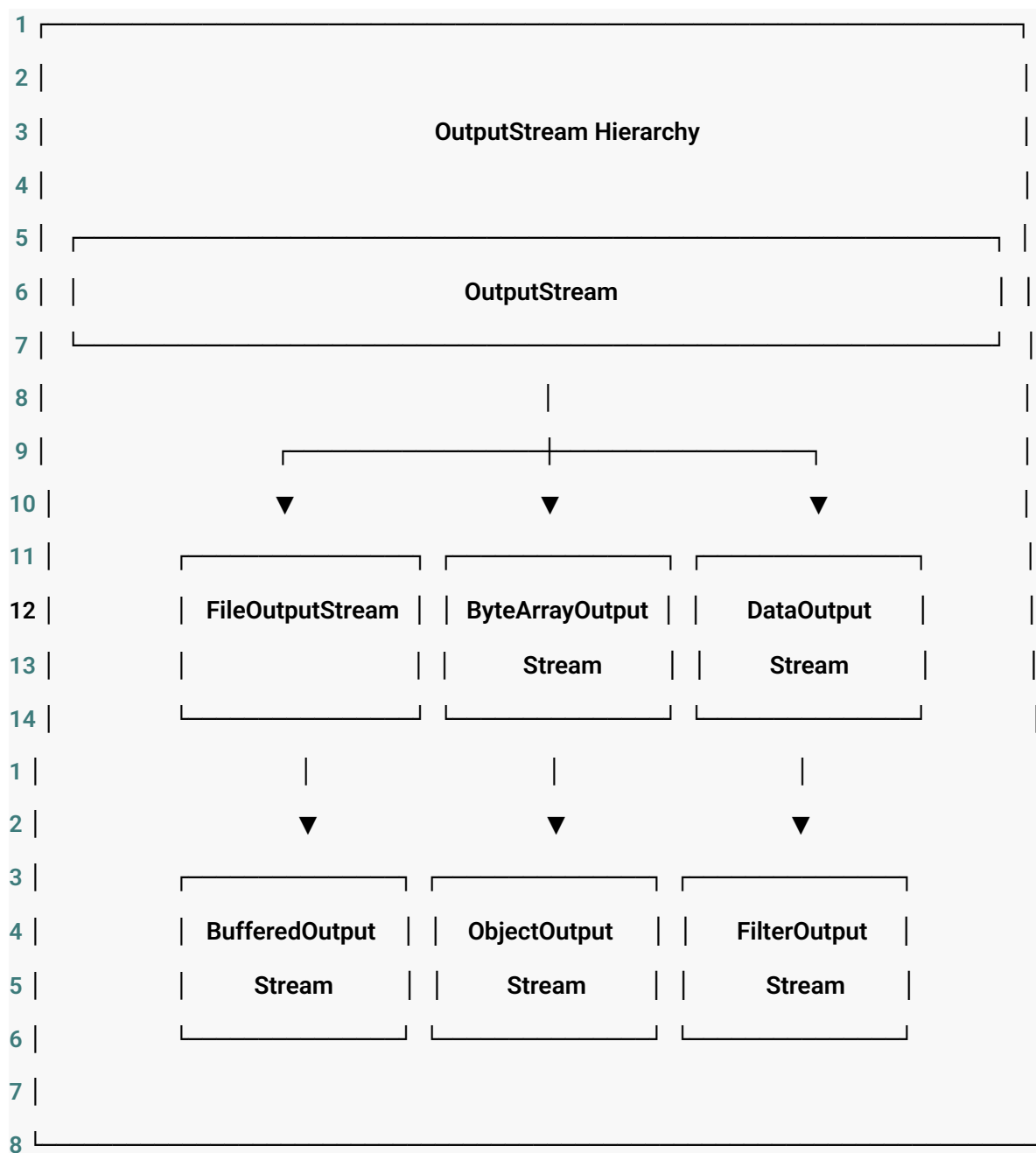
• Reader/Writer (کاراکتر): برای فایل‌های متنی (TXT, CSV, JSON, XML)

۱.۵.۲ Stream های بایت (Byte Streams)

InputStream (خواندن داده های باینری)



OutputStream (نوشتن داده‌های باینری)



مثال‌های عملی با Byte Streams

۱. FileInputStream (خواندن فایل)

```

1 // a
2 // روش ۱: خواندن بایت به بایت
3 try (FileInputStream fis = new FileInputStream("image.jpg")) {
4     int byteData;
5     while ((byteData = fis.read()) != -1) {
6         System.out.print(byteData); // مقادیر 0-255
7     }
8 } catch (IOException e) {
9     e.printStackTrace();
10 }
11
12 // روش ۲: خواندن تکه تکه (بهتر)
13 try (FileInputStream fis = new FileInputStream("image.jpg")) {
14     byte[] buffer = new byte[1024]; // KB بافر ۱
15     int bytesRead;
16     while ((bytesRead = fis.read(buffer)) != -1) {
17         System.out.println("خوانده شد: " + bytesRead + " بایت");
18         // buffer[0] تا buffer[bytesRead-1] پردازش
19     }
20 } catch (IOException e) {
21     e.printStackTrace();
22 }

```

```
1 // b
2 // نوشتن بایت‌ها در فایل
3 try (FileOutputStream fos = new FileOutputStream("output.jpg")) {
4     String text = "Hello World!";
5     byte[] data = text.getBytes();
6     fos.write(data);
7     System.out.println("فایل ذخیره شد");
8 } catch (IOException e) {
9     e.printStackTrace();
10 }
11
12 // کپی کردن فایل
13 try (FileInputStream fis = new FileInputStream("source.jpg");
14     FileOutputStream fos = new FileOutputStream("dest.jpg")) {
15     byte[] buffer = new byte[4096];
16     int bytesRead;
17     while ((bytesRead = fis.read(buffer)) != -1) {
18         fos.write(buffer, 0, bytesRead);
19     }
20     System.out.println("فایل کپی شد");
21 } catch (IOException e) {
22     e.printStackTrace();
23 }
```

۳. BufferedInputStream/OutputStream (بهبود سرعت)

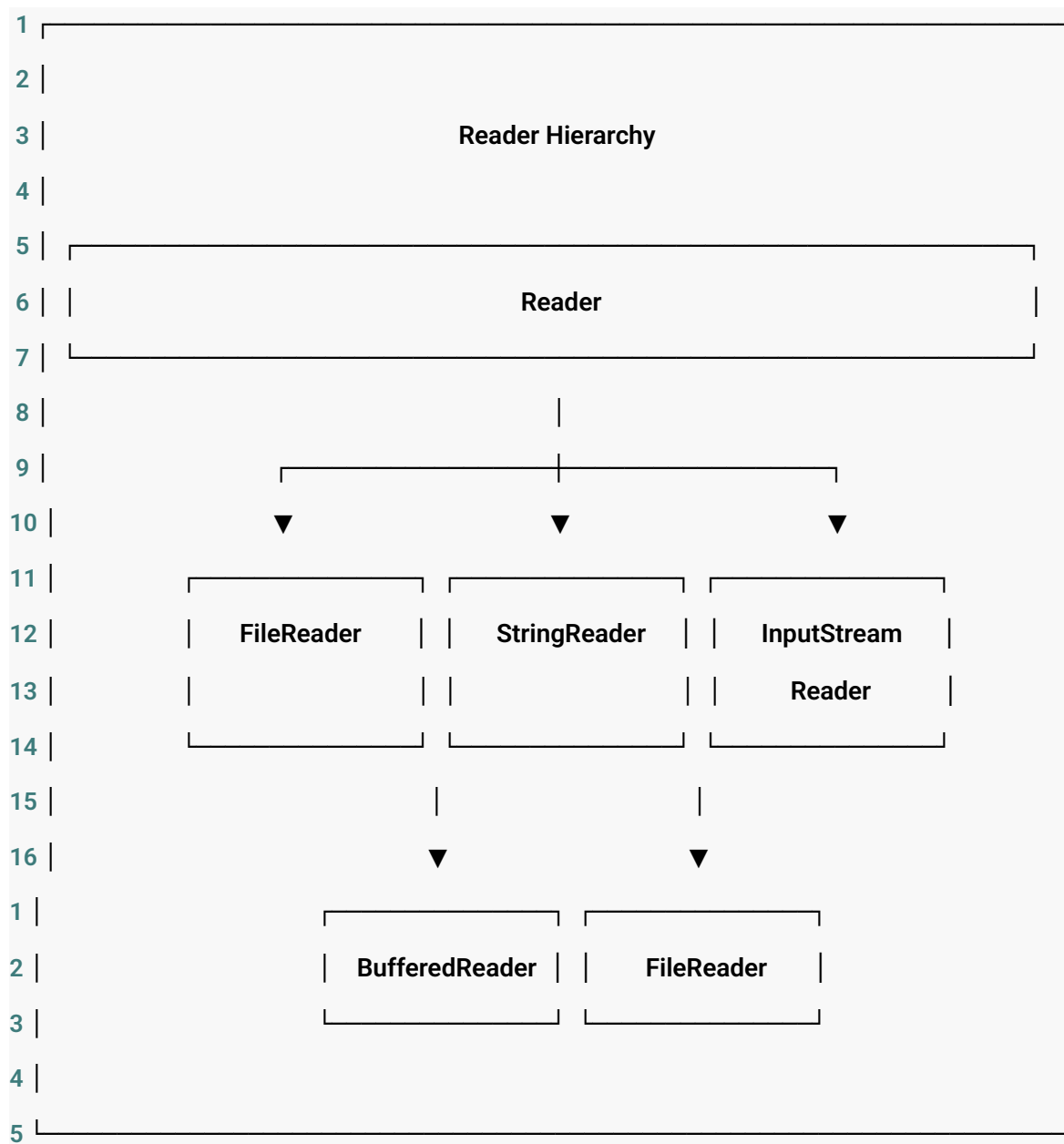
```

1 // c
2 با بافر - خیلی سریع تر //
3 try (BufferedInputStream bis = new BufferedInputStream(
4     new FileInputStream("large_file.zip"));
5     BufferedOutputStream bos = new BufferedOutputStream(
6         new FileOutputStream("copy.zip"))) {
7
8     byte[] buffer = new byte[8192];
9     int bytesRead;
10    while ((bytesRead = bis.read(buffer)) != -1) {
11        bos.write(buffer, 0, bytesRead);
12    }
13    System.out.println("فایل با بافر کپی شد");
14 } catch (IOException e) {
15     e.printStackTrace();
16 }

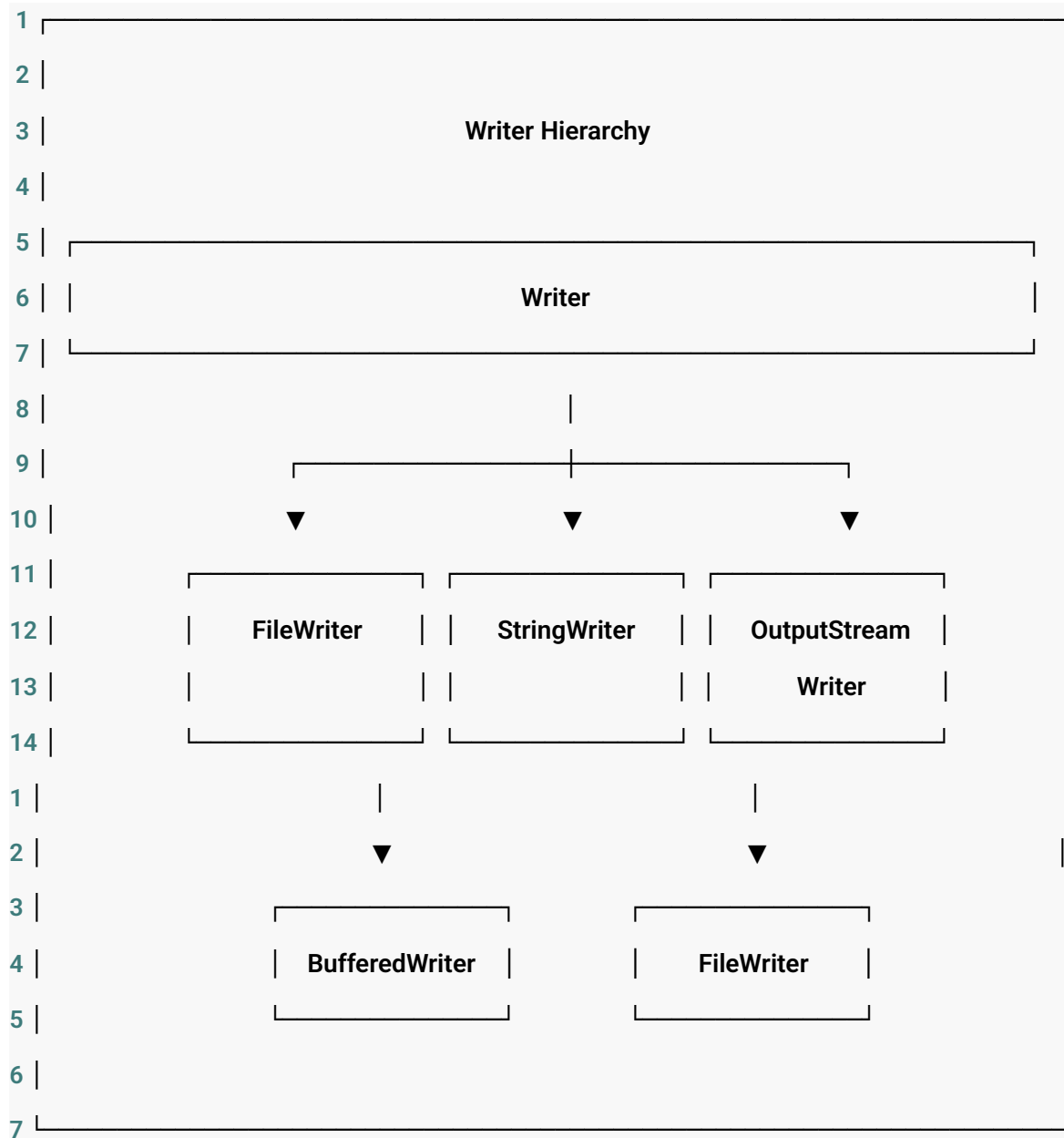
```

۳.۵.۱ Reader و Writer (داده‌های متنی)

Reader (خواندن متن)



Writer (نوشتن متن)



مثال‌های عملی با Reader/Writer

۱. FileReader (خواندن فایل متنی)

```
1 // a
2 // روش ۱: کاراکتر به کاراکتر (کند)
3 try (FileReader fr = new FileReader("file.txt")) {
4     int charData;
5     while ((charData = fr.read()) != -1) {
6         System.out.print((char) charData);
7     }
8 } catch (IOException e) {
9     e.printStackTrace();
10 }
```

۲. BufferedReader (توصیه شده)

```
1 // b
2 // روش ۲: BufferedReader - سریع‌تر و خط به خط
3 try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
4     String line;
5     while ((line = br.readLine()) != null) {
6         System.out.println(line);
7     }
8 } catch (IOException e) {
9     e.printStackTrace();
10 }
```

1-Java Interview Question

9

10 *روش ۳: با مشخص کردن کدگذاری //*

```
11 try (BufferedReader br = new BufferedReader(  
12     new InputStreamReader(new FileInputStream("file.txt"), "UTF-8"))) {  
13     String line;  
14     while ((line = br.readLine()) != null) {  
15         System.out.println(line);  
16     }  
17 } catch (IOException e) {  
18     e.printStackTrace();  
19 }
```

۳. FileWriter (نوشتن فایل متنی)

1 *// c*

2 *// ۱ ساده: FileWriter روش*

```
3 try (FileWriter fw = new FileWriter("output.txt")) {  
4     fw.write("Hello World!\n");  
5     fw.write("خط دوم");  
6 } catch (IOException e) {  
7     e.printStackTrace();  
8 }  
9
```

5 *توصیه شده) BufferedWriter روش ۲ //*

```
6 try (BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"))) {  
7     bw.write("Hello World!");  
8     bw.newLine();  
9 }
```


1-Java Interview Question

```
9  bw.write("خط دوم");
10 bw.write("و خط سوم");
11 } catch (IOException e) {
12  e.printStackTrace();
13 }
14
15 // اضافه کردن به انتهای فایل (Append روش ۳: با)
16 try (BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt", true))) {
17  bw.write("این به انتها اضافه میشه");
18  bw.newLine();
19 } catch (IOException e) {
20  e.printStackTrace();
21 }
```

۴. PrintWriter (نوشتن با فرمت)

```
1 // d
2 // PrintWriter - شبیه System.out
3 try (PrintWriter pw = new PrintWriter(new FileWriter("output.txt"))) {
4  pw.println("Hello World!");
5  pw.printf("نام: %s, سن: %d\n", "علی", 25);
6  pw.print("newline خط بدون");
7  pw.flush(); // فرستادن به فایل
8 } catch (IOException e) {
9  e.printStackTrace();
10 }
11 }
```

۱.۵.۴ Java NIO (New I/O) - مدرن تر

(*) java.nio از Java 1.4 اضافه شد و راه‌های مدرن‌تری برای I/O ارائه می‌دهد.

مقایسه IO و NIO

مقایسه IO و NIO در جاوا

ویژگی	IO (java.io)	NIO (java.nio)
جهت‌گیری	Stream-oriented (یک‌طرفه)	Buffer-oriented (دوطرفه)
بلوک‌شدن	Blocking (مسدود)	Non-blocking (غیرمسدود)
انتخاب‌گر	ندارد	Selector (چند کانال)
مناسب برای	فایل‌های معمولی	سرورهای با ترافیک بالا
API	ساده‌تر	پیچیده‌تر ولی قدرتمندتر

مثال‌های NIO

```

1 import java.nio.file.*;
2 import java.nio.*;
3
4 // ۱. (راحت‌ترین!) خواندن همه خطوط
5 List<String> lines = Files.readAllLines(Paths.get("file.txt"));
6 lines.forEach(System.out::println);
7
8 // ۲. (راحت‌ترین!) نوشتن
9 List<String> content = Arrays.asList("Hello", "World", "Line 3");

```

1-Java Interview Question

```
10 Files.write(Paths.get("output.txt"), content, StandardCharsets.UTF_8);
11
12 // ۳. کپی کردن فایل
13 Files.copy(Paths.get("source.txt"), Paths.get("dest.txt"),
14     StandardCopyOption.REPLACE_EXISTING);
15
16 // ۴. حذف فایل
17 Files.deleteIfExists(Paths.get("temp.txt"));
18
19 // ۵. چک کردن وجود فایل
20 if (Files.exists(Paths.get("file.txt"))) {
21     System.out.println("فایل وجود دارد");
22 }
23
24 // ۶. خواندن کل محتوا به صورت String (Java 11+)
25 String content = Files.readString(Paths.get("file.txt"));
26 System.out.println(content);
27
28 // ۷. نوشتن String به فایل (Java 11+)
29 Files.writeString(Paths.get("output.txt"), "Hello World!",
30     StandardCharsets.UTF_8);
31
32 // ۸. لیست کردن فایل‌های یک پوشه
33 try (Stream<Path> paths = Files.list(Paths.get("."))) {
34     paths.forEach(path -> System.out.println(path.getFileName()));
35 }
```

11

12 // ۹. ایجاد پوشه

13 Files.createDirectories(Paths.get("myapp/temp/logs"));

14

15 // ۱۰. دریافت اطلاعات فایل

16 Path path = Paths.get("file.txt");

17 System.out.println("نام: " + path.getFileName());

18 System.out.println("سایز: " + Files.size(path) + " بایت");

19 System.out.println("آخرین تغییر: " + Files.getLastModifiedTime(path));

20 System.out.println("نوع: " + Files.probeContentType(path));

۱.۵.۵ Object Serialization (ذخیره سازی اشیاء)

Serialization یعنی تبدیل یک شیء به جریان بایت برای ذخیره یا انتقال.

قوانین Serialization

- کلاس باید Serializable پیاده سازی کنه

- فیلدهای transient ذخیره نمیشن

- فیلدهای static ذخیره نمیشن

- کلاس باید serialVersionUID داشته باشه

مثال کامل

```
1 import java.io.*;
```

```
2
```

1-Java Interview Question

```
3 // ۱. Serializable کلاس
4 class Person implements Serializable {
5     private static final long serialVersionUID = 1L;
6
7     private String name;
8     private int age;
9     private transient String password; // ذخیره نمیشه
10    private static String company = "ACME"; // ذخیره نمیشه
11
12    public Person(String name, int age, String password) {
13        this.name = name;
14        this.age = age;
15        this.password = password;
16    }
17
18    // getters and setters
19 }
20
21 // ۲. Serialize (ذخیره)
22 public class SerializeExample {
23     public static void main(String[] args) {
24         Person person = new Person("Ali", 25, "secret123");
25
26         try (ObjectOutputStream oos = new ObjectOutputStream(
27             new FileOutputStream("person.ser"))) {
28             oos.writeObject(person);
29         }
```

1-Java Interview Question

```
16     System.out.println("شء ذخیره شد");
17 } catch (IOException e) {
18     e.printStackTrace();
19 }
20 }
21 }
22
23 // ۳. Deserialize (بازیابی)
24 public class DeserializeExample {
25     public static void main(String[] args) {
26         try (ObjectInputStream ois = new ObjectInputStream(
27             new FileInputStream("person.ser"))) {
28             Person person = (Person) ois.readObject();
29             System.out.println("نام: " + person.getName()); // Ali
30             System.out.println("سن: " + person.getAge()); // 25
31             System.out.println("پسورد: " + person.getPassword()); // null
32         } catch (IOException | ClassNotFoundException e) {
33             e.printStackTrace();
34         }
35     }
36 }
37 }
```

سفرشی سازی Serialization

```
1 class Person implements Serializable {
2     private static final long serialVersionUID = 1L;
3     private String name;
```

1-Java Interview Question

```
4 private int age;
5 private transient String password;
6
7 // نوشتن سفارشی
8 private void writeObject(ObjectOutputStream out) throws IOException {
9     out.defaultWriteObject();
10    out.writeUTF(password != null ? password : "default");
11 }
12
13 // خواندن سفارشی
14 private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException {
15     in.defaultReadObject();
16     password = in.readUTF();
17 }
18 }
```

۱.۵.۶ Console (ورودی از کاربر)

۱. Scanner (راحت‌ترین)

```
1 Scanner scanner = new Scanner(System.in);
2
3 System.out.print("نام: ");
4 String name = scanner.nextLine();
5
6 System.out.print("سن: ");
```

1-Java Interview Question

```
7 int age = scanner.nextInt();
8
9 System.out.print("حقوق: ");
10 double salary = scanner.nextDouble();
11
12 System.out.println("سال دارید " + age + " شما !" + name + " سلام");
13 scanner.close();
```

۲. BufferedReader (سریع تر)

```
1 try (BufferedReader reader = new BufferedReader(new InputStreamReader(System.in))) {
2     System.out.print("نام: ");
3     String name = reader.readLine();
4     System.out.print("سن: ");
5     int age = Integer.parseInt(reader.readLine());
6     System.out.println("سلام " + name);
7 } catch (IOException e) {
8     e.printStackTrace();
9 }
```

۳. Console (برای پسورد)

```
1 Console console = System.console();
2 if (console != null) {
3     String username = console.readLine("نام کاربری: ");
4     char[] password = console.readPassword("رمز عبور: ");
5     System.out.println("سلام " + username);
6 }
```



```

6 } else {
7   System.out.println("در دسترس نیست Console");
8 }

```

۱.۵.۷ ! سوالات مصاحبه‌ای پرتکرار

? سوال ۱: فرق Reader و InputStream چیه؟

جواب:

مقایسه Reader و InputStream در جاوا

ویژگی	InputStream	Reader
کاربرد	برای داده‌های باینری	برای داده‌های متنی
واحد خواندن	بایت (byte) می‌خواند	کاراکتر (char) می‌خواند
کدگذاری	ندارد	پشتیبانی از کدگذاری
مقدار بازگشتی read()	int read () برمی‌گردونه (0-255)	int read () برمی‌گردونه (0-65535)
مثال	تصویر، ویدیو	فایل TXT

? سوال ۲: فرق FileReader و FileInputStream چیه؟

جواب:

- 1 FileInputStream → برای فایل‌های **باینری** (تصویر، صدا، ویدیو)
- 2 FileReader → برای فایل‌های **متنی** (TXT, CSV)

? سوال ۳: BufferedReader چه مزیتی داره؟

جواب:

• بافرینگ: داده‌ها رو تکه‌تکه (بافر) می‌خونه، نه بایت‌به‌بایت

• سرعت بیشتر: به شدت سریع‌تر از FileReader معمولی

• متد readLine(): راحت‌ترین راه برای خواندن خط‌به‌خط

? سوال ۴: فرق static و transient در Serialization چیه؟

جواب:

مقایسه static و transient در جاوا

ویژگی	transient	static
محدوده	مخصوص هر شیء	متعلق به کلاس
ذخیره در Serialization	ذخیره نمیشه	ذخیره نمیشه
کاربرد	برای فیلدهای حساس (پسورد)	برای داده‌های اشتراکی
مقدار پس از Deserialize	null (یا مقدار پیش‌فرض)	مقدار کلاس (همان مقدار جاری)

? سوال ۵: Try-with-Resources چه مشکلی رو حل می‌کنه؟

جواب:

• قبلش برای بستن منابع کلی کد تکراری می‌نوشتیم

• مدیریت دستی استثناها دردسر داشت

• ممکن بود یادمون بره منابع رو ببندیم (Memory Leak)

• حالا خودکار و امن بسته میشن

؟ سوال ۶: IO و NIO چه فرقی دارن؟

جواب:

مقایسه IO و NIO در جاوا

ویژگی	IO	NIO
مبنای کار	Stream-based (یک طرفه)	Buffer-based (دو طرفه)
نوع عملکرد	Blocking (مسدود)	Non-blocking (غیرمسدود)
کاربرد مناسب	فایل های معمولی	سرورهای بزرگ
پیچیدگی API	ساده تر	قدرتمندتر
سرعت	کندتر	سریع تر

۱.۵.۸ خلاصه یک صفحه ای (دسته بندی کلاس های I/O در جاوا)

کاربرد	کلاس ها	مفهوم
فایل های باینری	FileInputStream, BufferedInputStream, DataInputStream	Byte Input
ذخیره باینری	FileOutputStream, BufferedOutputStream, DataOutputStream	Byte Output
فایل های متنی	FileReader, BufferedReader, InputStreamReader	Text Input
ذخیره متن	FileWriter, BufferedWriter, PrintWriter	Text Output
ذخیره اشیاء	ObjectInputStream, ObjectOutputStream	Serialization
روش مدرن I/O	Files, Paths, Path	NIO
ورودی از کاربر	Scanner, BufferedReader, Console	User Input

فاز ۲: برنامه‌نویسی شی‌گرا (Object-Oriented)

(Programming) – کامل و عمیق

برنامه‌نویسی شی‌گرا فقط یاد گرفتن ۴ تا کلمه نیست (کپسوله‌سازی، وراثت، چندریختی، انتزاع). باید بفهمی این مفاهیم چه مشکلی رو حل می‌کنن، چطور توی کد واقعی پیاده میشن، و چرا اینقدر مهم هستن.

2.0 چرا اصلاً شی‌گرایی؟ مشکل برنامه‌نویسی رویه‌ای (Procedural) چی بود؟

تصور کن یه برنامه بانکی داری. هزاران تابع داری: `calculateInterest()`, `deposit()`, `withdraw()`... همه این توابع روی داده‌های سراسری (Global Variables) کار می‌کنن.

• مشکل ۱: هر تابعی می‌تونه داده رو خراب کنه. هیچ محافظی نیست.

• مشکل ۲: اگه ساختار داده عوض بشه (مثلاً حساب پس‌انداز بشه دو نوع: کوتاه‌مدت و بلندمدت)، باید تمام توابع رو عوض کنی. این همیشه یه فاجعه.

• مشکل ۳: کد برای تیم‌های بزرگ و پروژه‌های عظیم غیرقابل مدیریت میشه.

شی‌گرایی این راه حل رو میده: داده و توابعی که روی اون داده کار می‌کنن رو بذار توی یه بسته به اسم شیء (Object) و بگو هیچکی حق نداره مستقیماً به داده دست بزنه. فقط از طریق اون توابع می‌تونی.

2.1 کلاس و شیء (Class & Object)

• کلاس: یه قالب یا نقشه هست. هیچ حافظه‌ای اشغال نمی‌کنه. (مثل نقشه ساختمان یا فرم خالی ثبت‌نام).

• شیء: یه نمونه واقعی از روی اون قالب که حافظه اشغال می‌کنه. (مثل ساختمونی که ساخته شده یا فرم پر شده).

! سوال مصاحبه‌ای ۱: توی حافظه، وقتی به شیء می‌سازی دقیقاً چه اتفاقی می‌افته؟

جواب:

1. وقتی JVM به دستور new میرسد، میره توی Heap فضای کافی برای فیلدهای اون کلاس (و کلاس‌های والدی) کنار میذاره.

2. اون فضا رو با مقادیر پیش‌فرض (0, null, false) پر می‌کنه.

3. Constructor (سازنده) کلاس رو صدا میزنه تا فیلدها رو با مقادیر دلخواه ما مقداردهی اولیه کنه.

4. در نهایت، آدرس این شیء توی Heap، به یه متغیر Reference در Stack داده میشه.

ساخته میشه. هنوز هیچ شیئی وجود نداره Stack اینجا فقط یه اشاره‌گر توی a. // 1

2 `BankAccount myAccount;`

3

ذخیره میشه myAccount اینجا اتفاقی بالا میفته و آدرس شیء جدید توی b. // 4

5 `myAccount = new BankAccount();`

2.2 سازنده‌ها (Constructors)

سوال: Constructor چیه و فرقیش با متد معمولی چیه؟

جواب:

• هیچ نوع بازگشتی‌ای نداره (حتی void).

• همنام کلاسه.

• برای مقداردهی اولیه به فیلدهای شیء هنگام ساختش استفاده میشه.

• اگه هیچ سازنده‌ای نسازی، جاوا یه سازنده پیش‌فرض (Default Constructor) بدون پارامتر برات میسازه. ولی به محض این که خودت یه سازنده (حتی با پارامتر) بسازی، اون پیش‌فرض دیگه ساخته نمیشه.

.

```

1 public class Person {
2     private String name;
3
4     // سازنده (Constructor)
5     public Person(String name) {
6         this.name = name; // this.name به فیلد کلاس اشاره میکند
7     }
8 }
9 // حالا این غلطه
10 // این درسته: Person p = new Person("Ali");

```

کلمه کلیدی **this**: همیشه به شیء فعلی (همینی که باهاش کار می‌کنیم) اشاره دارد. برای فرق گذاشتن بین فیلد کلاس و پارامتر ورودی استفاده میشه

2.3 اصل اول: کپسوله‌سازی (Encapsulation)

تعریف ساده: بسته‌بندی داده‌ها (متغیرها) و متدهایی که روی آن‌ها کار می‌کنند، در یک واحد (کلاس)، و مخفی کردن جزئیات پیاده‌سازی از دنیای بیرون.

چرا مهمه؟

• امنیت: نمی‌داری بقیه مستقیم بیان و فیلدهای کلاس رو خراب کنن.

• انعطاف‌پذیری: هر وقت خواستی می‌تونی منطق داخلی رو عوض کنی. تا وقتی امضای متدهای **public** عوض نشه، هیچ کد دیگه‌ای در برنامه نمی‌شکنه.

چطور پیاده‌سازی میشه؟ با Access Modifiers (سطوح دسترسی):

1-Java Interview Question

Modifier	داخل کلاس	داخل Package	زیرکلاس (ارث‌بری)	همه جا
private	✓	✗	✗	✗
default (بدون کلمه)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

قانون طلایی برنامه‌نویسان حرفه‌ای

• فیلدها رو همیشه private کن.

• دسترسی بهشون رو فقط از طریق متدهای public (getter و setter) بده. به اینا POJO یا Java Bean هم میگن.

```

1 public class Employee {
2     // ۱. مخفی‌سازی داده
3     private double salary;
4
5     // ۲. دسترسی کنترل‌شده (Getter)
6     public double getSalary() {
7         // اجازه دیدن بدی HR اینجا می‌تونی منطق بذاری. مثلاً فقط به
8         return salary;
9     }
10
11    // ۳. تغییر کنترل‌شده (Setter)
12    public void setSalary(double newSalary) {
13        if (newSalary > 0) { // اعتبارسنجی
14            this.salary = newSalary;
15        }
16    }

```

2.4 اصل دوم: وراثت (Inheritance)

تعریف ساده: ساختن کلاس جدید بر اساس کلاس موجود. کلاس جدید (زیرکلاس/فرزند) فیلدها و متدهای کلاس موجود (آبَرکلاس/والد) را به ارث می‌برد. رابطه IS-A داریم: Dog IS-A Animal.

مزیت اصلی: استفاده مجدد از کد (Code Reusability).

! سوال مصاحبه‌ای: تفاوت Association،

Aggregation و Composition رو کامل توضیح بده

جواب: این سه تا رابطه، درجه‌های مختلف وابستگی بین دو کلاس رو نشون میدن. از ضعیف به قوی:

۱. Association (ارتباط) – ضعیف‌ترین

تعریف: وقتی دو کلاس با هم ارتباط دارن، اما هیچکدوم مالک دیگری نیست و هیچ وابستگی عمری (Lifecycle Dependency) بینشون نیست.

تشبیه: من و یه دکتر. من می‌رم پیش دکتر، ولی دکتر مال من نیست و من مال دکتر نیستم. هرکدوم می‌تونیم بدون دیگری وجود داشته باشیم.

علامت: Uses a / Is associated with

1-Java Interview Question

```
1 // Association - کاملاً مستقل
2 class Doctor {
3     public void treat(Patient patient) {
4         // دکتر بیمار رو درمان می‌کنه
5     }
6 }
7
8
9 class Patient {
10     private String name;
11     // بیمار می‌تونه بدون دکتر وجود داشته باشه
12 }
13
14 // استفاده
15 Doctor doc = new Doctor();
16 Patient p = new Patient();
17 doc.treat(p); // ارتباط، ولی مالکیت نیست
```

ویژگی‌ها:

- هر دو شیء مستقل هستن
- هیچکدوم مالک دیگری نیست
- عمر هرکدوم به دیگری وابسته نیست
- می‌تونه یک‌طرفه یا دو‌طرفه باشه

۲. Aggregation (تجميع) – متوسط

تعريف: رابطه HAS-A ضعیف. یعنی یه کلاس شامل کلاس دیگه ست، ولی اون کلاس می تونه مستقل از مالکش هم وجود داشته باشه.

تشبيه: دانشگاه و دانشجوها. دانشگاه دانشجوها رو داره، ولی اگه دانشگاه تعطیل بشه، دانشجوها همچنان وجود دارن. (قبلاً گفتیم)

علامت: Has a (weak)

```

1 // Aggregation - جزء می تونه مستقل وجود داشته باشه
2 class Student {
3     private String name;
4     // Student وجود داشته باشه University می تونه بدون
5 }
6
7 class University {
8     private List<Student> students;
9
10    public University(List<Student> students) {
11        this.students = students; // از بیرون داده میشه
12        // ها هنوز هستن Student، از بین بره University اگه
13    }
14 }
15
16 // استفاده
17 Student s1 = new Student();
18 Student s2 = new Student();

```

```
1 List<Student> list = Arrays.asList(s1, s2);
```

```
2 University uni = new University(list); // دانشجویها قبل از دانشگاه بودن
```

ویژگی‌ها:

• شیء جزء می‌تونه بدون شیء کل وجود داشته باشه

• در Constructor از بیرون دریافت میشه (نه اینکه داخل ساخته بشه)

• رابطه "کل-جزء" (Whole-Part) هست

۳. Composition (ترکیب) – قوی‌ترین

تعریف: رابطه HAS-A قوی. یعنی جزء، بخشی از کل هست و بدون کل معنی نداره. اگه کل از بین بره، جزء هم از بین می‌ره.

تشبیه: خانه و اتاق‌ها. اتاق‌ها بدون خانه معنی ندارن. اگه خانه خراب بشه، اتاق‌ها هم از بین میرن.

علامت: Has a (strong) / Part of

```
1 // Composition - جزء بدون کل وجود نداره
2 class Room {
3     // Room معنی نداره House بدون Room
4 }
5
6 class House {
7     private Room room; // Composition
8
9     public House() {
10         this.room = new Room(); // داخل خودش ساخته میشه
```

1-Java Interview Question

```

1 // هم می‌میره Room، از بین بره House اگه
2 }
3 }
4
5 // استفاده
6 House house = new House(); // هم ساخته میشه House، Room با ساخت
7 // هم از بین می‌ره room، از بین بره house وقتی

```

ویژگی‌ها:

• شیء جزء بدون شیء کل وجود نداره

• داخل Constructor خود کل ساخته میشه (نه از بیرون)

• با نابودی کل، جزء هم نابود میشه

جدول مقایسه سه رابطه

ویژگی	Association	Aggregation	Composition
قدرت	ضعیف ●	متوسط ●	قوی ●
وابستگی عمری	هیچی	کم	کامل
جزء مستقل؟	کاملاً مستقل	مستقل	وابسته به کل
مثال	دکتر و بیمار	دانشگاه و دانشجو	خانه و اتاق
در کد	پارامتر متد	از بیرون دریافت	داخل ساخته میشه
نماد UML	خط ساده —▶	لوزی توخالی — — —	لوزی پر — — —

🎯 خلاصه کاربردی برای مصاحبه

- 1 Association (ارتباط) → "Uses a" → ضعیف‌ترین
- 2 Aggregation (تجمیع) → "Has a" → متوسط (جزء مستقل)
- 3 Composition (ترکیب) → "Has a" → قوی‌ترین (جزء وابسته)

نکته طلایی برای مصاحبه:

"همیشه سعی کن به جای Inheritance (IS-A) از Composition (HAS-A) استفاده کنی. این قانون طلایی طراحی شی‌گراست."

مثال کامل در یک کلاس 📝

```

1 // ۱. Association - فقط استفاده
2 class Driver {
3     public void drive(Car car) { // Association
4         car.move();
5     }
6 }
7
8 // ۲. Aggregation - جمع شدن
9 class Player {
10     private String name;
11     // Player می‌تونه تنها هم باشه
12 }
13

```

1-Java Interview Question

```
1 class Team {
2     private List<Player> players; // Aggregation
3
4     public Team(List<Player> players) {
5         this.players = players; // از بیرون میاد
6     }
7 }
8
9 // ۳. Composition - بخشی از کل
10 class Engine {
11     // Engine بدون Car معنی نداره
12 }
13
14 class Car {
15     private Engine engine; // Composition
16
17     public Car() {
18         this.engine = new Engine(); // داخل ساخته میشه
19     }
20 }
```

! سوال مصاحبه‌ای ۲: چرا "ترکیب (Composition) را بر وراثت ترجیح بده" به اصل طراحی معروفه؟ مشکل وراثت چیه؟

جواب حیاتی: وراثت باعث ایجاد جفت‌شدگی شدید (Tight Coupling) بین والد و فرزند میشه. یعنی:

• تغییر در کلاس والد می‌تونه تمام کلاس‌های فرزند رو بشکند. (بهبش میگن مشکل Fragile Base Class).

• وراثت، جزئیات پیاده‌سازی والد رو برای فرزند فاش می‌کنه (نقض کپسوله‌سازی).

• در جاوا که فقط یه والد می‌تونی داشته باشی، خیلی محدودکننده است.

ترکیب (Composition): یعنی به جای extends کردن، یه نمونه از اون کلاس رو به عنوان یه فیلد در کلاس خودت داشته

باشی (HAS-A). اینطوری انعطاف‌پذیری خیلی بالاتری داری و می‌تونی رفتار رو در زمان اجرا عوض کنی.

1 // روش بد (وراثت) a.

2 class TaxCalculator {

3 double calculate() { ... }

4 }

5 class IranTaxCalculator extends TaxCalculator { ... } // tight coupling

6

7 // روش خوب (ترکیب) b.

8 interface TaxStrategy { double calculate(); }

9 class IranTax implements TaxStrategy { ... }

10 class USATax implements TaxStrategy { ... }

11

12 class Invoice {

13 private TaxStrategy tax; // Composition!

14

15 public Invoice(TaxStrategy tax) {

16 this.tax = tax;

17 }

18 }

19 // حالا هر وقت خواستی، استراتژی مالیاتی رو عوض می‌کنی c.

20 new Invoice(new IranTax());

```
21 new Invoice(new USATax());
```

! سوال مصاحبه‌ای ۲.۵: فرق Composition و Aggregation چیه؟

جواب: هر دو نوعی رابطه HAS-A هستند، ولی تفاوت در طول عمر هست:

• Composition (ترکیب - قوی): شیء فرزند بدون والد وجود نداره. اگر والد بمیره، فرزند هم می‌میره.

• Aggregation (تجمع - ضعیف): شیء فرزند می‌تونه مستقل از والد وجود داشته باشه.

1 // Composition - خانه و اتاق‌ها

2 class Room { ... }

3 class House {

4 private Room room; // با ساخت خانه، اتاق ساخته میشه

5 public House() {

6 this.room = new Room(); // اتاق فقط درون خانه معنی داره

7 }

8 }

9

10 // Aggregation - دانشگاه و دانشجو

11 class Student { ... }

12 class University {

13 private List<Student> students; // دانشجو قبل از دانشگاه وجود داشته

14 public University(List<Student> students) {

15 this.students = students; // دانشجو مستقل از دانشگاه هست

16 }

17 }

18

19 ## 2.5 چندریختی (Polymorphism) اصل سوم: چندریختی

1-Java Interview Question

1

2 از نوع والد، برای اشاره به اشیاء از نوع فرزند. یا صدا زدن به (Reference) تعریف ساده: **چندشکلی**. توانایی به **ارجاع** **
متد یکسان و گرفتن رفتارهای متفاوت بر اساس نوع شیء.

3

4 دو نوع داریم:

5

6 Method Overloading یا (Compile-Time) الف چندریختی زمان **کامپایل** ###

7

8 به کلاس، چندین متد با نام یکسان ولی پارامترهای **متفاوت** (تعداد، نوع، یا ترتیب) دارد. کامپایلر با توجه به آرگومان‌های
ورودی، تشخیص میدهد کدام متد صدا زده بشه.

9

10 ```java

11 class Printer {

12 void print(String s) { ... }

13 void print(int i) { ... }

14 void print(String s, int copies) { ... }

15 }

! سوال مصاحبه‌ای ۳: آیا با تغییر نوع بازگشتی می‌تونیم Overload کنیم؟

جواب: نه! نوع بازگشتی امضای متد نیست و به تنهایی برای Overloading کافی نیست. کامپایلر موقع صدا زدن متد، فقط
به اسم متد و پارامترها نگاه می‌کنه.

ب) چندریختی زمان اجرا (Runtime) یا Method Overriding

کلاس فرزند، به متد از کلاس والد رو با دقیقاً همون امضا (اسم، پارامترها، نوع بازگشتی) بازنویسی (@Override) می‌کنه. تصمیم‌گیری برای اینکه کدوم نسخه از متد اجرا بشه، در زمان اجرا و بر اساس نوع واقعی شیء گرفته میشه. به این میگن Dynamic Method Dispatch.

```

1 class Animal {
2     void sound() { System.out.println("Some sound"); }
3 }
4 class Dog extends Animal {
5     @Override void sound() { System.out.println("Woof!"); }
6 }
7 class Cat extends Animal {
8     @Override void sound() { System.out.println("Meow"); }
9 }
10
11 public class Main {
12     public static void main(String[] args) {
13         Animal myPet; // به ارجاع از نوع Animal
14
15         myPet = new Dog();
16         myPet.sound(); // خروجی: Woof! (رفتار Dog)
17
18         myPet = new Cat();
19         myPet.sound(); // خروجی: Meow (رفتار Cat)
20     }
21 }

```

! سوال مصاحبه‌ای ۴: @Override چیه و آیا اجباریه؟

جواب: اجباری نیست، ولی به شدت توصیه میشه. این Annotation به کامپایلر میگه "من قصد دارم یه متد از والد رو بازنویسی کنم". اگه اشتباهی اسم متد رو غلط بنویسی یا پارامترهاش رو اشتباه بگیری، کامپایلر بهت خطا میده و نمی‌ذاره. بدون @Override، متد یه متد جدید محسوب میشه و چندریختی اتفاق نمی‌افته.

2.6 اصل چهارم: انتزاع (Abstraction)

تعریف ساده: پنهان کردن پیچیدگی و نمایش فقط اطلاعات ضروری. یعنی بگی چه کاری انجام میشه، نه چطور.

چرا مهمه؟ پیچیدگی سیستم رو کم میکنه و به کاربر (یا کلاس دیگه) اجازه میده با یه رابط ساده کار کنه.

چطور پیاده میشه؟

• کلاس انتزاعی (Abstract Class): کلاسیه که نمی‌تونی ازش شیء بسازی (new کنی). فقط برای اینه که

زیرکلاس‌ها ازش ارث‌بری کنن. می‌تونه متدهای abstract (بدون بدنه) و متدهای معمولی داشته باشه.

• رابط (Interface): یه "قرارداد" کامله. قبل از Java 8 فقط می‌تونست متدهای abstract و ثابت‌ها (static final) داشته باشه. حالا می‌تونه متدهای default و static هم داشته باشه.

! سوال مصاحبه‌ای ۵ (حیاتی): تفاوت Abstract Class و Interface رو کامل و با مثال

توضیح بده

جواب:

کلاس انتزاعی (Abstract Class):

• با extends استفاده میشه (فقط یکی).

• می‌تونه constructor داشته باشه (برای مقداره‌ی به فیلدهای مشترک والد).

• می‌تونه متدهای معمولی (با بدنه) داشته باشه.

• فیلدهاش می‌تونن private, protected و غیره باشن.

• برای وقتی خوبه که کلاس‌های مرتبط، یه پیاده‌سازی مشترک دارن. (مثلاً HttpServlet که کلی کد مشترک داره).

رابط (Interface):

• با implements استفاده میشه (چندین تا می‌تونن پیاده‌سازی کنن).

• constructor نداره.

• متدهاش ذاتاً public abstract بودن. متدهای default و static هم الان داریم.

• فیلدهاش فقط public static final هستن (یعنی ثابت).

• برای وقتی خوبه که می‌خوای یه "قابلیت" یا "رفتار" مشترک بین کلاس‌های کاملاً نامرتبط تعریف کنی

(مثلاً Serializable, Comparable).

```

1 // قرارداد "چه کاری"
2 interface Payable {
3     void pay(int amount); // همه‌شون باید بلد باشن پرداخت کنن
4 }
5
6 // Abstract Class = پایه مشترک
7 abstract class Employee {
8     protected String name;
9     public Employee(String name) { this.name = name; }
10    abstract double calculateSalary(); // هر کارمندی حقوقش فرق داره
11    public void clockIn() { System.out.println(name + " clocked in."); } // کار مشترک
12 }
13
14 // حالا استفاده

```

```

1 class FullTimeEmployee extends Employee implements Payable {
2     public FullTimeEmployee(String name) { super(name); }
3     @Override double calculateSalary() { return 5000; }
4     @Override public void pay(int amount) { System.out.println("Paid via bank"); }
5 }

```

2.7 مفاهیم تکمیلی و حیاتی OOP

! سوال مصاحبه‌ای ۶: static چیست؟ متد static با متد غیر static چه فرقی دارد؟

جواب: static یعنی متعلق به کلاس، نه به نمونه (شیء) خاص.

• فیلد static: به متغیر که بین همه اشیاء اون کلاس مشترک. تغییرش در یکی، در همه اعمال میشه. (مثل شمارنده تعداد اشیاء ساخته شده).

• متد static: می‌تونی بدون ساختن شیء صداش بزنی (ClassName.method). به فیلدهای غیر static (فیلد شیء) دسترسی نداره، چون اصلاً شیئی وجود نداره که بخواد بهش دسترسی داشته باشه! main هم static هست چون JVM بدون ساختن هیچ شیئی باید بتونه اجراش کنه.

! سوال مصاحبه‌ای ۷: final چیست و کجاها استفاده میشه؟

جواب:

• متغیر final: به ثابت میسازه. بعد از مقداردهی اولیه، دیگه نمیشه تغییرش داد.

• متد final: نمی‌شه در زیرکلاس Override اش کرد.

• کلاس final: نمی‌شه ازش ارث‌بری کرد (مثل String).

! سوال مصاحبه‌ای ۸: رابطه IS-A و HAS-A رو با مثال بگو

• IS-A: وراثت (extends). Dog IS-A Animal.

• HAS-A: ترکیب (Composition). Car HAS-A Engine. (ماشین یه موتور داره، نه اینکه ماشین یه نوع موتور هست). همیشه اولویت با HAS-A است.

! سوال مصاحبه‌ای ۸.۵: چرا Multiple Inheritance در جاوا ممنوعه؟

جواب: به خاطر مشکل Diamond Problem (الماس).

```

      A
     /\
    B  C
     \/
      D
  
```

اگه B و C هر دو یه متد از A رو Override کنن، D باید کدوم رو استفاده کنه؟ جاوا با حذف Multiple Inheritance برای کلاس‌ها، این مشکل رو حل کرد. ولی با Interface‌ها این مشکل وجود نداره چون متدهای Interface بدنه ندارن (قبلاً).

راه حل جاوا: یک کلاس می‌تونه چندین Interface پیاده‌سازی کنه.

```

1 interface Flyable { void fly(); }
2 interface Swimmable { void swim(); }
3 class Duck implements Flyable, Swimmable {
4     // می‌تونه هر دو رو پیاده‌سازی کنه بدون مشکل
5 }
  
```

📖 فاز ۳: مباحث پایه‌ای پیشرفته در جاوا – کامل و

عمیق

این فاز شامل سه ستون اصلی: مدیریت استثناها (Exceptions)، جنریک‌ها (Generics) و کالکشن‌ها (Collections). هر سه توی هر پروژه واقعی روزانه باهاشون سر و کار داری.

3.1 مدیریت استثناها (Exception Handling)

سلسله مراتب استثناها در جاوا

همه چیز از کلاس Throwable ارث می‌بره که دو تا زیرکلاس اصلی داره:

• **Error**: مشکلات جدی و غیرقابل بازیابی (مثل `OutOfMemoryError`, `StackOverflowError`). برنامه‌نویس نباید و نمی‌تونه کاری براشون بکنه.

• **Exception**: مشکلاتی که برنامه‌نویس می‌تونه مدیریت کنه. خودش دو دسته داره:

• **Checked Exception** (استثناهای کنترل‌شده): زیرکلاس مستقیم `Exception` هستند (نه `RuntimeException`). کامپایلر مجبورت می‌کنه یا `try-catch` کنی، یا با `throws` پرتاب کنی.

• **Unchecked Exception** (استثناهای کنترل‌نشده): زیرکلاس `RuntimeException` هستند (مثل `NullPointerException`). کامپایلر اجباری برای مدیریتشون نداره.

1 Throwable

2 |— Error (Unchecked)

3 |— Exception

4 |— Checked **Exception** (مثل `IOException`, `SQLException`)

5 |— `RuntimeException` (Unchecked - مثل `NullPointerException`)

! سوال مصاحبه‌ای ۱: Checked Exception و Unchecked Exception رو با مثال توضیح بده. فلسفه وجودیشون چیه؟

جواب:

• **Checked:** برای شرایطی که قابل پیش‌بینی و خارج از کنترل برنامه هستن، ولی برنامه می‌تونه از پششون بریاد. (مثلاً فایل پیدا نشد: `FileNotFoundException`، یا قطع شدن شبکه). طراح جاوا می‌خواسته برنامه‌نویس رو مجبور کنه به این فکر کنه که اگه فایل نبود چی بشه.

• **Unchecked:** معمولاً ناشی از باگ‌های برنامه‌نویسی هست (مثلاً دسترسی به ایندکس اشتباه، فراخوانی متد روی `null`). انتظار میره برنامه‌نویس کد رو درست بنویسه که این اتفاق نیفته، نه اینکه همش `try-catch` بذاره.

```
1 // Checked Exception: مجبوری مدیریتش کنی
2 try {
3     FileReader file = new FileReader("file.txt"); // اگه فایل نباشه چی؟
4 } catch (FileNotFoundException e) {
5     // بازیابی از خطا
6 }
7
8 // Unchecked Exception: اجباری نیست
9 String s = null;
10 s.length(); // خودت مقصری - NullPointerException پرتاب
```

! سوال مصاحبه‌ای ۲: finally چه زمانی اجرا می‌شه؟ تنها زمانی که اجرا نمی‌شه کی هست؟

جواب: بلوک `finally` همیشه بعد از بلوک `try` و `catch` اجرا می‌شه، چه استثنا رخ بده، چه نده، چه `catch` بشه، چه نشه. حتی اگه توی `try` دستور `return` داشته باشیم، `finally` درست قبل از خروج از متد اجرا میشه.

تنها موردی که اجرا نمی‌شود: اگر `System.exit()` تو `try` یا `catch` صدا بشه، JVM کلاً خاموش میشه و `finally` اجرا نمی‌شه. یا اگر نخ اعدام بشه. کاربرد اصلیش بستن منابع (فایل، کانکشن دیتابیس) هست.

```
1 try {
2     System.out.println("Try block");
3     return; // اجرا میشه finally هم return حتی با
4 } finally {
5     System.out.println("Finally block"); // این پرینت میشه
6 }
```

! سوال مصاحبه‌ای ۳: Try-with-Resources چه و چه مشکلی رو حل می‌کنه؟

جواب: از Java 7 اضافه شد. مشکل اصلی `finally` این بود که برای بستن هر منبع، کلی کد تکراری و زشت می‌نوشتیم و خود بستن هم می‌تونست استثنا بده (Suppressed Exceptions). Try-with-Resources اینو اتوماتیک می‌کنه. هر کلاسی که `AutoCloseable` رو پیاده‌سازی کنه، می‌تونه تو `try` پرانتز قرار بگیره و `close()` ش به طور خودکار و امن صدا زده میشه.

```
1 // روش قدیمی و زشت
2 FileInputStream fis = null;
3 try {
4     fis = new FileInputStream("file.txt");
5 } catch (IOException e) { ... }
6 finally {
7     if (fis != null) {
8         try { fis.close(); } catch (IOException e) { ... }
9     }
10 }
```

```

1
2 // روش جدید و ترجیح داده شده (Try-with-Resources)
3 try (FileInputStream fis = new FileInputStream("file.txt")) {
4     // استفاده از fis
5 } catch (IOException e) {
6     // کار بسته می‌شود
7 }

```

3.2 جنریک‌ها (Generics)

هدف اصلی: فراهم کردن Type Safety (امنیت نوع) در زمان کامپایل، و حذف نیاز به Cast کردن دستی.

تصور کن قبل از Java 5 چه فاجعه‌ای بود:

```

1 // a. قبل از جنریک (Java 1.4)
2 List list = new ArrayList();
3 list.add("Hello");
4 list.add(123); // کامپایلر هیچ اعتراضی ندارد!
5
6 String s = (String) list.get(0); // نیاز به Cast
7 String s2 = (String) list.get(1); // خطا در زمان اجرا! ClassCastException

```

اما با جنریک

```

1 List<String> list = new ArrayList<>();
2 list.add("Hello");
3 // list.add(123); // خطای کامپایلر! عالیه

```

1 String s = list.get(0); // نیست Cast نیازی به

! سوال مصاحبه‌ای ۴: Type Erasure (پاک‌سازی نوع) چیه؟

جواب: جنریک‌ها یه ویژگی زمان کامپایل هستن! برای حفظ سازگاری با نسخه‌های قدیمی جاوا، کامپایلر:

1. نوع‌های جنریک رو چک می‌کنه.

2. همه اطلاعات نوع رو پاک می‌کنه.

3. هر جا لازم باشه، Cast اضافه می‌کنه.

پس در زمان اجرا (Runtime)، JVM اصلاً خبر نداره که List<String> بوده یا List<Integer>. هر دو فقط List هستن. برای همین نمی‌تونن instanceof روی نوع جنریک انجام بدی ((if (list instanceof List<String>) غلطه)).

! سوال مصاحبه‌ای ۵: Wildcard ها (?) در جنریک چیا هستن؟ extends و ? super رو

توضیح بده (PECS)

جواب: وقتی یه متد می‌نویسی، نمی‌خوای محدود به یه نوع خاص باشی، اما نمی‌خوای Type Safety رو هم از دست بدی. اینجا Wildcard ها میان:

• ? extends T (Upper-Bounded Wildcard): یعنی "هر نوعی که زیرکلاس T باشه (یا خود T)". از این ساختار

فقط می‌تونن بخونی (get). نمی‌تونن چیزی بذاری (add)، چون نمیدونی دقیقاً چه نوعیه.

• ? super T (Lower-Bounded Wildcard): یعنی "هر نوعی که اَبَرکلاس T باشه (یا خود T)". توی این ساختار

می‌تونن بذاری (add) از نوع T).

PECS: Producer Extends, Consumer Super

• اگه می‌خوای از یه کالکشن بخونی (Producer)، از extends استفاده کن.

• اگه می‌خوای توی یه کالکشن بنویسی (Consumer)، از super استفاده کن.

```

1 // Producer Extends: فقط میتونم اعداد رو بخونم
2 public void printNumbers(List<? extends Number> list) {
3     for (Number n : list) { // فقط خواندن
4         System.out.println(n);
5     }
6     // list.add(10); // خطا!
7 }
8
9 // Consumer Super: اضافه کنم Integer میتونم بهش
10 public void addNumbers(List<? super Integer> list) {
11     list.add(10); // نوشتن Integer
12     // Integer x = list.get(0); // بازگشتی! نوع بازگشتی
13 }

```

3.3 کالکشن‌ها (Collections Framework)

این چارچوب یه معماری یکپارچه برای کار با گروهی از اشیاء است. نقشه راه:

• Collection (Interface): ریشه. زیرمجموعه‌هاش: List, Set, Queue.

• Map (Interface): مستقل از Collection. برای زوج‌های کلید-مقدار.

3.3.1 لیست‌ها (Lists)

ArrayList در مقابل LinkedList

! سوال مصاحبه‌ای ۶: ArrayList و LinkedList رو کامل مقایسه کن. کی کدوم رو انتخاب

می‌کنی؟

جواب:

ArrayList: در داخل یه آرایه پویا هست. المان‌ها پشت سر هم توی حافظه‌ان.

• $O(1)$: $get(index)$ — دسترسی تصادفی فوق‌العاده سریع.

• $O(1)$ amortized: $add(element)$ — اضافه کردن به آخر سریع، ولی اگه ظرفیت آرایه پر بشه، یه آرایه بزرگتر ساخته و همه چیز کپی میشه.

• $O(n)$: $add(index, element)$ — اضافه کردن به وسط کند هست چون بقیه المان‌ها باید جابجا بشن.

LinkedList: در داخل یه لیست پیوندی دوطرفه هست. هر گره به قبلی و بعدی اشاره داره.

• $O(n)$: $get(index)$ — باید از اول لیست تا اون ایندکس حرکت کنه.

• $O(1)$: $add(index, element)$ — به شرطی که قبلاً به اون گره رسیده باشی. خود عمل درج/حذف سریع.

• $O(1)$: $addFirst/removeFirst$. برای پیاده‌سازی Stack و Queue خیلی خوبه.

چه موقع کدوم؟

• عملیات غالب get و set هست: حتماً ArrayList.

• عملیات غالب اضافه/حذف از وسط یا ابتدای یه لیست خیلی بزرگ: LinkedList.

• توی عمل و دنیای واقعی، ArrayList به خاطر حافظه کش CPU معمولاً انتخاب بهتری هست و LinkedList کمتر استفاده میشه.

3.3.2 مجموعه‌ها (Sets)

TreeSet و HashSet، LinkedHashMap

• HashSet: از HashMap در داخل استفاده می‌کند. ترتیب رو تضمین نمی‌کند.

عملیات add, remove, contains با $O(1)$.

• LinkedHashMap: مثل HashSet، ولی به لیست پیوندی هم نگه میداره که ترتیب درج (Insertion Order) رو حفظ کند.

• TreeSet: از به درخت سرخ-سیاه (Red-Black Tree) استفاده می‌کند. ترتیب طبیعی (Natural Ordering) یا ترتیب به Comparator که خودت میدی. عملیات اصلی با $O(\log n)$. برای وقتی خوبه که نیاز به چیزای مرتب داری.

3.3.3 نقشه‌ها (Maps)

Hashtable و HashMap، LinkedHashMap، TreeMap

! سوال مصاحبه‌ای ۷: HashMap دقیقاً چطور کار می‌کند؟ (سوال امتیازآور)

جواب:

1. HashMap به آرایه از سطل‌ها (Buckets) هست. هر سطل به Node (کلید، مقدار، اشاره‌گر به بعدی).

2. وقتی put(key, value) می‌کنیم:

• متد hashCode() روی کلید صدا زده میشه.

• به تابع هش ثانویه، این عدد رو به به ایندکس در آرایه تبدیل می‌کند.

• اگه اون سطل خالی بود، گره جدید اونجا ذخیره میشه.

•اگه پر بود (Collision)، به انتهای لیست پیوندی در اون سطل اضافه میشه. در Java 8، اگه تعداد گره‌های یه سطل از ۸ بیشتر بشه، اون لیست پیوندی به یه درخت سرخ-سیاه (Red-Black Tree) تبدیل میشه تا جستجو از $O(n)$ به $O(\log n)$ بهبود پیدا کنه.

3. وقتی `get(key)` می‌کنیم:

•دوباره `hashCode()` صدا زده میشه و ایندکس سطل پیدا میشه.

•اول با `hashCode` مقایسه می‌کنه، سپس برای اطمینان کامل، با `equals()` کلید داخل اون سطل رو با کلید ورودی مقایسه می‌کنه.

! سوال مصاحبه‌ای ۸: چرا باید `hashCode()` و `equals()` رو همزمان با هم Override کنیم؟ (سوال حیاتی)

جواب: بخاطر قرارداد (Contract) بین این دو متد:

•اگه `equals()` میگه دو شیء برابر هستن، `hashCode()` آن‌ها باید یک عدد یکسان برگردونه.

•اگه `hashCode()` دو شیء یکسان باشه، `equals` ممکنه برابر باشه یا نباشه.

عاقبت نقض قرارداد: اگه کلیدی رو با `put` ذخیره کنی، و بعداً همون کلید منطقی (با `equals` برابر) رو با `get` جستجو کنی، ولی `hashCode` ش فرق کنه، `HashMap` توی سطل اشتباه می‌گرده و چیزی پیدا نمی‌کنه. انگار کلیدت گم شده!

! سوال مصاحبه‌ای ۹: `Hashtable` و `ConcurrentHashMap` چه فرقی دارن؟ کدوم بهتره؟

جواب:

•`Hashtable`: قدیمی و از Java 1.0 هست. تمام متدهاش با `synchronized` هستن. یعنی در هر لحظه فقط یک نخ می‌تونه به کل `Map` دسترسی داشته باشه. `Performance` خیلی پایینی داره. کلید و مقدار `null` رو قبول نمی‌کنه.

•ConcurrentHashMap: مدرن و از Java 5. برای محیط‌های هم‌رند طراحی شده. به جای قفل کردن کل ساختار، فقط سطل مورد نظر (bucket-level locking) رو قفل می‌کنه. این کار باعث میشه چندین نخ بتونن همزمان روی سطل‌های مختلف کار کنن. Performance بسیار بالاتری داره. null رو هم قبول نمی‌کنه.

! سوال مصاحبه‌ای ۱۰: متدهای equals و hashCode برای به کلاس سفارشی چطور

درست بنویسیم؟

```

1 public class Student {
2     private int id;
3     private String name;
4
5     // equals مقایسه منطقی
6     @Override
7     public boolean equals(Object o) {
8         if (this == o) return true; // بهینه‌سازی
9         if (o == null || getClass() != o.getClass()) return false; // نوع چک
10        Student student = (Student) o; // Cast
11        return id == student.id; // مقایسه بر اساس کلید اصلی (id)
12    }
13
14    // hashCode مطابق با equals
15    @Override
16    public int hashCode() {
17        return Objects.hash(id); // استفاده کن id فقط از
18    }
19 }
```


📖 فاز ۴: جاوا پیشرفته (Advanced Java) –

برنامه‌نویسی تابعی و همروندی

تسلط به این مباحث نشون میده که با روندهای جدید جاوا همراهی و می‌تونی کد تمیز، خواناتر و بهینه‌تری بنویسی.

این فاز شامل دو بخش بزرگه: برنامه‌نویسی تابعی (Functional Programming) با Lambda و Stream، و برنامه‌نویسی همروند (Concurrent Programming) با Thread ها. هر دوتاش توی مصاحبه و توی کار واقعی خیلی مهمن.

4.1 برنامه‌نویسی تابعی (Functional Programming) در جاوا

از Java 8 به بعد، جاوا دیگه فقط یه زبان شی‌گرا نیست. مفاهیم برنامه‌نویسی تابعی بهش اضافه شدن. هدف: نوشتن کدی که خواناتر، کوتاه‌تر و کم‌خطاتر باشه و راحت‌تر بتونیم پردازش‌های موازی انجام بدیم.

4.1.1 لامبدا (Lambda Expressions)

تعریف ساده: لامبدا یه تابع بی‌نام (Anonymous Function) هست. یعنی یه متد بدون اسم که می‌تونیم مثل یه شیء باهاش رفتار کنیم (به عنوان آرگومان پاسش بدیم، توی متغیر ذخیرش کنیم).

چرا لامبدا اومد؟ قبل از Java 8، برای تعریف یه رفتار ساده (مثلاً یه Thread جدید) مجبور بودیم یه کلاس بی‌نام (Anonymous Inner Class) بنویسیم که پر از کدهای اضافی و زشت بود.

1-Java Interview Question

```
1 // a. روش قدیمی با Anonymous Inner Class
2 Runnable oldWay = new Runnable() {
3     @Override
4     public void run() {
5         System.out.println("Running old way");
6     }
7 };
8
9 // روش جدید با Lambda (Java 8+)
10 Runnable newWay = () -> System.out.println("Running lambda way");
```

ساختار لامبدا

```
1 (param1, param2) -> { body }
```

• (پارامترها): ورودی‌های تابع. اگر فقط یک پارامتر باشد و نوعش رو ننویسی، می‌تونی پرانتز رو حذف کنی.

• <-: علامت فلش.

• { بدنه }: کد اجرایی. اگر فقط یک خط باشد، می‌تونی {} و return رو حذف کنی.

```
1 // a. مثال‌های معتبر لامبدا
2 () -> System.out.println("Hello"); // بدون پارامتر
3 x -> x * 2; // یک پارامتر، یک خط
4 (x, y) -> { // چند پارامتر، چند خط
5     int sum = x + y;
6     return sum;
7 };
```

! سوال مصاحبه‌ای ۱: Functional Interface (واسط تابعی) چیه و چه ربطی به لامبدا داره؟

داره؟

جواب: یه اینترفیس که فقط و فقط یک متد انتزاعی داشته باشه. به این متد میگن Abstract Method. لامبدا دقیقاً پیاده‌سازی همین یک متد رو فراهم می‌کنه.

• کامپایلر می‌فهمه که لامبدا باید اون یک متد رو پیاده‌سازی کنه.

• برای جلوگیری از خطا، بهتره از Annotation مخصوص @FunctionalInterface استفاده کنیم. این Annotation کامپایلر رو مجبور می‌کنه چک کنه که اینترفیس واقعاً فقط یه متد انتزاعی داره.

```

1 @FunctionalInterface
2 interface Calculator {
3     int calculate(int x, int y); // فقط یک متد انتزاعی
4 }
5
6 public class Main {
7     public static void main(String[] args) {
8         // calculate استفاده از لامبدا برای پیاده‌سازی متد
9         Calculator adder = (a, b) -> a + b;
10        Calculator multiplier = (a, b) -> a * b;
11
12        System.out.println(adder.calculate(5, 3)); // خروجی: 8
13    }
14 }
```

بسته java.util.function: جاوا به سری Functional Interface آماده داره که توی Stream API خیلی استفاده میشن. مهم‌ترین‌هاش

• `Predicate<T>`: به T میگیره، boolean برمیگردونه. (مثل filter که تست شرطی انجام میده).

• `Function<T, R>`: به T میگیره، به R برمیگردونه. (مثل map که تبدیل انجام میده).

• `Consumer<T>`: به T میگیره و هیچی برنمیگردونه (void). (مثل forEach).

• `Supplier<T>`: هیچی نمیگیره، به T جدید میسازه و برمیگردونه. (مثل `Collectors.toList()`).

Stream API 4.1.2

تعریف ساده: Stream به جریان (رودخونه) از داده‌هاست که به صورت اعلامی (Declarative) روشن عملیات انجام میدیم. به جای اینکه بگیم چطور (با حلقه for)، میگیم چه کاری (filter, map, collect).

! سوال مصاحبه‌ای ۲: Stream با Collection چه فرقی داره؟

جواب حیاتی:

• ذخیره‌سازی: Collection به ساختمان داده است که داده رو ذخیره می‌کنه. Stream اصلاً داده ذخیره نمی‌کنه،

فقط به مسیر برای انتقال و پردازش داده از به منبع (مثل Collection) هست.

• عملیات: Stream عملیات رو به دو دسته تقسیم می‌کنه: Terminal و Intermediate.

• Lazy Evaluation: به عملیات Intermediate (مثل filter) تا زمانی که به عملیات Terminal (مثل collect) صدا

زده نشه، اصلاً اجرا نمی‌شه. این بهینه‌سازی خیلی مهمیه.

عملیات‌های Stream

ساخت Stream:

```
1 // a. از Collection
2 List<String> names = Arrays.asList("Ali", "Reza", "Sara");
3 Stream<String> stream1 = names.stream();
4
5 // از آرایه
6 Stream<String> stream2 = Arrays.stream(new String[]{"X", "Y"});
```

عملیات میانی (Intermediate): به Stream جدید برمیگردونن. Lazy هستند

• filter(Predicate): فیلتر بر اساس شرط.

• map(Function): تبدیل هر عنصر به یه چیز دیگه.

• sorted(): مرتب‌سازی.

• distinct(): حذف موارد تکراری.

• limit(n): فقط n عنصر اول.

عملیات پایانی (Terminal): به نتیجه (غیر Stream) برمیگردونن و جریان رو می‌بندن

• collect(Collectors.toList()): جمع‌آوری المان‌ها در یه List.

• forEach(Consumer): انجام عملی روی هر عنصر.

• reduce(BinaryOperator): ترکیب کل المان‌ها در یک مقدار واحد (مثلاً جمع کل).

1-Java Interview Question

```
1 // a. مثال کامل
2 List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
3
4 List<Integer> evenSquares = numbers.stream()
5   .filter(n -> n % 2 == 0)    // ۱. Intermediate: فقط زوج‌ها
6   .map(n -> n * n)           // ۲. Intermediate: به توان ۲ برسودن
7   .sorted((a, b) -> b - a)    // ۳. Intermediate: نزولی مرتب کن
8   .limit(3)                  // ۴. Intermediate: فقط ۳ تا اول
9   .collect(Collectors.toList()); // ۵. Terminal: بریز توی یه لیست
10
11 // نتیجه: [36, 64, 100] (بزرگترین مربع‌های زوج: ۱۰، ۸ و ۶)
```

! سوال مصاحبه‌ای ۳: متد reduce چیکار می‌کنه؟

جواب: عملیات پایانی (Terminal) هست که عناصر Stream رو یکی یکی با هم طبق یه قاعده ترکیب می‌کنه تا به یه مقدار واحد برسیم. یه جورایی for حلقه‌ای هست که یه accumulator داریم.

```
1 // a. جمع اعداد به لیست
2 List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
3 int sum = numbers.stream()
4   .reduce(0, (subtotal, element) -> subtotal + element);
5 // 0 + 1 = 1
6 // 1 + 2 = 3
7 // 3 + 3 = 6
8 // 6 + 4 = 10 -> sum = 10
```

4.2 برنامه‌نویسی هم‌رند و چندریسمانی (Concurrency & Multithreading)

هدف: اجرای همزمان چندین بخش از برنامه برای استفاده بهتر از CPU و افزایش سرعت پاسخگویی. (به جای اینکه کاربر منتظر بمونه، به Thread دیگه کار رو انجام بده).

4.2.1 ساختن و اجرای Thread

دو روش اصلی:

1. extends Thread: کلاس خودت رو از کلاس Thread ارث ببر و متد run() رو Override کن. (روش قدیمی، محدودیت‌ها چون نمی‌تونی از کلاس دیگه‌ای ارث ببری).

2. implements Runnable (توصیه شده): کلاس خودت Runnable رو پیاده‌سازی کنه، بعد به شیء Thread بسازی و Runnable رو بهش پاس بدی.

```

1 // روش توصیه شده با Runnable
2 class MyTask implements Runnable {
3     @Override
4     public void run() {
5         System.out.println("اجرا توسط: " + Thread.currentThread().getName());
6     }
7 }
8 public class Main {
9     public static void main(String[] args) {
10         Thread t1 = new Thread(new MyTask());
11         Thread t2 = new Thread(new MyTask());
12

```

```

13    t1.start(); // رو مستقیم صدا نزن run() هیچوقت
14    t2.start(); // start() به Thread و جدید میسازه و run() صدا میزنه
15 }
16 }

```

4.2.2 مشکلات رایج در همروندی

! سوال مصاحبه‌ای ۴: Race Condition (شرایط مسابقه) چیه؟

جواب: وقتی دو یا چند Thread به یه منبع مشترک (Shared Resource) دسترسی همزمان دارن و حداقل یکی از اون‌ها منبع رو تغییر میده. نتیجه نهایی به ترتیب زمان‌بندی Thread ها بستگی داره و غیرقابل پیش‌بینی و خطرناک میشه.

```

1 public class Counter {
2     private int count = 0;
3
4     // داره Race Condition نیست! Thread-Safe این متد
5     public void increment() {
6         count++; // این یه عملیات ۳ مرحله‌ایه: خواندن، افزودن، نوشتن
7     }
8 }

```

! سوال مصاحبه‌ای ۵: synchronized چطور مشکل Race Condition رو حل می‌کنه؟

جواب: کلمه کلیدی synchronized تضمین می‌کنه که در هر لحظه فقط یک Thread بتونه متد یا بلوک مشخصی از کد رو برای یک شیء خاص اجرا کنه. این کار با قفل کردن (Lock) انجام میشه.


```

1 public class SafeCounter {
2     private int count = 0;
3
4     public synchronized void increment() { // در هر لحظه Thread فقط یک
5         count++;
6     }
7
8     // روی یه بلوک synchronized یا
9     public void incrementBlock() {
10        synchronized (this) {
11            count++;
12        }
13    }
14 }

```

نکته: synchronized باعث کاهش Performance میشه، چون Thread ها باید منتظر هم بمونن. باید فقط جایی که لازمه استفاده بشه

! سوال مصاحبه‌ای ۶: Deadlock (بن بست) چیه و چطور رخ میده؟

جواب: وضعیتی که دو یا چند Thread برای همیشه منتظر یکدیگر می‌مونن. معمولاً زمانی رخ میده که:

• Thread A قفل منبع ۱ رو داره و قفل منبع ۲ رو میخواهد.

• Thread B قفل منبع ۲ رو داره و قفل منبع ۱ رو میخواهد.

• هیچکدام قفل خودشون رو آزاد نمی‌کنن، پس هیچکدام به قفل بعدی نمیرسن. برنامه هنگ می‌کنه.

راه‌های جلوگیری:

1. ترتیب قفل‌گیری رو یکسان کن (همه اول قفل منبع ۱ رو بگیرن، بعد ۲).

2. از tryLock () با زمان مشخص استفاده کن تا اگر نتونست قفل کنه، قفل‌های قبلی رو آزاد کنه و دوباره تلاش کنه.

! سوال مصاحبه‌ای ۷: تفاوت Callable و Runnable؟

جواب:

• Runnable: متد run () داره که مقدار برمیگردونه و نمی‌تونه Checked Exception پرتاب کنه.

• Callable<V>: متد call () داره که یه مقدار از نوع V برمیگردونه و می‌تونه استثنا پرتاب کنه.

```
1 Callable<String> task = () -> {
```

```
2 Thread.sleep(1000);
```

```
3 return "نتیجه کار";
```

```
4};
```

```
5 // استفاده کرد ExecutorService باید از Callable برای اجرای
```

! سوال مصاحبه‌ای ۸: ExecutorService و Thread Pool چیه؟ چرا ازشون استفاده

می‌کنیم؟

جواب: ساختن و از بین بردن Thread ها هزینه‌ی زیادی داره. به جای ساختن یه Thread جدید برای هر کار کوچیک (که ممکنه هزاران کار همزمان باشه)، از یه استخر (Pool) از Thread های آماده استفاده می‌کنیم.

• ExecutorService یه اینترفیس جاوا برای مدیریت این استخرهاست.

• کارها رو به صورت Runnable یا Callable بهش میدیم (submit).

• خودش یه Thread آزاد از استخر برمی‌داره و کار رو اجرا می‌کنه.

• بعد از اتمام، Thread رو نمی‌کشه، برش میگردونه به استخر برای کار بعدی.

```
1 ExecutorService executor = Executors.newFixedThreadPool(5); // استخری با ۵ Thread
```

```
2
```

```
3 for (int i = 0; i < 10; i++) {
```

```
4     executor.submit(() -> {
```

```
1         System.out.println(Thread.currentThread().getName() + " is running");
```

```
2     });
```

```
3 }
```

```
4
```

```
5 executor.shutdown(); // بعد از اتمام کارها، استخر رو خاموش کن
```

این هم از فاز ۴. با Stream API و Lambda می‌تونی کد خیلی تمیزتری بنویسی، و با Concurrency می‌فهمی برنامه‌های بزرگ چطور سریع و پاسخگو باقی میمونن.

! سوال مصاحبه‌ای ۹: CompletableFuture چیه و چه کاربردی داره؟

جواب: CompletableFuture یه راه مدرن برای برنامه‌نویسی ناهمگام (Asynchronous) در جاواست. به ما اجازه می‌ده:

• چندین Task رو به صورت زنجیره‌ای اجرا کنیم

• Taskها رو ترکیب کنیم

• خطاها رو مدیریت کنیم

```
1 // به صورت ناهمگام Task اجرای به a.
```

```
2 CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> {
```

```
3     // کار سنگین
```

```
4     return "Result";
```

```
5 });
```

6

7 // b. زنجیره‌ای کردن عملیات

8 `CompletableFuture.supplyAsync(() -> "Hello")`9 `.thenApply(String::toUpperCase) // HELLO`10 `.thenApply(s -> s + " World") // HELLO World`11 `.thenAccept(System.out::println); // چاپ نتیجه`

1

2 // c. ترکیب دو Task

3 `CompletableFuture<Integer> f1 = CompletableFuture.supplyAsync(() -> 5);`4 `CompletableFuture<Integer> f2 = CompletableFuture.supplyAsync(() -> 3);`5 `f1.thenCombine(f2, (a, b) -> a + b) // 8`6 `.thenAccept(System.out::println);`

! سوال مصاحبه‌ای ۱۰: Optional چیست و چرا استفاده می‌شود؟

جواب: Optional یک Container هست که می‌تونه یه مقدار داشته باشه یا خالی باشه (null). هدفش جلوگیری از NullPointerException هست.

1 // a. روش قدیمی (خطرناک)

2 `String name = getUsername();`3 `if (name != null) {`4 `System.out.println(name.length()); // NPE ممکنه`

5 }

6

7 // b. روش جدید با Optional

8 `Optional<String> optionalName = Optional.ofNullable(getUsername());`9 `optionalName.ifPresent(name -> System.out.println(name.length()));`

10

11 *// Optional متدهای مفید c.*12 `optionalName.orElse("Default");` *// مقدار پیش فرض null اگر بود،*13 `optionalName.orElseThrow(() -> new RuntimeException("Name not found"));`14 `optionalName.map(String::toUpperCase).orElse("EMPTY");`

! سوال مصاحبه‌ای ۱۱: متدهای Default و Static در Interface (از Java 8) چین؟

جواب: قبل از جاوا ۸، Interface ها فقط می‌توانستند متدهای abstract داشته باشند. از جاوا ۸:

Default Method: متدی با بدنه که می‌تونه در Interface باشه و کلاس‌های implement کننده می‌تونن override کنن یا نکنن.

Static Method: متدی که به خود Interface تعلق داره و با اسم Interface صدا زده میشه.

```

1  interface Vehicle {
2      // Abstract Method
3      void start();
4
5      // Default Method (بدنه داره)
6      default void stop() {
7          System.out.println("Vehicle stopped");
8      }
9
10     // Static Method
11     static void service() {
12         System.out.println("Vehicle in service");
13     }
14 }
15

```

1-Java Interview Question

```
16 class Car implements Vehicle {
17     @Override
18     public void start() {
22         System.out.println("Car started");
23     }
1 // stop() رو override نکرده، پس از Interface استفاده می‌کنه
2 }
3
4 // استفاده
5 Car car = new Car();
6 car.start(); // Car started
7 car.stop(); // Vehicle stopped
8 Vehicle.service(); // Vehicle in service
```

📘 فاز ۵: ساختمان داده و طراحی الگوریتم – کامل و

عمیق

هدف این بخش حفظ کردن کد نیست، بلکه درک نحوه کار و توانایی انتخاب ابزار مناسب برای هر مسئله است. در مصاحبه، ازت می‌خواند که مسئله رو روی تخته حل کنی و بعد تحلیلش کنی که چرا این راه حل خوبه.

5.1 تحلیل پیچیدگی (Big O Notation)

سوال: Big O چیست و چرا اهمیت دارد؟ (این سوال O همیشه می‌پرسن)

جواب: Big O یک نمادگذاری ریاضی است که نرخ رشد (حد بالا) زمان اجرا یا فضای مصرفی یک الگوریتم را بر اساس اندازه ورودی (n) توصیف می‌کند.

هدفش چیست؟ مقایسه مقیاس‌پذیری (Scalability) الگوریتم‌ها. وقتی n به سمت بی‌نهایت میل می‌کند، کدام الگوریتم بهتر عمل می‌کند؟ (ثابت‌ها را فاکتور می‌گیریم).

! سوال مصاحبه‌ای ۱: همه پیچیدگی‌های زمانی معروف را به ترتیب از بهترین به بدترین نام

ببر و برای هر کدام یک مثال واقعی در جاوا بزن

نماد	نام	مثال واقعی (جاوا / دنیای واقعی)	توضیح
$O(1)$	ثابت (Constant)	دسترسی به یه خانه آرایه <code>arr[5]</code> ، درج در <code>HashMap</code>	مستقل از n، همیشه زمان ثابت است. بهترین.
$O(\log n)$	لگاریتمی (Logarithmic)	جستجوی دودویی (<code>binarySearch</code>)، جستجو در <code>TreeMap</code>	با افزایش n، زمان خیلی کم زیاد می‌شود. عالی.
$O(n)$	خطی (Linear)	یک حلقه <code>for</code> ساده، جستجوی خطی در	زمان به نسبت مستقیم با n زیاد

نماد	نام	مثال واقعی (جاوا / دنیای واقعی)	توضیح
		ArrayList	می‌شود. خوب.
$O(n \log n)$	خطی-لگاریتمی (Linearithmic)	مرتب‌سازی سریع (QuickSort)، مرتب‌سازی ادغامی (MergeSort)	بهترین پیچیدگی برای الگوریتم‌های مرتب‌سازی عمومی.
$O(n^2)$	مربعی (Quadratic)	دو حلقه for تو در تو، مرتب‌سازی حبابی (BubbleSort)	بد. با بزرگ شدن n خیلی زود کند می‌شود.
$O(2^n)$	نمایی (Exponential)	محاسبه بازگشتی فیبوناچی بدون memoization	فاجعه. فقط برای n های خیلی کوچک قابل استفاده است.

چطور محاسبه کنیم؟

• حلقه for ساده: $O(n)$

• حلقه‌های تو در تو: توان n (مثلاً ۲ حلقه $= O(n^2)$)

• نصف کردن متوالی فضای جستجو: $O(\log n)$

• عملیات پشت سر هم: پیچیدگی بزرگترین عملیات غالب می‌شود. $(O(n) + O(\log n) = O(n))$

5.2 ساختمان داده‌های اصلی (Core Data Structures)

مصاحبه‌کننده می‌خواهد بداند تو فقط ArrayList و HashMap بلد نیستی، بلکه می‌دانی کی از کدام استفاده کنی.

۱. آرایه (Array)

• چیست؟ بلوکی از خانه‌های پشت سر هم در حافظه.

• مزیت: دسترسی تصادفی (get) با $O(1)$.

• عیب: سایز ثابت. درج/حذف از وسط با $O(n)$.

• کاربرد جاوا: String، int[].

۲. لیست پیوندی (Linked List)

• چیست؟ زنجیره‌ای از گره‌ها. هر گره داده و یک اشاره‌گر به گره بعدی (و قبلی در حالت دوطرفه) دارد.

• مزیت: درج/حذف از ابتدا/انتها با $O(1)$. سائز پویا.

• عیب: دسترسی تصادفی (get) با $O(n)$ چون باید از اول لیست حرکت کرد.

• کاربرد جاوا: LinkedList (حافظه بیشتر نسبت به ArrayList).

۳. پشته (Stack)

• چیست؟ ساختمان داده LIFO (Last-In, First-Out). آخرین چیزی که می‌ذاری، اولین چیزی که برمی‌داری. مثل پشته کتاب.

• عملیات اصلی:

• push(): گذاشتن روی پشته ($O(1)$).

• pop(): برداشتن از روی پشته ($O(1)$).

• peek(): دیدن بالاترین عنصر بدون برداشتن ($O(1)$).

• کاربرد: دکمه Back در مرورگر، Call Stack در JVM، ارزیابی عبارات ریاضی.

• جاوا: کلاس Stack قدیمی است. به جایش Deque (مثلاً ArrayDeque) توصیه می‌شود.

۴. صف (Queue)

• چیست؟ ساختمان داده FIFO (First-In, First-Out). مثل صف نانوايي. اولین کسی که می‌آید، اولین نفر خارج می‌شود.

• عملیات اصلی:

• enqueue()/offer(): اضافه به انتها ($O(1)$).

• dequeue()/poll(): برداشتن از ابتدا ($O(1)$).

• کاربرد: مدیریت درخواست‌ها در سرور، صف چاپگر، Message Brokers (مثل RabbitMQ).

• جاوا: Queue یک Interface است. پیاده‌سازی‌ها: LinkedList, PriorityQueue.

۵. هاش مپ (Hash Map) / جدول درهم‌سازی (Hash Table)

• چیست؟ ذخیره‌سازی زوج‌های کلید-مقدار. با استفاده از یک تابع هاش (hashCode())، کلید به یک ایندکس در یک آرایه (Bucket) تبدیل می‌شود.

• مزیت: میانگین $O(1)$ برای get, put, delete.

• عیب: بدترین حالت $O(n)$ در صورت برخورد (Collision) زیاد. بینظم.

• جزئیات حیاتی (در جاوا):

• برخورد (Collision): وقتی دو کلید، hashCode یکسان داشته باشند. راه حل: زنجیره‌سازی (Chaining).

هر سطل آرایه، یک لیست پیوندی است. در Java 8، اگر لیست طولانی شود (> 8)، برای بهبود به $O(\log n)$ به درخت سرخ-سیاه تبدیل می‌شود.

• اهمیت equals و hashCode: تابع هاش می‌گوید در کدام سطل بگردد. equals می‌گوید کدام کلید در آن سطل دقیقاً همانی است که می‌خواهیم.

5.3 الگوریتم‌های پایه

۱. جستجو (Searching)

• جستجوی خطی (Linear Search): تک‌تک خونه‌ها را پیمایش کن. $O(n)$. فقط برای داده‌های نامرتب.

• جستجوی دودویی (Binary Search): $O(\log n)$. عالی است، اما فقط روی داده‌های مرتب کار می‌کند. وسط چک می‌کند. اگر هدف بزرگتر بود، نیمه چپ را دور بریز. اگر کوچکتر بود، نیمه راست را دور بریز.

۲. مرتب‌سازی (Sorting)

• مرتب‌سازی حبابی (Bubble Sort): $O(n^2)$. فقط آموزشی!

• مرتب‌سازی سریع (Quick Sort): متوسط $O(n \log n)$. از یک عنصر محوری (pivot) استفاده می‌کند. معمولاً در عمل سریع‌ترین است. (الگوریتم Arrays.sort () جاوا برای primitives).

• مرتب‌سازی ادغامی (Merge Sort): همیشه $O(n \log n)$. پایدار (Stable) است. از روش تقسیم و حل (Divide & Conquer) استفاده می‌کند. نیاز به حافظه اضافی دارد. (الگوریتم Collections.sort () و Arrays.sort () برای اشیاء).

5.4 رویکردهای طراحی الگوریتم

! سوال مصاحبه‌ای ۲: مسائل را با چه رویکردهایی می‌توان حل کرد؟

۱. تقسیم و حل (Divide and Conquer):

• ایده: مسئله بزرگ را به زیرمسئله‌های کوچکتر و مشابه بشکن. زیرمسئله‌ها را حل کن. نتایج را ترکیب کن.

• مثال: Merge Sort، Quick Sort، جستجوی دودویی.

۲. برنامه‌نویسی پویا (Dynamic Programming - DP):

• ایده: وقتی زیرمسئله‌ها همپوشانی (Overlap) دارند (مثلاً تابع فیبوناچی، fib(5) و fib(4) هر دو به fib(3) نیاز دارند). به جای محاسبه مجدد، نتایج زیرمسئله‌ها را در یک حافظه (Memo) ذخیره کن.

• مثال کلاسیک: محاسبه فیبوناچی. از $O(2^n)$ به $O(n)$ کاهش می‌یابد.

۳. حریصانه (Greedy):

• ایده: در هر قدم، بهترین انتخاب محلی را انجام بده، به این امید که به بهترین جواب کلی برسی. همیشه جواب بهینه کلی را نمی‌دهد.

• مثال: الگوریتم دایکسترا (کوتاه‌ترین مسیر)، مسئله بازگرداندن پول خرد با کمترین تعداد سکه.

۴. پسگرد (Backtracking):

• ایده: مانند یک هزارتو، تمام مسیرهای ممکن را امتحان کن و هر جا به بن‌بست رسیدی، به عقب برگرد و مسیر دیگری را امتحان کن.

• مثال: مسئله هشت وزیر (Eight Queens)، حل جدول سودوکو.

5.5 جمع‌بندی مصاحبه‌ای: مقایسه‌های کلیدی

! سوال مصاحبه‌ای ۳: ArrayList در مقابل LinkedList؟

• ArrayList: دسترسی تصادفی $O(1)$ ، درج در وسط $O(n)$.

• LinkedList: دسترسی تصادفی $O(n)$ ، درج در وسط $O(1)$ (بعد از پیدا کردن مکان).

• قانون: در ۹۰٪ موارد ArrayList به دلیل بهینگی برای حافظه کش CPU و کارایی عمومی بهتر است.

! سوال مصاحبه‌ای ۴: HashMap در مقابل TreeMap؟

• HashMap: میانگین $O(1)$ برای عملیات اصلی. بدون ترتیب.

• TreeMap: $O(\log n)$ برای عملیات اصلی. کلیدها را به ترتیب (مرتب‌شده) نگه می‌دارد.

📖 فاز ۶: شبکه‌های کامپیوتری (Computer)

(Networks) – ویژه بک‌اند کاران

این بخش برای به توسعه‌دهنده بک‌اند حیاتی، مخصوصاً تو دنیای اینترپرایز که با REST API، میکروسرویس‌ها، پروتکل‌های امن و Load Balancer سروکار داری. مصاحبه‌کننده ازت انتظار داره بدونی وقتی به ریکوئست از فرانت‌اند میزنه بیرون، چه مسیری رو طی می‌کنه تا به بک‌اند تو برسه.

6.1 مدل OSI و TCP/IP

سوال: مدل OSI چیه و چرا باید بدونیمش؟

جواب: مدل OSI به مدل مرجع ۷ لایه‌س که ارتباطات شبکه رو استاندارد می‌کنه. هر لایه وظیفه مشخصی داره و با لایه بالا و پایین خودش در تعامله.

#	لایه (Layer)	وظیفه اصلی	مثال‌ها / پروتکل‌ها
۷	Application	رابط کاربر و شبکه	HTTP, HTTPS, FTP, SMTP, DNS
۶	Presentation	ترجمه، رمزنگاری، فشرده‌سازی	SSL/TLS, ASCII, JPEG
۵	Session	مدیریت نشست‌ها (Session)	NetBIOS, RPC
۴	Transport	انتقال مطمئن داده (Port)	TCP, UDP
۳	Network	مسیریابی بسته‌ها (IP)	IP, ICMP, Router
۲	Data Link	آدرس‌دهی فیزیکی (MAC)	Ethernet, Switch
۱	Physical	انتقال سیگنال‌های الکتریکی/نوری	کابل‌ها، Hub

اما مدل ساده‌تر TCP/IP که عملاً استفاده میشه ۴ لایه‌س:

• Application: ترکیب لایه‌های ۵-۷ (HTTP و غیره)

• Transport: لایه ۴ (TCP/UDP)

•Internet: لایه ۳ (IP)

•Network Access: ترکیب لایه‌های ۱-۲

? سوال مصاحبه‌ای ۱: توی مرورگر `https://example.com` رو می‌زنی. از لحظه Enter تا نمایش صفحه، چه اتفاقی می‌فته؟ (سوال طلایی و حیاتی)

جواب کامل:

1. DNS Lookup: مرورگر اول کش خودش، بعد کش OS و در نهایت DNS Resolver رو چک می‌کنه تا اسم دامنه (`example.com`) رو به IP تبدیل کنه.

2. TCP Connection (SYN, SYN-ACK, ACK): مرورگر یه درخواست اتصال TCP روی پورت ۴۴۳ (HTTPS) به اون IP می‌فرسته. این فرایند به "دست‌دهی سه‌مرحله‌ای" معروفه.

3. TLS Handshake (برای HTTPS): یه فرایند جداگونه برای تبادل گواهی (Certificate) و توافق روی کلیدهای رمزنگاری انجام میشه.

4. ارسال درخواست HTTP: مرورگر یه ریکوئست HTTP (مثلاً `GET /index.html`) می‌سازه و از طریق این تونل امن می‌فرسته.

5. پاسخ سرور: وب‌سرور (مثل Nginx) ریکوئست رو می‌گیره، به بک‌اند (مثل Spring Boot) پاس میده، Response می‌سازه و برمی‌گردونه.

6. رندر صفحه: مرورگر HTML رو می‌گیره، CSS و JS‌های ضمیمه رو هم دانلود می‌کنه، و صفحه رو رندر می‌کنه.

6.2 پروتکل‌های TCP و UDP (قلب لایه Transport)

? سوال مصاحبه‌ای ۲: تفاوت TCP و UDP چیه؟ کی کدوم رو استفاده می‌کنیم؟

جواب:

TCP (Transmission Control Protocol): اتصال‌گرا و مطمئن.

• تحویل بسته‌ها رو تضمین می‌کنه (با Acknowledgement).

• ترتیب بسته‌ها رو حفظ می‌کنه.

• کنترل جریان (Flow Control) و ازدحام (Congestion Control) داره.

• سربار (Overhead) بیشتری داره و کندتره.

• مناسب برای: HTTP، ایمیل، انتقال فایل (FTP)، هرچیزی که گم شدن یه بیتش فاجعه‌ست.

UDP (User Datagram Protocol): غیراتصال‌گرا و نامطمئن.

• بسته رو می‌فرسته، بدون تضمین برای رسیدن یا ترتیب.

• خیلی سریعتره و سربارش کمه.

• مناسب برای: استریم ویدئو، بازی‌های آنلاین، VoIP، DNS. (از دست رفتن چند بسته اهمیتی نداره، سرعت

مهمه).

? سوال مصاحبه‌ای ۳: Handshake سه مرحله‌ای TCP (Three-way Handshake) رو

توضیح بده

جواب: فرآیندی که قبل از ارسال داده، یک اتصال TCP برقرار می‌کنه:

1.SYN: کلاینت به بسته با پرچم SYN (همگام‌سازی) به سرور می‌فرسته. ("میخوام وصل شم، شماره ترتیب من X هست.")

2.SYN-ACK: سرور با به بسته با پرچم SYN و ACK (تأیید) جواب می‌دهد. ("باشه. شماره ترتیب من Y هست، و منتظر X+1 هستم.")

3.ACK: کلاینت به بسته با پرچم ACK می‌فرسته. ("رسید. شروع کنیم.")

6.3 پروتکل HTTP (زبان مادری بک‌اند)

ساختار یک درخواست HTTP:

```
1 GET /api/users/1 HTTP/1.1
2 Host: www.example.com
3 Authorization: Bearer <token>
4 Content-Type: application/json
5 Accept: application/json
6
7 (empty line - body for POST/PUT)
```

? سوال مصاحبه‌ای ۴: متدهای اصلی (Verbs) HTTP رو با کاربردها توضیح بده

• GET: دریافت یک منبع (Resource). (امن و Idempotent)

• POST: ایجاد یک منبع جدید. (نه امن، نه Idempotent)

• PUT: جایگزینی کامل یک منبع موجود. (Idempotent)

• PATCH: به‌روزرسانی بخشی از یک منبع. (نه Idempotent لزوماً)

• DELETE: حذف یک منبع. (Idempotent)

تفاوت Safe و Idempotent:

• Safe: عملیات فقط خواندنی است و سرور رو تغییر نمیده (GET, HEAD, OPTIONS).

• Idempotent (خودتوان): یعنی اگر یه ریکوئست رو چند بار بفرستی، نتیجه فرقی نکنه. (GET, PUT, DELETE).

POST خودتوان نیست (چندتا POST = چندتا رکورد جدید).

? سوال مصاحبه‌ای ۵: Status Code های مهم و پرتکرار رو نام ببر

2xx (Success)

• 200 OK: موفقیت در GET/PUT.

• 201 Created: موفقیت در ساخت منبع جدید (POST).

• 204 No Content: عملیات موفق ولی بدنه پاسخ خالیه (مثلاً بعد از DELETE).

3xx (Redirection)

• 301 Moved Permanently: تغییر مسیر دائم.

• 302 Found: تغییر مسیر موقت.

4xx (Client Error)

• 400 Bad Request: درخواست مشکل داره (مثلاً JSON اشتباه).

• 401 Unauthorized: احراز هویت نشده (لاگین نکرده).

• 403 Forbidden: دسترسی نداره (لاگین کرده ولی نقشش اجازه نداره).

• 404 Not Found: منبع پیدا نشد.

• 405 Method Not Allowed: متد اشتباهی (مثلاً GET به جای POST).

5xx (Server Error)

• 500 Internal Server Error: خطای عمومی و پیش‌بینی نشده سرور (یه خورده! Exception).

• 502 Bad Gateway: سرور بالادستی پاسخ نامعتبر داده.

• 503 Service Unavailable: سرور موقتاً در دسترس نیست.

6.4 مفاهیم پیشرفته و پرتکرار شبکه

? سوال مصاحبه‌ای ۶: REST API چیه؟ چه ویژگی‌هایی داره؟

جواب: (REST (Representational State Transfer) یه سبک معماری برای طراحی API‌های تحت وب هست.

• Stateless (بی‌وضعیت): هر درخواست از کلاینت به سرور باید تمام اطلاعات لازم رو داشته باشه. سرور وضعیت

(Session) کلاینت رو ذخیره نمی‌کنه. (این فلسفه JWT هم هست).

• Resource-Based: همه چیز یک منبع (Resource) محسوب میشه و با URL مشخص میشه (/users/1).

• استفاده از HTTP Verbs به صورت معنی‌دار: (GET برای خواندن، POST برای ساختن...).

• نمایش‌های چندگانه: یه منبع می‌تونه با فرمت‌های مختلف (JSON, XML) برگردونده بشه.

? سوال مصاحبه‌ای ۷: تفاوت بین HTTP/1.1 و HTTP/2 رو توضیح بده

HTTP/1.1:

• Head-of-Line Blocking: هر اتصال TCP فقط یک درخواست رو در یک زمان پردازش می‌کنه.

• محدودیت اتصال: مرورگرها مجبورن ۶-۸ اتصال همزمان به هر دامنه باز کنن.

• متن‌گرا (Text-based).

:HTTP/2

- Multiplexing (تقسیم‌بندی): چندین درخواست و پاسخ می‌تونن همزمان روی یک اتصال TCP جریان داشته باشن. (مشکل Head-of-Line Blocking رو در لایه Application حل کرد).
- Header Compression (HPACK): هدرها فشرده میشن.
- Server Push: سرور می‌تونه منابعی که کلاینت نیاز داره رو قبل از درخواست ارسال کنه.
- باینری (Binary).

? سوال مصاحبه‌ای ۸: CORS (Cross-Origin Resource Sharing) چیه و چرا وجود داره؟

جواب: یه مکانیزم امنیتی در مرورگرهاست (Same-Origin Policy). مرورگرها به طور پیش‌فرض نمی‌ذارن یه اسکریپت از دامنه A، به API دامنه B درخواست بده.

مشکل ما: وقتی فرانت‌اند (مثلاً روی localhost:3000) و بک‌اند (روی localhost:8080) جدا هستن.

راه حل از سمت بک‌اند: سرور باید در هدر پاسخ، Access-Control-Allow-Origin رو با دامنه مجاز (یا * برای همه) بفرسته. برای متدهای خاص مثل PUT/DELETE، یه درخواست "پیش‌پرواز" (Preflight) با متد OPTIONS چک می‌کنه که اجازه‌ها رو بررسی کنه.

6.5 مفاهیم امنیتی شبکه (مکمل فاز امنیت)

- SSL/TLS: پروتکل رمزنگاری که امنیت رو در لایه Transport فراهم می‌کنه. اسم رمز HTTPS هست. کاراش: (۱) رمزنگاری داده، (۲) تضمین یکپارچگی داده، (۳) احراز هویت سرور (با Certificate).
- DNS (Domain Name System): دفترچه تلفن اینترنت. اسم دامنه را به IP تبدیل می‌کنه.

📖 فاز ۷: سیستم عامل (Operating System) –

ویژه بک اند کاران

اینجا هدف این نیست که به ادمین لینوکس بشی، بلکه هدف اینه که بفهمی برنامه‌ای که می‌نویسی (مثلاً به اپلیکیشن Spring Boot) چطور با سیستم عامل تعامل می‌کنه، حافظه رو مدیریت می‌کنه، و درخواست‌های همزمان رو پردازش می‌کنه. مصاحبه‌کننده می‌خواد ببینه درک عمیقی از محیط اجرای کدت داری.

7.1 پردازش (Process) و ریس (Thread)

این پایه‌ای‌ترین و مهم‌ترین مفهوم در سیستم عامله که مستقیماً به جاوا ربط داره.

! سوال مصاحبه‌ای ۱: تفاوت پردازش (Process) و ریس (Thread) رو کامل توضیح بده

جواب:

• پردازش (Process): به برنامه در حال اجراست. وقتی Spring Boot رو اجرا می‌کنی، به Process ساخته میشه. هر Process دارای فضای حافظه مستقل و مجزا هست (شامل Stack، Heap، کد و داده). پردازش‌ها از هم ایزوله هستن و ارتباط بینشون (Inter-Process Communication - IPC) سنگین و کند هست (مثل Socket، Pipe). سیستم عامل، Process رو مدیریت می‌کنه.

• ریس (Thread): به واحد اجرایی درون به پردازش است. هر Process حداقل به Thread داره (Thread اصلی یا main). تمام Thread‌های به Process، فضای حافظه مشترک دارن (همون Heap و داده‌ها)، اما هر کدوم Stack مخصوص به خودشون رو دارن. ارتباط بین Thread‌ها خیلی سبک‌تر و سریع‌تره (چون حافظه مشترکه). اما این اشتراک، مشکل هم‌روندی (Race Condition) رو ایجاد می‌کنه.

ویژگی	پردازه (Process)	ریسه (Thread)
حافظه	فضای مجزا و مستقل	فضای مشترک (Heap) + Stack مجزا
ارتباطات	کند و پیچیده (IPC)	سریع و ساده (حافظه مشترک)
ساخت	زمان بر و پرهزینه	سریع و کم هزینه
ایزوله بودن	ایزوله کامل	ایزوله نیستن (مشکل امنیتی/پایداری)
تعریف واقعی	اجرای یک برنامه (java -jar app.jar)	یک مسیر اجرا درون آن برنامه (هر ریکوئست در Tomcat)

7.2 مدیریت حافظه (Memory Management)

؟ سوال مصاحبه‌ای ۲: حافظه در یک Process چطور سازماندهی میشه؟ بخش‌های اصلی

حافظه (Memory Layout) رو نام ببر

جواب: وقتی یه برنامه (Process) اجرا میشه، سیستم عامل یه فضای حافظه مجازی بهش اختصاص میده که به بخش‌های زیر تقسیم میشه:

1. Text (Code Segment): شامل کدهای ماشین برنامه (دستورالعمل‌ها). معمولاً فقط خواندنی (Read-only).
2. Data Segment: شامل متغیرهای سراسری (Global) و ایستا (Static) که در طول عمر برنامه وجود دارن.
3. Stack (پشته): شامل متغیرهای محلی توابع، پارامترها و آدرس‌های بازگشت. هر Thread یه Stack مجزا داره. ساختار LIFO. سایز محدود (معمولاً چند مگابایت). پر شدنش باعث StackOverflowError میشه.
4. Heap (پشته): بزرگترین بخش حافظه که برای تخصیص حافظه پویا (Dynamic Memory Allocation) استفاده میشه. هر چیزی که در جاوا با new می‌سازی، توی Heap ساخته میشه. تمام Thread‌ها در یک Process به Heap مشترک دسترسی دارن. اینجاست که Garbage Collector جاوا کار می‌کنه.

7.3 همروندی و بن‌بست (Concurrency & Deadlock)

اینجا مکمل مباحث Concurrency در جاوا (فاز ۴) هست.

? سوال مصاحبه‌ای ۳: Deadlock (بن‌بست) چیه و ۴ شرط لازم برای رخ دادنش رو نام ببر.
(سوال کلاسیک سیستم عامل)

جواب: Deadlock وضعیتی که دو یا چند Process/Thread برای همیشه منتظر یکدیگر می‌مونن، چون هر کدوم منبعی رو در اختیار داره که دیگری بهش نیاز داره.

۴ شرط لازم (Coffman Conditions) که هر چهارتا باید همزمان برقرار باشن:

1. Mutual Exclusion (انحصار متقابل): حداقل یک منبع باید به صورت اشتراک‌ناپذیر باشه (در هر لحظه فقط یک Thread بتونه ازش استفاده کنه).

2. Hold and Wait (نگه‌داشتن و انتظار): یک Thread، حداقل یک منبع رو در اختیار داره و منتظر منابعی که در اختیار Thread های دیگه هست.

3. No Preemption (عدم قبضه کردن): سیستم عامل نمی‌تونه به زور منبع رو از یه Thread بگیره. Thread باید خودش داوطلبانه آزادش کنه.

4. Circular Wait (انتظار دایره‌ای): یه حلقه از Thread ها وجود داشته باشه که هرکدوم منتظر منبعی از Thread بعدی در حلقه باشه. (A منتظر B، B منتظر C، C منتظر A).

راه‌های مقابله (از دید برنامه‌نویس جاوا):

• همیشه قفل‌ها رو به یک ترتیب مشخص بگیر.

• از tryLock() با زمان انتظار مشخص استفاده کن.

• از قفل‌های تو در تو تا حد امکان پرهیز کن.

7.4 زمان‌بندی (CPU Scheduling) CPU

? سوال مصاحبه‌ای ۴: تفاوت بین Context Switch در Process و Thread چیست؟

جواب: Context Switch عملیاتی که CPU از اجرای یک Process/Thread به Process/Thread دیگر سوئیچ می‌کند. این کار توسط Dispatcher در سیستم عامل انجام می‌شود.

- Context Switch پردازش: خیلی پرهزینه است. چون باید کل فضای حافظه (Page Tables)، وضعیت ثابت‌ها و ... عوض بشود. (Translation Lookaside Buffer) TLB در CPU هم باید Flush بشود.
- Context Switch ریسک: به مراتب سبک‌تر. چون Thread‌های یک Process، فضای حافظه مشترک دارند. فقط باید Stack Pointer و Program Counter و ثابت‌ها عوض بشوند. به همین خاطر Thread رو Lightweight Process هم می‌گویند.

7.5 مفاهیم سیستمی پراکنده ولی مفید

• Virtual Memory (حافظه مجازی): تکنیکی که سیستم عامل با کمک سخت‌افزار، حافظه اصلی (RAM) و حافظه جانبی (Disk) رو ترکیب می‌کند تا برنامه بتونه حافظه‌ای بیشتر از RAM فیزیکی موجود استفاده کند. باعث میشه هر Process تصور کنه که فضای حافظه بزرگ و پیوسته فقط برای خودش داره.

• Cache (حافظه پنهان): حافظه‌ای سریع و کوچک نزدیک CPU که برای کاهش تأخیر در دسترسی به داده‌های پرکاربرد از RAM استفاده میشه. در طراحی نرم‌افزار (مثل Redis, Ehcache) هم همین مفهوم به کار میره.

• Lock و Semaphore:

• Lock (قفل): به مکانیزم انحصاری. فقط یک Thread می‌تونه در یک زمان Lock رو بگیره.
(مثل ReentrantLock در جاوا).

• Semaphore (چراغ راهنما): به شمارنده است که اجازه می‌ده تعداد مشخصی Thread به طور همزمان به یک منبع دسترسی داشته باشند. (مثلاً اتصال به دیتابیس با حداکثر ۱۰ ارتباط همزمان).

این هم از فاز ۷. همونطور که می‌بینی، مفاهیم سیستم عامل مستقیماً روی درک ما از JVM، Multi-threading و حتی Performance Tuning تأثیر دارن.

فاز ۸: پایگاه داده (Database) – کامل و عمیق

این فاز به دو بخش بزرگ تقسیم میشه:

• بخش اول: اصول طراحی پایگاه داده (Database Design)

• بخش دوم: آژ پایگاه داده (SQL)

بخش اول: اصول طراحی پایگاه داده (Database Design)

هدف این بخش اینه که یاد بگیری چطور یه مسئله دنیای واقعی رو به یه مدل داده‌ای بهینه و بدون افزونگی (Redundancy) تبدیل کنی. مصاحبه‌کننده می‌خواد ببینه تو فقط SQL بلد نیستی، بلکه می‌تونی یه دیتابیس رو از صفر طراحی کنی.

8.1.1 مدل‌سازی داده (Data Modeling)

۱. مدل (ER (Entity-Relationship Model):

• موجودیت (Entity): یه شیء یا مفهوم در دنیای واقعی که می‌خوایم اطلاعاتش رو ذخیره کنیم. (مثلاً Customer, Order, Product). در نهایت به جدول تبدیل میشه.

• صفت (Attribute): ویژگی‌های یه موجودیت. (مثلاً CustomerName, OrderDate). به ستون تبدیل میشن.

• رابطه (Relationship): ارتباط بین دو موجودیت.

• One-to-One (1:1): یه User یک Profile داره.

• One-to-Many (1:N): یه Customer می‌تونه چندین Order داشته باشه. (پرکاربردترین).

• Many-to-Many (M:N): یه Student چندین Course داره، و یه Course چندین Student داره. (نیاز به جدول واسطه داره).

کلیدها (Keys):

• Primary Key (PK - کلید اصلی): ستونی که به طور یکتا (Unique) هر سطر رو مشخص می‌کند. (مثلاً CustomerID). نمی‌تونه NULL باشه. هر جدول فقط یک PK داره.

• Foreign Key (FK - کلید خارجی): ستونی در جدول فرزند که به کلید اصلی جدول والد اشاره می‌کند. (مثلاً در جدول Order، ستون CustomerID به FK هست که به Customer.CustomerID اشاره می‌کند). برای حفظ یکپارچگی ارجاعی (Referential Integrity) به کار میره.

8.1.2 نرمال سازی (Normalization)

؟ سوال مصاحبه‌ای ۱: نرمال سازی چیه و چرا انجامش میدیم؟ (سوال تکراری در همه

مصاحبه‌ها)

جواب: فرآیند سازماندهی ستون‌ها و جداول برای کاهش افزونگی (Redundancy) و بهبود یکپارچگی (Integrity) داده‌ها. هدف اصلی اینه که هر داده فقط و فقط در یک جا ذخیره بشه. این کار از بروز ناهنجاری‌ها (Anomalies) در عملیات INSERT, UPDATE, DELETE جلوگیری می‌کند.

سه شکل نرمال اول (3NF) رو باید مثل آب خوردن بلد باشی:

1NF (First Normal Form):

• قانون: هر خانه از جدول باید تک‌مقداری (Atomic) باشه. یعنی نمی‌تونی توی یه سلول، چند تا شماره تلفن بذاری که با کاما جدا شدن.

• همچنین: هر سطر باید یکتا باشه (پس حتماً یه کلید اصلی نیاز داریم).

غلط (نقض 1NF)	درست (1NF)
StudentID, Courses	StudentID, CourseID
Math, Physics ,1	Math ,1
	Physics ,1

:2NF (Second Normal Form)

•پیش‌نیاز: جدول باید در 1NF باشد.

•قانون: هیچ صفت غیرکلیدی (Non-prime attribute) نباید فقط به بخشی از کلید اصلی وابسته باشد. (این موضوع فقط وقتی مطرح میشه که کلید اصلی ما ترکیبی (Composite Key) باشه).

•مثال: جدولی با کلید ترکیبی (EmployeeID, SkillID). اگه ستون EmployeeName توی این جدول باشه، این فقط به EmployeeID وابسته است، نه به کل کلید. پس باید این جدول بشکنه و EmployeeName به جدول Employee بره.

:3NF (Third Normal Form)

•پیش‌نیاز: جدول باید در 2NF باشد.

•قانون: هیچ صفت غیرکلیدی نباید به یک صفت غیرکلیدی دیگه وابسته باشه (بیش میگویند وابستگی گذرا یا Transitive Dependency). یعنی همه ستون‌های غیرکلیدی باید فقط و فقط به کلید اصلی وابسته باشن.

•مثال کلاسیک: جدول Employee: EmployeeID (PK), DepartmentID (FK), DepartmentName

•اینجا DepartmentName به DepartmentID وابسته است، نه به EmployeeID. پس باید DepartmentName رو ببریم توی یه جدول Department.

8.1.3 ایندکس‌ها (Indexes)

؟ سوال مصاحبه‌ای ۲: ایندکس چیه؟ مزایا و معایبش چیه؟ کی باید ایندکس بسازیم؟

جواب: ایندکس یه ساختمان داده (معمولاً B-Tree) هست که روی یک یا چند ستون از جدول ساخته میشه تا سرعت جستجو و واکنشی داده (SELECT با شرط WHERE) رو به شدت افزایش بده. مثل فهرست انتهای کتاب می‌مونه.

•مزیت: خواندن (SELECT) رو سریع‌تر می‌کنه.

•معایب:

•نوشتن (INSERT, UPDATE, DELETE) رو کندتر می‌کنه، چون خود ایندکس هم باید به‌روز بشه.

•فضای اضافی روی دیسک اشغال می‌کنه.

•کی ایندکس بسازیم؟ روی ستون‌هایی که زیاد توی WHERE، JOIN، ORDER BY و GROUP BY استفاده میشن.

•ستون‌های کلید خارجی (FK) کاندیدای عالی برای ایندکس هستن.

بخش دوم: آزمون پایگاه داده (SQL)

اینجا دیگه عملاته. باید بتونی کوئری بنویسی، Join بزنی، و داده‌ها رو گروه‌بندی کنی.

8.2.1 عملیات CRUD

•SELECT: خواندن داده.

•INSERT: اضافه کردن داده.

•UPDATE: به‌روزرسانی داده.

•DELETE: حذف داده.

8.2.2 JOINها (حیاتی)

? سوال مصاحبه‌ای ۳: انواع JOIN رو با مثال توضیح بده

جواب: فرض کن دو جدول داریم: Customers (مشتریان) و Orders (سفارشات).

•INNER JOIN (یا JOIN): فقط سطرهایی که در هر دو جدول تطابق دارن رو برمی‌گردونه.

- 1 SELECT * FROM Customers
- 2 INNER JOIN Orders ON Customers.CustID = Orders.CustID;
- 3 -- فقط مشتریانی که سفارش دارند

LEFT JOIN (یا LEFT OUTER JOIN): همه سطرهای جدول چپ (Customers) رو برمی‌گردونه. اگه جدول راست تطابقی نداشته، ستون‌های راست NULL میشن

SELECT * FROM Customers LEFT JOIN Orders ON Customers.CustID = Orders.CustID -- همه مشتریان.
آنهایی که سفارش ندارند، اطلاعاتشون کنارشون NULL هست.

RIGHT JOIN (یا RIGHT OUTER JOIN): عکس LEFT JOIN. همه سطرهای جدول راست. (کمتر استفاده میشه، میتونی با LEFT JOIN و جابجا کردن جدول‌ها همون کار رو بکنی)

8.2.3 گروه‌بندی و توابع تجمعی

? سوال مصاحبه‌ای ۴: GROUP BY و HAVING چه فرقی دارن؟

- GROUP BY: سطرها رو بر اساس یک یا چند ستون گروه‌بندی می‌کنه. معمولاً با توابع تجمعی (COUNT, SUM, AVG, MAX, MIN) استفاده میشه تا به خلاصه از هر گروه به دست بیاد.
- WHERE در مقابل HAVING:

•WHERE: سطرها رو قبل از گروه‌بندی فیلتر می‌کنه. نمی‌تونه روی توابع تجمعی شرط بذاره.

•HAVING: گروه‌ها رو بعد از گروه‌بندی فیلتر می‌کنه. برای شرط گذاشتن روی توابع تجمعی طراحی شده (مثلاً بگو فقط گروه‌هایی رو نشون بده که تعدادشون بیشتر از ۵ تاست).

1 -- a. مثال: تعداد سفارشات هر مشتری، فقط برای مشتریانی که بیش از ۲ سفارش دارن

2 SELECT CustomerID, COUNT(*) AS OrderCount

3 FROM Orders

4 GROUP BY CustomerID

5 HAVING COUNT(*) > 2;

8.2.4 زیرپرس‌وجوها (Subqueries)

? سوال مصاحبه‌ای ۵: Subquery چیه و کجا استفاده میشه؟

جواب: یه کوئری که داخل یه کوئری دیگه قرار میگیره. معمولاً داخل WHERE، FROM یا SELECT.

1 -- a. پیدا کردن مشتریانی که سفارش‌هاشون از میانگین کل بیشتره

2 SELECT * FROM Orders

3 WHERE Amount > (SELECT AVG(Amount) FROM Orders);

4

5 ## ترجیح داده میشه JOIN هست و JOIN معمولاً کندتر از WHERE در Subquery: نکته

8.2.5 مفاهیم پیشرفته SQL

•UNION و UNION ALL:

•این ستون‌ها رو روی هم می‌چسبونن (نه مثل JOIN که کنار هم می‌چسبونن). یعنی نتیجه دو کوئری رو باهم ترکیب می‌کنن. تعداد و نوع ستون‌ها باید یکسان باشه.

• UNION ALL: همه سطرها را نگه میدارد (حتی تکراری). سریعتر.

• UNION: سطرهای تکراری را حذف می‌کند (کندتر چون باید مرتب‌سازی و حذف انجام بده).

• EXISTS vs IN

• IN: بررسی می‌کند که به مقدار درون به لیست یا نتیجه Subquery هست یا نه. برای لیست‌های کوچک خوبه.

• EXISTS: بررسی می‌کند که آیا Subquery حداقل یک سطر برمی‌گردونه یا نه. معمولاً برای Subquery های بزرگ سریعتر از IN هست، چون به محض پیدا کردن اولین سطر تطابق، متوقف میشه.

اینم از فاز ۸. همونطور که می‌بینی، خیلی وسیع و مهمه. ولی نگران نباش، با تمرین SQL توی LeetCode یا HackerRank کاملاً مسلط میشی.

؟ سوال مصاحبه‌ای ۶: تفاوت SQL و NoSQL چیه؟ کی از کدوم استفاده کنیم؟

جواب: این یکی از مهم‌ترین سوالات مصاحبه‌ست.

ویژگی	SQL (Relational)	NoSQL (Non-Relational)
ساختار	جدولی (Table) با Schema ثابت	مستند (Document)، کلید-مقدار، گراف
زبان	SQL (استاندارد)	API اختصاصی هر دیتابیس
تراکنش	ACID (قوی)	BASE (انعطاف‌پذیر)
مقیاس‌پذیری	عمودی (Vertical)	افقی (Horizontal)
نمونه‌ها	MySQL, PostgreSQL, Oracle	MongoDB, Redis, Cassandra

کی از SQL استفاده کنیم؟

• داده‌ها ساختار مشخص دارن

• نیاز به Join و Query پیچیده داریم

• یکپارچگی داده مهمه (مثل بانک، حسابداری)

کی از NoSQL استفاده کنیم؟

• داده‌ها ساختار مشخصی ندارند (مثل JSON)

• سرعت و مقیاس‌پذیری مهمه

• داده‌ها به راحتی Shard میشن (مثل سیستم‌های بزرگ)

? سوال مصاحبه‌ای V: Redis چیه و چه کاربردی داره؟

جواب: Redis یه دیتابیس In-Memory (در حافظه) از نوع Key-Value هست. خیلی سریع‌ه چون داده‌ها رو در RAM نگه می‌داره.

کاربردهای اصلی:

1. Cache (کش): ذخیره نتایج پرتکرار برای سرعت بخشیدن

2. Session Storage: ذخیره اطلاعات نشست کاربر

3. Rate Limiting: محدود کردن تعداد درخواست‌ها

4. Message Queue: صف پیام‌های ساده (با Pub/Sub)

5. Counter: شمارنده‌های سریع (مثل تعداد بازدید)

Redis دستورات ساده # 1

2 SET user:1 "Ali" # ذخیره

3 GET user:1 # دریافت "Ali" ->

4 INCR page_views # افزایش شمارنده

5 EXPIRE session:1 3600 # منقضی شدن بعد از ۱ ساعت

مقایسه با دیتابیس معمولی:

Redis: فوق‌العاده سریع، ولی داده‌ها موقتی هستند و گروانه

دیتابیس معمولی: کندتر، ولی دائمی و ارزون

? سوال مصاحبه‌ای ۸: Transaction Isolation Level ها رو توضیح بده

جواب: وقتی چندین تراکنش همزمان اجرا میشن، ممکنه مشکلاتی پیش بیاد. ۴ سطح ایزوله‌سازی وجود داره:

سطح ایزولاسیون	Dirty Read	Non-Repeatable Read	Phantom Read
READ_UNCOMMITTED	ممکنه ❌	ممکنه ❌	ممکنه ❌
READ_COMMITTED	غیرممکن ✅	ممکنه ❌	ممکنه ❌
REPEATABLE_READ	غیرممکن ✅	غیرممکن ✅	ممکنه ❌
SERIALIZABLE	غیرممکن ✅	غیرممکن ✅	غیرممکن ✅

توضیح مشکلات:

Dirty Read: خواندن داده‌ای که هنوز Commit نشده

Non-Repeatable Read: دو بار خواندن یه ردیف، مقادیر متفاوت

Phantom Read: یه Query دو بار اجرا میشه و تعداد ردیف‌ها فرق داره

1 SQL تنظیم سطح ایزوله‌سازی در --

2 SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

فاز ۹: امنیت (Security) – کامل و عمیق

امنیت به لایه نیست، به اصل هست که باید توی تمام لایه‌های برنامه (از کد تا سرور) رعایت بشه. ما اینجا روی مفاهیمی تمرکز می‌کنیم که به بک‌اند کار جاوا (مخصوصاً با Spring Security) باید بلد باشه.

9.1 مفاهیم پایه و حیاتی امنیت

? سوال مصاحبه‌ای ۱: Authentication و Authorization چه فرقی دارن؟ (سوال تکراری و

پایه‌ای)

جواب:

• **Authentication** (احراز هویت): پاسخ به سوال "تو کی هستی؟". فرآیند تأیید هویت کاربر (مثلاً با بررسی نام کاربری و رمز عبور). اگر هویت تأیید نشه، خطای **401 Unauthorized** برگردونده میشه.

• **Authorization** (مجوزدهی/کنترل دسترسی): پاسخ به سوال "تو چه کاری می‌تونی انجام بدی؟". بعد از احراز هویت، مشخص می‌کنیم که این کاربر به چه منابعی (APIها، صفحات) دسترسی داره. مثلاً کاربر عادی نمیتونه پنل ادمین رو ببینه. اگر دسترسی نداشته باشه، خطای **403 Forbidden** برگردونده میشه.

مثال ساده:

• پاسپورت تو = **Authentication** (ثابت می‌کنه تو شهروندی).

• ویزای تو = **Authorization** (ثابت می‌کنه اجازه داری وارد به کشور خاص بشی).

؟ سوال مصاحبه‌ای ۲: چرا نباید پسورد رو به صورت Plain Text ذخیره کنیم؟ راه درستش

چیه؟

جواب: اگر دیتابیس به هر دلیلی (هک، خطای انسانی) لو بره، پسورد تمام کاربران فاش میشه. چون خیلی از کاربران از یه پسورد برای سایت‌های مختلف استفاده می‌کنن، این یه فاجعه‌ست.

راه حل: Hashing (درهم‌سازی)

• هش کردن: یه الگوریتم ریاضی یک‌طرفه (One-way) است. یعنی از روی هش، هرگز نمیشه به پسورد اصلی رسید.

• Salt (نمک): یه رشته تصادفی یکتا که به هر پسورد اضافه میشه و بعد ازش هش گرفته میشه. این کار حمله با جدول‌های رنگین‌کمان (Rainbow Table Attacks) رو خنثی می‌کنه.

• الگوریتم‌های مناسب: BCrypt، SCrypt یا Argon2. این الگوریتم‌ها عمداً کند (Slow) طراحی شدن تا حملات Brute-Force رو سخت کنن.

• در عمل (جاوا/Spring Security): از BCryptPasswordEncoder استفاده می‌کنیم.

9.2 احراز هویت مدرن: JWT (JSON Web Token)

؟ سوال مصاحبه‌ای ۳: JWT چیه، چه ساختاری داره و چطور کار می‌کنه؟ (سوال طلایی)

مصاحبه‌های Spring

جواب: JWT یه استاندارد باز (RFC 7519) برای انتقال امن اطلاعات (Claims) بین دو طرف به صورت یه JSON فشرده و امضا شده است. بزرگترین مزیتش Stateless (بدون وضعیت) بودنش هست، یعنی سرور نیازی به ذخیره Session ندارد و مقیاس‌پذیری رو بالا می‌بره.

ساختار یه JWT از سه بخش Base64-encoded تشکیل شده که با نقطه جدا میشن: HEADER.PAYLOAD.SIGNATURE

1.Header: نوع توکن (JWT) و الگوریتم امضا (مثلاً HS256).

```
1 { "alg": "HS256", "typ": "JWT" }
```

```
2 { "sub": "123", "name": "Ali", "role": "ADMIN", "exp": 1680000000 }
```

Signature (امضا): مهم‌ترین بخش برای امنیت. با این فرمول ساخته میشه

```
1 HMACSHA256( base64UrlEncode(header) + "." + base64UrlEncode(payload), SECRET_KEY )
```

این امضا تضمین می‌کنه که توکن در مسیر بین کلاینت و سرور دستکاری (Tamper) نشده. چون اگه Payload تغییر کنه، امضا دیگه معتبر نخواهد بود

فرآیند کار

• کاربر با username/password لاگین می‌کنه.

• سرور هویت رو چک می‌کنه و یک JWT امضاءشده برمی‌گردونه.

• کلاینت JWT رو ذخیره می‌کنه (مثلاً توی Local Storage یا Cookie).

• در هر درخواست بعدی، کلاینت توکن رو در هدر Authorization می‌فرسته:

```
1 Authorization: Bearer <token>
```

• سرور بدون مراجعه به دیتابیس، با استفاده از SECRET_KEY امضای توکن رو چک می‌کنه. اگر معتبر بود،

اطلاعات کاربر رو از Payload استخراج می‌کنه و پردازش انجام میشه.

نکته حیاتی: Payload فقط Base64 Encode شده، نه Encrypt! بنابراین هرگز اطلاعات

حساس مثل پسورد رو توی JWT قرار نده. هر کسی که توکن رو داشته باشه، می‌تونه

Payload رو بخونه

9.3 آسیب‌پذیری‌های رایج (OWASP Top 10)

? سوال مصاحبه‌ای ۴: SQL Injection چیه و چطور در جاوا ازش جلوگیری می‌کنی؟

جواب: وقتی ورودی کاربر مستقیماً (مثلاً از طریق concatenation رشته‌ها) به یک کوئری SQL اضافه بشه، و مهاجم بتونه با تزریق کد SQL ساختار کوئری رو تغییر بده.

1 -- a.

2 -- username: ' OR '1'='1 ورودی کاربر در فیلد

3 -- :کوئری نهایی

4 SELECT * FROM users WHERE username = ' OR '1'='1' AND password = '...'

راه حل در جاوا

• هرگز و مطلقاً هرگز با + زدن یا String.format() کوئری SQL نساز!

• همیشه از Parameterized Queries (Prepared Statements) استفاده کن.

• در JDBC: استفاده از PreparedStatement

1 String query = "SELECT * FROM users WHERE username = ?";

2 PreparedStatement ps = connection.prepareStatement(query);

3 ps.setString(1, userInput); // اینجا ورودی امن می‌شود

? سوال مصاحبه‌ای ۵: XSS (Cross-Site Scripting) و CSRF رو خیلی خلاصه بگو و راه

حلشون چیه؟

• XSS (حمله با اسکریپت بین سایتی): مهاجم یه اسکریپت مخرب جاوااسکریپت رو توی وبسایت قرار میده که در مرورگر قربانی اجرا میشه.

• راه حل: مهم‌ترین کار همیشه Escape/Encode کردن خروجی‌ها قبل از نمایش در HTML هست. کتابخانه‌هایی مثل Thymeleaf این کار رو خودکار انجام میدن.

• CSRF (جعل درخواست بین سایتی): مهاجم قربانی رو فریب میده تا وقتی توی سایت A لاگین هست، ناخواسته یک درخواست مخرب (مثلاً انتقال پول) به سایت A بفرسته.

• راه حل: استفاده از CSRF Token (یه مقدار تصادفی مخفی در فرم‌ها که توسط سرور تولید و بررسی میشه). نکته برای بک‌اند کار: اگر API تو Stateless هست و با JWT کار می‌کنی (توکن در هدر میاد، نه Cookie)، به طور پیش‌فرض در برابر CSRF ایمن هستی و معمولاً CSRF protection رو غیرفعال می‌کنن. اما برای برنامه‌های سنتی که با Session و Cookie کار می‌کنن، حتماً باید فعال باشه.

9.4 جمع‌بندی بهترین روش‌های امنیتی (Security Best Practices)

• اصل حداقل دسترسی (Principle of Least Privilege): به هر کاربر یا سرویس فقط و فقط کمترین سطح دسترسی که برای انجام کارش نیاز داره رو بده.

• لاگ‌گیری و مانیتورینگ: تمام تلاش‌های ناموفق برای ورود، خطاهای ۴۰۱ و ۴۰۳، و فعالیت‌های مشکوک رو لاگ کن.

• به‌روزرسانی وابستگی‌ها: همیشه کتابخانه‌ها رو آپدیت نگه دار. آسیب‌پذیری‌های معروف (CVEs) دائماً کشف میشن.

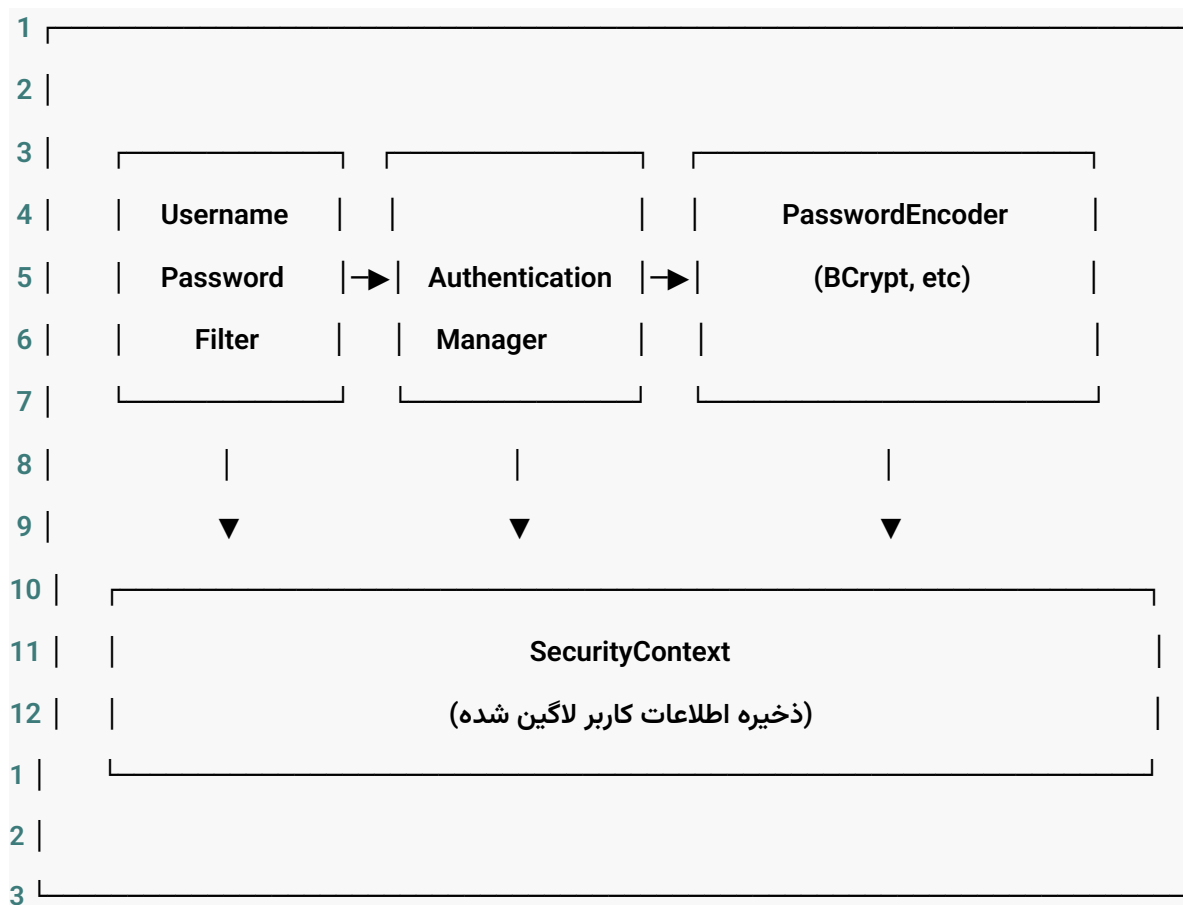
• HTTPS همه جا: هرگز، تأکید می‌کنم هرگز از HTTP ساده در Production استفاده نکن. همیشه SSL/TLS رو فعال کن.

۲. Spring Security

۹.۵ Spring Security (امنیت در Spring Boot)

Spring Security فریم‌ورک اصلی امنیت در اکوسیستم Spring هست.

۹.۵.۱ مفاهیم اصلی Spring Security



۹.۵.۲ PasswordEncoder (رمزنگاری پسورد)

```

1 @Configuration
2 class SecurityConfig {
3
4     @Bean
5     public PasswordEncoder passwordEncoder() {
6         return new BCryptPasswordEncoder(); // قوی و استاندارد
7         //یا: return new SCryptPasswordEncoder();
8         // یا: return new Argon2PasswordEncoder();
9     }
10 }
11 // استفاده
12 @Service
13 class UserService {
14     @Autowired
15     private PasswordEncoder passwordEncoder;
16
17     public void registerUser(String username, String rawPassword) {
18         String encodedPassword = passwordEncoder.encode(rawPassword);
19         // در دیتابیس encodedPassword ذخیره
20     }
21
22     public boolean checkPassword(String rawPassword, String encodedPassword)
23     {
24         return passwordEncoder.matches(rawPassword, encodedPassword);
25     }
26 }

```


۹.۵.۳ UserDetailsService (بارگذاری کاربر)

```

1 @Service
2 class CustomUserDetailsService implements UserDetailsService {
3
4     @Autowired
5     private UserRepository repository;
6
7     @Override
8     public UserDetails loadUserByUsername(String username)
9         throws UsernameNotFoundException {
10
11         User user = repository.findByUsername(username)
12             .orElseThrow(() -> new UsernameNotFoundException("User not found"));
13
14         return User.builder()
15             .username(user.getUsername())
16             .password(user.getPassword())
17             .authorities(user.getRoles().stream()
1             .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
1             .toArray(String[]::new))
19             .build();
20     }
21 }

```

```
1 @Component
2 class JwtService {
3
4     @Value("${jwt.secret}")
5     private String secretKey;
6
7     public String generateToken(String username, List<String> roles) {
8         return Jwts.builder()
9             .setSubject(username)
10            .claim("roles", roles)
11            .setIssuedAt(new Date())
12            .setExpiration(new Date(System.currentTimeMillis() + 86400000)) // 24h
13            .signWith(SignatureAlgorithm.HS256, secretKey)
14            .compact();
15     }
16
17     public String extractUsername(String token) {
18         return Jwts.parser()
19             .setSigningKey(secretKey)
20             .parseClaimsJws(token)
21             .getBody()
22             .getSubject();
23     }
24
25     public boolean isTokenValid(String token) {
```

1-Java Interview Question

```
7     try {
8         Jwts.parser().setSigningKey(secretKey).parseClaimsJws(token);
9         return true;
10    } catch (JwtException e) {
11        return false;
12    }
13 }
14 }
15
16 // JWT Filter
17 @Component
18 class JwtAuthenticationFilter extends OncePerRequestFilter {
19
20     @Autowired
21     private JwtService jwtService;
22
23     @Autowired
24     private CustomUserDetailsService userDetailsService;
25
26     @Override
27     protected void doFilterInternal(HttpServletRequest request,
1         HttpServletResponse response,
2         FilterChain filterChain) {
3         String authHeader = request.getHeader("Authorization");
4
5         if (authHeader != null && authHeader.startsWith("Bearer ")) {
```

1-Java Interview Question

```
6     String token = authHeader.substring(7);
7     String username = jwtService.extractUsername(token);
8
9     if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
10         UserDetails userDetails = userDetailsService.loadUserByUsername(username);
11         SecurityContextHolder.getContext().setAuthentication(
12             new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities())
13         );
14     }
15 }
16 filterChain.doFilter(request, response);
17 }
18 }
```

۹.۵.۵ تنظیمات امنیتی (Security Filter Chain)

```
1 @Configuration
2 @EnableWebSecurity
3 @EnableMethodSecurity
4 class SecurityConfig {
5
6     @Bean
7     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
8         return http
9             // ۱. CSRF (غیرفعال کن JWT برای)
10             .csrf(csrf -> csrf.disable())
11         ;
12     }
13 }
```

1-Java Interview Question

```
6      // تنظیم دسترسی ها ۲.
7      .authorizeHttpRequests(auth -> auth
8          .requestMatchers("/api/auth/**").permitAll() // لاگین اثبات نام
9          .requestMatchers("/api/admin/**").hasRole("ADMIN")
10         .requestMatchers("/api/user/**").hasAnyRole("USER", "ADMIN")
11         .anyRequest().authenticated()
12     )
13
14     // مدیریت Session (برای JWT Stateless)
15     .sessionManagement(session -> session
16         .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
17     )
18
19     // JWT Filter اضافه کردن ۴.
20     .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
21
22     .build();
23 }
24 }
```

۹.۵.۶ استفاده از @PreAuthorize و @Secured

```
1 @RestController
2 @RequestMapping("/api/users")
3 class UserController {
4
5     // می تونه ADMIN فقط
```

1-Java Interview Question

```
6  @PreAuthorize("hasRole('ADMIN')")
7  @GetMapping("/all")
8  public List<User> getAllUsers() { ... }
9
10 // USER یا ADMIN
11 @PreAuthorize("hasAnyRole('USER', 'ADMIN')")
12 @GetMapping("/{id}")
13 public User getUser(@PathVariable Long id) { ... }
14
15 // بررسی اینکه کاربر خودش باشد
16 @PreAuthorize("#id == authentication.principal.id")
17 @PutMapping("/{id}")
18 public User updateUser(@PathVariable Long id, @RequestBody User user) { ... }
19
20 // فقط USER
21 @Secured("ROLE_USER")
22 @PostMapping("/profile")
23 public User updateProfile() { ... }
24 }
```

9.5.7 Method Security (در Service)

```
1 @Service
2 class OrderService {
3
4  // فقط ADMIN
5  @PreAuthorize("hasRole('ADMIN')")
```

```

6 public List<Order> getAllOrders() { ... }
7
8 // مدیر شرکت یا صاحب سفارش
9 @PreAuthorize("hasRole('ADMIN') or #userId == authentication.principal.id")
10 public Order getOrder(Long userId, Long orderId) { ... }
11 }

```

۹.۵.۸ Authentication Entry Point (مدیریت خطاهای امنیتی)

```

1 @Component
2 class CustomAuthenticationEntryPoint implements AuthenticationEntryPoint {
3
4     @Override
5     public void commence(HttpServletRequest request,
6                           HttpServletResponse response,
7                           AuthenticationException authException) {
8         response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
9         response.setContentType(MediaType.APPLICATION_JSON_VALUE);
10        response.getWriter().write("""
11            {
12                "error": "Unauthorized",
13                "message": "Please login first"
14            }
15        """);
16    }
17 }

```

راهنمای کاربردی دیزاین پترن‌های پرکاربرد در جاوا

دیزاین پترن یا الگوی طراحی، یک راه حل کلی و قابل تکرار برای حل یک مشکل رایج در طراحی نرمافزار است، که به جای کدنویسی از صفر، یک قالب یا نقشهٔ اثبات شده برای ساختاردهی کد و بهبود انعطافپذیری و نگهداری آن در اختیار برنامه‌نویس قرار میدهد.

ساختار:

• برای هر الگو:

• مشکل رایج (نشانه استفاده)

• راه حل (ایده پترن)

• مثال ساده در جاوا

• مزایا

• معایب / هشدارها

۱. Creational (ایجاد)

این الگوها به نحوه ساخته شدن اشیاء می‌پردازند. کارشان این است که فرایند `new` کردن را هوشمندانه‌تر کنند تا سیستم از جزئیات ساخت شیء بی‌خبر باشد و کد انعطاف‌پذیرتر شود. مثال معروف: Singleton (تضمین یک نمونه منحصر به فرد)، Factory (ساخت اشیاء بدون مشخص کردن کلاس دقیق آنها).

۲. Structural (ساختاری)

این الگوها به ترکیب کلاس‌ها و اشیاء برای ساختن ساختارهای بزرگ‌تر می‌پردازند. هدفشان این است که با استفاده از ارث‌بری یا ترکیب، رابط‌های ساده‌تری برای کار با سیستم‌های پیچیده فراهم کنند. مثال معروف: Adapter (هماهنگ‌کننده دو رابط ناسازگار)، Decorator (افزودن قابلیت جدید به اشیاء به صورت پویا).

۳. Behavioral (رفتاری)

این الگوها به ارتباط و تعامل بین اشیاء می‌پردازند. کارشان تخصیص مسئولیت‌ها بین اشیاء و مدیریت الگوریتم‌ها و جریان داده است تا ارتباطات به هم گره نخورده و قابل توسعه باشند. مثال معروف: Observer (اعلام تغییرات به اشیاء وابسته)، Strategy (تغییر الگوریتم در زمان اجرا).

۴. Concurrency (همروندی / همزمانی)

این الگوها به مدیریت همزمانی و چندنخی (Multi-threading) می‌پردازند. کارشان این است که به اشیاء کمک کنند در محیط‌های همروند (که چندین کار با هم اجرا می‌شوند) بدون بروز مشکل کار کنند و از تداخل نخ‌ها جلوگیری شود. مثال معروف: Active Object (جداسازی اجرای متد از فراخوانی آن در نخ جداگانه)، Monitor Object (همگام‌سازی دسترسی به متدهای یک شیء).

خلاصه اینکه: ایجاد = چطور بسازیم، ساختاری = چطور کنار هم بچینیم، رفتاری = چطور با هم حرف بزنیم.

۲۳ الگوی کلاسیک

Singleton-1

فقط یک نمونه از کلاس در کل برنامه بساز و یک نقطه دسترسی سراسری به آن بده. (مثال: مدیریت اتصال پایگاه داده)

Factory Method-2

یک متد برای ساخت شیء تعریف کن، ولی نوع دقیق شیء را به زیرکلاس‌ها واگذار کن. (مثال: ساخت دکمه‌های مختلف در سیستم‌عامل‌های گوناگون)

Abstract Factory-3

یک خانواده از اشیاء مرتبط را بدون مشخص کردن کلاس‌های عینی‌شان بساز. (مثال: ساخت دکمه + پنجره + منو برای یک تم خاص)

Builder-4

ساخت یک شیء پیچیده را از نمایش (نمایشگر) آن جدا کن تا بتوانی با یک فرایند، اشکال مختلف بسازی. (مثال: ساخت پیتزا با ترکیبات مختلف)

Prototype-5

یک شیء را با کپی‌کردن یک نمونه موجود (الگو) بساز، نه با new کردن. (مثال: کپی کردن یک سند ورد با تمام محتوا)

Adapter-6

رابط یک کلاس را به رابط دیگری که مشتری انتظار دارد، تبدیل کن (مثل مبدل برق). (مثال: تبدیل دوشاخه گرد به دو شاخه تخت)

Bridge-7

بعد انتزاعی را از بعد پیاده‌سازی جدا کن تا هر دو بتوانند مستقل تغییر کنند. (مثال: کنترل‌های از راه دور با پیاده‌سازی‌های مختلف برای تلویزیون و رادیو)

Composite-8

اشیاء را به صورت درخت بچین تا مشتری با یک شیء منفرد و یک ترکیب از اشیاء، یکسان رفتار کند. (مثال: فایل ها و پوشه ها در سیستم فایل)

Decorator-9

به یک شیء، قابلیت های جدید به صورت پویا اضافه کن بدون اینکه به ساختار آن آسیب بزنی. (مثال: قهوه ساده + شیر + شکلات + خامه)

Facade-10

یک رابط ساده تر برای یک زیرسیستم پیچیده فراهم کن. (مثال: دکمه "استارت" در ماشین که کلیه مراحل روشن کردن را پشت صحنه انجام می دهد)

Flyweight-11

با اشتراک گذاری اشیاء کوچک و سبک، حافظه را بهینه کن (مخصوص اشیاء زیاد). (مثال: حروف یک متن که بارها استفاده می شوند، هر کدام فقط یک بار ساخته می شوند)

Proxy-12

یک جانشین یا دروازه بان برای یک شیء دیگر قرار بده تا دسترسی به آن را کنترل کند. (مثال: تصویر کوچک (thumbnail) که جای تصویر سنگین را می گیرد تا بارگذاری نشده)

Strategy-13

یک خانواده از الگوریتم‌ها را تعریف کن و آنها را قابل تعویض در زمان اجرا کن. (مثال: پرداخت با کارت، پی‌پی، یا نقدی)

Observer-14

وقتی یک شیء تغییر می‌کند، به همه اشیاء وابسته به آن اطلاع بده. (مثال: خبرگزاری که به همه مشترکین خبر می‌دهد)

Command-15

درخواست را به صورت یک شیء بسته‌بندی کن تا قابلیت صف‌بندی، ثبت تاریخچه (Undo/Redo) داشته باشی. (مثال: دکمه‌های Ctrl+C و Ctrl+V که هر کدام یک فرمان هستند)

Iterator-16

به ترتیب به تمام عناصر یک مجموعه دسترسی پیدا کن بدون اینکه به ساختار داخلی آن دست بزنی. (مثال: حلقه for-each که آرایه یا لیست را پیمایش می‌کند)

Mediator-17

ارتباط بین چندین شیء را با یک واسطه مرکزی مدیریت کن تا اشیاء مستقیماً به هم متصل نباشند. (مثال: برج مراقبت که بین هواپیماها ارتباط برقرار می‌کند)

Memento-18

حالت داخلی یک شیء را ذخیره کن تا بتوانی بعداً آن را به حالت قبلی برگردانی. (مثال: دکمه Undo در ویرایشگر متن که به نقاط ذخیره‌شده برمی‌گردد)

State-19

رفتار یک شیء را با تغییر وضعیت داخلی اش عوض کن (انگار که کلاسش عوض شده). (مثال: پخش کننده موسیقی در حالت Play، Pause و Stop رفتار متفاوتی دارد)

Template Method-20

اسکلت یک الگوریتم را در متد پایه تعریف کن و بعضی قدمها را به زیرکلاسها واگذار کن. (مثال: دستور پخت غذا که مراحل ثابت است ولی مواد اولیه در هر غذا متفاوت است)

Visitor-21

یک عملیات جدید به یک ساختار از اشیاء اضافه کن بدون اینکه کلاسهای آن اشیاء را تغییر دهی. (مثال: افزودن عملیات "ارسال ایمیل" به همه کاربران بدون تغییر کلاس User)

Chain of Responsibility-22

درخواست را از یک زنجیره از پردازشگرها عبور بده تا هر کدام توانست آن را پردازش کند. (مثال: سیستم پشتیبانی که درخواست از سطح ۱ به سطح ۲ و سطح ۳ می رود تا حل شود)

1. Creational Patterns (الگوهای ساخت اشیاء)

Singleton 1.1

• خلاصه کاربرد: وقتی فقط «یکی» از یک چیز باید وجود داشته باشد

مثل:

- 1 GameManager
- 2 ConfigManager
- 3 InputManager

و نمی‌خواهی چندتا ازش ساخته بشه.

این‌جا:

• می‌رسیم به Singleton

• (البته باید با دقت و فقط وقتی واقعاً لازمه استفاده بشه)

مشکل:

• فقط باید یک نمونه از یک کلاس در کل برنامه وجود داشته باشد.

• مثال: GameManager, Logger, GameConfig.

ایده پترن:

• سازنده را private می‌کنیم.

• یک نمونه‌ی ثابت (static) از کلاس داخل خودش نگه می‌داریم.

• یک متد getInstance() برای دسترسی به آن می‌نویسیم.

کد ساده:

```
1 public class GameConfig {
2     private static GameConfig instance;
3     private GameConfig() {
4         // تنظیمات اولیه
5     }
6     public static GameConfig getInstance() {
```

1-Java Interview Question

```
7         if (instance == null) {
8             instance = new GameConfig();
9         }
10        return instance;
11    }
12}
```

مزایا:

- کنترل کامل روی تعداد نمونه‌ها (فقط یکی).

- مناسب برای مدیریت تنظیمات سراسری.

معایب:

- اگر زیاد استفاده شود، تبدیل به Global Variable می‌شود.

- تست‌نویسی (Unit Test) را سخت می‌کند.

- وابستگی پنهان در سیستم ایجاد می‌کند.

مثال کامل: انتخاب کاپیتان

```
1 // Singleton
2 public final class Captain {
3     private static Captain captain;
4
5     private Captain() {
6
7     }
8
9     public static synchronized Captain getInstance(){
10         if (captain == null){
11             captain = new Captain();
12         }
13     }
14 }
```

```
12     System.out.println("New Captain is elected for your team");
13     }
14     else {
15         System.out.println("You already have a Captain for your team");
16         System.out.println("Send him for the toss");
17     }
18
19     return captain;
20 }
21 }
22
23 // CLient
24 public class App
25 {
26     public static void main( String[] args )
27     {
28         Captain captain1 = Captain.getInstance();
29
30         Captain captain2 = Captain.getInstance();
31
32         if (captain1 == captain2) {
33             System.out.println("Captains are equal");
34         }
35     }
36 }
```

Factory Method 1.2

• خلاصه کاربرد: جلوگیری از تکرار new کردن کلاس‌های مشخص (مثلاً ساخت دشمن‌ها).

مشکل:

• کد شما نباید مستقیماً با new کردن کلاس‌های مشخص، درگیر جزئیات ساخت شود.

• می‌خواهید بسته به شرایط، نوع مشخصی از یک آبجکت ساخته شود.

ایده پترن:

• یک متد کارخانه (factory) می‌سازید که شیء مناسب را برمی‌گرداند.

• کد کلاینت فقط آن متد را صدا می‌زند، نه new SomeClass().

کد ساده:

```
1 abstract class Enemy {
2     abstract void attack();
3 }
4
5 class Orc extends Enemy {
6     @Override
7     void attack() { System.out.println("Orc attacks!"); }
8 }
9
10 class Troll extends Enemy {
11     @Override
12     void attack() { System.out.println("Troll attacks!"); }
13 }
1
2 class EnemyFactory {
3     public static Enemy createEnemy(String type) {
```

1-Java Interview Question

```
4  switch (type) {
5      case "orc": return new Orc();
6      case "troll": return new Troll();
7      default: throw new IllegalArgumentException("Unknown type");
8  }
9  }
10 }
```

مزایا:

- کد اصلی از جزئیات ساخت اشیاء جدا می‌شود.

- افزودن نوع جدید را ساده‌تر می‌کند (با کمی تغییر در کارخانه).

معایب:

- اگر شرط‌ها زیاد شوند، خود کارخانه می‌تواند شلوغ شود.

- هنوز هم برای هر نوع جدید باید کارخانه را تغییر دهید (نقض جزئی Open/Closed).

مثال کامل: رابط حمل و نقل

```
1 // Product
2 public interface Transport {
3     void deliver();
4 }
5
6 // Concrete Products
7 public class Truck implements Transport {
8     @Override
9     public void deliver() {
10         System.out.println("Delivering by truck...");
11     }
12 }
```

1-Java Interview Question

```
11  }
12 }
13
14 public class Ship implements Transport {
15     @Override
16     public void deliver() {
17         System.out.println("Delivering by ship...");
18     }
19 }
20
21 public class Airplane implements Transport {
22     @Override
23     public void deliver() {
24         System.out.println("Delivering by airplane...");
25     }
26 }
27
28 // Creator
29 public abstract class Logistics {
30     protected abstract Transport createTransport();
31
32     //core business logic
33     public void planDelivery(){
34         Transport transport = createTransport();
35         transport.deliver();
36     }
37 }
```

```
38
39 // Concrete Creators
40 public class RoadLogistics extends Logistics {
41     @Override
42     protected Transport createTransport() {
43         return new Truck();
44     }
45 }
46
47 public class SeaLogistics extends Logistics {
48     @Override
49     protected Transport createTransport() {
50         return new Ship();
51     }
52 }
53
54 public class SkyLogistics extends Logistics {
55     @Override
56     protected Transport createTransport() {
57         return new Airplane();
58     }
59 }
60
61 // Client
62 public class Main {
63     public static void main(String[] args) {
64         Logistics logistics;
```

```
65
66    //Road transportation
67        logistics = new RoadLogistics();
68
69        logistics.planDelivery();
70
71    //Sea transportation
72    logistics= new SeaLogistics();
73        logistics.planDelivery();
74
75        //sky logistics
76    logistics = new SkyLogistics();
77    logistics.planDelivery();
78 }
79 }
```

Abstract Factory 1.3

• خلاصه کاربرد: ساخت خانواده‌ای از اشیاء مرتبط (مثلاً ست کامل UI ویندوز در مقابل مک).

مشکل:

• می‌خواهید خانواده‌ای از اشیاء مرتبط بسازید که با هم کار می‌کنند.

• مثلاً: GUI برای Windows و Mac؛ هر کدام Button, Checkbox مخصوص خود را دارند.

ایده پترن:

• یک اینترفیس کارخانه کلی می‌سازید.

• هر پیاده‌سازی کارخانه، یک «خانواده کامل» از اشیاء را تولید می‌کند.

کد ساده:

```

1 interface GUIFactory {
2     Button createButton();
3     Checkbox createCheckbox();
4 }
5
6 interface Button { void render(); }
7 interface Checkbox { void render(); }
8
9 class WindowsButton implements Button {
10     public void render() { System.out.println("Windows Button"); }
11 }
12
13 class MacButton implements Button {
14     public void render() { System.out.println("Mac Button"); }
15 }
16
17 class WindowsFactory implements GUIFactory {
18     public Button createButton() { return new WindowsButton(); }
19     public Checkbox createCheckbox() { return () -> System.out.println("Windows Checkbox"); }
20 }
21
22 class MacFactory implements GUIFactory {
23     public Button createButton() { return new MacButton(); }
24     public Checkbox createCheckbox() { return () -> System.out.println("Mac Checkbox"); }
25 }

```

8 }

مزایا:

• تضمین می‌کند اشیاء مربوط به هم، هماهنگ ساخته شوند.

• وابستگی به اینترفیس‌ها، نه پیاده‌سازی‌ها.

معایب:

• برای پروژه‌های کوچک، اضافه‌کاری محسوب می‌شود.

• ساختار کلاس‌ها زیاد می‌شود.

Builder 1.4

• خلاصه کاربرد: ساخت کلاس‌های پیچیده با تنظیمات زیاد (مثل ساخت Character بازی).

وقتی می‌خوای شیء را بسازی، ولی سازنده‌ات شاخ‌دار شده

مثلاً:

```
1 new Character("Player1", 100, 20, 5, true, false, true, 0.2, "RANGER", ... );
```

این‌جا:

• خوندن و فهمیدن این سازنده سخت شده

• پارامتر زیاد = اشتباه انسانی زیاد

مشکل:

• سازنده‌ی کلاس بسیار طولانی و پر از پارامتر است.

• بعضی پارامترها اختیاری‌اند و خواندن کد سخت می‌شود.

ایده پترن:

- یک کلاس Builder می‌سازید که مرحله به مرحله پارامترها را تنظیم می‌کند.

• در نهایت با build () شیء آماده را برمی‌گرداند.

کد ساده:

```
1 class Character {
2     private String name;
3     private int hp;
4     private int damage;
5
6     private Character(Builder builder) {
7         this.name = builder.name;
8         this.hp = builder.hp;
9         this.damage = builder.damage;
10    }
11
12    public static class Builder {
13        private String name;
14        private int hp;
15        private int damage;
16
17        public Builder name(String name) {
18            this.name = name; return this;
19        }
20
21        public Builder hp(int hp) {
22            this.hp = hp; return this;
23        }
24
25        public Builder damage(int damage) {
26            this.damage = damage; return this;
27        }
28
29        public Character build() {
30            return new Character(this);
31        }
32    }
33 }
```


1-Java Interview Question

```
7    }
8    public Builder damage(int dmg) {
9        this.damage = dmg; return this;
10   }
11
12   public Character build() {
13       return new Character(this);
14   }
15 }
16 }
17
18 // استفاده:
19 Character hero = new Character.Builder()
20     .name("Hero")
21     .hp(100)
22     .damage(20)
23     .build();
```

مزایا:

• خوانایی بالا، مخصوصاً برای اشیاء پیچیده.

• جلوگیری از سازنده‌های طولانی.

معایب:

• کلاس اضافی (Builder) نیاز دارد.

• برای اشیاء ساده، ممکن است غیرضروری باشد.

مثال کامل: ساخت خانه

```
1 // Product
2 public class House implements HousePlan {
3
4     private String basement;
5     private String structure;
6     private String roof;
7     private String interior;
8
9     @Override
10    public void setBasement(String basement) {
11        this.basement = basement;
12    }
13
14    @Override
15    public void setStructure(String structure) {
16        this.structure = structure;
17    }
18
19    @Override
20    public void setRoof(String roof) {
21        this.roof = roof;
22    }
23
24    @Override
25    public void setInterior(String interior) {
26        this.interior = interior;
```

1-Java Interview Question

```
27  }
28
29
30  @Override
31  public String toString() {
32      return "House{" +
33          "basement=" + basement + "\" +
34          ", structure=" + structure + "\" +
35              ", roof='" + roof + '\"' +
36          ", interior=" + interior + "\" +
37              '}'';
38  }
39 }
40
41 // Builder Interface
42 public interface HouseBuilder {
43     void buildBasement();
44     void buildStructure();
45     void buildRoof();
46     void buildInterior();
47
48     House getHouse();
49 }
50
51 // Concrete Builders
52 public class IglooHouseBuilder implements HouseBuilder {
53     private House house;
54 }
```

1-Java Interview Question

```
55 public IglooHouseBuilder() {
56     this.house = new House();
57 }
58
59 @Override
60 public void buildBasement() {
61     house.setBasement("ice");
62 }
64 @Override
65 public void buildStructure() {
66     house.setStructure("ice");
67 }
68
69 @Override
70 public void buildRoof() {
71     house.setRoof("ice");
72 }
73
74 @Override
75 public void buildInterior() {
76     house.setInterior("ice");
77 }
78
79 @Override
80 public House getHouse() {
81     return house;
82 }
83 }
```

1-Java Interview Question

```
84
85 public class TipiHouseBuilder implements HouseBuilder {
86     private House house;
87
88     public TipiHouseBuilder() {
89         this.house = new House();
90     }
91
92     @Override
93     public void buildBasement() {
94         house.setBasement("wood");
95     }
96
97     @Override
98     public void buildStructure() {
99         house.setStructure("wood");
100     }
101
102     @Override
103     public void buildRoof() {
104         house.setRoof("wood");
105     }
106
107     @Override
108     public void buildInterior() {
109         house.setInterior("wood");
110     }
```

1-Java Interview Question

```
111
112  @Override
113  public House getHouse() {
114      return house;
115  }
116 }
117
118 // Director
119 public class CivilEngineer {
120     private HouseBuilder houseBuilder;
121
122     public CivilEngineer(HouseBuilder houseBuilder) {
123         this.houseBuilder = houseBuilder;
124     }
125
126     public House getHouse() {
127         return this.houseBuilder.getHouse();
128     }
129
130     public void constructHouse() {
131         this.houseBuilder.buildBasement();
132         this.houseBuilder.buildStructure();
133         this.houseBuilder.buildRoof();
134         this.houseBuilder.buildInterior();
135     }
136 }
137
```

```
138 // Main
139 public class Main {
140     public static void main(String[] args) {
141         HouseBuilder iglooBuilder = new IglooHouseBuilder();
142         CivilEngineer engineer = new CivilEngineer(iglooBuilder);
143
144         engineer.constructHouse();
145
146         House house = engineer.getHouse();
147
148         System.out.println(house);
149
150         StringBuilder builder = new StringBuilder("Hello");
151
152         String data = builder.append(1)
153             .append(true)
154             .append("friend")
155             .toString();
156
157         System.out.println(data);
158
159     }
160 }
```

Prototype 1.5

• خلاصه کاربرد: ساخت اشیاء جدید از روی نمونه‌های آماده، وقتی new کردن هزینه‌بر است.

مشکل:

• ساخت یک شیء جدید، هزینه بر است (از نظر زمان یا منابع).

• ولی یک شیء مشابه در اختیار دارید و می‌خواهید از روی آن کپی بسازید.

ایده پترن:

• به جای `new`, از `clone()` یا کپی کردن شیء موجود استفاده می‌کنید.

کد ساده:

```
1 class Enemy implements Cloneable {
2     private String type;
3     private int hp;
4
5     public Enemy(String type, int hp) {
6         this.type = type;
7         this.hp = hp;
8     }
9
10    @Override
11    public Enemy clone() {
12        try {
13            return (Enemy) super.clone();
14        } catch (CloneNotSupportedException e) {
15            throw new RuntimeException(e);
16        }
17    }
18 }
19
```


1-Java Interview Question

```
20 // استفاده :
21 Enemy orc = new Enemy("orc", 100);
22 Enemy clonedOrc = orc.clone();
```

مزایا:

- ایجاد اشیاء جدید سریع‌تر از ساخت از صفر.
- مناسب برای سیستم‌هایی مانند بازی که دشمن‌های مشابه زیاد دارد.

معایب:

- پیاده‌سازی clone() در جاوا کمی حساس و پیچیده است.
- برای اشیاء دارای وابستگی عمیق (Deep Copy)، دقت زیادی لازم است.

2. Structural Patterns (الگوهای ساختاری)

Adapter 2.1

- خلاصه کاربرد: اتصال دو کلاس با اینترفیس‌های ناسازگار بدون تغییر کد اصلی.

مشکل:

- دو کلاس/کتابخانه با هم ناسازگار هستند (متدها یا نام‌ها متفاوتند).
- می‌خواهید بدون تغییر کد اصلی، از آن‌ها با هم استفاده کنید.

ایده پترن:

1-Java Interview Question

• یک «آداپتر» می‌نویسید که رابط (interface) مورد انتظار شما را پیاده‌سازی کرده و پشت‌صحنه، کلاس خارجی را صدا بزند.

کد ساده:

```
1 interface GameInput {
2     void jump();
3 }
4
5 // کتابخانه خارجی:
6 class ExternalInputSystem {
7     void doJump() {
8         System.out.println("External jump");
9     }
10 }
11
12 class InputAdapter implements GameInput {
13     private ExternalInputSystem external;
14
15     public InputAdapter(ExternalInputSystem external) {
16         this.external = external;
17     }
18
19     @Override
20     public void jump() {
21         external.doJump();
22     }
23 }
```

مزایا:

- اتصال کد قدیمی به کتابخانه‌های جدید بدون تغییر هسته.

- جداسازی وابستگی‌ها.

معایب:

- اگر زیاد استفاده شود، تعداد کلاس‌ها زیاد می‌شود.

- می‌تواند ساختار را پیچیده کند.

Bridge 2.2

• خلاصه کاربرد: جدا کردن انتزاع از پیاده‌سازی وقتی کلاس دو بُعد تغییر دارد (مثل شکل و نوع رندر).

مشکل:

- یک کلاس در دو بُعد تغییر می‌کند (مثلاً نوع سلاح و نوع رندر).

- اگر برای هر ترکیب یک کلاس بنویسی، تعداد کلاس‌ها منفجر می‌شود.

ای‌دِا پترن:

- یک کلاس را به دو بخش: «Abstraction» و «Implementation» تقسیم می‌کنی.

- این دو از طریق ترکیب (composition) به هم وصل می‌شوند، نه ارث‌بری.

کد ساده:

```
1 interface Renderer {  
2     void renderCircle(float x, float y, float radius);  
3 }  
4
```

1-Java Interview Question

```
5  class VectorRenderer implements Renderer {
6  public void renderCircle(float x, float y, float radius) {
7      System.out.println("Drawing circle as vector");
8  }
9  }
10
11 class RasterRenderer implements Renderer {
12 public void renderCircle(float x, float y, float radius) {
13     System.out.println("Drawing circle as pixels");
14 }
15 }
16
17 class Circle {
18     private Renderer renderer;
19     private float x, y, radius;
20
21     public Circle(Renderer renderer, float x, float y, float radius) {
22         this.renderer = renderer;
23         this.x = x;
24         this.y = y;
25         this.radius = radius;
26     }
27
28     public void draw() {
29         renderer.renderCircle(x, y, radius);
30     }
31 }
```

مزایا:

• کاهش انفجار تعداد کلاس‌ها.

• امکان تغییر مستقل دو بُعد (مثلاً نوع رندر و نوع شکل).

معایب:

• ساختار اولیه کمی پیچیده‌تر است.

Composite 2.3

• خلاصه کاربرد: مدیریت ساختارهای درختی (مثل صحنه بازی: Node, Child, Group) با یک API یکسان.

مشکل:

• می‌خواهید با ساختارهای درختی (مثل صحنه‌ی بازی: Node, Child Node) کار کنید.

• می‌خواهید "یک شیء" و "مجموعه‌ای از اشیاء" را با یک API مدیریت کنید.

ایده پترن:

• یک اینترفیس مشترک تعریف می‌کنید.

• هم اشیاء منفرد و هم مجموعه‌ها آن را پیاده‌سازی می‌کنند.

کد ساده:

```
1 interface GameObject {  
2     void update();  
3 }  
4  
5 class Sprite implements GameObject {  
6     public void update() {
```

1-Java Interview Question

```
7   System.out.println("Sprite update");
8   }
9 }
10
11 class Group implements GameObject {
12     private List<GameObject> children = new ArrayList<>();
13
14     public void add(GameObject obj) { children.add(obj); }
15
16     public void update() {
17         for (GameObject obj : children) {
18             obj.update();
19         }
20     }
21 }
```

مزایا:

- یکسان شدن کار با تکی و مجموعه.

- مناسب برای ساختارهای صحنه (Scene Graph).

معایب:

- اشکال‌زدایی ساختارهای پیچیده ممکن است سخت شود.

Decorator 2.4

- خلاصه کاربرد: افزودن قابلیت‌های جدید به شیء بدون تغییر کلاس اصلی (مانند شمشیر آتشین).

مشکل:

• می‌خواهید به یک شیء، رفتار جدید اضافه کنید بدون اینکه کلاس اصلی را تغییر دهید.

• مثلاً روی یک سلاح، قابلیت «آتش» یا «یخ» اضافه کنید.

ایده پترن:

• کلاسی می‌سازید که اینترفیس شیء اصلی را پیاده‌سازی کرده و یک شیء از همان نوع را در خود نگه دارد؛ سپس قبل/بعد از صدا زدن متدهای آن، رفتار اضافه می‌کند.

کد ساده:

```
1 interface Weapon {
2     void attack();
3 }
4
5 class BasicSword implements Weapon {
6     public void attack() {
7         System.out.println("Slash");
8     }
9 }
1
2 class FireDecorator implements Weapon {
3     private Weapon base;
4
5     public FireDecorator(Weapon base) {
6         this.base = base;
7     }
```

1-Java Interview Question

```
8
9 public void attack() {
10     base.attack();
11     System.out.println("+ Fire damage");
12 }
13 }
```

مزایا:

- افزودن رفتار بدون تغییر کلاس اصلی.

- ترکیب پذیری بالا (مثلاً Fire + Ice + Poison).

معایب:

- ممکن است زنجیره‌ی دکوراتورها پیچیده شود.

- ردیابی رفتار واقعی سخت می‌شود.

Facade 2.5

- خلاصه کاربردی: ایجاد یک رابط ساده برای یک سیستم پیچیده و شلوغ.

مشکل:

- سیستم شما از چندین کلاس و زیرسیستم تشکیل شده و استفاده مستقیم از آن‌ها سخت است.

ایده پترن:

- یک کلاس ساده (Facade) می‌سازید که متدهای ساده و سطح بالا ارائه می‌کند و پشت صحنه بقیه کلاس‌ها را صدا می‌زند.

کد ساده:

```
1 class AudioSystem {
2     void init() {}
3     void playSound(String name) {}
4 }
5
6 class PhysicsSystem {
7     void init() {}
8 }
9
10 class GameEngineFacade {
11     private AudioSystem audio = new AudioSystem();
12     private PhysicsSystem physics = new PhysicsSystem();
13
14     public void startGame() {
15         audio.init();
16         physics.init();
17         System.out.println("Game started");
18     }
19 }
```

مزایا:

• ساده سازی استفاده از سیستم های پیچیده.

• مناسب برای API های سطح بالا.

معایب:

• اگر بیش از حد بزرگ شود، خودش تبدیل به God Object می‌شود.

Flyweight 2.6

• خلاصه کاربرد: صرفه‌جویی در رم با اشتراک‌گذاری داده‌های مشترک بین میلیون‌ها شیء مشابه.

مشکل:

• تعداد بسیار زیادی از اشیاء مشابه دارید (مثلاً هزاران سرباز در یک بازی)، و هر کدام داده‌های تکراری زیادی دارند.

ایده پترن:

• داده‌های مشترک (invariant) را جدا کرده و فقط یک نسخه نگه می‌دارید.

• داده‌های خاص هر شیء را جداگانه ذخیره می‌کنید.

کد ساده (خیلی ساده‌شده):

```
1 class TreeType {
2     String name;
3     String texture;
4
5     public TreeType(String name, String texture) {
6         this.name = name;
7         this.texture = texture;
8     }
9 }
10
11 class Tree {
12     int x, y;
13     TreeType type;
```

14 }

مزایا:

• کاهش مصرف حافظه.

معایب:

• پیچیدگی در مدیریت داده‌های مشترک و اختصاصی.

Proxy 2.7

• خلاصه کاربرد: کنترل دسترسی یا مدیریت پیش‌از-اجرا (مانند Lazy Loading تصویر).

مشکل:

• می‌خواهید قبل از دسترسی به یک شیء، کنترل اضافه انجام دهید (مثلاً Lazy Loading، یا کنترل دسترسی، یا کش).

ایده پترن:

• یک کلاس Proxy می‌سازید که اینترفیس همان شیء اصلی را دارد، و درخواست‌ها را قبل از رسیدن به شیء اصلی مدیریت می‌کند.

کد ساده:

```
1 interface Image {
2     void display();
3 }
4
5 class RealImage implements Image {
6     private String filename;
7 }
```

1-Java Interview Question

```
8     public RealImage(String filename) {
9         this.filename = filename;
10        loadFromDisk();
11    }
12
13    private void loadFromDisk() {
14        System.out.println("Loading " + filename);
15    }
16
17    public void display() {
18        System.out.println("Displaying " + filename);
19    }
20 }
21
22 class ImageProxy implements Image {
23     private String filename;
24     private RealImage realImage;
25
26     public ImageProxy(String filename) {
27         this.filename = filename;
28     }
29
30     public void display() {
31         if (realImage == null) {
32             realImage = new RealImage(filename);
33         }
34         realImage.display();
35     }
36 }
```

مزایا:

•کنترل دسترسی، Lazy Loading، کش کردن.

معایب:

•افزودن یک لایه اضافی بین کلاینت و شیء اصلی.

3. Behavioral Patterns (الگوهای رفتاری)

Strategy 3.1

•خلاصه کاربرد: جایگزینی if-else های طولانی برای رفتارهای مختلف (مثل الگوریتم های حرکت دشمن).

مثلاً:

```
1 if (type.equals("CREDIT_CARD")) { ... }
1 else if (type.equals("PAYPAL")) { ... }
2 else if (type.equals("CRYPTO")) { ... }
```

و هر جا، همین داستان تکرار می شه.

این نشونه ی اینه که:

•داری به نوع (type) حساسیت نشون می دی

•هر "نوع" رفتار متفاوتی داره

مشکل:

•چندین روش مختلف برای انجام یک کار دارید (مثلاً روش های مختلف پرداخت، یا روش های مختلف حرکت

دشمن)، و نمی خواهید با if-else تشخیص دهید.

ایده پترن:

• یک اینترفیس برای رفتار تعریف می‌کنید.

• چندین پیاده‌سازی مختلف برایش می‌نویسید.

• در زمان اجرا، یکی از آن‌ها را به شیء اصلی تزریق می‌کنید.

کد ساده (مثال Payment):

```
1 interface PaymentStrategy {
2     void pay(double amount);
3 }
4
5 class CreditCardPayment implements PaymentStrategy {
6     public void pay(double amount) {
7         System.out.println("Paying " + amount + " by credit card");
8     }
9 }
10
11 class CryptoPayment implements PaymentStrategy {
12     public void pay(double amount) {
13         System.out.println("Paying " + amount + " by crypto");
14     }
15 }
16
17 class PaymentProcessor {
18     private PaymentStrategy strategy;
19
20     public PaymentProcessor(PaymentStrategy strategy) {
21         this.strategy = strategy;
22     }
23 }
```

1-Java Interview Question

```
14 }
15
16 public void process(double amount) {
17     strategy.pay(amount);
18 }
19 }
```

مزایا:

• جلوگیری از if-else های طولانی.

• امکان تغییر رفتار در زمان اجرا.

معایب:

• افزایش تعداد کلاس‌ها.

مثال کامل: مسیر حرکت

```
1 // Strategy interface
2 public interface RouteStrategy
3 {
4     void buildRoute(String from, String to);
5 }
6
7 // Navigator
8 public class Navigator
9 {
10     private RouteStrategy routeStrategy;
11
12     public void setRouteStrategy(RouteStrategy routeStrategy)
13     {
14         this.routeStrategy = routeStrategy;
```

1-Java Interview Question

```
15     }
16
17     // Wrapper (📌)
18     public void buildRoute(String from, String to)
19     {
20         this.routeStrategy.buildRoute(from, to);
21     }
22 }
23
24 // Car Concrete
25 public class CarRouteStrategy implements RouteStrategy
26 {
27     @Override
28     public void buildRoute(String from, String to)
29     {
30         System.out.println("Driving...");
31     }
32 }
33
34 // Walking Concrete
35 public class WalkingRouteStrategy implements RouteStrategy
36 {
37     @Override
38     public void buildRoute(String from, String to)
39     {
40         System.out.println("Walking...");
41     }
42 }
43
44 // Main
45 public class Main
```


1-Java Interview Question

```
46 {
47     public static void main(String[] args)
48     {
49         RouteStrategy carStrategy = new CarRouteStrategy();
50
51         Navigator navigator = new Navigator();
52         navigator.setRouteStrategy(carStrategy);
53         navigator.buildRoute("Tehran", "Qazvin");
54     }
55 }
```

Observer 3.2

• خلاصه کاربرد: خبردار کردن سایر اشیاء از تغییرات (مانند آپدیت UI هنگام تغییر جان بازیکن).

مشکل:

• یک شیء (Subject) دارد تغییر می‌کند و چندین بخش دیگر باید از این تغییر باخبر شوند (مثلاً UI، لاگ، سیستم امتیاز).

ایده پترن:

• لیستی از ناظرها (Observers) در Subject نگه می‌دارید.

• وقتی Subject تغییر کرد، همه Observerها را خبر می‌کند.

کد ساده:

```
1 interface Observer {
2     void update(int hp);
3 }
4
```

1-Java Interview Question

```
5 class Player {
6     private int hp;
7     private List<Observer> observers = new ArrayList<>();
8
9     public void addObserver(Observer o) { observers.add(o); }
10
11     public void setHp(int hp) {
12         this.hp = hp;
13         notifyObservers();
14     }
15
16     private void notifyObservers() {
17         for (Observer o : observers) {
18             o.update(hp);
19         }
20     }
21 }
```

مزایا:

• Loose Coupling بین Subject و Observer ها.

معایب:

• ممکن است چرخه‌های پیچیده‌ای از اطلاع‌رسانی ایجاد شود.

Command 3.3

• خلاصه کاربرد: پیمایش روی کالکشن‌ها بدون درگیری با جزئیات داخلی آن‌ها.

مشکل:

• می‌خواهید درخواست (عملیاتی مثل "حمله"، "پريدن") را به شکل یک شیء ذخیره کنید (برای Undo، Redo، Queue کردن دستورات).

ایده پترن:

• هر دستور را در یک کلاس جداگانه می‌ریزید که یک متد execute() دارد.

کد ساده:

```

1 interface Command {
2     void execute();
3 }
4
5 class JumpCommand implements Command {
6     public void execute() {
7         System.out.println("Player jumps");
8     }
9 }
10
11 class InputHandler {
12     private Command jumpCommand;
13
14     public InputHandler(Command jumpCommand) {
15         this.jumpCommand = jumpCommand;
16     }
17
18     public void onJumpPressed() {
19         jumpCommand.execute();

```

1-Java Interview Question

20 }

21 }

مزایا:

• مناسب برای Undo/Redo و Queue از دستورات.

معایب:

• افزایش تعداد کلاس‌های کوچک.

مثال کامل: تلویزیون

```
1 // Command
2 public interface Command
3 {
4     void execute();
5 }
6
7 // Receiver
8 public class Television
9 {
10     public void turnOn()
11     {
12         System.out.println("TV is on.");
13     }
14
15     public void turnOff()
16     {
17         System.out.println("TV is off.");
18     }
19 }
20
```

1-Java Interview Question

```
21 // Concerete Command
22 public class TurnOnCommand implements Command
23 {
24     private Television television;
25
26     public TurnOnCommand(Television television)
27     {
28         this.television = television;
29     }
30
31     // Wrappper
32     @Override
33     public void execute()
34     {
35         this.television.turnOn();
36     }
37 }
38
39 // Concerete Command
40 public class TurnOffCommand implements Command
41 {
42     private Television television;
43
44     public TurnOffCommand(Television television)
45     {
46         this.television = television;
47     }
48
49     @Override
50     public void execute()
51     {
```

1-Java Interview Question

```
52         this.television.turnOff();
53     }
54 }
55
56 // Invoker
57 public class RemoteControllerInvoker implements Command
58 {
59     private Command command;
60
61     public void setCommand(Command command)
62     {
63         this.command = command;
64     }
65
66     @Override
67     public void execute()
68     {
69         this.command.execute();
70     }
71 }
72
73 // Main
74 public class Main
75 {
76     public static void main(String[] args)
77     {
78         Television television = new Television();
79
80         Command turnOn = new TurnOnCommand(television);
81         Command turnOff = new TurnOffCommand(television);
82         Command volumeUp = new VolumeUpCommand(television);
```

1-Java Interview Question

```
83     Command volumeDown = new VolumeDownCommand(television);
84
85     RemoteControllerInvoker remote = new RemoteControllerInvoker();
86     remote.setCommand(turnOn);
87     remote.execute();
88
89     remote.setCommand(turnOff);
90     remote.execute();
91
92     remote.setCommand(volumeUp);
93     remote.execute();
94
95     remote.setCommand(volumeDown);
96     remote.execute();
97 }
98 }
```

Iterator 3.4

• خلاصه کاربرد: پیمایش روی کالکشن‌ها بدون درگیری با جزئیات داخلی آن‌ها.

مشکل:

• می‌خواهید روی عناصر یک مجموعه حرکت کنید بدون اینکه جزئیات ساختار داخلی آن را بدانید.

ایده پترن:

• یک اینترفیس Iterator تعریف می‌کنید که `hasNext()` و `next()` داشته باشد.

کد:

در جاوا، Iterator از قبل وجود دارد (java.util.Iterator).

```
1 List<String> list = new ArrayList<>();
2 Iterator<String> it = list.iterator();
3 while (it.hasNext()) {
4     System.out.println(it.next());
5 }
```

مزایا:

- ساده‌سازی پیمایش روی ساختارهای مختلف.

معایب:

- تقریباً همیشه ضمنی در جاوا استفاده می‌شود.

Mediator 3.5

• خلاصه کاربرد: کاهش وابستگی‌های مستقیم بین کلاس‌ها با استفاده از یک واسط مرکزی.

مشکل:

- چندین کلاس با هم زیاد در ارتباط‌اند و هر کدام دیگری را صدا می‌زنند.

- این باعث وابستگی شدید و کد پیچیده می‌شود.

ایده پترن:

- به جای اینکه مستقیم همدیگر را صدا بزنند، از یک واسط (Mediator) استفاده می‌کنند.

کد ساده (مفهومی):

```
1 interface Mediator {  
2     void notify(Component sender, String event);  
3 }  
4  
5 abstract class Component {  
6     protected Mediator mediator;  
7     public Component(Mediator mediator) { this.mediator = mediator; }  
8 }
```

مزایا:

• کاهش وابستگی مستقیم بین کلاس‌ها.

معایب:

• Mediator ممکن است خیلی بزرگ و پیچیده شود.

Memento 3.6

• خلاصه کاربرد: ذخیره وضعیت فعلی برای بازگشت به عقب (Save/Undo).

مشکل:

• می‌خواهید وضعیت یک شیء را ذخیره کنید تا بعداً به آن برگردید (Undo).

ایده پترن:

• یک کلاس Memento می‌سازید که وضعیت را نگه می‌دارد.

• شیء اصلی می‌تواند وضعیتش را به Memento تبدیل و برگردان کند.

کد ساده:

```
1  class GameState {
2  private int level;
3
4  public GameState(int level) {
5      this.level = level;
6  }
7
8  public int getLevel() { return level; }
9  }
10
11 class Game {
12 private int level;
13
14 public GameState save() {
15     return new GameState(level);
16 }
17
18 public void restore(GameState state) {
19     this.level = state.getLevel();
20 }
21 }
```

مزایا:

• پیاده‌سازی Undo/Save در بازی‌ها.

معایب:

• مصرف حافظه زیاد اگر وضعیت‌ها بزرگ باشند.

State 3.7

• خلاصه کاربرد: مدیریت رفتار شیء بر اساس وضعیتش (مثل Standing, Attacking) به جای if.

مشکل:

• رفتار یک شیء بسته به وضعیت (State) آن تغییر می‌کند.

• کد پر از if (state == ...) می‌شود.

ایده پترن:

• برای هر وضعیت، یک کلاس جداگانه می‌سازید که رفتار را تعریف می‌کند.

کد ساده:

```
1 interface PlayerState {
2     void handleInput();
3 }
4
5 class StandingState implements PlayerState {
6     public void handleInput() {
7         System.out.println("Standing");
8     }
9 }
10
```

1-Java Interview Question

```
11 class RunningState implements PlayerState {
12     public void handleInput() {
13         System.out.println("Running");
14     }
15 }
16
17 class Player {
18     private PlayerState state;
19
20     public void setState(PlayerState state) {
21         this.state = state;
22     }
23
24     public void handleInput() {
25         state.handleInput();
26     }
27 }
```

مزایا:

• حذف if-else های مربوط به وضعیت.

*معایب:

• افزایش تعداد کلاس ها.

مثال کامل: ثبت مقاله در سایت

```
1 // State
2 public interface DocumentState
3 {
4     void publish();
```

1-Java Interview Question

```
5
6  void render();
7
8  void approve();
9}
10
11 // Document
12 public class Document
13 {
14     private DocumentState documentState;
15
16     public Document()
17     {
18         // Always Draft for first
19         this.documentState = new DraftState(this);
20     }
21
22     public void changeState(DocumentState state)
23     {
24         this.documentState = state;
25     }
26
27     public void publish()
28     {
29         this.documentState.publish();
30     }
31
32     public void render()
33     {
34         this.documentState.render();
```

1-Java Interview Question

```
35     }
36
37     public void approve()
38     {
39         this.documentState.approve();
40     }
41 1 }
42
43 1 // Draft
44 1 public class DraftState implements DocumentState
45 1 {
46     private Document document;
47
48     public DraftState(Document document)
49     {
50         this.document = document;
51     }
52
53     @Override
54     public void approve()
55     {
56         System.out.println("Draft can not be Approved...");
57     }
58
59     @Override
60     public void publish()
61     {
62         System.out.println("Document is send moderation...");
63         this.document.changeState(new ModerationState(document));
64     }
65
```

1-Java Interview Question

```
66     @Override
67     public void render()
68     {
69         System.out.println("Document Render...");
70     }
71 1 }
72
73 1 // Moderation
74 1 public class ModerationState implements DocumentState
75 1 {
76     private Document document;
77
78     public ModerationState(Document document)
79     {
80         this.document = document;
81     }
82
83     @Override
84     public void approve()
85     {
86         System.out.println("Document approve and publish...");
87         this.document.changeState(new PublishState(document));
88     }
89
90     @Override
91     public void publish()
92     {
93         System.out.println("Document is waiting for approval...");
94     }
95
96     @Override
```

1-Java Interview Question

```
97     public void render()
98     {
99         System.out.println("Rendering document under moderation state...");
100     }
101 1 }
102
103 1 // Publish
104 1 public class PublishState implements DocumentState
105 1 {
106     private Document document;
107
108     public PublishState(Document document)
109     {
110         this.document = document;
111     }
112
113     @Override
114     public void approve()
115     {
116         System.out.println("Document is already approced");
117     }
118
119     @Override
120     public void publish()
121     {
122         System.out.println("Document is already published");
123     }
124
125     @Override
126     public void render()
```


1-Java Interview Question

```
127     {
128         System.out.println("Rendering published document...");
129     }
130 1 }
131
132 1 // Main
133 1 public class Main
134 1 {
135     public static void main(String[] args)
136     {
137         Document document = new Document();
138
139         // Draft
140         document.render();
141         document.publish();
142
143         // Moderation
144         document.render();
145         document.publish();
146         document.approve();
147
148         // Publish
149         document.render();
150         document.publish();
151     }
152 1 }
```

Template Method 3.8

• خلاصه کاربرد: تعریف اسکلت کلی الگوریتم در کلاس پایه و سپردن جزئیات به فرزندان.

مشکل:

• الگوریتم کلی در چند کلاس مشابه است، اما بعضی جزئیات فرق می‌کند.

ایده پترن:

• در کلاس پایه، الگوریتم کلی را به صورت متد `final` می‌نویسید.

• بخش‌هایی که باید متفاوت باشند را به صورت متدهای `abstract` یا قابل `override` تعریف می‌کنید.

کد ساده:

```
1 abstract class GameLoop {
2     public final void run() {
3         init();
4         update();
5         render();
6     }
7
8     protected abstract void init();
9     protected abstract void update();
10    protected abstract void render();
11 }
```

مزایا:

• اشتراک الگوریتم کلی، جلوگیری از تکرار کد.

معایب:

• وابستگی به ارث‌بری (Inheritance).

Visitor 3.9

• خلاصه کاربرد: افزودن عملیات جدید به کلاس‌ها بدون تغییر ساختار داخلی آن‌ها.

مشکل:

• می‌خواهید روی مجموعه‌ای از کلاس‌های مختلف عملیاتی انجام دهید بدون اینکه کلاس‌های اصلی را تغییر دهید.

ایده پترن:

• یک Visitor تعریف می‌کنید که متد visit برای هر نوع کلاس دارد.

• کلاس‌ها یک متد accept(Visitor v) دارند.

کد ساده (خیلی خلاصه):

```
1 interface Visitor {  
2     void visitEnemy(Enemy e);  
3     void visitPlayer(Player p);  
4 }  
5  
6 interface Element {  
7     void accept(Visitor v);  
8 }
```

مزایا:

• افزودن عملیات جدید بدون تغییر کلاس‌های موجود.

معایب:

• اگر انواع جدیدی از Element اضافه کنید، باید Visitor ها را تغییر دهید.

Chain of Responsibility 3.10

• خلاصه کاربرد: عبور درخواست از زنجیره ای از هندلرها تا یکی مسئولیت آن را بپذیرد.

مشکل:

• می خواهید چندین Handler داشته باشید که هر کدام ممکن است یک درخواست را پردازش کنند.

• نمی خواهید یک Handler خاص به صورت ثابت مشخص باشد.

ایده پترن:

• Handler ها را در یک زنجیره قرار می دهید.

• هر Handler درخواست را گرفته، اگر نتوانست رسیدگی کند، به بعدی پاس می دهد.

کد ساده:

```
1 abstract class Handler {  
2     protected Handler next;  
3  
4     public Handler setNext(Handler next) {  
5         this.next = next;  
6         return next;  
7     }  
8  
9     public void handle(String request) {
```

```

10  if (next != null) next.handle(request);
11  }
12 }
13
14 class AuthHandler extends Handler {
15     @Override
16     public void handle(String request) {
17         System.out.println("Auth checked");
18         super.handle(request);
19     }
20 }

```

مزایا:

• انعطاف بالا در تعیین ترتیب و نوع Handler ها.

معایب:

• دیدگاه کردن مسیر عبور درخواست گاهی سخت است.

نکته پایانی برای فالكون

• از اسم پترن شروع نکن؛ از مشکل شروع کن.

• هر وقت دیدی:

• if-else بر اساس نوع زیاد شده → به Strategy / State فکر کن.

• ساخت اشیاء پیچیده است → به Builder / Factory فکر کن.

• رفتار مشترک با تفاوت‌های کوچک داری → به Template Method فکر کن.

• چندین آبجکت باید از تغییر یک آبجکت باخبر شوند → به Observer فکر کن.

این فایل را می‌توانی به عنوان `design-pattern-cheatsheet.md` در GitHub یا Gist ذخیره کنی و هر وقت وسط پروژه گیر کردی، سریع نگاهی به آن بیندازی.

فاز ۱۰: روز مصاحبه و سوالات غیرفنی (HR & Behavioral)

اینجا قراره یاد بگیری چطور خودت رو بفروشی، چطور به سوالات سخت جواب بدی و چطور به تأثیر اولیه عالی بذاری. خیلی از شرکت‌های اینترپرایز به اندازه‌ی مهارت فنی، روی شخصیت، انگیزه و طرز فکر آدم‌ها سرمایه‌گذاری می‌کنن.

10.1 سوالات رایج درباره خودت و مسیر شغلی

? سوال ۱: "از خودت بگو." (Tell me about yourself)

این سوال با اینکه ساده به نظر میرسه، سرنوشت‌سازه. مصاحبه‌کننده نمی‌خواد داستان زندگیت رو بشنوه. یه فرمول ۳ مرحله‌ای به اسم گذشته-حال-آینده رو استفاده کن:

• گذشته (زمینه): مختصر بگو چه خوندي و چه تجربه‌ای داری.

• حال (نقطه اتصال): چرا و چطور وارد برنامه‌نویسی شدی و الان چه مهارت‌هایی داری.

• آینده (هدف): چی می‌خوای و چرا این شرکت و این موقعیت شغلی برات جذابه.

✓ مثال عالی برای تو: "من لیسانس معماری دارم و اونجا یاد گرفتم چطور مسائل پیچیده رو ساختمند حل کنم. بعد از اتمام سربازی، به خاطر علاقه‌ام به تکنولوژی و حل مسئله، به صورت جدی و خودآموز وارد دنیای برنامه‌نویسی شدم. توی این مسیر روی جاوا و فریم‌ورک Spring متمرکز شدم و تا JPA و Hibernate پیش اومدم. الان یه پروژه شخصی [اسم پروژه رو بگو] رو پیاده‌سازی کردم که می‌تونید روی گیت‌هابم ببینید. الان به دنبال یه چالش جدی‌تر در یه تیم حرفه‌ای مثل تیم شما هستم تا بتونم دانشم رو در مقیاس سازمانی به کار بگیرم و رشد کنم."

؟ سوال ۲: "چرا از رشته معماری به برنامه‌نویسی اومدی؟"

نکته: اینو به نقطه ضعف نبین، تبدیلش کن به یه نقطه قوت.

جواب: "معماری به من یاد داد که چطور قبل از ساختن، طراحی کنم. به مسائل از بالا نگاه کنم، ساختارمند فکر کنم و به جزئیات دقت کنم. توی برنامه‌نویسی هم دقیقاً همین مهارت‌ها نیازم. من عاشق ساختن چیزها از صفر و دیدن نتیجه کارم هستم، و حس می‌کنم توی دنیای نرم‌افزار این ساختن سریع‌تر و با تأثیر بیشتری اتفاق می‌افته. به نظرم این تغییر مسیر، تلفیقی از علاقه و مهارت‌های ذاتیم بود."

؟ سوال ۳: "بزرگترین نقطه ضعف و نقطه قوتت چیه؟"

• نقطه قوت: یه مهارت مرتبط با شغل رو انتخاب کن و برایش دلیل بیار. "بهترین نقطه قوتم پشتکارم در حل مسائل هست. وقتی به یه باگ می‌خورم، تا مشکل رو ریشه‌یابی نکنم و راه حل تمیز براش پیدا نکنم، ولش نمی‌کنم."

• نقطه ضعف: یه ضعف واقعی (اما نه خیلی خطرناک برای شغل) رو بگو و مهم‌تر از اون، راه‌حلی که برای بهبودش در پیش گرفتی رو توضیح بده. "گاهی اوقات انقدر درگیر جزئیات کد میشم که از تصویر بزرگ غافل میشم. برای حل این موضوع، الان سعی می‌کنم در بازه‌های منظم یه قدم به عقب بردارم و از خودم بپرسم که آیا این راه‌حل با هدف اصلی پروژه هم‌خوانی داره یا نه."

10.2 سوالات رفتاری (Behavioral) - روش STAR

مصاحبه‌کننده می‌خواهد بفهمد در موقعیت‌های واقعی چطور عمل می‌کنی. از روش STAR برای جواب دادن استفاده کن:

• Situation (موقعیت): اون موقعیت چه بود؟

• Task (وظیفه): وظیفه تو چی بود؟

• Action (اقدام): چه اقدام مشخصی انجام دادی؟

• Result (نتیجه): نتیجه کار چی شد؟

؟ سوال ۴: "از یه چالش فنی که باهاش روبرو شدی و حلش کردی بگو."

- S/T: "توی پروژه شخصی خودم، یه باگ عجیب داشتم که باعث می‌شد برنامه به صورت رندوم کند بشه."
- A: "با بررسی لاگ‌ها متوجه شدم مشکل از N+1 Query در Hibernate هست. پس شروع کردم به مطالعه و فهمیدم که باید از JOIN FETCH یا EntityGraph برای کوئری‌های خاص استفاده کنم. کد رو بازنویسی کردم."
- R: "زمان پاسخگویی اون API خاص از ۳ ثانیه به ۲۰۰ میلی‌ثانیه کاهش پیدا کرد و کلی مطلب جدید یاد گرفتم."

10.۳ سوالاتی که تو باید از مصاحبه‌کننده بپرسی

این بخش خیلی مهمه. وقتی مصاحبه‌کننده می‌پرسه "آیا سوالی داری؟"، هرگز نگو "نه". این نشون‌دهنده علاقه و هوش توئه.

چند سوال عالی برای پرسیدن:

- "تیم شما معمولاً برای مدیریت پروژه از چه متدولوژی‌ای (مثل Agile/Scrum) استفاده می‌کنه؟"
- "بزرگترین چالش فنی که تیم شما در حال حاضر باهاش روبروست چیه؟"
- "شخصی که در این موقعیت شغلی موفق باشه، در ۶ ماه یا یک سال اول چه دستاوردهایی داشته که شما رو خوشحال می‌کنه؟"
- "فرآیند Code Review در تیم شما چطور انجام میشه؟"
- "مسیر پیشرفت فنی (مثلاً Senior شدن) در شرکت شما چطور تعریف شده؟"

10.۴ تاکتیک‌های روز مصاحبه

- پروژه‌ها رو آماده کن: لپ‌تاپ آماده باشه و بتونی پروژه‌هایی که توی رزومت نوشتی رو سریع اجرا کنی و تکه‌کدهای مهمش رو نشون بدی.

• قبل از مصاحبه تحقیق کن: در مورد شرکت بخون. محصولاتشون چیه؟ با چه تکنولوژی‌هایی کار می‌کنن؟ این نشون میده که واقعاً بهشون علاقه داری.

• سوال رو متوجه نشدی؟ اشکال نداره بپرسی. این خیلی بهتر از جواب اشتباه دادنه. می‌تونی بگی: "میشه یه کم بیشتر توضیح بدید؟ می‌خوام مطمئن بشم درست متوجه شدم."

• جواب یه سوال رو بلد نیستی؟ اعتراف کن، اما مسیر رو نشون بده. بگو: "راستش با این کتابخونه خاص کار نکردم، اما با یه مفهوم مشابه (فلان) کار کردم. می‌تونم سریع یادش بگیرم. اینطور فکر می‌کنم که احتمالاً اینطوری کار می‌کنه..."

• یه جمع‌بندی آخر کار رو داشته باش. آخرش بگو: "ممنون بابت وقتی که گذاشتید. این گفتگو واقعاً اشتیاق من رو برای پیوستن به تیم شما بیشتر کرد."

System Design Interview - راهنمای کامل

برای افراد غیر کامپیوتر

این مجموعه شامل طراحی کامل ۵ سیستم کلیدی به زبان ساده و قابل فهم برای افرادی است که رشته‌شان کامپیوتر نیست. هر سیستم با تشبیه‌های دنیای واقعی، نمودارهای متنی و نکات مصاحبه‌ای ارائه شده است.

هدف این مجموعه

- درک مفاهیم بدون نیاز به کدنویسی
- توانایی طراحی روی تخته سفید در مصاحبه
- آشنایی با اصطلاحات کلیدی سیستم دیزاین
- آمادگی برای سوالات سطح بالا (High-Level Design)

سیستم ۱: طراحی سرویس کوتاه‌کننده لینک (URL Shortener)

(Shortener)

مشکل چیست؟

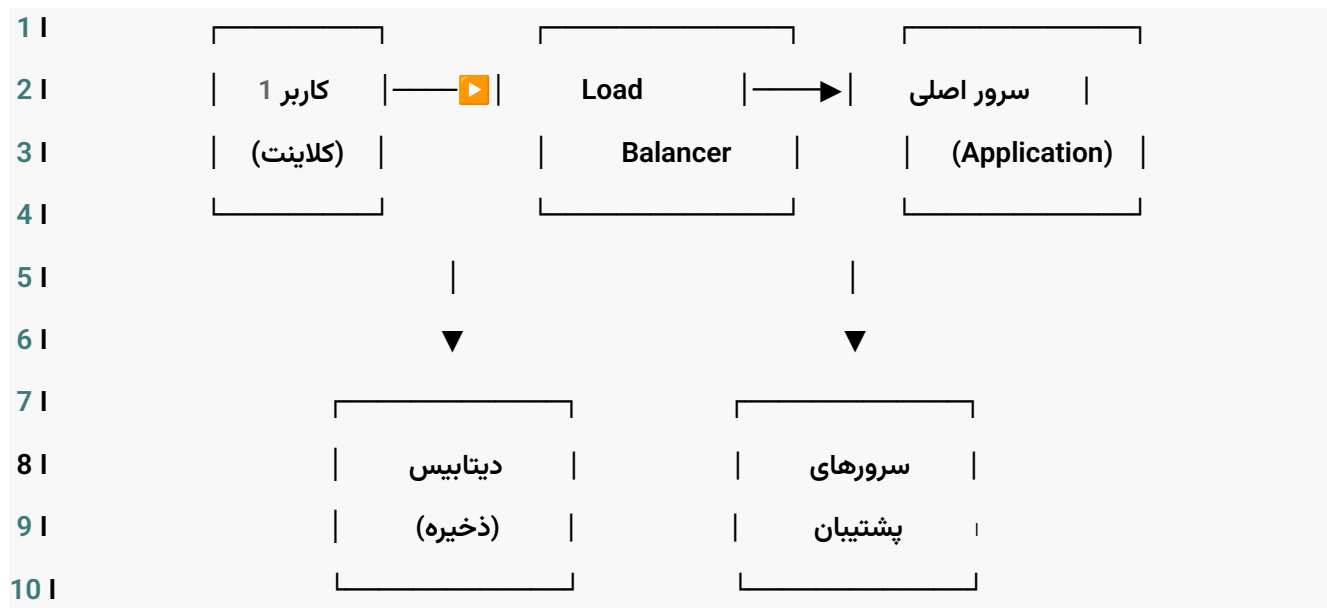
ما می‌خواهیم یک سرویس بسازیم که یک لینک بلند (مثل <https://www.example.com/blog/post/12345>) را به یک لینک کوتاه (مثل tinyurl.com/abc123) تبدیل کند. وقتی کاربر روی لینک کوتاه کلیک کند، به لینک اصلی هدایت شود.

معماری کلی (با تشبیه دنیای واقعی) 🏗️

تصور کنید یک کتابخانه دارید:

- کتابدار (سرور): درخواست‌ها را دریافت می‌کند
- فهرست کتاب‌ها (دیتابیس): لینک بلند و کوتاه را ذخیره می‌کند
- کدهای مخفف (ID Generator): شماره‌های منحصر به فرد برای هر لینک تولید می‌کند

نمودار سطح بالا (High-Level Diagram) 📊



تیکه ۶: Application Server و Load Balancer

اجزای اصلی (Components)

۱. Load Balancer (منشی ورودی)

- وظیفه: توزیع درخواست‌ها بین چند سرور
- تشبیه: منشی یک شرکت که مشتریان را به کارشناسان مختلف ارجاع می‌دهد
- نکته مصاحبه: از Round Robin یا Least Connections استفاده می‌کنیم

۲. Application Server (سرور برنامه)

- وظیفه: پردازش منطق اصلی
- کارهایی که انجام می‌دهد:
- دریافت لینک بلند از کاربر
- تولید یک کد یکتا (مثل abc123)
- ذخیره کردن در دیتابیس
- برگرداندن لینک کوتاه به کاربر

۳. Database (دیتابیس)

- وظیفه: ذخیره‌سازی زوج‌های (لینک کوتاه ← لینک بلند)
- ساختار جدول:

1			
2	short_code	original_url	created_at
3			
4	abc123	https://example.com/...	2024-01-01
5	xyz789	https://google.com/...	2024-01-02
6			

تیکه ۸: الگوریتم تولید کد کوتاه

الگوریتم تولید کد کوتاه

روش ۱: Base62 Encoding (توصیه شده)

• ایده: عددی را به حروف و اعداد تبدیل کنیم

• مثال: عدد 123456 را به w7e تبدیل می‌کنیم

• مزیت: خیلی کوتاه و خوانا

• تشبیه: مثل تبدیل شماره به کد پستی

روش ۲: Hash Function (تابع هش)

• ایده: از لینک اصلی یک هش می‌گیریم

• مشکل: احتمال برخورد (Collision) وجود دارد

• تشبیه: مثل اثر انگشت که ممکن است دو نفر اثر انگشت یکسان داشته باشند (بسیار نادر)

مقیاس پذیری (Scalability)

مشکل: اگر روزانه ۱۰۰ میلیون لینک بسازیم، دیتابیس پر می شود!

راه حل ۱: Sharding (تقسیم دیتابیس)

• تشبیه: به جای یک انبار بزرگ، چندین انبار در شهرهای مختلف

• پیاده سازی: بر اساس اولین حرف کد کوتاه

1 کدهای شروع با A: دیتابیس

2 کدهای شروع با B: دیتابیس

تیکه ۱۰: Cache و سوالات مصاحبه ۱

راه حل ۲: Cache (حافظه موقت)

• تشبیه: دفترچه یادداشت کنار دست برای لینک های پرطرفدار

• ابزار: Redis یا Memcached

• نکته: ابتدا کش را چک می کنیم، اگر نبود به دیتابیس می رویم

سوالات مصاحبه ای !

سوال ۱: چطور از تکراری شدن کدها جلوگیری می کنیم؟

جواب: از یک شمارنده مرکزی (مثل Redis INCR) استفاده می کنیم تا هر بار یک عدد یکتا بگیریم و آن را به Base62 تبدیل کنیم.

سوال ۲: اگر کاربر لینک تکراری بدهد چه می‌شود؟

جواب: دو رویکرد:

• رویکرد ۱: همیشه لینک جدید بساز (ساده‌تر)

• رویکرد ۲: چک کن اگر لینک قبلاً ثبت شده، همان کد را برگردان (پیچیده‌تر)

سوال ۳: آمار بازدید (Analytics) را چطور ثبت کنیم؟

جواب: یک جدول جداگانه برای ثبت هر کلیک:

1			
2	short_code	user_ip	click_time
3			
4	abc123	192.168.1.1	2024-01-01
5			

تیکه ۱۲: خلاصه سیستم ۱

خلاصه یک صفحه‌ای برای مصاحبه

سوال: یک سرویس کوتاه‌کننده لینک طراحی کنید.

پاسخ سطح بالا:

• Load Balancer برای توزیع ترافیک

• سرورهای Application برای پردازش

• دیتابیس برای ذخیره (با Sharding و Cache)

• ID Generator برای تولید کدهای یکتا

•سیستم کش برای لینک‌های پرطرفدار

تخمین اعداد:

•۱۰۰ میلیون لینک جدید در روز

•هر لینک متوسط ۱KB فضا

•۱۰۰GB فضای ذخیره‌سازی در روز!

سیستم ۲: طراحی سیستم پیام‌رسان (مثل تلگرام/واتساپ)

مشکل چیست؟

می‌خواهیم سیستمی طراحی کنیم که کاربران بتوانند پیام‌های متنی، تصاویر و فایل‌ها را به صورت لحظه‌ای با هم به اشتراک بگذارند. ویژگی‌های کلیدی: ارسال پیام، دریافت لحظه‌ای، وضعیت آنلاین/آفلاین، تاریخچه پیام‌ها.

تشبیه دنیای واقعی

سیستم پیام‌رسان مثل یک اداره پست با ویژگی‌های ویژه است:

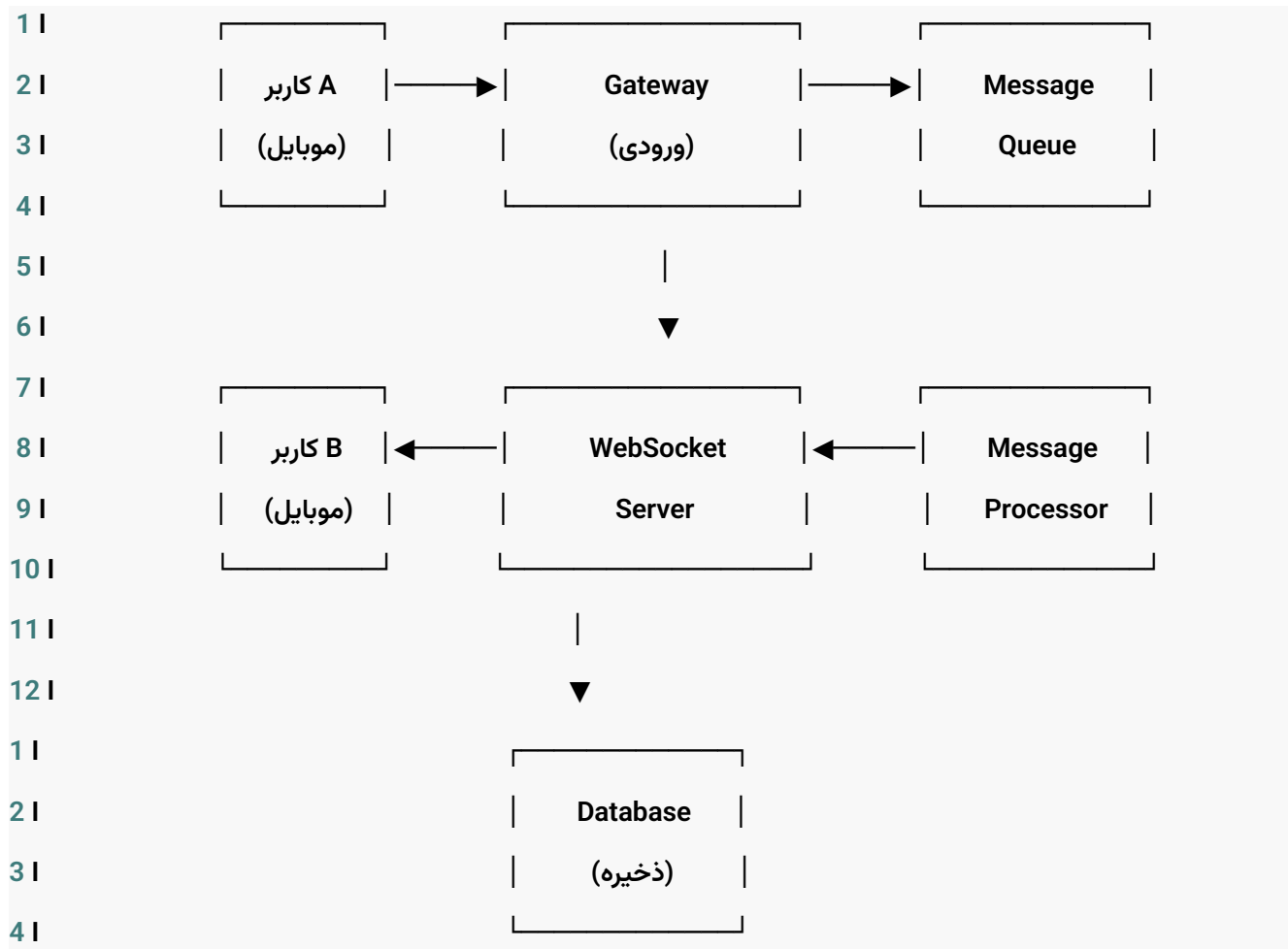
•کاربر: فرستنده/گیرنده نامه

•صندوق پستی (سرور): محل ذخیره نامه‌ها

•پستچی (WebSocket): نامه را فوری می‌رساند

•آدرس‌نامه (دیتابیس): لیست مخاطبین

معماری کلی



تیکه ۱۵: Message Queue و Gateway

اجزای اصلی

۱. Gateway (ورودی سیستم)

•وظیفه: اولین نقطه برخورد با کاربر

• تشبیه: ورودی یک ساختمان بزرگ

• کارها:

• احراز هویت (Authentication)

• هدایت درخواست‌ها به سرور مناسب

• محدودیت نرخ درخواست (Rate Limiting)

۲. Message Queue (صف پیام‌ها)

• وظیفه: پیام‌ها را در صف قرار دهد تا یکی‌یکی پردازش شوند

• تشبیه: صف نانوايي - همه وارد می‌شوند ولی یکی‌یکی خارج می‌شوند

• چرا مهم است؟ اگر یک کاربر ۱۰۰۰ پیام بفرستد، سرور را خراب نمی‌کند

۳. WebSocket Server (سرور اتصال لحظه‌ای)

• وظیفه: ارتباط دوطرفه و لحظه‌ای با کاربر

• تشبیه: تلفن همراه - هر دو طرف می‌توانند همزمان صحبت کنند

• تفاوت با HTTP: HTTP مثل پست است (بفرست و منتظر جواب باش)، WebSocket مثل تلفن (همیشه وصل هستی)

۴. Message Processor (پردازشگر پیام)

• وظیفه: گرفتن پیام از صف و ذخیره کردن در دیتابیس

• تشبیه: کارمند پشت باجه که نامه‌ها را بایگانی می‌کند

۵. Database (دیتابیس)

• ساختار پیام:

1						
2	id	sender_id	receiver_id	content	timestamp	
3						
4	1	101	102	12:30:00	سلام چطوری؟	
5	2	102	101	12:30:15	خوبم تو؟	
6						

مدیریت وضعیت آنلاین/آفلاین

روش: Heartbeat Mechanism (ضربان قلب)

• تشبیه: مثل چراغ حیاط خانه - اگر روشن باشد، شما خانه‌اید

• کار: هر ۵ ثانیه، کاربر یک سیگنال ("من زنده‌ام") به سرور می‌فرستد

• اگر ۱۵ ثانیه سیگنال نیاید → کاربر آفلاین محسوب می‌شود

تیکه ۱۸: مقیاس‌پذیری و امنیت سیستم ۲

ذخیره‌سازی پیام‌ها (مقیاس‌پذیری)

مشکل: هر روز میلیاردها پیام تولید می‌شود!

راه حل ۱: شاردينگ بر اساس User ID

دیتابیس ۱: کاربران ۱ تا ۱۰۰۰

دیتابیس ۲: کاربران ۱۰۰۱ تا ۲۰۰۰

راه حل ۲: جداسازی پیام‌های قدیمی و جدید

• پیام‌های ۳ ماه اخیر: در دیتابیس اصلی (سریع)

• پیام‌های قدیمی‌تر: در دیتابیس بایگانی (آرشیو)

امنیت 

۱. احراز هویت (Authentication)

• هر کاربر یک Token دارد (مثل کارت شناسایی)

• هر درخواست باید Token معتبر داشته باشد

۲. رمزنگاری (Encryption)

• توصیه: پیام‌ها را در مسیر (HTTPS) و در دیتابیس رمز کنید

• تشبیه: نامه را در پاکت قفل‌دار بگذارید

تیکه ۱۹: سوالات مصاحبه سیستم ۲ (بخش اول)

! سوالات مصاحبه‌ای

سوال ۱: چطور پیام را به چندین نفر (گروهی) بفرستیم؟

جواب: به جای ارسال به هر کاربر، پیام را یک بار در دیتابیس ذخیره می‌کنیم و یک جدول `group_members` داریم که مشخص می‌کند هر کاربر در کدام گروه‌هاست. فقط کافی است پیام را به همه اعضای گروه اطلاع‌رسانی کنیم.

سوال ۲: اگر کاربر آفلاین باشد، پیام را چطور دریافت می‌کند؟

جواب:

• پیام در دیتابیس ذخیره می‌شود با وضعیت `delivered: false`

• وقتی کاربر آنلاین شد، سرور پیام‌های تحویل‌نشده را برایش می‌فرستد

• کاربر پیام را دریافت می‌کند و وضعیت به `delivered: true` تغییر می‌کند

سوال ۳: چطور از دوبار ارسال شدن پیام جلوگیری کنیم؟ (Idempotency)

جواب: هر پیام یک `message_id` یکتا دارد. سرور چک می‌کند اگر پیامی با این ID قبلاً پردازش شده، آن را نادیده می‌گیرد.

سوال ۴: تفاوت WebSocket و HTTP در این سیستم چیست؟

جواب:

• HTTP: برای ارسال پیام (فرستنده → سرور)

• WebSocket: برای دریافت لحظه‌ای پیام (سرور → گیرنده)

خلاصه یک صفحه‌ای

سوال: یک سیستم پیام‌رسان طراحی کنید.

پاسخ سطح بالا:

- Gateway برای ورود و احراز هویت
- Message Queue برای مدیریت بار
- WebSocket Server برای ارتباط لحظه‌ای
- Message Processor برای ذخیره‌سازی
- دیتابیس با Sharding و Cache

تخمین اعداد:

- ۵۰۰ میلیون کاربر فعال
- هر کاربر روزانه ۵۰ پیام → ۲۵ میلیارد پیام در روز
- نیاز به سیستم ذخیره‌سازی ابری عظیم

سیستم ۳: طراحی شبکه اجتماعی (مثل توییتر/X)

مشکل چیست؟

طراحی یک شبکه اجتماعی که کاربران بتوانند:

- پست (Tweet) منتشر کنند
- پست دیگران را بخوانند (Timeline/Feed)
- دنبال کنندگان (Followers) داشته باشند

•پست‌ها را لایک و کامنت کنند

تشبیه دنیای واقعی 🏠

شبکه اجتماعی مثل یک شهر بزرگ با روزنامه‌های شخصی است:

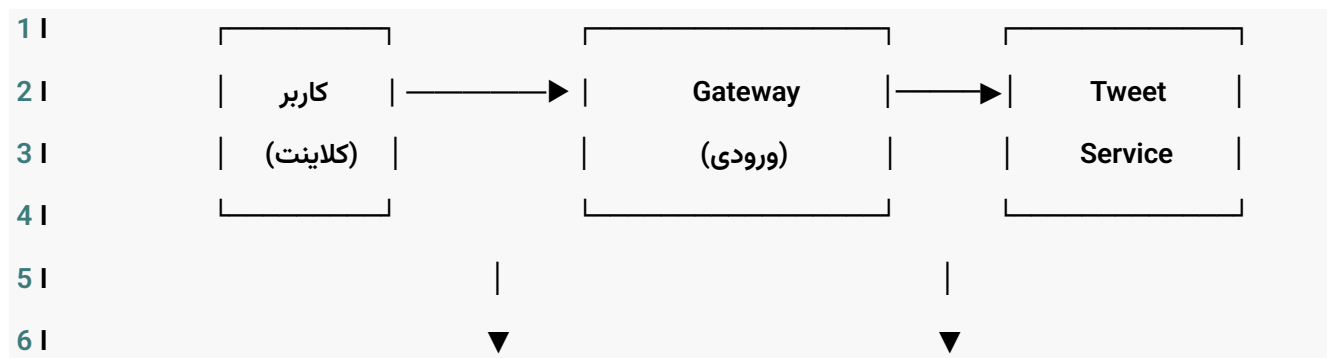
•هر کاربر: یک روزنامه‌نگار

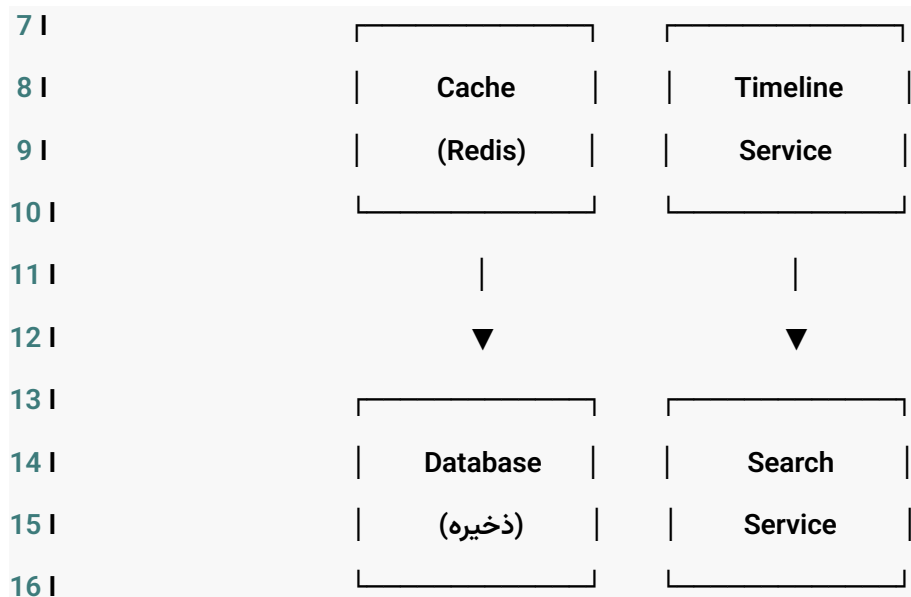
•توییت: یک مقاله کوتاه

•دنبال کردن: مشترک شدن در روزنامه آن شخص

•تایم‌لاین: صفحه اول روزنامه شخصی شما

معماری کلی 📊





تیکه ۲۳: Timeline Service و Tweet

اجزای اصلی

۱. Tweet Service (سرویس پست)

•وظیفه: ایجاد، حذف و دریافت توییت‌ها

•تشبیه: روزنامه‌نگار که مقاله می‌نویسد

•کارها:

•ذخیره توییت در دیتابیس

•ارسال توییت به سیستم تایم‌لاین

۲. Timeline Service (سرویس تایم‌لاین)

•وظیفه: نمایش آخرین پست‌های افرادی که دنبال می‌کنید

•تشبیه: صفحه اول روزنامه شما

۳. Follow Service (سرویس دنبال کردن)

•وظیفه: مدیریت روابط بین کاربران (چه کسی چه کسی را دنبال می‌کند)

•ساختار:

1			
2	follower_id	following_id	created_at
3			
4	101	102	2024-01-01
5	101	103	2024-01-02
6			

🧠 دو رویکرد طراحی تایم لاین

رویکرد ۱: Pull-based (کشیدن - توصیه شده برای توییتر)

•تشبیه: شما هر روز به غرفه‌های مختلف می‌روید و روزنامه می‌خرید

•کار: وقتی تایم لاین را باز می‌کنید، سرور توییت‌های افرادی که دنبال می‌کنید را جمع‌آوری می‌کند

•مزیت: ساده و کم‌هزینه

•عیب: اگر ۵۰۰۰ نفر را دنبال کنید، کند می‌شود

رویکرد ۲: Push-based (فشردن - توصیه شده برای اینستاگرام)

•تشبیه: روزنامه‌ها برای شما پست می‌شوند (اشتراک)

•کار: وقتی کسی توییت می‌زند، برای همه دنبال‌کنندگان یک کپی در تایم‌لاینشان ذخیره می‌شود

•مزیت: بسیار سریع

•عیب: اگر سلبریتی با ۱۰۰ میلیون دنبال‌کننده توییت بزند، ۱۰۰ میلیون کپی ذخیره می‌شود!

تیکه ۲۵: ترکیب رویکردها و Caching

ترکیب هر دو (توصیه نهایی)

•برای سلبریتی‌ها: Pull-based (چون تعداد کپی‌ها زیاد است)

•برای کاربران عادی: Push-based (چون دنبال‌کننده‌ها کم است)

کش (Caching) در شبکه اجتماعی

چرا کش مهم است؟

•توییت‌های داغ (Viral) میلیون‌ها بار دیده می‌شوند

•بدون کش، دیتابیس می‌سوزد!

لایه‌های کش

1 کش مرورگر (Browser Cache) لایه ۱: کش مرورگر

2 |

3 (برای عکس‌ها و فیلم‌ها) CDN: لایه ۲

4 |

5 (برای توییت‌های داغ) Redis: لایه ۳

6 |

7 لایه ۴: دیتابیس اصلی

تیکه ۲۶: استراتژی کش و جستجو

استراتژی کش: Cache-Aside

- کاربر توپیت را درخواست می‌کند
- اول کش را چک می‌کنیم
- اگر بود → برمی‌گردانیم (خیلی سریع)
- اگر نبود → از دیتابیس می‌خوانیم و در کش ذخیره می‌کنیم

جستجو (Search)

- مشکل: جستجوی توپیت بین میلیاردها توپیت
- راه حل: Elasticsearch
- تشبیه: یک کتابخانه با فهرست‌های پیشرفته
- چرا؟ برای جستجوی متن و هشتک‌ها عالی است
- کار: توپیت‌ها را Index (فهرست‌بندی) می‌کند تا جستجو سریع شود

! سوالات مصاحبه‌ای

سوال ۱: چطور توپیت‌های داغ (Trending) را پیدا می‌کنیم؟

جواب:

- هر توپیت یک وزن دارد (بر اساس لایک، کامنت، بازتوپییت)
- سیستم یک کرون جاب (هر ۵ دقیقه) محاسبه می‌کند

• از الگوریتم‌های کشف روند مثل Count-Min Sketch استفاده می‌کنیم

سوال ۲: چطور از اسپم و محتوای نامناسب جلوگیری کنیم؟

جواب:

• فیلتر کلمات کلیدی

• سیستم یادگیری ماشین برای تشخیص محتوای نامناسب

• محدودیت نرخ (Rate Limiting) - هر کاربر در روز فقط ۱۰۰۰ توییت

سوال ۳: اگر یک سلبریتی ۲۰۰ میلیون دنبال‌کننده داشته باشد، چطور توییت می‌فرستد؟

جواب:

• از رویکرد Pull استفاده می‌کنیم

• توییت را فقط یک بار ذخیره می‌کنیم

• وقتی هر کاربر تایم‌لاینش را باز می‌کند، توییت را می‌خواند

خلاصه یک صفحه‌ای 

سوال: یک شبکه اجتماعی مثل توییتر طراحی کنید.

پاسخ سطح بالا:

• Gateway برای ورود و احراز هویت

• Tweet Service برای مدیریت پست‌ها

• Timeline Service با ترکیب Push/Pull

• Cache (Redis) برای سرعت

•Elasticsearch برای جستجو

•دیتابیس با Sharding بر اساس User ID

تخمین اعداد:

•۵۰۰ میلیون کاربر فعال

•هر کاربر روزانه ۱۰ توییت → ۵ میلیارد توییت در روز

•هر توییت متوسط ۱۴۰ کاراکتر → ۷۰۰GB متن در روز!

سیستم ۴: طراحی سیستم فروشگاه‌های (مثل دیجیکالا)

مشکل چیست؟

طراحی یک فروشگاه اینترنتی که کاربران بتوانند:

•محصولات را مرور کنند

•به سبد خرید اضافه کنند

•سفارش ثبت کنند

•پرداخت انجام دهند

•وضعیت سفارش را پیگیری کنند

تشبیه دنیای واقعی

فروشگاه اینترنتی مثل یک فروشگاه بزرگ فیزیکی است:

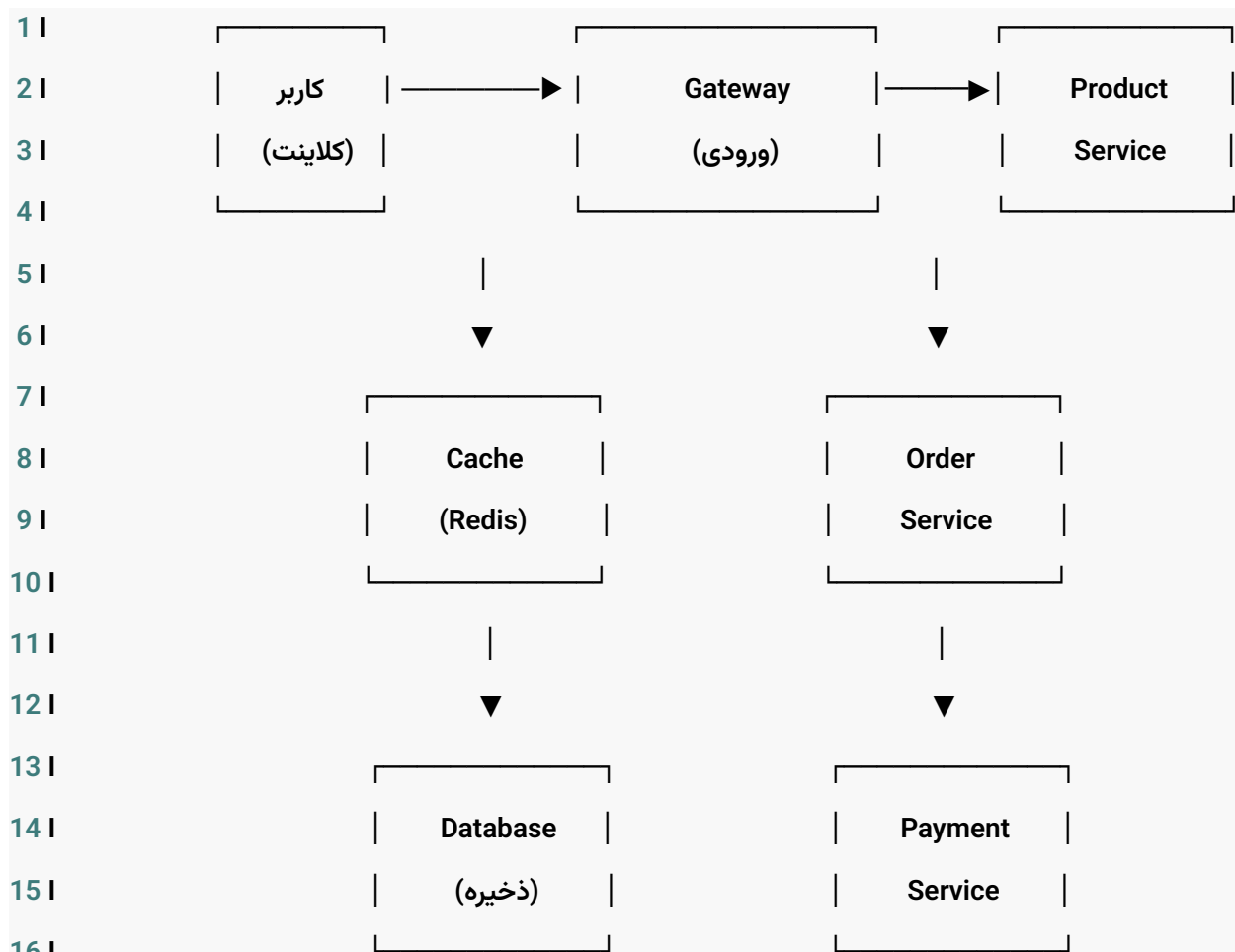
• محصولات: کالاهای موجود در قفسه‌ها

• سبد خرید: سبد خرید فیزیکی شما

• انبار: انبار پشته‌ی فروشگاه

• پرداخت: صندوق فروش

معماری کلی



تیکه ۳۰: Cart Service و Product

اجزای اصلی

۱. Product Service (سرویس محصولات)

•وظیفه: مدیریت اطلاعات محصولات

•تشبیه: قفسه‌های فروشگاه با برچسب قیمت

•اطلاعات محصول:

1					
2	product_id	name	price	stock	
3					
4	101	100 عدد	20,000,000	گوشی سامسونگ	
5	102	500 عدد	2,500,000	هدفون بلوتوث	
6					

۲. Cart Service (سرویس سبد خرید)

•وظیفه: مدیریت سبد خرید کاربر

•تشبیه: سبد خرید در فروشگاه

•نکته: معمولاً در Redis ذخیره می‌شود (سریع و موقت)

تیکه ۳۱: Order و Payment Service و مدیریت موجودی

۳. Order Service (سرویس سفارشات)

•وظیفه: ثبت سفارش نهایی

•تشبیه: خرید نهایی در صندوق

•مراحل:

1. دریافت اطلاعات سبد خرید

2. چک کردن موجودی

3. محاسبه قیمت نهایی

4. ثبت سفارش در دیتابیس

۴. Payment Service (سرویس پرداخت)

•وظیفه: پردازش پرداخت

•تشبیه: کارت خوان فروشگاه

•نکته: معمولاً با درگاه بانکی ارتباط برقرار می‌کند

مدیریت موجودی (Inventory Management)

مشکل: دو کاربر همزمان آخرین محصول را می‌خرند! (Race Condition)

راه حل: قفل خوش‌بینانه (Optimistic Lock)

•هر محصول یک version دارد

•وقتی کاربر می‌خواهد بخرد، موجودی را چک می‌کند

•اگر موجودی $0 <$ بود، خرید انجام می‌شود

•version را ۱ افزایش می‌دهیم

•اگر version تغییر کرده بود (کاربر دیگری خرید)، سفارش را Cancel می‌کنیم

راه حل: قفل بدبینانه (Pessimistic Lock)

• وقتی کاربر محصولی را به سبد خرید اضافه می‌کند، محصول را قفل می‌کنیم

• کاربر دیگری نمی‌تواند آن را بخرد

• بعد از ۱۵ دقیقه، قفل آزاد می‌شود

پرداخت و تراکنش (Transaction)

ویژگی‌های ACID (مهم در مصاحبه)

ویژگی	توضیح	تشبیه
Atomicity	یا همه مراحل انجام شود یا هیچکدام	اگر پول از حساب شما کم شد، باید به حساب فروشنده واریز شود
Consistency	داده‌ها همیشه معتبر هستند	موجودی منفی نمی‌شود
Isolation	تراکنش‌ها از هم جدا هستند	خرید شما بر خرید من تأثیر نمی‌گذارد
Durability	بعد از انجام، از بین نمی‌رود	حتی با قطع برق، سفارش ثبت شده باقی می‌ماند

مدیریت تراکنش در کد

```

1 try {
2   start transaction
3   check inventory
4   decrease inventory
5   create order
6   process payment
7   commit transaction
8 } catch (Exception e) {

```

9 rollback transaction // همه چیز به حالت اول برمی‌گردد

10 }

تیکه ۳۳: توصیه‌ها و سوالات مصاحبه ۱

توصیه‌ها و تخفیف‌ها (Recommendation)

سیستم Recommendation چطور کار می‌کند؟

• سابقه خرید کاربر: محصولات مشابه خریده است

• تخفیف‌های فصلی: مثل تخفیف نوروزی

• محصولات پرفروش: دیگران چه خریده‌اند

ذخیره‌سازی اطلاعات کاربر برای شخصی‌سازی

• تشبیه: فروشنده که مشتریان همیشگی را می‌شناسد

! سوالات مصاحبه‌ای

سوال ۱: چطور از فروش محصولی که موجودی ندارد جلوگیری کنیم؟

جواب: از Inventory Service مستقل استفاده می‌کنیم که:

• قبل از ثبت سفارش، موجودی را چک می‌کند

• موجودی را به‌روز نگه می‌دارد

• اگر موجودی کم بود، به کاربر هشدار می‌دهد

سوال ۲: اگر پرداخت ناموفق بود، چه می‌شود؟

جواب:

- تمام تراکنش Rollback می‌شود
- موجودی به حالت اول برمی‌گردد
- سفارش با وضعیت failed ذخیره می‌شود
- به کاربر پیام خطا نمایش داده می‌شود

سوال ۳: چطور از کلاهبرداری (Fraud) جلوگیری کنیم؟

جواب:

- محدودیت نرخ (Rate Limiting)
- تشخیص آدرس‌های IP مشکوک
- تأیید دو مرحله‌ای برای خریدهای بزرگ
- الگوریتم‌های تشخیص الگوی خرید غیرعادی

سوال ۴: سبد خرید را در کجا ذخیره کنیم؟ دیتابیس یا کش؟

جواب:

- پاسخ صحیح: در Redis (کش)
- دلیل: خیلی سریع است و موقتی بودن سبد خرید را نشان می‌دهد
- نکته: اگر کاربر لاگین کرده باشد، سبد خرید در دیتابیس هم بکاپ می‌شود

خلاصه یک صفحه‌ای

سوال: یک فروشگاه اینترنتی مثل دیجیکالا طراحی کنید.

پاسخ سطح بالا:

• Gateway برای ورود و احراز هویت

• Product Service با کش (Redis)

• Redis در Cart Service

• Order Service با تراکنش ACID

• Payment Service متصل به درگاه بانکی

• Inventory Service برای مدیریت موجودی

تخمین اعداد:

• ۱۰۰ میلیون محصول

• ۱۰۰۰ میلیون کاربر

• ۱۰ میلیون سفارش در روز

• ۵۰۰ میلیون درخواست محصول در روز

سیستم ۵: طراحی پلتفرم اشتراک ویدیو (مثل)

یوتیوب/آپارات)

مشکل چیست؟

طراحی پلتفرمی که کاربران بتوانند:

• ویدیو آپلود کنند

• ویدیو تماشا کنند (استریم)

• ویدیو را لایک/کامنت کنند

• کیفیت ویدیو را تغییر دهند (۴۸۰p, 720p, 1080p)

تشبیه دنیای واقعی 🏠

پلتفرم ویدیو مثل یک شبکه تلویزیون با کتابخانه فیلم است:

• ویدیو: یک فیلم سینمایی

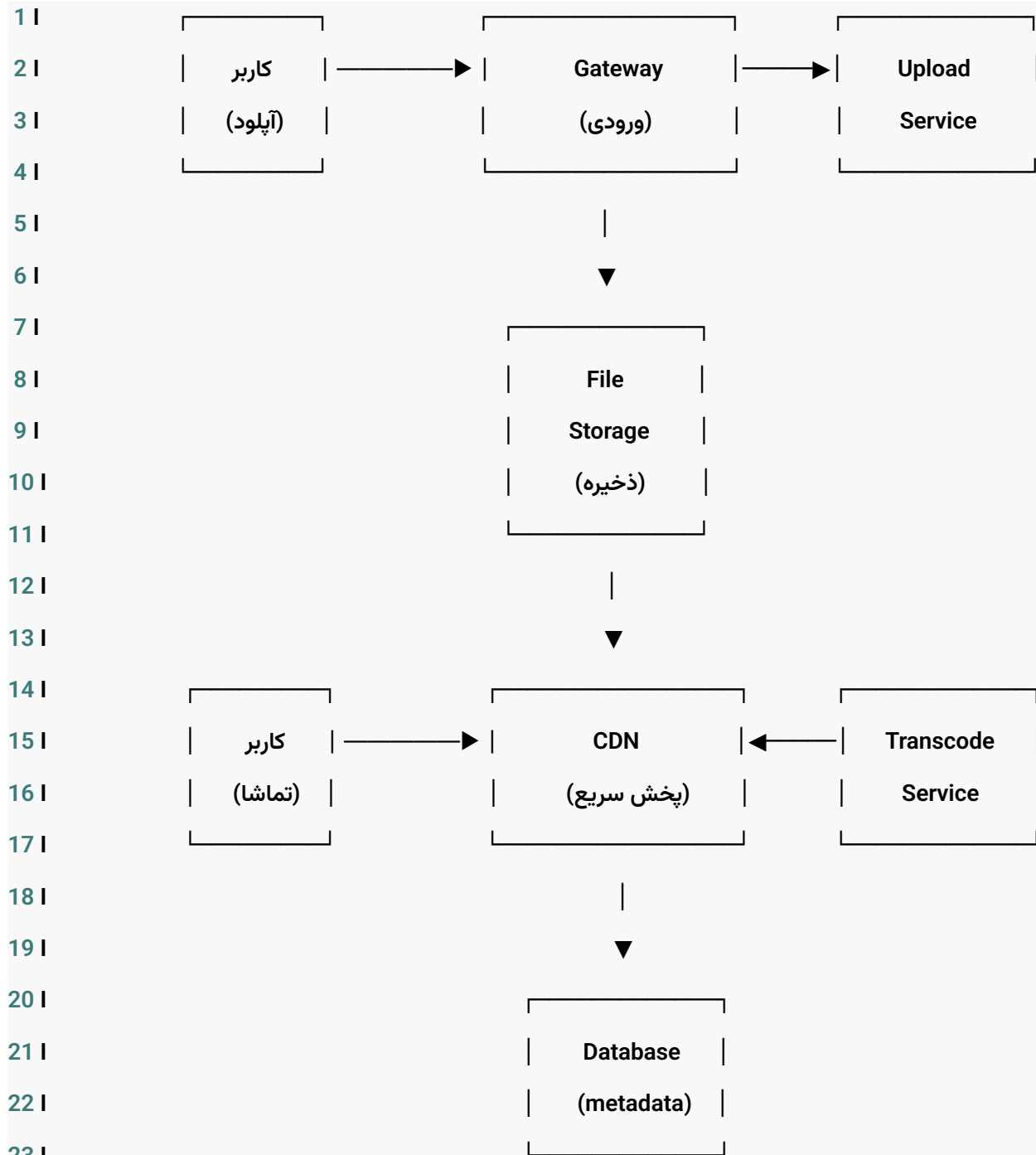
• آپلود: کارگردان فیلم را به تلویزیون می‌دهد

• تماشا: تماشاگر کانال را عوض می‌کند

• کیفیت‌های مختلف: SD, HD, 4K

معماری کلی 📊

1-Java Interview Question



تیکه ۳۷: Upload, File Storage و Transcode Service

اجزای اصلی

۱. Upload Service (سرویس آپلود)

- وظیفه: دریافت ویدیو از کاربر
- تشبیه: دریافت فیلم توسط تلویزیون
- چالش: ویدیوها بزرگ هستند (GBها)
- راه حل: آپلود تکه تکه (Chunked Upload)

۲. File Storage (ذخیره سازی فایل)

- وظیفه: ذخیره ویدیوها در فضای ابری
- ابزار: AWS S3، Google Cloud Storage
- تشبیه: بایگانی فیلم ها در انبار

۳. Transcode Service (سرویس تبدیل کیفیت)

- وظیفه: ویدیو را به کیفیت های مختلف تبدیل کند
- تشبیه: تبدیل فیلم به فرمت های مختلف
- کیفیت ها: ۲۴۰p, ۴۸۰p, ۷۲۰p, ۱۰۸۰p, ۴K

۴. CDN (شبکه توزیع محتوا)

- وظیفه: پخش سریع ویدیو به کاربران در سراسر جهان
- تشبیه: چندین سینما در شهرهای مختلف که یک فیلم را نمایش می دهند

•مزیت: ویدیو به نزدیکترین سرور به کاربر می‌رود

۵. Metadata Service (سرویس اطلاعات)

•وظیفه: ذخیره اطلاعات ویدیو (عنوان، توضیحات، تعداد بازدید)

•تشبیه: جلد و برچسب پشت فیلم

معماری آپلود ویدئو 🧠

فرآیند آپلود تکه‌تکه (Chunked Upload)

- 1 | مرحله ۱: کاربر شروع به آپلود می‌کند
- 2
- 3 | تقسیم می‌شود MB مرحله ۲: ویدیو به تکه‌های ۲
- 4
- 5 | مرحله ۳: هر تکه جداگانه آپلود می‌شود
- 6
- 7 | مرحله ۴: سرور تکه‌ها را دریافت و ترکیب می‌کند
- 8
- 9 | مرحله ۵: اگر آپلود قطع شد، از همان تکه ادامه می‌دهد

مزیت

•اگر اینترنت قطع شود، کل ویدیو از دست نمی‌رود

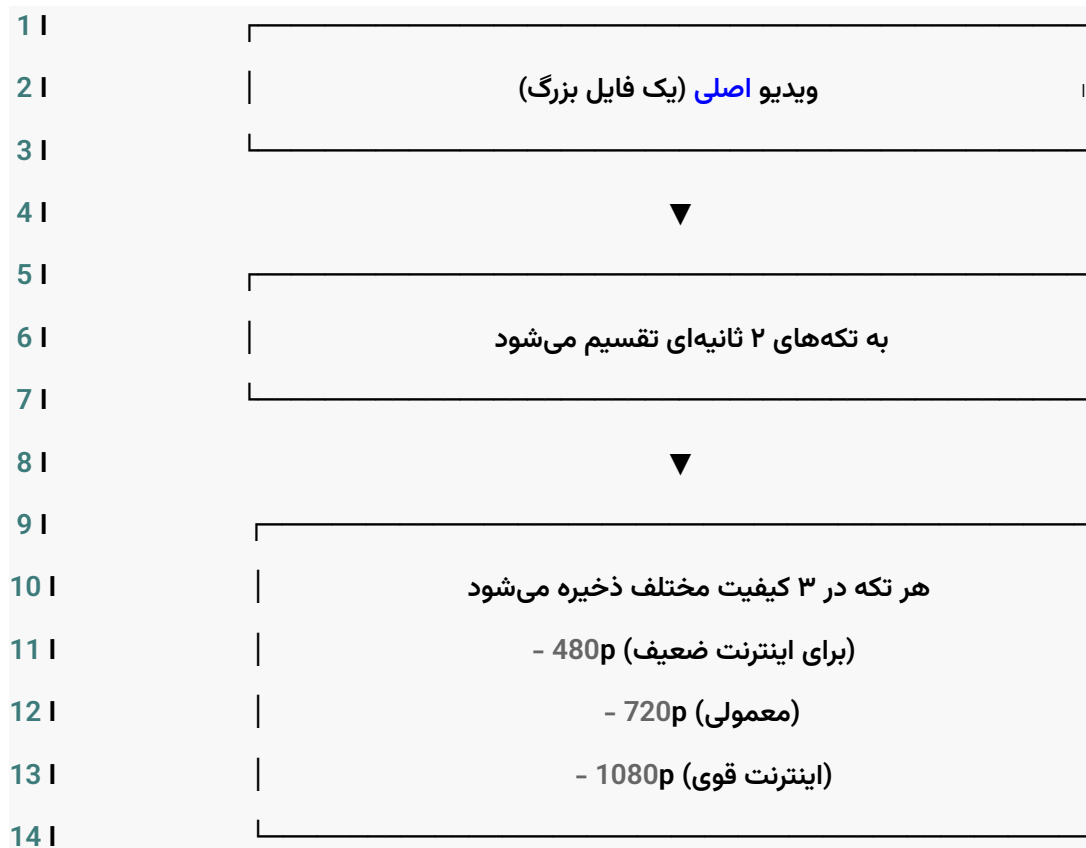
•می‌توان سرعت آپلود را بهینه کرد

تیکه ۳۹: معماری پخش ویدئو (Streaming)

معماری پخش ویدئو (Streaming)

پروتکل HLS (HTTP Live Streaming)

تشبیه: یک کتاب به چندین فصل تقسیم شده است



مزیت

• کاربر نیازی به دانلود کل ویدئو ندارد

• ویدئو تکه‌تکه پخش می‌شود

• کیفیت بر اساس سرعت اینترنت تغییر می‌کند (Adaptive Bitrate)

تیکه ۴۰: ذخیره‌سازی و جستجوی ویدیو

ذخیره‌سازی ویدیوها

آمار تقریبی:

- هر دقیقه ویدیو با کیفیت ۱۰۸۰p $\approx 100\text{MB}$
- ۱۰۰۰۰ ویدیو در روز $\approx 100\text{GB}$ فضای ذخیره‌سازی
- بعد از ۱ سال $\approx 36\text{TB}$ فضای ذخیره‌سازی!

راه‌حل‌های ذخیره‌سازی

۱. ذخیره‌سازی سرد (Cold Storage)

- تشبیه: بایگانی فیلم‌های قدیمی در انبار
- ویژگی: ارزان ولی کند
- کاربرد: ویدیوهای قدیمی (بیش از ۱ سال)

۲. ذخیره‌سازی گرم (Hot Storage)

- تشبیه: فیلم‌های جدید در قفسه اصلی
- ویژگی: گران ولی سریع
- کاربرد: ویدیوهای جدید و پرطرفدار

۳. سیستم توزیع شده (Distributed Storage)

• تشبیه: فیلم‌ها در چندین سینما پخش شده‌اند

• ابزار: HDFS یا GlusterFS

جستجوی ویدیو

• چالش: جستجو بین میلیون‌ها ویدیو

• راه حل: Elasticsearch

• تشبیه: فهرست کتابخانه با سیستم جستجوی سریع

• اطلاعات Index شده:

• عنوان ویدیو

• توضیحات

• هشتک‌ها

• کامنت‌های محبوب

سیستم پیشنهاد ویدیو (Recommendation)

چطور یوتیوب ویدیو پیشنهاد می‌دهد؟

• سابقه تماشای شما

• ویدیوهای مشابه (هم‌تگ)

• ویدیوهای محبوب (تعداد بازدید زیاد)

• ویدیوهای دوستان (اگر دنبال می‌کنید)

الگوریتم ساده:

$$1 \text{ score} = (\text{views} \times 0.3) + (\text{likes} \times 0.2) + (\text{comments} \times 0.1) + (\text{watch_time} \times 0.4)$$

! سوالات مصاحبه‌ای

سوال ۱: چطور ویدیو را با سرعت کم اینترنت پخش کنیم؟

جواب:

- از Adaptive Bitrate Streaming استفاده می‌کنیم
- سرعت اینترنت کاربر را اندازه می‌گیریم
- کیفیت مناسب را انتخاب می‌کنیم (اگر سرعت کم است $\rightarrow 480p$)

سوال ۲: چطور از کپی‌رایت جلوگیری کنیم؟

جواب:

- سیستم Content ID (مثل یوتیوب)
- مقایسه هش ویدیوها با ویدیوهای ثبت شده
- تشخیص خودکار صدا و تصویر با AI

سوال ۳: آپلود همزمان میلیون‌ها ویدیو در روز را چطور مدیریت کنیم؟

جواب:

- Message Queue (مثل RabbitMQ) برای صف‌بندی
- چندین Transcode Worker برای پردازش موازی
- Auto-scaling در فضای ابری

خلاصه یک صفحه‌ای

سوال: یک پلتفرم اشتراک ویدیو مثل یوتیوب طراحی کنید.

پاسخ سطح بالا:

- Gateway برای ورود و احراز هویت
- Upload Service با آپلود تکه تکه
- File Storage (S3/Google Cloud)
- Transcode Service برای کیفیت‌های مختلف
- CDN برای پخش سریع
- Metadata Service برای اطلاعات ویدیو
- Recommendation System برای پیشنهاد ویدیو

تخمین اعداد:

- ۲۰ میلیارد کاربر در ماه
- ۵۰۰ ساعت ویدیو در هر دقیقه آپلود می‌شود
- ۱۰ میلیارد ساعت ویدیو در روز تماشا می‌شود
- نیاز به فضای ذخیره‌سازی در مقیاس Exabyte

جمع‌بندی نهایی برای مصاحبه

الگوی پاسخ به هر سوال System Design

1. درک نیازمندی‌ها (Requirements)

• فانکشنال (Functional) (چه کاری باید انجام دهد)

• غیر فانکشنال (Non-Functional) (مقیاس، امنیت، سرعت)

2. طراحی سطح بالا (High-Level Design)

• رسم نمودار با مستطیل و فلش

• توضیح هر کامپوننت

3. جزئیات مهم (Core Components)

• دیتابیس (نوع، شاردینگ)

• کش (Redis)

• پیام‌رسانی (Message Queue)

4. مقیاس‌پذیری (Scalability)

• چطور با افزایش کاربران کار می‌کند؟

• چطور سرعت را حفظ می‌کند؟

5. تخمین اعداد (Back-of-the-envelope)

• تعداد کاربران

• فضای ذخیره‌سازی

• پهنای باند

منابع تکمیلی برای شما

• ویدیوهای کانال ByteByteGo (انیمیشن‌های فوق‌العاده)

• Grokking the System Design Interview (کتاب)

• کانال Gaurav Sen (یوتیوب - با زیرنویس فارسی)

• System Design Interview - Alex Xu (کتاب دو جلدی)

سیستم ۶: طراحی سیستم بانکی (مثل بانک ملی/درگاه پرداخت)

مشکل چیست؟

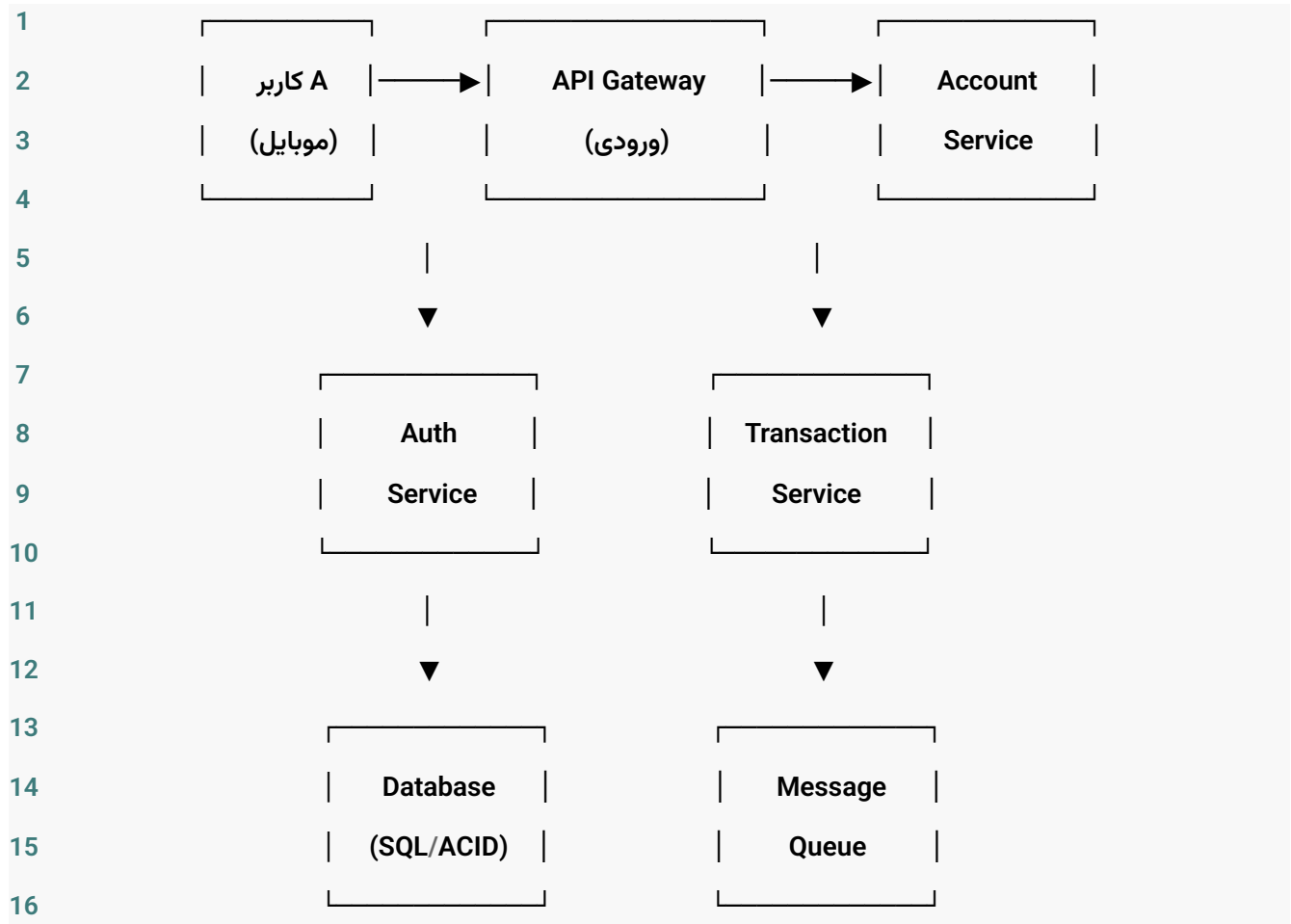
ما می‌خواهیم یک سیستم بانکی طراحی کنیم که کاربران بتوانند:

- حساب بانکی داشته باشند
 - موجودی حساب را ببینند
 - پول انتقال دهند (حواله/کارت به کارت)
 - تاریخچه تراکنش‌ها را مشاهده کنند
- مهم‌ترین چالش: پول نه گم شود، نه دوباره ساخته شود. هر تراکنش باید دقیق باشد.

تشبیه دنیای واقعی

سیستم بانکی مثل یک گاوصندوق بزرگ با دفتر کل حسابداری است:

- حساب بانکی: یک کشو در گاوصندوق
- تراکنش: یک سند حسابداری که باید مهر و امضا شود
- دفتر کل: دیتابیس که همه تراکنش‌ها را ثبت می‌کند
- کارمند بانک: سرور که درخواست‌ها را پردازش می‌کند
- ممیزی: بازرس که همه اسناد را چک می‌کند



Account Service و API Gateway

اجزای اصلی

۱. API Gateway (ورودی سیستم)

•وظیفه: اولین نقطه برخورد با کاربر

•تشبیه: درب ورودی بانک با نگهبان

•کارها:

•احراز هویت (Authentication) – چک کردن Token کاربر

•محدودیت نرخ (Rate Limiting) – جلوگیری از حمله

•هدایت درخواست‌ها به سرویس مناسب

۲. Account Service (سرویس حساب)

•وظیفه: مدیریت حساب‌های کاربران

•تشبیه: کارمند بانک که حساب‌ها را باز می‌کند

•اطلاعات حساب:

account_id	user_id	balance	currency
1001	501	5,000,000	تومان
1002	502	10,000,000	تومان

Database و Transaction Service

۳. Transaction Service (سرویس تراکنش)

•وظیفه: انجام عملیات انتقال وجه

•تشبیه: کارمند باجه که حواله صادر می‌کند

•مراحل:

1. دریافت درخواست انتقال

2. چک کردن موجودی حساب مبدا

3. کم کردن پول از حساب مبدا

4. اضافه کردن پول به حساب مقصد

5. ثبت تراکنش در دیتابیس

۴. Database (دیتابیس)

•وظیفه: ذخیره‌سازی حساب‌ها و تراکنش‌ها

•نوع: حتماً SQL (مثل PostgreSQL) به خاطر پشتیبانی از ACID

•جدول تراکنش‌ها:

txn_id	from_acc	to_acc	amount	status
90001	1001	1002	500,000	SUCCESS
90002	1002	1001	200,000	FAILED

جزئیات حیاتی - جریان انتقال وجه

جریان انتقال وجه (Core Transaction Flow)

این بخش قلب سیستم است. باید نشان دهید که چطور یک تراکنش را امن و اتمیک می‌کنید.

سناریو: کاربر A می‌خواهد ۵۰۰,۰۰۰ تومان به کاربر B انتقال دهد.

1 درخواست انتقال می‌دهد A مرحله ۱: کاربر

2 |

3 (معتبر است؟ Token) را چک می‌کند A مرحله ۲: سرور هویت کاربر

4 |

5 چک می‌شود (کافی است؟) A مرحله ۳: موجودی حساب

6 |

7 در دیتابیس زده می‌شود B و حساب A مرحله ۴: قفل روی ردیف حساب

8 |

9: مرحله ۵: عملیات اتمیک (در یک تراکنش دیتابسی)

10 - کم می‌شود (۴,۵۰۰,۰۰۰ → ۵,۰۰۰,۰۰۰) A موجودی -

11 - زیاد می‌شود (۱۰,۵۰۰,۰۰۰ → ۱۰,۰۰۰,۰۰۰) B موجودی -

12 - درج می‌شود transactions یک رکورد در جدول -

13 |

14 (ثبت نهایی) COMMIT → مرحله ۶: اگر همه چیز موفق بود

15 |

16 مرحله ۷: قفل‌ها آزاد می‌شوند

17 |

18 به صف پیام فرستاده می‌شود (Event) مرحله ۸: یک رویداد

مفهوم ACID در بانکداری

🔒 مفهوم ACID (مهم‌ترین مفهوم در سیستم بانکی)

ویژگی	توضیح ساده	مثال در بانک
Atomicity	یا همه مراحل انجام شود یا هیچکدام	اگر پول از حساب A کم شد، حتماً به حساب B اضافه شود
Consistency	داده‌ها همیشه معتبر می‌مانند	موجودی هیچ حسابی منفی نمی‌شود
Isolation	تراکنش‌ها از هم جدا هستند	دو تراکنش همزمان روی یک حساب، همدیگر را خراب نمی‌کنند
Durability	بعد از ثبت، از بین نمی‌رود	حتی با قطع برق، تراکنش ثبت شده باقی می‌ماند

جلوگیری از تراکنش تکراری (Idempotency)

🛡️ جلوگیری از تراکنش تکراری (Idempotency)

مشکل: اگر کاربر دو بار روی دکمه «انتقال» کلیک کند، چه می‌شود؟

تشبیه: اگر دو بار بگویید «پول را بفرست»، نباید دو بار پول کم شود.

راه حل: هر درخواست انتقال یک idempotency_key یکتا دارد.

- 1 (UUID مثل) کاربر یک کلید یکتا تولید می‌کند
- 2 |
- 3 درخواست با کلید به سرور می‌رسد
- 4 |
- 5 سرور چک می‌کند: آیا قبلاً با این کلید تراکنشی انجام شده؟
- 6 |
- 7 بله → همان نتیجه قبلی را برمی‌گرداند (پول دوباره کم نمی‌شود)
- 8 |
- 9 خیر → تراکنش را انجام می‌دهد و کلید را ذخیره می‌کند

• ابزار: می‌توانیم کلیدها را در Redis با زمان انقضا ذخیره کنیم.

امنیت در سیستم بانکی

امنیت (Security)

۱. رمزنگاری (Encryption)

• در حالت استراحت (At Rest): داده‌های حساس مثل شماره حساب و کارت با AES-256 رمزنگاری می‌شوند.

• در حال انتقال (In Transit): همه ارتباطات با TLS 1.3 انجام می‌شود.

• تشبیه: پول را در گاوصندوق قفل‌دار بگذارید و با ماشین ضدگلوله جابجا کنید.

۲. احراز هویت دو مرحله‌ای (2FA)

برای تراکنش‌های بالای یک مبلغ مشخص (مثلاً ۵۰ میلیون تومان)، حتماً کد تایید پیامکی یا اپلیکیشن لازم است.

۳. ممیزی (Audit Log)

هر تغییری در سیستم باید ثبت شود: چه کسی، چه زمانی، چه کاری انجام داد.

تشبیه: دوربین مدار بسته در بانک که همه چیز را ضبط می‌کند.

۴. تشخیص تقلب (Fraud Detection)

خدمت جداگانه که تراکنش‌ها را تحلیل می‌کند.

الگوهای مشکوک: انتقال مبلغ زیاد در ساعت غیرعادی، به حساب جدید، از IP خارجی.

تشبیه: کارمند امنیتی که رفتار مشتریان را زیر نظر دارد.

مقیاس‌پذیری و در دسترس بودن

مقیاس‌پذیری (Scalability)

۱. پارتیشن‌بندی (Sharding)

• مشکل: میلیون‌ها حساب در یک دیتابیس جا نمی‌شود.

• راه حل: تقسیم بر اساس `account_id`

دیتابیس ۱: حساب‌های ۱۰۰۰ تا ۵۰۰۰۰ 1

دیتابیس ۲: حساب‌های ۵۰۰۰۱ تا ۱۰۰۰۰۰ 2

دیتابیس ۳: حساب‌های ۱۰۰۰۰۱ تا ۱۵۰۰۰۰ 3

۲. تکثیر (Replication)

• معماری Master-Slave:

• Master: همه نوشتن‌ها اینجا انجام می‌شود.

• Slave: فقط خواندن‌ها (مثل نمایش موجودی) از اینجا انجام می‌شود.

• تشبیه: دفتر اصلی بانک (نوشتن) و شعبه‌های محلی (خواندن).

۳. در دسترس بودن بالا (High Availability)

• اگر دیتابیس Master از کار بیفتد، یکی از Slave‌ها سریعاً Master می‌شود.

• تشبیه: اگر مدیر شعبه مرکزی بیمار شود، معاونش سریعاً جای او را می‌گیرد.

سوالات مصاحبه سیستم بانکی

! سوالات مصاحبه‌ای

سوال ۱: چطور از منفی شدن موجودی جلوگیری می‌کنیم؟

جواب: در تراکنش دیتابسی، قبل از کم کردن پول، شرط $balance \geq amount$ را چک می‌کنیم. اگر موجودی کافی نبود، تراکنش Rollback می‌شود.

سوال ۲: اگر سرور بعد از کم کردن پول از حساب A اما قبل از اضافه کردن به حساب B از کار بیفتد، چه می‌شود؟

جواب: چون هر دو عملیات در یک تراکنش دیتابیزی ACID انجام می‌شود، اگر سرور بین این دو مرحله از کار بیفتد، دیتابیس به طور خودکار کل تراکنش را Rollback می‌کند. هیچ پولی گم نمی‌شود.

سوال ۳: چطور تراکنش‌های بین‌المللی (ارزهای مختلف) را مدیریت می‌کنیم؟

جواب: یک سرویس نرخ تبدیل ارز (Exchange Rate Service) داریم که نرخ لحظه‌ای را می‌گیرد. مبلغ تبدیل می‌شود و بعد تراکنش انجام می‌شود. چالش اصلی: نرخ ارز لحظه‌ای تغییر می‌کند.

سوال ۴: تفاوت سیستم بانکی با سیستم فروشگاه‌های چیست؟

جواب:

- در بانک، تراکنش مالی قلب سیستم است.
- سازگاری (Consistency) از در دسترس بودن (Availability) مهم‌تر است.
- در فروشگاه، اگر سبد خرید لحظه‌ای درست نباشد، فاجعه نیست. اما در بانک، اگر موجودی اشتباه باشد، فاجعه است.

سوال ۵: چطور با تراکنش‌های همزمان روی یک حساب برخورد می‌کنیم؟

جواب: از قفل ردیف (Row Locking) در دیتابیس استفاده می‌کنیم. وقتی تراکنشی روی حساب A در حال انجام است، تراکنش دیگر باید صبر کند تا قفل آزاد شود.

خلاصه یک صفحه‌ای

سوال: یک سیستم بانکی طراحی کنید.

پاسخ سطح بالا:

• API Gateway برای ورود و احراز هویت

• Account Service برای مدیریت حساب‌ها

• Transaction Service برای انجام تراکنش‌ها

• دیتابیس (PostgreSQL) SQL با ACID

• Redis برای کش و Idempotency

• Message Queue برای اطلاع‌رسانی غیرهمزمان

• Fraud Detection Service برای تشخیص تقلب

تخمین اعداد:

• ۵۰ میلیون کاربر فعال

• هر کاربر ۵ تراکنش در روز → ۲۵۰ میلیون تراکنش در روز

• هر تراکنش متوسط ۱KB فضا → ۲۵۰GB داده در روز!

سیستم ۷: طراحی سیستم رزرواسیون (مثل Booking.com / علی بابا)

مشکل چیست؟

ما می‌خواهیم یک سیستم رزرو طراحی کنیم که کاربران بتوانند:

- لیست آیتم‌های قابل رزرو (هتل، بلیط هواپیما، صندلی سینما) را ببینند

- موجودی (اتاق خالی / صندلی خالی) را چک کنند

- یک آیتم را برای خودشان رزرو کنند

- رزرو خود را لغو کنند

مهم‌ترین چالش: دو نفر همزمان آخرین اتاق یا صندلی را رزرو نکنند! (Double Booking)

تشبیه دنیای واقعی

سیستم رزرو مثل یک سینما با صندلی‌های محدود است:

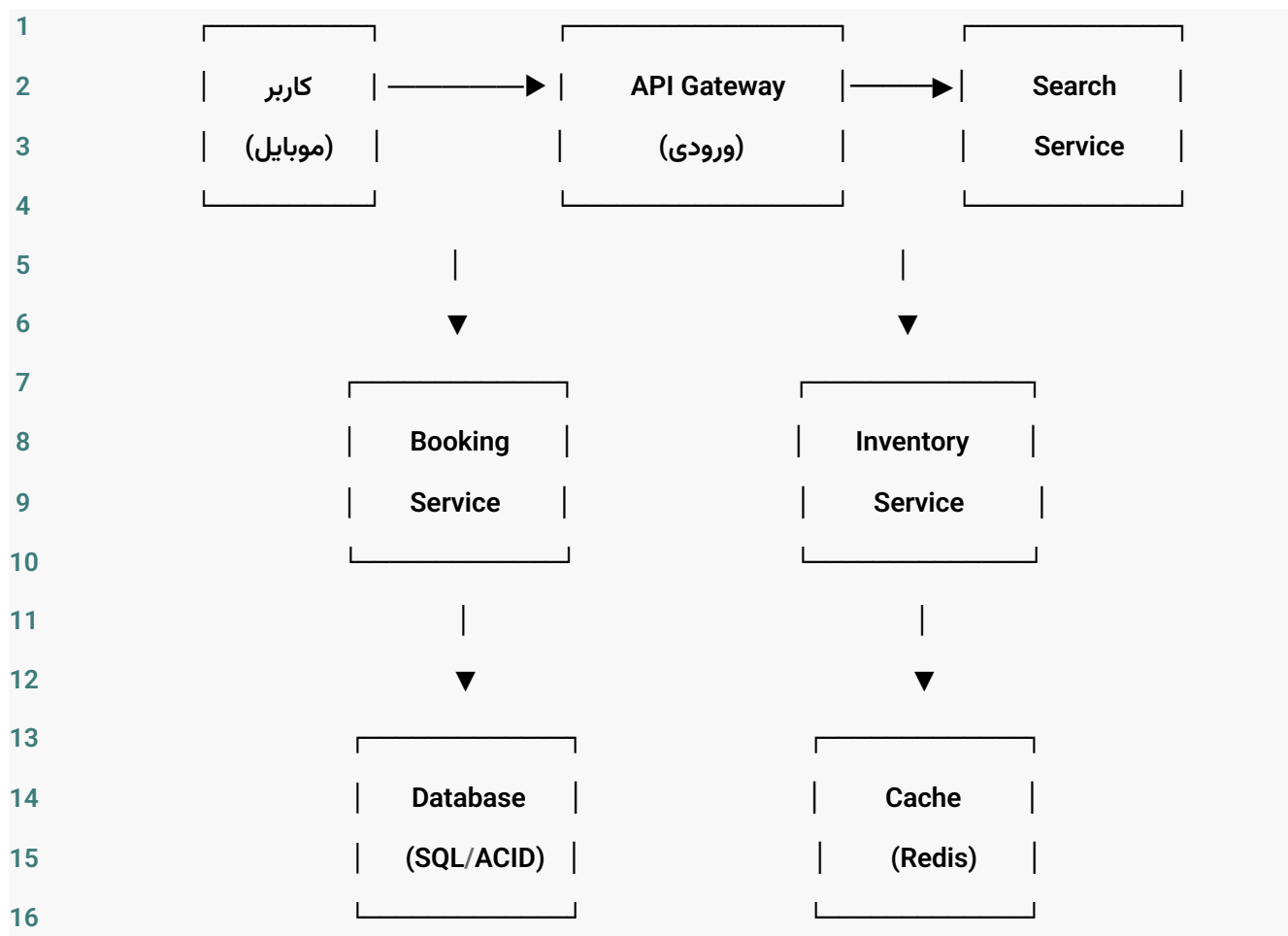
- صندلی سینما: آیتم قابل رزرو (اتاق هتل، صندلی هواپیما)

- بلیط: رزرو شما

- گیشه فروش: سرور که درخواست‌ها را پردازش می‌کند

- نقشه صندلی‌ها: دیتابیس که وضعیت هر صندلی را نشان می‌دهد

- قفل روی صندلی: وقتی کسی صندلی را انتخاب می‌کند، موقتاً قفل می‌شود

اجزای اصلی

۱. Search Service (سرویس جستجو)

•وظیفه: نمایش آیتم‌های موجود بر اساس فیلتر کاربر

•تشبیه: تابلو اعلانات سینما که فیلم‌ها و سانس‌ها را نشان می‌دهد

•کارها:

•دریافت فیلترها (تاریخ، شهر، قیمت، نوع اتاق)

•جستجو در دیتابیس

•نمایش نتایج با وضعیت موجودی

۲. Inventory Service (سرویس موجودی)

•وظیفه: مدیریت موجودی آیتم‌ها

•تشبیه: نقشه صندلی‌های سینما که نشان می‌دهد کدام صندلی خالی است

•اطلاعات:

item_id	date	total_count	booked_count
101 (هتل A)	2024-05-01	100	45
101 (هتل A)	2024-05-02	100	100 ← پر شده
102 (هتل B)	2024-05-01	50	10

Database و Booking Service

۳. Booking Service (سرویس رزرو)

•وظیفه: ثبت رزرو نهایی

•تشبیه: گیشه فروش بلیط که بلیط را صادر می‌کند

•مراحل:

1. دریافت درخواست رزرو از کاربر

2. چک کردن موجودی با Inventory Service

3. اگر موجود بود، کاهش موجودی

4. ثبت رزرو در دیتابیس

5. صدور بلیط / تاییدیه

4. Database (دیتابیس)

• نوع: SQL (مثل MySQL یا PostgreSQL) به خاطر پشتیبانی از تراکنش ACID

• جدول رزروها:

booking_id	user_id	item_id	date	status
70001	501	101	2024-05-01	CONFIRMED
70002	502	101	2024-05-01	CANCELLED
70003	503	102	2024-05-02	PENDING

⚠ مشکل اصلی: Double Booking (رزرو همزمان)

سناریوی فاجعه: دو کاربر همزمان آخرین اتاق هتل را می‌بینند:

- 1 (از 100 99 = booked_count) اتاق 101 را می‌بیند A لحظه ۱: کاربر
- 2 (از 100 99 = booked_count) اتاق 101 را می‌بیند B لحظه ۲: کاربر
- 3! لحظه ۳: هر دو رزرو می‌کنند
- 4 ❌ از 101 100 = booked_count: نتیجه

راه‌حل‌های مدیریت همزمانی در رزرو منابع

راه‌حل ۱: قفل بدبینانه (Pessimistic Locking)

ایده:

وقتی کاربری در حال رزرو است، ردیف مربوطه را در دیتابیس قفل می‌کنیم. بقیه باید صبر کنند.

تشبیه:

صندلی سینما را با زنجیر موقتاً می‌بندیم تا کسی نتواند بنشیند.

مزیت:

۱۰۰٪ امن

عیب:

کند می‌شود چون همه باید در صف بمانند

```
1 SELECT * FROM inventory
2 WHERE item_id = 101 AND date = '2024-05-01'
3 FOR UPDATE;
4 -- حالا ردیف قفل است. هیچ‌کس دیگر نمی‌تواند آن را تغییر دهد --
```

راه‌حل ۲: قفل خوش‌بینانه (Optimistic Locking)

ایده:

هر ردیف یک version دارد. موقع تغییر، چک می‌کنیم version عوض نشده باشد.

تشبیه:

روی صندلی یک برچسب زمان می‌چسبانیم. وقتی می‌خواهید بنشینید، چک می‌کنید برچسب همان است که دیدید.

مزیت:

سریع‌تر

عیب:

اگر conflict زیاد باشد، کاربر باید دوباره تلاش کند

1 کاربر A: version = 5 → رزرو می‌کند → version = 6

2 عوض شده version (version) !رزرو می‌کند → خطا → version = 5 کاربر B

3 باید دوباره تلاش کند کاربر B

راه حل ۳: Redis با SETNX (پیشنهادی برای سرعت بالا) - پیاده‌سازی

در جاوا

ایده:

قبل از رزرو، یک کلید یکتا در Redis می‌سازیم. اگر کلید وجود داشت، یعنی کسی دیگر در حال رزرو است.

تشبیه:

یک چراغ قرمز روی صندلی روشن می‌کنیم. اگر چراغ روشن بود، صندلی اشغال است.

مزیت:

سرعت بسیار بالا به دلیل استفاده از حافظه (In-Memory)

عیب:

• مناسب سناریوهای کوتاه مدت (مثل زمان اتمام خرید)

• نیاز به مدیریت انقضای کلید (TTL) برای جلوگیری از قفل های بی نهایت

1 SET inventory:101:2024-05-01:user501 "locked" NX EX 60

2 اگر موفق بود: کاربر قفل را گرفته و می تواند رزرو کند --

3 اگر ناموفق بود: یکی دیگر در حال رزرو است --

رزرو موقت (Temporary Hold)

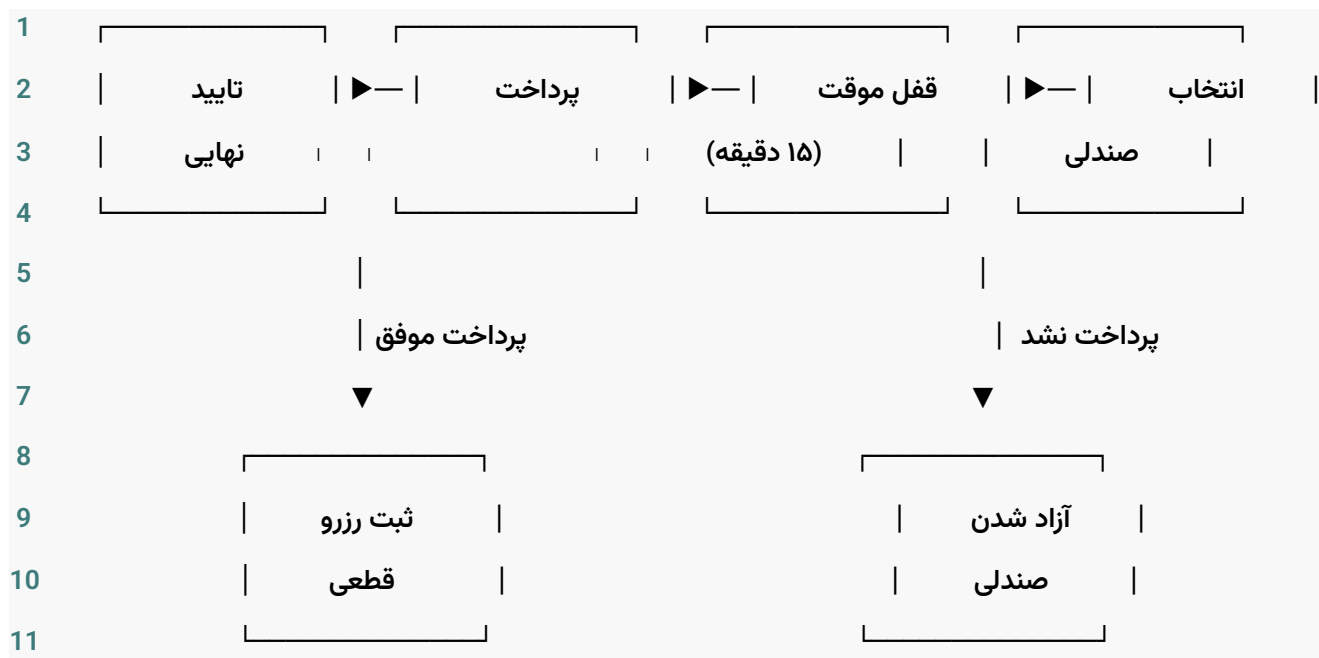
مشکل: کاربر صندلی را انتخاب می کند ولی هنوز پول نداده است. چقدر صبر کنیم؟

راه حل: صندلی را برای مدت کوتاه (مثلاً ۱۵ دقیقه) قفل می کنیم:

• اگر در این ۱۵ دقیقه پول داد → رزرو قطعی می شود

• اگر نداد → قفل آزاد می شود و صندلی دوباره قابل رزرو است

تشبیه: در سینما، صندلی را انتخاب می کنید، به شما ۱۵ دقیقه فرصت می دهند پول بدهید. اگر ندهید، صندلی آزاد می شود.



مقیاس پذیری

۱. کش کردن موجودی (Caching Inventory)

- مشکل: چک کردن موجودی در دیتابیس اصلی کند است.
- راه حل: موجودی را در Redis نگه می داریم.
- تشبیه: یک تابلو دیجیتال که تعداد صندلی های خالی را لحظه ای نشان می دهد.
- نکته: همیشه دیتابیس اصلی مرجع نهایی است. Redis فقط برای سرعت است.

۲. شاردینگ بر اساس آیتم (Sharding by Item)

- مشکل: میلیون ها آیتم قابل رزرو داریم.
- راه حل: تقسیم بر اساس item_id

دیتابیس ۱: آیتم های ۱ تا ۱۰۰۰۰۰ 1

دیتابیس ۲: آیتم های ۱۰۰۰۰۱ تا ۲۰۰۰۰۰ 2

۳. جداسازی رزروهای قدیمی

- رزروهای ۶ ماه اخیر: در دیتابیس اصلی (سریع)
- رزروهای قدیمی تر: در دیتابیس بایگانی (ارزان)

! سوالات مصاحبه‌ای

سوال ۱: اگر کاربر رزرو کند ولی پرداخت ناموفق باشد، چه می‌شود؟

جواب:

- رزرو با وضعیت PENDING ثبت می‌شود
- قفل موقت (مثلاً ۱۵ دقیقه) فعال است
- اگر پرداخت ناموفق بود، قفل آزاد می‌شود
- موجودی به حالت اول برمی‌گردد
- رزرو با وضعیت FAILED ذخیره می‌شود

سوال ۲: چطور از رزرو بیش از ظرفیت جلوگیری می‌کنیم؟

جواب:

- از قفل‌گذاری (Locking) استفاده می‌کنیم
- قبل از ثبت، شرط $\text{booked_count} < \text{total_count}$ را چک می‌کنیم
- عملیات کاهش موجودی و ثبت رزرو در یک تراکنش ACID انجام می‌شود

سوال ۳: اگر دو کاربر همزمان آخرین صندلی را بخواهند چه می‌شود؟

جواب:

• با قفل بدبینانه (Pessimistic): یکی از آنها منتظر می ماند تا قفل آزاد شود. نفر اول رزرو می کند، نفر دوم خطای «ظرفیت تکمیل» می گیرد.

• با قفل خوش بینانه (Optimistic): نفر دوم باید دوباره تلاش کند و موجودی جدید را ببیند.

سوال ۴: چطور رزرو گروهی (مثلاً ۵ اتاق با هم) انجام دهیم؟

جواب:

• همه ۵ اتاق باید در یک تراکنش ACID رزرو شوند

• اگر حتی یک اتاق موجود نبود، کل تراکنش Rollback می شود

• به کاربر پیام «رزرو گروهی ناموفق» نمایش داده می شود

سوال ۵: تفاوت سیستم رزرو با سیستم فروشگاه چیست؟

جواب:

• در رزرو، موجودی محدود و تاریخ محور است (اتاق هتل برای تاریخ خاص)

• در فروشگاه، موجودی فیزیکی است ولی تاریخ محور نیست

• در رزرو، زمان نقش کلیدی دارد (قفل موقت، انقضای رزرو)

خلاصه یک صفحه ای



سوال: یک سیستم رزرو (هتل/بلیط) طراحی کنید.

پاسخ سطح بالا:

• API Gateway برای ورود

• Search Service برای نمایش آیتم‌های موجود

• Inventory Service برای مدیریت موجودی

• Booking Service برای ثبت رزرو

• دیتابیس SQL با ACID

• Redis برای کش موجودی و قفل موقت

• Message Queue برای اطلاع‌رسانی

تخمین اعداد:

• ۱۰ میلیون آیتم قابل رزرو

• ۵۰۰ هزار رزرو در روز

• هر رزرو متوسط ۵۰۰MB → ۱KB داده در روز

• نرخ تبدیل جستجو به رزرو: حدود ۲٪

سیستم ۸: طراحی سیستم چت زنده / پشتیبانی آنلاین (Live Chat / Customer Support)

 مشکل چیست؟

ما می‌خواهیم یک سیستم چت زنده طراحی کنیم که:

• کاربران بتوانند با پشتیبانی صحبت کنند (مثل چت آنلاین دیجی‌کالا یا اسنپ)

• اپراتورها بتوانند همزمان به چند کاربر پاسخ دهند

• اگر اپراتور آزاد نبود، کاربر در صف انتظار بماند

• تاریخچه چت برای پیگیری بعدی ذخیره شود

تفاوت با پیامرسان (واتساپ): در پیامرسان، دو کاربر عادی با هم چت می‌کنند. در سیستم پشتیبانی، یک اپراتور (کارمند) به چند کاربر پاسخ می‌دهد و صف انتظار وجود دارد.

تشبیه دنیای واقعی 🏠

سیستم پشتیبانی آنلاین مثل مرکز تماس (Call Center) است:

• کاربر: مشتری که زنگ زده و منتظر پاسخ است

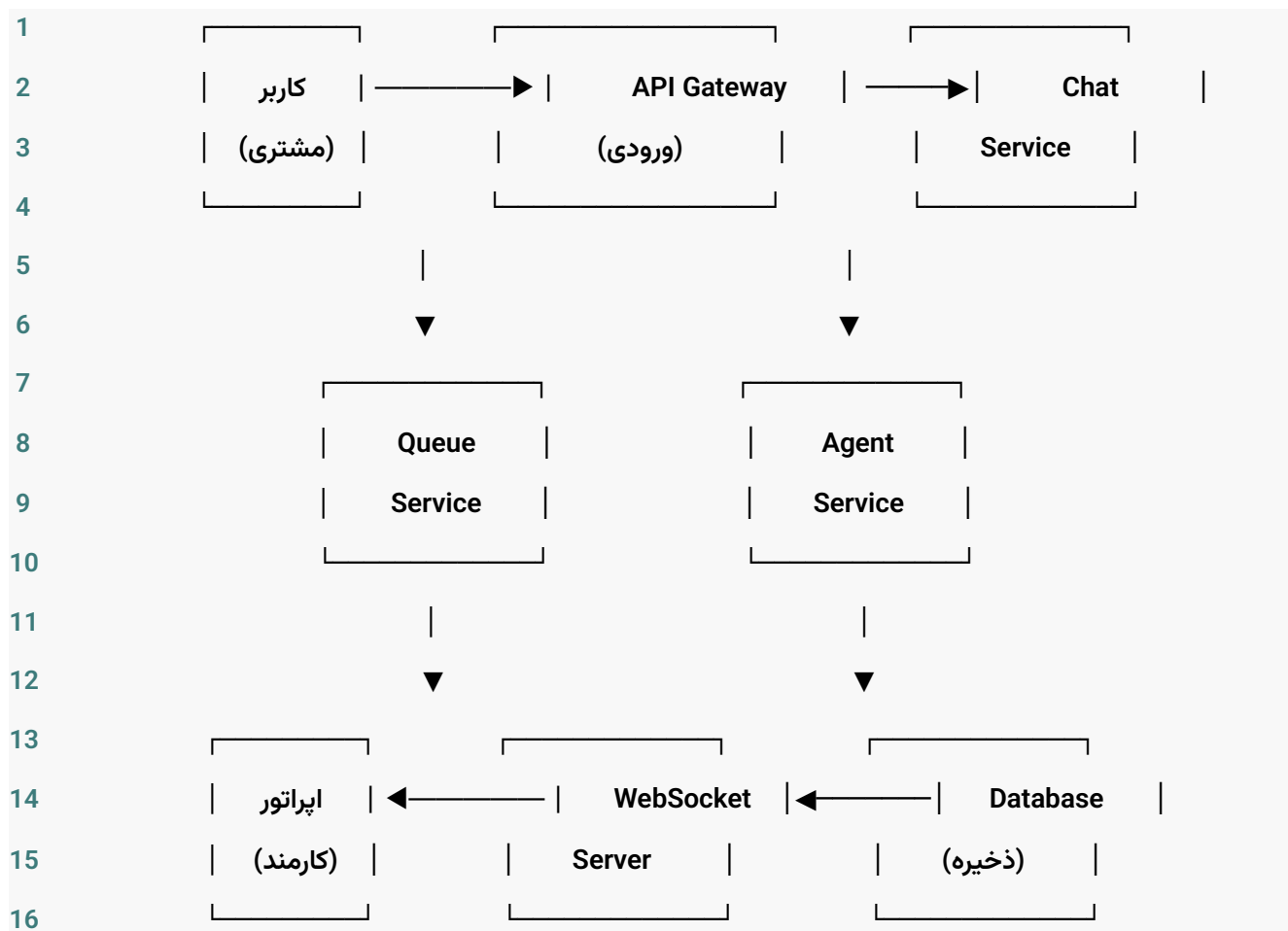
• اپراتور: کارمند پشتیبانی که به مشتریان پاسخ می‌دهد

• صف انتظار: مشتریانی که پشت خط منتظرند

• مدیر شیفت: سیستمی که مشتریان را به اپراتورهای آزاد متصل می‌کند

• ضبط مکالمات: دیتابیس که چت‌ها را ذخیره می‌کند

معماری کلی 📊



اجزای اصلی

۱. Chat Service (سرویس چت)

•وظیفه: مدیریت پیام‌های چت بین کاربر و اپراتور

•تشبیه: خط تلفن که صدا را منتقل می‌کند

•کارها:

•دریافت پیام از کاربر

•ارسال پیام به اپراتور مربوطه

•ذخیره پیام در دیتابیس

•نمایش وضعیت «در حال تایپ...»

۲. Queue Service (سرویس صف انتظار)

•وظیفه: مدیریت صف کاربران منتظر

•تشبیه: صف پشت خط تلفن که به ترتیب وصل می‌شوند

•اطلاعات:

queue_id	user_id	entered_at	priority
1	501	14:30:00	normal
2	502	14:31:15	high
3	503	14:32:45	normal

۳. Agent Service (سرویس اپراتور)

•وظیفه: مدیریت اپراتورها و وضعیت آنها

•تشبیه: مدیر مرکز تماس که می‌داند کدام کارمند آزاد است

•وضعیت اپراتور:

agent_id	name	status	current_chat
8001	علی رضایی	AVAILABLE	null
8002	مریم احمدی	BUSY	501
8003	حسین کریمی	OFFLINE	null

WebSocket و ارتباط لحظه‌ای

۴. WebSocket Server (سرور اتصال لحظه‌ای)

•وظیفه: ارتباط دوطرفه و لحظه‌ای بین کاربر و اپراتور

•تشبیه: تلفن که هر دو طرف می‌توانند همزمان صحبت کنند

•تفاوت با HTTP:

•HTTP: مثل نامه فرستادن (بفرست و منتظر جواب باش)

•WebSocket: مثل تلفن (همیشه وصل هستی، هر لحظه می‌توانی صحبت کنی)

۵. Database (دیتابیس)

•نوع: ترکیبی از SQL و Redis

•SQL: برای ذخیره تاریخچه چت‌ها

•Redis: برای ذخیره وضعیت لحظه‌ای (چه کسی آنلاین است، چه کسی در حال چت است)

جدول چت‌ها:

chat_id	user_id	agent_id	message	timestamp
50001	501	8002	سلام	14:35:00
50001	8002	501	سلام چطور؟	14:35:10
50002	502	8001	مشکل سفارش	14:36:00

جریان اتصال کاربر به اپراتور

این قلب سیستم پشتیبانی است. بیایید ببینیم از لحظه ورود کاربر چه اتفاقی می افتد:

- 1 مرحله ۱: کاربر وارد صفحه چت پشتیبانی می شود
- 2 |
- 3 مرحله ۲: کاربر دکمه "شروع چت" را می زند
- 4 |
- 5 قرار می دهد Queue Service کاربر را در Chat Service: مرحله ۳
- 6 |
- 7 وجود دارد؟ (AVAILABLE) مرحله ۴: سیستم چک می کند آیا اپراتور آزاد
- 8 |
- 9 بله → کاربر به اپراتور آزاد متصل می شود —
- 10 | تغییر می کند BUSY وضعیت اپراتور به -
- 11 | کاربر از صف خارج می شود -
- 12 | بین آنها برقرار می شود WebSocket -
- 13 |
- 14 خیر → کاربر در صف می ماند —
- 15 در صف هستید" را می بیند X پیام "شما نفر -
- 16 وقتی اپراتور آزاد شد، به او متصل می شود -

تشبیه: مثل تماس با مرکز پشتیبانی تلفنی. اول پیام "همه اپراتورها مشغول هستند، لطفاً منتظر بمانید" را می شنوید، بعد به محض آزاد شدن یک اپراتور، وصل می شوید.

مدیریت صف (Queue Management)

۱. صف عادی (FIFO - First In First Out)

• تشبیه: صف نانوائی - هر کس زودتر آمده، زودتر نوبتش می‌شود

• مزیت: عادلانه است

۲. اولویت‌بندی (Priority Queue)

• تشبیه: در بیمارستان، بیمار اورژانسی زودتر از بیمار عادی ویزیت می‌شود

• کاربرد:

• کاربران VIP (خرید زیاد) اولویت بالاتر دارند

• مشکل قطعی پرداخت (که فوری است) نسبت به سوال عمومی اولویت دارد

1 صف اولویت بالا: [کاربر ۱۰۵] [کاربر ۱۰۸] ← اینها زودتر وصل می‌شوند

2 صف عادی: [کاربر ۱۰۱] [کاربر ۱۰۲] [کاربر ۱۰۳]

۳. حداکثر زمان انتظار (Timeout)

اگر کاربر ۱۰ دقیقه در صف بماند چه می‌شود؟

• گزینه ۱: پیام "بعداً تماس بگیرید" نمایش داده می‌شود

• گزینه ۲: سیستم یک تیکت (Ticket) می‌سازد و کاربر از صف خارج می‌شود

قابلیت‌های ویژه ✨

۱. وضعیت "در حال تایپ" (Typing Indicator)

- تشبیه: صدای تق تق کیبورد در تلفن که نشان می‌دهد طرف مقابل در حال نوشتن است
- پیاده‌سازی: وقتی کاربر شروع به تایپ می‌کند، یک پیام کوتاه WebSocket می‌فرستد: "کاربر در حال تایپ است"
- اپراتور: یک انیمیشن سه نقطه (...) می‌بیند

۲. انتقال چت (Transfer Chat)

- مشکل: اپراتور نمی‌تواند مشکل تخصصی (مثلاً مشکل پرداخت) را حل کند
- راه‌حل: چت را به اپراتور متخصص منتقل می‌کند
- تشبیه: در مرکز تماس، اگر مشکل فنی است، به واحد فنی منتقل می‌کنند

۳. چت گروهی با چند اپراتور

- مشکل: مشکل پیچیده است و نیاز به نظر چند کارشناس دارد
- راه‌حل: یک چت گروهی با چند اپراتور و یک کاربر
- تشبیه: جلسه با چند کارشناس برای حل مشکل شما

۴. ربات چت (Chatbot)

- کاربرد: پاسخ به سوالات پرتکرار (ساعات کاری، وضعیت سفارش)

•تشبیه: منشی تلفنی که اول پاسخ می‌دهد و اگر نتوانست حل کند، به اپراتور انسانی منتقل می‌کند

امنیت

۱. احراز هویت

•کاربر: باید لاگین کرده باشد (Token معتبر)

•اپراتور: باید با پنل مدیریت و سطح دسترسی وارد شده باشد

•تشبیه: کارت شناسایی کارمند برای ورود به اتاق پشتیبانی

۲. حریم خصوصی

•کاربر A نباید چت کاربر B را ببیند

•اپراتور فقط به چت‌های مربوط به خودش دسترسی دارد

•تشبیه: هر مکالمه تلفنی فقط بین همان دو نفر است

۳. ممیزی (Audit Log)

•همه چت‌ها ذخیره می‌شوند

•مدیر می‌تواند چت‌ها را بررسی کند (برای کنترل کیفیت)

•تشبیه: ضبط مکالمات در مرکز تماس برای آموزش و کنترل کیفیت

مقیاس پذیری

۱. توزیع اپراتورها

• مشکل: ۱۰۰۰ اپراتور همزمان آنلاین هستند

• راه حل: اپراتورها بین چند سرور توزیع می شوند

• هر سرور مسئول تعدادی اپراتور است

۲. کش وضعیت اپراتورها

• وضعیت اپراتور (آزاد/مشغول/آفلاین) در Redis ذخیره می شود

• تشبیه: تابلو دیجیتال که نشان می دهد هر کارمند کجاست و چه می کند

• مزیت: سریع تر از دیتابیس SQL است

۳. آرشیو چت های قدیمی

• چت های ۳ ماه اخیر: در دیتابیس اصلی

• چت های قدیمی تر: در دیتابیس بایگانی (Cold Storage)

! سوالات مصاحبه‌ای

سوال ۱: چطور یک کاربر را به نزدیک‌ترین اپراتور آزاد متصل می‌کنیم؟

جواب:

- از Redis برای نگهداری لیست اپراتورهای آزاد استفاده می‌کنیم
- وقتی کاربر وارد صف می‌شود، سیستم اولین اپراتور AVAILABLE را انتخاب می‌کند
- معیار انتخاب: وضعیت (AVAILABLE)، تعداد چت‌های فعال، تخصص

سوال ۲: اگر اپراتور وسط چت اینترنتش قطع شود، چه می‌شود؟

جواب:

- سیستم با Heartbeat متوجه قطعی می‌شود
- چت به اپراتور آزاد دیگری منتقل می‌شود
- کاربر پیام "اپراتور جدید به چت پیوست" را می‌بیند
- تاریخچه چت برای اپراتور جدید نمایش داده می‌شود

سوال ۳: چطور از اسپم و پیام‌های تکراری جلوگیری می‌کنیم؟

جواب:

- Rate Limiting: هر کاربر حداکثر ۱۰ پیام در دقیقه
- فیلتر کلمات نامناسب

• اگر کاربر ۳ بار پیام تکراری فرستاد، اخطار می‌گیرد

سوال ۴: تفاوت این سیستم با پیام‌رسان (واتساپ) چیست؟

جواب:

ویژگی	پیام‌رسان	چت پشتیبانی
طرفین گفتگو	دو کاربر عادی	کاربر و اپراتور
صف انتظار	ندارد	دارد
اولویت‌بندی	ندارد	دارد (VIP/عادی)
انتقال گفتگو	ندارد	دارد (به اپراتور دیگر)
مقیاس کاربران	میلیاردها	میلیون‌ها

سوال ۵: اگر ۱۰۰۰ کاربر همزمان وارد صف شوند، چه می‌شود؟

جواب:

- صف در Redis ذخیره می‌شود (سریع)
- به هر کاربر شماره صف داده می‌شود
- کاربرانی که بیش از ۱۰ دقیقه منتظرند، پیام "بعداً مراجعه کنید" می‌گیرند
- سیستم Auto-scaling اپراتورهای رباتیک را فعال می‌کند

خلاصه یک صفحه‌ای



سوال: یک سیستم چت زنده / پشتیبانی آنلاین طراحی کنید.

پاسخ سطح بالا:

• API Gateway برای ورود کاربران

- Chat Service برای مدیریت پیام‌ها
- Queue Service برای صف انتظار
- Agent Service برای مدیریت اپراتورها
- WebSocket Server برای ارتباط لحظه‌ای
- Redis برای وضعیت اپراتورها و صف
- دیتابیس SQL برای تاریخچه چت‌ها

تخمین اعداد:

۱۰۰ میلیون کاربر

۱۰,۰۰۰ اپراتور همزمان

۱۰ میلیون چت در روز

هر چت متوسط ۲۰ پیام → ۲۰ میلیون پیام در روز

جمع‌بندی نهایی هر ۸ سیستم

#	سیستم	چالش اصلی	دیتابیس اصلی	ابزار کلیدی
۱	URL Shortener	تولید کد یکتا	SQL	Base62, Redis
۲	پیام‌رسان	ارتباط لحظه‌ای	SQL + NoSQL	WebSocket, Queue
۳	شبکه اجتماعی	تایم‌لاین سریع	SQL + NoSQL	Redis, CDN
۴	فروشگاهی	مدیریت موجودی	SQL (ACID)	Redis, Queue
۵	اشتراک ویدیو	ذخیره و پخش فایل	Metadata: SQL	S3, CDN, HLS
۶	بانکی	تراکنش اتمیک	SQL (ACID)	Locking, Audit
۷	رزرواسیون	جلوگیری از Double Booking	SQL (ACID)	Locking, Redis
۸	چت پشتیبانی	صف انتظار و اتصال	SQL + Redis	WebSocket, Queue

فاز ۱۳: Spring Boot – خلاصه و کاربردی

این بخش برای مصاحبه‌هایی که ازت Spring Boot می‌پرسن ضروریه. نیازی نیست همه جزئیات رو بدونی، ولی مفاهیم کلیدی رو باید بلد باشی.

۱۳.۱ چرخه حیات Bean ها در Spring

Bean: یه شیء که توسط Spring Container مدیریت میشه.

مراحل چرخه حیات:

1. Instantiation: سازنده صدا زده میشه

2. Dependency Injection: وابستگی‌ها تزریق میشن

3. Initialization: متدهای @PostConstruct و afterPropertiesSet اجرا میشن

4. Ready for Use: Bean آماده استفاده است

5. Destruction: متدهای @PreDestroy اجرا میشن

```

1 @Component
2 class MyBean {
3     public MyBean() {
4         System.out.println("1. سازنده");
5     }
6
7     @Autowired
8     public void setDependency(SomeService service) {
9         System.out.println("2. تزریق وابستگی");
10    }

```

```
11
12 @PostConstruct
13 public void init() {
14     System.out.println("3. مقداردهی اولیه");
15 }
16
17 @PreDestroy
18 public void destroy() {
19     System.out.println("4. پاک سازی");
20 }
21 }
```

۱۳.۲ انواع تزریق وابستگی (Dependency Injection)

۱. Constructor Injection (توصیه شده ✓)

```
1 @Service
2 class UserService {
3     private final UserRepository repository;
4
5     public UserService(UserRepository repository) {
6         this.repository = repository;
7     }
8 }
```

۲. Setter Injection

```

1 @Service
2 class UserService {
3     private UserRepository repository;
4
5     @Autowired
6     public void setRepository(UserRepository repository) {
7         this.repository = repository;
8     }
9 }

```

۳. Field Injection (کمتر توصیه میشه)

```

1 @Service
2 class UserService {
3     @Autowired
4     private UserRepository repository;
5 }

```

۳.۳ @Transactional چطور کار می‌کنه؟

Spring با Proxy دور کلاس می‌پیچه و قبل و بعد از متد، Transaction رو مدیریت می‌کنه.

```

1 @Service
2 class OrderService {
3     @Autowired

```

1-Java Interview Question

```
4 private OrderRepository orderRepository;
5
6 @Transactional
7 public void createOrder(Order order) {
8     // ۱. Transaction شروع همیشه
9     orderRepository.save(order);
10    // ۲. Transaction Commit (اگر خطا نبود) همیشه
11    // ۳. Rollback، اگر خطا بود همیشه
12 }
13 }
```

نکات مهم:

• @Transactional روی متدهای public کار می‌کند (به خاطر Proxy)

• روی متدهای private کار نمی‌کند

• RuntimeException باعث Rollback همیشه، Checked Exception نه (مگر اینکه تنظیم کنی)

```
1 // Rollback برای همه Exception ها
2 @Transactional(rollbackFor = Exception.class)
3 public void save() { ... }
4
5 // فقط RuntimeException برای
6 @Transactional
7 public void save() { ... }
```

Repository@ و Service@ و RestController@ ۱۳.۴

Annotation	کاربرد
RestController@	کنترلر REST API (ترکیب @Controller + @ResponseBody)
Service@	لایه Business Logic
Repository@	لایه دسترسی به دیتابیس (با ترجمه خودکار Exception)
Component@	عمومی‌ترین، برای هر Bean دلخواه

```

1 @RestController
2 @RequestMapping("/api/users")
3 class UserController {
4     @Autowired
5     private UserService userService;
6
7     @GetMapping("/{id}")
8     public User getUser(@PathVariable Long id) {
9         return userService.findById(id);
10    }
11 }
12
13 @Service
14 class UserService {
15     @Autowired
16     private UserRepository repository;
17
18     public User findById(Long id) {
19         return repository.findById(id).orElseThrow();

```

1-Java Interview Question

```
20 }  
21 }  
22  
23 @Repository  
24 interface UserRepository extends JpaRepository<User, Long> {  
25 }
```

۱۳.۵ application.properties مهم ترین تنظیمات

```
1 # properties config  
2 # سرور  
3 server.port=8080  
4 server.servlet.context-path=/api  
5 # دیتابیس  
6 spring.datasource.url=jdbc:mysql://localhost:3306/mydb  
7 spring.datasource.username=root  
8 spring.datasource.password=1234  
9 # JPA  
10 spring.jpa.hibernate.ddl-auto=update  
11 spring.jpa.show-sql=true  
12 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect  
13 # لاگ  
14 logging.level.org.springframework=INFO  
15 logging.level.com.myapp=DEBUG
```

۱۳.۶ Scope های Bean در Spring

Scope	توضیح
singleton (پیش فرض)	فقط یک نمونه در کل برنامه
prototype	هر بار که درخواست بشه، یه نمونه جدید
request	برای هر درخواست HTTP، یه نمونه (فقط در Web)
session	برای هر نشست کاربر، یه نمونه (فقط در Web)

```

1 @Component
2 @Scope("prototype")
3 class MyPrototypeBean { ... }

```

۳. Spring Bean ها (مفصل تر)

۱۳.۷ Bean های Spring – کامل

۱۳.۷.۱ چرخه حیات کامل Bean

```

1 |
2 |
3 | 1. Instantiation (ساخت شیء)
4 |   ↓
5 | 2. Populate Properties (تزریق وابستگی ها)
6 |   ↓

```


1-Java Interview Question

```
7 | 3. BeanNameAware (پیا‌ده‌سازی شده‌ اگر‌ setBeanName)
8 |   ↓
9 | 4. BeanClassLoaderAware (پیا‌ده‌سازی شده‌ اگر‌ setBeanClassLoader)
10 |   ↓
11 | 5. BeanFactoryAware (پیا‌ده‌سازی شده‌ اگر‌ setBeanFactory)
12 |   ↓
13 | 6. @PostConstruct (متد‌ init)
14 |   ↓
15 | 7. InitializingBean (پیا‌ده‌سازی شده‌ اگر‌ afterPropertiesSet)
16 |   ↓
17 | 8. Bean Ready! (آماده‌ استفاده‌)
18 |   ↓
19 | 9. @PreDestroy (متد‌ destroy)
20 |   ↓
21 | 10. DisposableBean (پیا‌ده‌سازی شده‌ اگر‌ destroy)
22 |
23 |
```

۱۳.۷.۲ انواع Scopes

```
1 // ۱. Singleton (پیش‌فرض) - یک نمونه برای کل برنامه
2 @Component
3 @Scope("singleton")
4 class SingletonBean { ... }
5
6 // ۲. Prototype - هر بار یک نمونه جدید
7 @Component
8 @Scope("prototype")
```

1-Java Interview Question

```
9 class PrototypeBean { ... }
10
11 // ۳. Request - برای هر درخواست HTTP
12 @Component
13 @Scope("request")
14 class RequestBean { ... }
15
16 // ۴. Session - برای هر نشست کاربر
17 @Component
18 @Scope("session")
19 class SessionBean { ... }
20 // ۵. Application - یک نمونه برای کل ServletContext
21 @Component
22 @Scope("application")
23 class ApplicationBean { ... }
```

۱۳.۷.۳ Bean Definition (روش‌های تعریف Bean)

```
1 // a. با @Component
2 @Component
3 class MyComponent { ... }
4
5 // b. با @Service
6 @Service
7 class MyService { ... }
8
9 // c. با @Repository
10 @Repository
```

1-Java Interview Question

```
11 class MyRepository { ... }
12
13 // d. با @Controller / @RestController
14 @RestController
15 class MyController { ... }
16
17 // e. در کلاس @Bean با
18 @Configuration
19 class AppConfig {
20     @Bean
21     public MyBean myBean() {
22         return new MyBean();
23     }
24
25     @Bean
26     public MyBean myBeanWithParams(AnotherBean another) {
27         return new MyBean(another);
28     }
29 }
30
31 // f. با @Import
32 @Import({MyConfig.class, AnotherConfig.class})
33 class MainConfig { ... }

34 ۱۳.۷.۴ Bean Wiring (ها Bean اتصال)

35 // ۱. Constructor Injection (توصیه شده)
```

```
36 @Service
37 class UserService {
38     private final UserRepository repository;
39
40     // نیست @Autowired اگه فقط یک سازنده باشه، نیازی به
41     public UserService(UserRepository repository) {
42         this.repository = repository;
43     }
44 }
45
46 // ۲. Setter Injection
47 @Service
48 class UserService {
49     private UserRepository repository;
50
51     @Autowired
52     public void setRepository(UserRepository repository) {
53         this.repository = repository;
54     }
55 }
56
57 // ۳. Field Injection
58 @Service
59 class UserService {
60     @Autowired
61     private UserRepository repository;
```

1-Java Interview Question

```
62 }
63
64 // ۴. @Qualifier (از یه نوع هستن Bean وقتی چند تا)
65 @Service
66 class ReportService {
67     @Autowired
68     @Qualifier("pdfReport")
69     private ReportGenerator pdfGenerator;
70
71     @Autowired
72     @Qualifier("excelReport")
73     private ReportGenerator excelGenerator;
74 }
75
76 // ۵. @Primary (Bean اولویت با این)
77 @Configuration
78 class ReportConfig {
79     @Bean
80     @Primary
81     public ReportGenerator pdfGenerator() {
82         return new PdfGenerator();
83     }
84
85     @Bean
86     public ReportGenerator excelGenerator() {
87         return new ExcelGenerator();
```

```

88 }
89 }
90
91 // ۶. @Lazy (ساخت دیر هنگام)
92 @Service
93 class UserService {
94     @Autowired
95     @Lazy
96     private HeavyService heavyService; // فقط وقتی استفاده بشه ساخته میشه
97 }

```

۱۳.۷.۵ Conditional Beans (ساخت شرطی)

```

1 @Configuration
2 class ConditionalConfig {
3
4     // ۱. اگه کلاس مشخصی در Classpath بود
5     @ConditionalOnClass(Driver.class)
6     @Bean
7     public DataSource mysqlDataSource() { ... }
8
9     // ۲. اگه کلاس مشخصی در Classpath نبود
10    @ConditionalOnMissingClass("com.mysql.jdbc.Driver")
11    @Bean
12    public DataSource h2DataSource() { ... }
13
14    // ۳. اگه پراپرتی مشخصی تنظیم شده بود

```

1-Java Interview Question

```
15 @ConditionalOnProperty(name = "app.use.mysql", havingValue = "true")
16 @Bean
17 public DatabaseService mysqlService() { ... }
18
19 // ۴. خاصی موجود بود Bean اگه
20 @ConditionalOnBean(MyService.class)
21 @Bean
22 public AnotherService anotherService() { ... }
23
24 // ۵. ترکیب شرطها
25 @ConditionalOnProperty(name = "app.cache.enabled")
26 @ConditionalOnBean(CacheManager.class)
27 @Bean
28 public CacheService cacheService() { ... }
29 }
```

فاز ۱۴-۲: JPA و Hibernate – کامل و عمیق

Hibernate محبوب‌ترین فریمورک ORM توی جاواست. اگه Persistence Context، Lazy Loading و Cascade رو عمیق نفهمی، توی Spring Data JPA به مشکل می‌خوری.

۲.۱-۱۴ Hibernate چیه؟

Hibernate یه فریمورک (Object-Relational Mapping) ORM هست که وظیفه‌ش نقشه‌برداری خودکار از آبجکت‌های جاوا به جدول‌های دیتابیسسه.

چرا Hibernate به جای JDBC؟

ویژگی	JDBC	Hibernate
نوشتن SQL	دستی	خودکار
Mapping	دستی (ResultSet → Object)	خودکار
Cache	✗	Level 1 / Level 2 ✓
استقلال از دیتابیس	✗ وابسته به SQL	✓ با Dialect عوض میشه
مدیریت Transaction	دستی	خودکار

1 java

2 // روش JDBC

3 String sql = "INSERT INTO users (name, email) VALUES (?, ?)";

4 PreparedStatement ps = conn.prepareStatement(sql);

5 ps.setString(1, user.getName());

6 ps.setString(2, user.getEmail());

7 ps.executeUpdate();

8

9 // روش Hibernate

10 session.persist(user); // SQL تولید میشه خودکار

JPA vs Hibernate ۱۴-۲.۲

! سوال مصاحبه‌ای ۱: فرق JPA و Hibernate چیه؟

جواب:

ویژگی	JPA	Hibernate
ماهیت	استاندارد (Specification)	پیاده‌سازی (Implementation)
پکیج	javax.persistence / jakarta.persistence	org.hibernate
اینترفیس اصلی	EntityManager	Session
زبان Query	JPQL	HQL
رابطه	استاندارد رو تعریف می‌کنه	استاندارد JPA رو پیاده می‌کنه

نکته کلیدی: JPA اینترفیس، Hibernate پیاده‌سازی. می‌تونن پیاده‌سازی رو عوض کنی (مثل EclipseLink بدون تغییر کد JPA).

1 java

2 // روش JPA (استاندارد)

3 @PersistenceContext

4 private EntityManager entityManager;

5 entityManager.persist(user);

6

7 // روش Hibernate خالص

8 @Autowired

9 private SessionFactory sessionFactory;

10 sessionFactory.getCurrentSession().persist(user);

۱۴-۲.۳ کامپوننت‌های اصلی Hibernate

SessionFactory vs Session

ویژگی	SessionFactory	Session
دامنه	سطح اپلیکیشن (Singleton)	سطح درخواست
Thread-Safe	بله 	خیر 
هزینه ساخت	بالا	پایین
کاربرد	تولید Session	اجرای CRUD

1 java

2 // SessionFactory: یکی برای کل اپ

3 SessionFactory factory = new Configuration()

4 .configure("hibernate.cfg.xml")

5 .buildSessionFactory();

6

7 // Session: برای هر عملیات

8 Session session = factory.openSession();

9 Transaction tx = session.beginTransaction();

10

11 User user = new User("Ali", "ali@test.com");

12 session.persist(user);

13

14 tx.commit();

15 session.close();

Entity Persistence Context ۱۴-۲.۴ و چرخه حیات Entity

چرخه حیات Entity

1		
2		
3	1. Transient (گذرا)	
4	↓ persist()	
5	2. Persistent (مدیریت شده)	
6	↓ detach() / close() / clear()	
7	3. Detached (جدا شده)	
8	↓ merge()	
9	2. Persistent (دوباره)	
10	↓ remove()	
11	4. Removed (حذف شده)	
12		
13		

1 java

```

2 //۱. Transient - شیء جدید - Hibernate نمی‌شناسه
3 User user = new User("Ali", "ali@test.com");
4
5 //۲. Persistent - Hibernate می‌کنه مدیریتش
6 session.persist(user); // هست Persistence Context حالا توی
7
8 // Dirty Checking - تغییر خودکار
9 user.setName("Ali Reza"); // می‌زنه SQL خودکار commit نیست! موقع update() نیازی به
10
11 //۳. Detached - خارج شد Session از
12 session.close(); // دیگه مدیریت نمیشه user حالا
13
14 //۴. Removed
15 session.remove(user);

```

! سوال مصاحبه‌ای ۲: Persistence Context چیه؟

جواب: Persistence Context به حافظه موقت (First-Level Cache) هست که Hibernate توش Entityهای مدیریت‌شده رو نگه می‌داره. مزایاش:

1. Dirty Checking: تغییرات Entityها رو خودکار تشخیص میده.

2. First-Level Cache: اگه به Entity رو دوبار بخونی، فقط به SQL می‌زنه.

3. Write-Behind: SQLها رو تا زمان commit عقب می‌ندازه (Batching).

```
1 java
```

```
2 User user1 = session.get(User.class, 1L); // SQL میزنه
3 User user2 = session.get(User.class, 1L); // SQL نمیزنه! از Cache میخونه
4 System.out.println(user1 == user2); // true
```

۵-۲-۱۴ Inheritance (وراثت) توی Hibernate

Hibernate چهار استراتژی برای Map کردن وراثت داره:

۱. SINGLE_TABLE (پیش فرض)

همه کلاس ها توی یه جدول ذخیره میشن. یه ستون DTYPE نوع رو مشخص می کنه.

```
1 java
```

```
2 @Entity
3 @Inheritance(strategy = InheritanceType.SINGLE_TABLE)
4 @DiscriminatorColumn(name = "type", discriminatorType = DiscriminatorType.STRING)
5 public abstract class Payment {
6     @Id @GeneratedValue
7     private Long id;
8     private double amount;
9 }
10
11 @Entity
12 @DiscriminatorValue("CARD")
13 public class CardPayment extends Payment {
14     private String cardNumber;
15 }
16
17 @Entity
18 @DiscriminatorValue("CASH")
19 public class CashPayment extends Payment {
20     private String currency;
21 }
```

مزایا: سریع‌ترین (بدون JOIN)، ساده.

معایب: ستون‌های NULL زیاد، نمی‌تونی NOT NULL بذاری.

۲. JOINED

هر کلاس جدول خودش رو داره. با JOIN به هم وصل میشن.

```
1 java
2 @Entity
3 @Inheritance(strategy = InheritanceType.JOINED)
4 public abstract class Payment {
5     @Id @GeneratedValue
6     private Long id;
7     private double amount;
8 }
9
10 @Entity
11 public class CardPayment extends Payment {
12     private String cardNumber;
13 }
```

مزایا: نرمال‌سازی، NOT NULL راحت.

معایب: JOIN زیاد، کندتر.

۳. TABLE_PER_CLASS

هر کلاس جدول کامل خودش رو داره (حتی فیلدهای پدر).

```
1 java
2 @Entity
3 @Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
4 public abstract class Payment {
5     @Id @GeneratedValue
6     private Long id;
7     private double amount;
8 }
```

1-Java Interview Question

```
9
10 @Entity
11 public class CardPayment extends Payment {
12     private String cardNumber;
13     // id و amount هم توی این جدول تکرار میشن
14 }
```

مزایا: بدون JOIN برای هر کلاس.

معایب: UNION برای Query های پدر، تکرار ستون.

۴. MappedSuperclass

کلاس پدر جدول نداره، فقط فیلدها رو به ارث میده.

```
1 java
2 @MappedSuperclass
3 public abstract class BaseEntity {
4     @Id @GeneratedValue
5     private Long id;
6
7     @Column(name = "created_at")
8     private LocalDateTime createdAt;
9 }
10
11 @Entity
12 public class User extends BaseEntity {
13     private String name;
14 }
```

! سوال مصاحبه‌ای ۳: کدوم استراتژی وراثت بهتره؟

جواب:

استراتژی	سرعت	نرمال سازی	کاربرد
SINGLE_TABLE	سریع ترین ⚡	✗	وراثت کم، فیلدهای مشترک زیاد
JOINED	کند	✓	وراثت پیچیده، داده‌های زیاد
TABLE_PER_CLASS	متوسط	✗	هر کلاس مستقل
MappedSuperclass	-	-	فقط اشتراک فیلد (بدون Polymorphism)

توصیه: اگر Query پدر زیاد داری و تعداد کلاس‌ها کمه، SINGLE_TABLE. اگر نرمال سازی مهمه، JOINED.

۲.۶-۱۴ Association (انجمن) توی Hibernate

انواع رابطه

رابطه	Annotation	مثال
One-to-One	OneToOne@	User ↔ Profile
One-to-Many	OneToMany@	User → Orders
Many-to-One	ManyToOne@	Orders → User
Many-to-Many	ManyToMany@	Student ↔ Course

One-to-One .1

```

1 java
2 @Entity
3 public class User {
4     @Id @GeneratedValue
5     private Long id;
6     private String name;
7
8     @OneToOne(cascade = CascadeType.ALL)
9     @JoinColumn(name = "profile_id", referencedColumnName = "id")
10    private Profile profile;
11 }
12
13 @Entity
14 public class Profile {
15     @Id @GeneratedValue
16     private Long id;
17     private String bio;
18
19     @OneToOne(mappedBy = "profile")
20     private User user;
21 }

```

One-to-Many / Many-to-One .2

```

1 java
2 @Entity
3 public class User {
4     @Id @GeneratedValue
5     private Long id;
6     private String name;
7
8     @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
9     private List<Order> orders = new ArrayList<>();
10
11    // متد كمكى

```

1-Java Interview Question

```
12 public void addOrder(Order order) {
13     orders.add(order);
14     order.setUser(this);
15 }
16 }
17
18 @Entity
19 public class Order {
20     @Id @GeneratedValue
21     private Long id;
22     private String product;
23
24     @ManyToOne(fetch = FetchType.LAZY)
25     @JoinColumn(name = "user_id")
26     private User user;
27 }
```

Many-to-Many .۳

```
1 java

2 @Entity
3 public class Student {
4     @Id @GeneratedValue
5     private Long id;
6     private String name;
7
8     @ManyToMany(cascade = CascadeType.ALL)
9     @JoinTable(
10         name = "student_course",
11         joinColumns = @JoinColumn(name = "student_id"),
12         inverseJoinColumns = @JoinColumn(name = "course_id")
13     )
14     private List<Course> courses = new ArrayList<>();
15 }
16
17 @Entity
18 public class Course {
19     @Id @GeneratedValue
```

1-Java Interview Question

```

20 private Long id;
21 private String title;
22
23 @ManyToMany(mappedBy = "courses")
24 private List<Student> students = new ArrayList<>();
25 }

```

! سوال مصاحبه‌ای ۴: mappedBy چیه و چرا لازمه؟

جواب: mappedBy به Hibernate می‌گه که این سمت رابطه، مالک نیست و از سمت دیگه پیروی می‌کنه. بدون mappedBy، Hibernate دو جدول واسط می‌سازه و دوبار SQL می‌زنه!

```

1 java
2 // غلط - دو جدول واسط ساخته میشه ❌
3 @OneToMany
4 private List<Order> orders;
5
6 // درست - یه جدول واسط، مالکیت با ✓
7 @OneToMany(mappedBy = "user")
8 private List<Order> orders;

```

قانون: سمت @ManyToOne همیشه مالکه. سمت @OneToMany باید mappedBy داشته باشه.

۲.۷-۱۴ FetchType (نوع واکنشی)

! سوال مصاحبه‌ای ۵: فرق EAGER و LAZY چیه؟

جواب:

ویژگی	LAZY	EAGER
زمان واکنشی	موقع دسترسی	همون لحظه لود Entity
کارایی	✓ بهتر	❌ کندتر
خطر	LazyInitializationException	N+1 Problem
پیش فرض	OneToMany, @ManyToMany@	ManyToOne, @OneToOne@

1-Java Interview Question

1 java

```
2 // LAZY - میزنه SQL، صدا بزنی getOrders() فقط وقتی -  
3 @OneToMany(fetch = FetchType.LAZY)  
4 private List<Order> orders;  
5  
6 // EAGER - ها هم لود میشن User, Order همون لحظه لود -  
7 @ManyToOne(fetch = FetchType.EAGER)  
8 private User user;
```

مشکل LazyInitializationException

1 java

```
2 // ❌ خطا  
3 User user = session.get(User.class, 1L);  
4 session.close(); // بسته شد Session  
5 user.getOrders().size(); // LazyInitializationException!  
6  
7 // ✅ دسترسی بزنی Session راه حل ۱: توی  
8 User user = session.get(User.class, 1L);  
9 user.getOrders().size(); // هنوز Session  
10 session.close();  
11  
12 // ✅ ۲: JOIN FETCH: راه حل  
13 @Query("SELECT u FROM User u JOIN FETCH u.orders WHERE u.id = :id")  
14 User findWithOrders(@Param("id") Long id);
```

مشکل N+1

1 java

```
2 // ❌ N+1 Problem  
3 List<User> users = userRepository.findAll(); // ۱ Query  
4 for (User u : users) {  
5     u.getOrders().size(); // N Query اضافه!  
6 }  
7 // مجموع: N+1 Query
```

1-Java Interview Question

8

9 //  راه حل: *JOIN FETCH*

10 `@Query("SELECT DISTINCT u FROM User u JOIN FETCH u.orders")`

11 `List<User> findAllWithOrders();` // *Query* فقط ۱

! سوال مصاحبه ای ۶: N+1 Problem چیه و چطور حلش می کنی؟

جواب: N+1 یعنی برای لود کردن N تا Entity، به جای ۱ N+1، Query تا Query زده میشه.

راه حل ها:

1. JPQL توی JOIN FETCH

2. Spring Data JPA توی EntityGraph

3. BatchSize برای لود دسته ای

4. Fetch(FetchMode.SUBSELECT)

5. تنظیم hibernate.default_batch_fetch_size

1 java

2 // *JOIN FETCH* راه حل ۱

3 `@Query("SELECT u FROM User u JOIN FETCH u.orders")`

4 `List<User> findAllWithOrders();`

5

6 // *EntityGraph* راه حل ۲

7 `@EntityGraph(attributePaths = {"orders"})`

8 `List<User> findAll();`

9

10 // *BatchSize* راه حل ۳

11 `@OneToMany`

12 `@BatchSize(size = 10)`

13 `private List<Order> orders;`

۲.۸-۱۴ Cascade (آبشاری)

! سوال مصاحبه‌ای ۷: Cascade چیه و انواعش؟

جواب: Cascade یعنی عملیات روی Entity پدر، به فرزندان هم اعمال بشه.

نوع Cascade	توضیح
PERSIST	ذخیره پدر → فرزندان هم ذخیره
MERGE	آپدیت پدر → فرزندان هم آپدیت
REMOVE	حذف پدر → فرزندان هم حذف
REFRESH	رفرش پدر → فرزندان هم رفرش
DETACH	جدا کردن پدر → فرزندان هم جدا
ALL	همه موارد بالا

1 java

2 //  با Cascade

3 @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)

4 private List<Order> orders = new ArrayList<>();

5

6 // استفاده

7 User user = new User("Ali");

8 Order order1 = new Order("Book");

9 Order order2 = new Order("Pen");

10 user.addOrder(order1);

11 user.addOrder(order2);

12

13 session.persist(user); // Order ها هم خودکار ذخیره میشن

orphanRemoval

1 java

2 @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)

3 private List<Order> orders = new ArrayList<>();

4

5 // حذف از لیست = حذف از دیتابیس

6 user.getOrders().remove(0); // Order همیشه حذف خودکار

! سوال مصاحبه‌ای ۸: فرق Cascade REMOVE و orphanRemoval چیه؟

جواب:

ویژگی	Cascade REMOVE	orphanRemoval
حذف پدر	فرزندان حذف میشن	فرزندان حذف میشن
حذف از لیست	✗ حذف نمیشن	✓ حذف میشن
جدا کردن رابطه	✗	✓ (اگه setUser(null) کنی)

1 java

2 // Cascade.REMOVE

3 user.getOrders().remove(order); // Order می‌مونه DB توی

4 session.remove(user); // Order میشن حالا

5

6 // orphanRemoval

7 user.getOrders().remove(order); // Order حذف میشه DB خودکار از

۲.۹-۱۴ Hibernate توی Caching

First-Level Cache (الزامی)

همون Persistence Context توی Session نگه داشته میشه.

```
1 java
2 User u1 = session.get(User.class, 1L); // SQL
3 User u2 = session.get(User.class, 1L); // Cache
```

Second-Level Cache (اختیاری)

توی SessionFactory نگه داشته میشه. بین Session ها مشترکه.

```
1 xml
2 <!-- hibernate.cfg.xml -->
3 <property name="hibernate.cache.use_second_level_cache">true</property>
4 <property name="hibernate.cache.region.factory_class">
5     org.hibernate.cache.ehcache.EhCacheRegionFactory
6 </property>
```

```
1 java
2 @Entity
3 @Cacheable
4 @org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
5 public class User { ... }
```

Query Cache

```
1 java
2 Query query = session.createQuery("FROM User WHERE name = :name");
3 query.setParameter("name", "Ali");
4 query.setCacheable(true); // فعال کردن Query Cache
```


۱۰-۲-۱۴ Locking (قفل گذاری)

Optimistic Locking

با به ستون version کار می‌کنه. تداخل رو موقع Commit تشخیص میده.

```
1 java
```

```
2 @Entity
3 public class Product {
4     @Id @GeneratedValue
5     private Long id;
6
7     @Version
8     private int version;
9
10    private int quantity;
11 }
```

```
1 java
```

```
2 // Thread 1
3 Product p1 = session.get(Product.class, 1L);
4 p1.setQuantity(5);
5 // Thread 2 همزمان
6 Product p2 = session.get(Product.class, 1L);
7 p2.setQuantity(10);
8
9 // Thread 1 commit → version = 2
10 // Thread 2 commit → OptimisticLockException!
```

Pessimistic Locking

دیتابیس رو قفل می‌کنه.

```
1 java
2 // Lock on Read
3 Product p = session.get(Product.class, 1L, LockMode.PESSIMISTIC_WRITE);
4
5 // Lock on Query
6 Query query = session.createQuery("FROM Product WHERE id = :id");
7 query.setLockMode("p", LockMode.PESSIMISTIC_WRITE);
```

! سوال مصاحبه‌ای ۹: فرق Pessimistic Locking و Optimistic چیه؟

جواب:

ویژگی	Optimistic	Pessimistic
روش	version column	DB Lock
کارایی	بالا ✓	پایین ✗
تداخل	موقع Commit	موقع Read
کاربرد	Read زیاد، Write کم	Write زیاد، تداخل زیاد
خطا	OptimisticLockException	زمان انتظار

Flush و Transaction ۱۴-۲.۱۱

Flush چیه؟

Flush یعنی SQL Hibernate های معلق رو به دیتابیس بفرسته (ولی Commit نمیشه).

```
1 java
2 session.persist(user); // زده SQL هنوز
3 session.flush(); // (نه Commit ولی) زده شد INSERT حالا
4 tx.commit(); // Commit
```

FlushMode

توضیح	حالت
قبل از Query و Commit	AUTO (پیش فرض)
فقط موقع Commit	COMMIT
دستی با flush()	MANUAL
قبل از هر Query	ALWAYS

! سوال مصاحبه‌ای ۱۰: فرق persist, save, saveOrUpdate, merge چیه؟

جواب:

توضیح	Detached	Transient	برگردوند	متد
JPA استاندارد	خطا ✗	✓ ذخیره	void	persist()
Hibernate	✗ INSERT جدید	✓ ذخیره	ID	save()
Hibernate	✓ آپدیت	✓ ذخیره	void	saveOrUpdate()
JPA استاندارد	✓ آپدیت	✓ ذخیره	Entity	merge()
Hibernate	✓ آپدیت	✗ خطا	void	update()

1 java

2 // *persist* فقط - *Transient*

3 User user = new User("Ali");

4 session.persist(user); // *INSERT*

5

6 // *merge* - هر دو حالت

7 User detached = ...; // قبلی *Session* از

8 User managed = session.merge(detached); // *SELECT + UPDATE*

JPQL vs HQL vs Native SQL ۱۴-۲.۱۲

1 java

2 // JPQL (استاندارد)

3 @Query("SELECT u FROM User u WHERE u.age > :age")

4 List<User> findOlderThan(@Param("age") int age);

5

6 // HQL (Hibernate-specific)

7 Query query = session.createQuery("FROM User u WHERE u.name LIKE :name");

8 query.setParameter("name", "%Ali%");

9

10 // Native SQL

11 @Query(value = "SELECT * FROM users WHERE age > ?1", nativeQuery = true)

12 List<User> findOlderThanNative(int age);

! سوال مصاحبه‌ای ۱۱: JPQL و SQL چه فرقی دارند؟

جواب:

ویژگی	JPQL	SQL
موجودیت‌ها	Field و Entity	Column و Table
حساسیت	به Case حساس	معمولاً نه
Polymorphism	✓	✗
قابل حمل	✓ بین DBها	✗

1 java

2 // JPQL

3 "SELECT u FROM User u WHERE u.name = 'Ali'"

4

5 // SQL معادل

6 "SELECT * FROM users WHERE name = 'Ali'"

Order و User : ۱۴-۲.۱۳ نمونه کامل:

```

1 java
2 @Entity
3 @Table(name = "users")
4 public class User {
5
6     @Id
7     @GeneratedValue(strategy = GenerationType.IDENTITY)
8     private Long id;
9
10    @Column(nullable = false, length = 100)
11    private String name;
12
13    @Column(unique = true)
14    private String email;
15
16    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL,
17        orphanRemoval = true, fetch = FetchType.LAZY)
18    private List<Order> orders = new ArrayList<>();
19
20    @OneToOne(cascade = CascadeType.ALL)
21    @JoinColumn(name = "profile_id")
22    private Profile profile;
23
24    @ManyToMany(cascade = CascadeType.PERSIST)
25    @JoinTable(name = "user_role",
26        joinColumns = @JoinColumn(name = "user_id"),
27        inverseJoinColumns = @JoinColumn(name = "role_id"))
28    private Set<Role> roles = new HashSet<>();
29
30    @Version
31    private int version;
32
33    @CreationTimestamp
34    private LocalDateTime createdAt;
35
36    @UpdateTimestamp

```

1-Java Interview Question

```
37 private LocalDateTime updatedAt;
38
39 // متدهای کمکی
40 public void addOrder(Order order) {
41     orders.add(order);
42     order.setUser(this);
43 }
44
45 public void removeOrder(Order order) {
46     orders.remove(order);
47     order.setUser(null);
48 }
49
50 // Getters/Setters
51 }
52
53 @Entity
54 @Table(name = "orders")
55 public class Order {
56
57     @Id
58     @GeneratedValue(strategy = GenerationType.IDENTITY)
59     private Long id;
60
61     private String product;
62     private double amount;
63
64     @ManyToOne(fetch = FetchType.LAZY)
65     @JoinColumn(name = "user_id", nullable = false)
66     private User user;
67
68     // Getters/Setters
69 }
```

۱۴-۲-۱۴ جمع‌بندی - نکات طلایی مصاحبه

1. JPA استاندارد، Hibernate پیاده‌سازی.

2. Session = Persistence Context - توی زندگی می‌کنه.

3. mappedBy سمت مالکیت رو مشخص می‌کنه - بدونش دو جدول واسط ساخته میشه.

4. LAZY پیش‌فرض، EAGER@، OneToMany، پیش‌فرض @ManyToOne.

5. N+1 Problem با JOIN FETCH یا EntityGraph حل میشه.

6. Cascade عملیات پدر → فرزندان، orphanRemoval حذف از لیست → حذف از DB.

7. SINGLE_TABLE سریع‌ترین استراتژی وراثت.

8. Optimistic Locking با @Pessimistic، Version، LockMode.

9. persist برای merge، Transient، Detached برای.

10. Flush SQL - Commit ≠ Flush می‌زنه، Commit تأیید می‌کنه.

نکته طلایی: توی مصاحبه اگه پرسیدن "چرا Hibernate استفاده می‌کنی؟" بگو: "چون ORM باعث میشه کد دیتابیس ننویسم، Dirty Checking خودکار دارم، و با Dialect می‌تونم دیتابیس رو عوض کنم بدون تغییر کد. مهم‌تر از همه، Persistence Context و Caching کارایی رو بالا می‌بره."

فاز ۱۴-۳: Servlet و مدیریت Session – کامل و عمیق



این بخش برای درک مفاهیم پایه‌ای وب در جاوا ضروریه. مفاهیمی مثل Stateless، Filter، Token و ServletContext، Session از سوالات پرتکرار مصاحبه‌های بک‌اند هستند.

۱۴-۳.۱ تفاوت HTTP و FTP (Stateless vs Stateful)

! سوال مصاحبه‌ای ۱: HTTP Stateless هست یا Stateful؟ FTP چطور؟

جواب:

ویژگی	HTTP	FTP
وضعیت (State)	Stateless (بدون حالت)	Stateful (با حالت)
معنی	هر درخواست مستقل از درخواست قبلیه. سرور هیچ چیزی از درخواست قبلی به خاطر نمی‌سپاره.	سرور وضعیت اتصال رو نگه می‌داره. مثلاً می‌دونه الان توی چه پوشه‌ای هستی.
اتصال	معمولاً Connectionless (هر درخواست یه اتصال جدید)	Connection-Oriented (یه اتصال باز می‌مونه)
پورت پیش فرض	80 (HTTP)، 443 (HTTPS)	20 و 21
کاربرد	وب، REST API	انتقال فایل

چرا HTTP Stateless طراحی شده؟

• سادگی و مقیاس‌پذیری (Scalability)

• سرور مجبور نیست برای هر کاربر حافظه اختصاص بده

• هر درخواست می‌تونه به هر سروری توی Cluster بره

مشکل Stateless بودن HTTP چیه؟

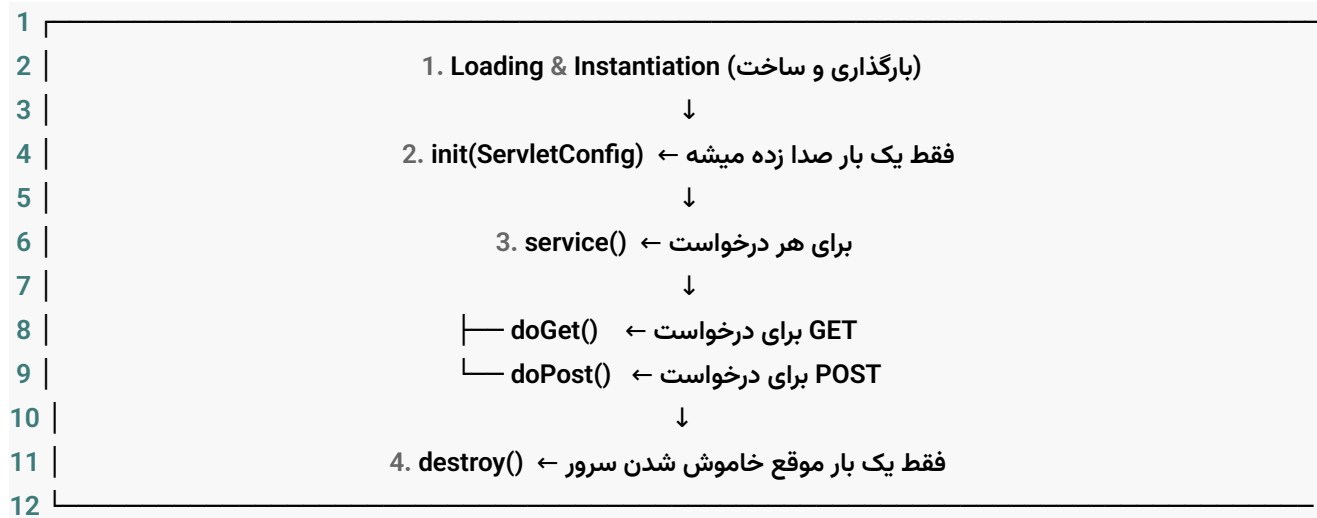
• سرور نمی‌دونه کاربر قبلاً Login کرده یا نه!

• راه‌حل: Session یا Token (JWT) برای نگه‌داشتن وضعیت.

۳.۲-۱۴ Servlet چیست؟

Servlet یه کلاس جاواییه که روی سرور اجرا میشه و درخواست‌های HTTP رو پردازش می‌کنه. Servlet Container (مثل Tomcat) مسئول مدیریت چرخه حیات Servlet‌هاست.

چرخه حیات Servlet



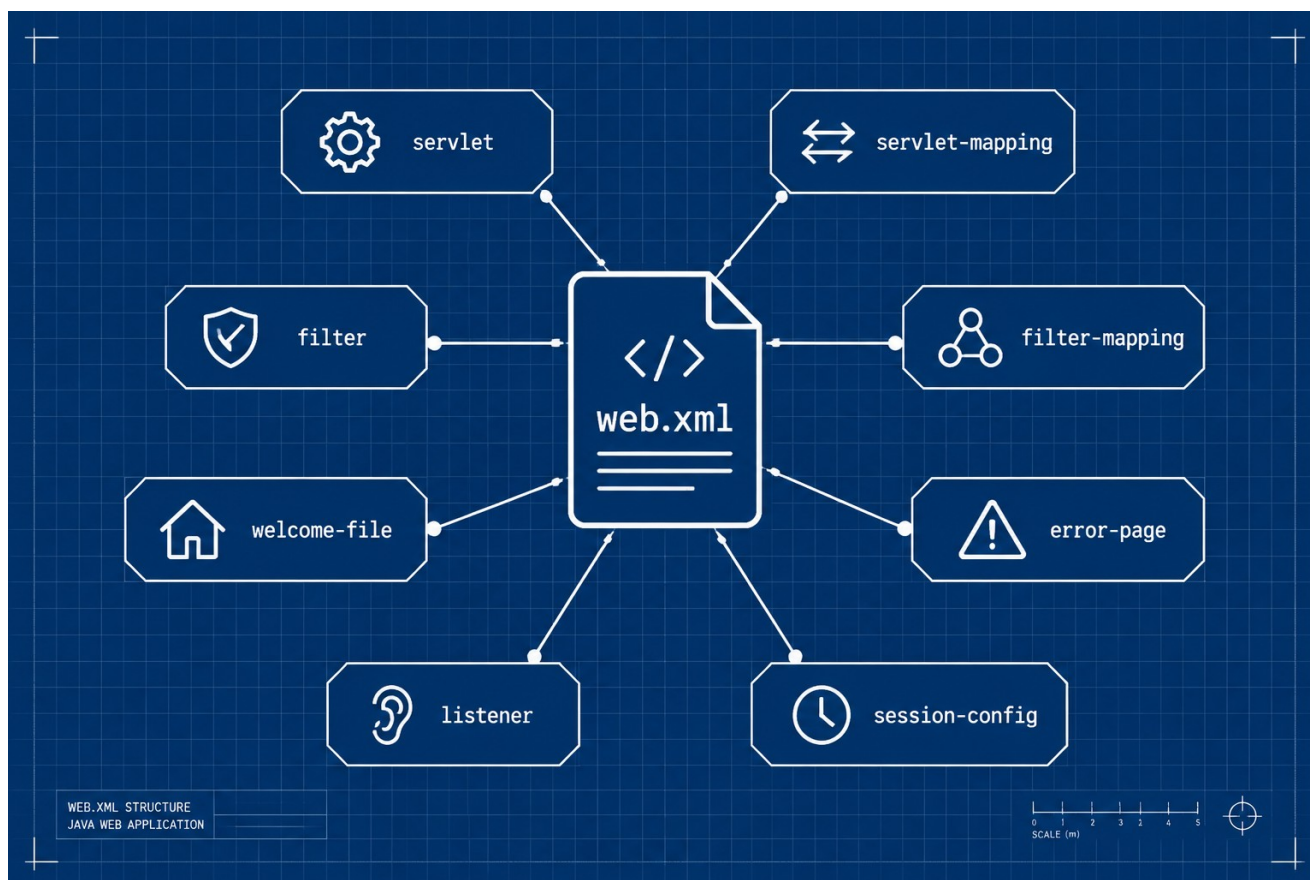
1-Java Interview Question

```
1 public class LoginServlet extends HttpServlet {
2
3     @Override
4     public void init() throws ServletException {
5         System.out.println("1. Servlet داره initialize همیشه");
6     }
7
8     @Override
9     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
10         throws ServletException, IOException {
11         resp.getWriter().println("دریافت شد GET درخواست");
12     }
13
14     @Override
15     protected void doPost(HttpServletRequest req, HttpServletResponse resp)
16         throws ServletException, IOException {
17         String username = req.getParameter("username");
18         String password = req.getParameter("password");
19         // پردازش لاگین
20     }
21
22     @Override
23     public void destroy() {
24         System.out.println("4. Servlet داره destroy همیشه");
25     }
26 }
```

Servlet ۱۴-۳.۳ – قلب تنظیمات web.xml

فایل تنظیمات Deployment Descriptor هست. توی پروژه‌های قدیمی و مصاحبه‌ها خیلی مهمه.

ساختار کامل web.xml



```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
5     http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
6     version="4.0">
7

```

1-Java Interview Question

```
8  <!-- ۱. تعریف Servlet -->
9  <servlet>
10     <servlet-name>login</servlet-name>
11     <servlet-class>ir.nas.LoginServlet</servlet-class>
12
13     <!-- init-param: پارامترهای اولیه برای Servlet -->
14     <init-param>
15         <param-name>dbUrl</param-name>
16         <param-value>jdbc:mysql://localhost:3306/mydb</param-value>
17     </init-param>
18     <init-param>
19         <param-name>maxRetries</param-name>
20         <param-value>3</param-value>
21     </init-param>
22
23     <!-- load-on-startup: موقع استارت سرور ساخته میشه -->
24     <load-on-startup>1</load-on-startup>
25 </servlet>
26
27 <!-- ۲. Mapping Servlet به URL -->
28 <servlet-mapping>
29     <servlet-name>login</servlet-name>
30     <url-pattern>/login</url-pattern>
31 </servlet-mapping>
32
33 <!-- ۳. تعریف Filter -->
34 <filter>
35     <filter-name>filter-login</filter-name>
36     <filter-class>ir.nas.LoginFilter</filter-class>
37
38     <!-- init-param برای Filter -->
39     <init-param>
40         <param-name>excludedPaths</param-name>
41         <param-value>/public,/css,/js</param-value>
42     </init-param>
43 </filter>
44
```

1-Java Interview Question

```
45 <!-- ۴. Mapping Filter به URL -->
46 <filter-mapping>
47     <filter-name>filter-login</filter-name>
48     <url-pattern>/login</url-pattern>
49     <!-- کنى map خاص هم Servlet می تونی به -->
50     <!-- <servlet-name>login</servlet-name> -->
51 </filter-mapping>
52
53 <!-- ۵. Listener -->
54 <listener>
55     <listener-class>ir.nas.MyAppContextListener</listener-class>
56 </listener>
57
58 <!-- ۶. تنظیمات Session -->
59 <session-config>
60     <session-timeout>30</session-timeout> <!-- دقیقه -->
61     <cookie-config>
62         <http-only>true</http-only>
63         <secure>false</secure>
64     </cookie-config>
65 </session-config>
66
67 <!-- ۷. صفحه خوش آمد -->
68 <welcome-file-list>
69     <welcome-file>index.html</welcome-file>
70     <welcome-file>index.jsp</welcome-file>
71 </welcome-file-list>
72
73 <!-- ۸. صفحه خطا -->
74 <error-page>
75     <error-code>404</error-code>
76     <location>/404.html</location>
77 </error-page>
78 <error-page>
79     <error-code>500</error-code>
80     <location>/500.html</location>
81 </error-page>
83 </web-app>
```

دسترسی به init-param توی Servlet

```

1 public class LoginServlet extends HttpServlet {
2     private String dbUrl;
3     private int maxRetries;
4
5     @Override
6     public void init(ServletConfig config) throws ServletException {
7         // روش ۱: از طریق ServletConfig
8         this.dbUrl = config.getInitParameter("dbUrl");
9         this.maxRetries = Integer.parseInt(config.getInitParameter("maxRetries"));
10
11         // روش ۲: از طریق getServletConfig()
12         String name = getServletConfig().getServletName();
13     }
14 }

```

۳.۴-۱۴ Filter – نگهبان درخواست‌ها

Filter مثل یه دروازه‌بان عمل می‌کنه: قبل از رسیدن درخواست به Servlet اجرا میشه و می‌تونه درخواست رو رد کنه، تغییر بده یا لاگ بگیره.

چرخه حیات Filter

```

1 Request → Filter1 → Filter2 → Servlet → Filter2 → Filter1 → Response

```

نمونه LoginFilter

```

1 public class LoginFilter implements Filter {
2
3     private List<String> excludedPaths;
4
5     @Override
6     public void init(FilterConfig config) throws ServletException {
7         // خوندن init-param
8         String paths = config.getInitParameter("excludedPaths");
9         this.excludedPaths = Arrays.asList(paths.split(","));
10        System.out.println("Filter initialized");
11    }
12
13    @Override
14    public void doFilter(ServletRequest request, ServletResponse response,
15                        FilterChain chain) throws IOException, ServletException {
16
17        HttpServletRequest req = (HttpServletRequest) request;
18        HttpServletResponse resp = (HttpServletResponse) response;
19
20        String path = req.getRequestURI();
21
22        // بود، رد کن excludedPaths اگه مسیر توی
23        if (excludedPaths.stream().anyMatch(path::startsWith)) {
24            chain.doFilter(request, response);
25            return;
26        }
27
28        // چک کردن Session
29        HttpSession session = req.getSession(false);
30        boolean isLoggedIn = (session != null && session.getAttribute("user") != null);
31
32        if (isLoggedIn) {
33            // کاربر لاگین کرده، برو به
34            chain.doFilter(request, response);
35        } else {

```

1-Java Interview Question

```
36      کاربر لاگین نکرده، بفرست به صفحه لاگین //
37      resp.sendRedirect(req.getContextPath() + "/login.jsp");
38  }
39  }
40
41  @Override
42  public void destroy() {
43      System.out.println("Filter destroyed");
44  }
45 }
```

ترتیب اجرای Filterها

ترتیب توی web.xml مهمه! Filter ها به ترتیبی که تعریف میشن اجرا میشن.

۱۴-۳.۵ Listener – شنونده رویدادها

Listener کلاسیه که به رویدادهای چرخه حیات وب اپ گوش میده.

انواع Listener

Listener	رویداد
ServletContextListener	استارت/خاموشی وب اپ
HttpSessionListener	ساخت/نابودی Session
ServletRequestListener	شروع/پایان درخواست
ServletContextAttributeListener	تغییر attribute در Context
HttpSessionAttributeListener	تغییر attribute در Session

نمونه ServletContextListener

```

1 java
2 public class MyAppContextListener implements ServletContextListener {
3
4     @Override
5     public void contextInitialized(ServletContextEvent sce) {
6         System.out.println("وب اپ داره استارت میشه");
7
8         // مثلاً راه اندازی Connection Pool
9         ServletContext ctx = sce.getServletContext();
10        ctx.setAttribute("appName", "MyApp");
11    }
12
13    @Override
14    public void contextDestroyed(ServletContextEvent sce) {
15        System.out.println("وب اپ داره خاموش میشه");
16        // بستن منابع
17    }
18 }

```

نمونه HttpSessionListener

```

1 java
2 public class MySessionListener implements HttpSessionListener {
3
4     private static int activeSessions = 0;
5
6     @Override
7     public void sessionCreated(HttpSessionEvent se) {
8         activeSessions++;
9         System.out.println("جدید ساخته شد. تعداد Session: " + activeSessions);
10    }
11
12    @Override
13    public void sessionDestroyed(HttpSessionEvent se) {
14        activeSessions--;
15        System.out.println("نابود شد. تعداد Session: " + activeSessions);
16    }
17 }

```

ServletConfig و ServletContext ۱۴-۳.۶

! سوال مصاحبه‌ای ۲: فرق ServletConfig و ServletContext چیه؟

جواب:

1-Java Interview Question

ویژگی	ServletConfig	ServletContext
دامنه	مخصوص به Servlet خاص	کل وب اپ
تعداد	یکی برای هر Servlet	یکی برای کل اپ
دسترسی	getServletConfig()	getServletContext()
init-param	مخصوص همون Servlet	برای کل اپ
کاربرد	تنظیمات خاص به Servlet	اشتراک داده بین همه Servletها

1 java

```

2 // ServletConfig - مخصوص به Servlet
3 public class MyServlet extends HttpServlet {
4     @Override
5     public void init(ServletConfig config) {
6         String value = config.getInitParameter("myParam"); // فقط برای این Servlet
7     }
8 }
9
10 // ServletContext - برای کل اپ
11 public class AnotherServlet extends HttpServlet {
12     @Override
13     protected void doGet(HttpServletRequest req, HttpServletResponse resp) {
14         ServletContext ctx = getServletContext();
15         ctx.setAttribute("sharedData", "value"); // می بینن همه Servletها
16
17         String appName = ctx.getInitParameter("appName"); // پارامتر کل اپ
18     }
19 }
```

تنظیم context-param توی web.xml

```

1 xml
2 <context-param>
3   <param-name>appName</param-name>
4   <param-value>MyApplication</param-value>
5 </context-param>

```

web.xml ۱۴-۳.۷ Annotation ها – جایگزین مدرن

از Servlet 3.0 به بعد، می‌تونن به جای web.xml از Annotation استفاده کنن.

WebServlet@

```

1 java
2 @WebServlet(
3   name = "login",
4   urlPatterns = {"/login", "/signin"},
5   initParams = {
6     @WebInitParam(name = "dbUrl", value = "jdbc:mysql://localhost:3306/mydb"),
7     @WebInitParam(name = "maxRetries", value = "3")
8   },
9   loadOnStartup = 1
10 )
11 public class LoginServlet extends HttpServlet {
12   // ...
13 }

```

WebFilter@

```

1 java
2 @WebFilter(
3     filterName = "filter-login",
4     urlPatterns = {"/login", "/admin/*"},
5     initParams = {
6         @WebInitParam(name = "excludedPaths", value = "/public,/css,/js")
7     }
8 )
9 public class LoginFilter implements Filter {
10     // ...
11 }

```

WebListener@

```

1 java
2 @WebListener
3 public class MySessionListener implements HttpSessionListener { // ... }

```

! سوال مصاحبه‌ای ۳: web.xml یا Annotation – کدام بهتره؟

جواب:

ویژگی	web.xml	Annotation
خوانایی	همه تنظیمات یه جا	پراکنده توی کد
تغییر بدون Recompile	✓ همیشه	✗ نمیشه
محیط‌های مختلف	راحت (چند web.xml)	سخت
مدرن بودن	قدیمی	✓ مدرن
Spring Boot	استفاده نمیشه	✓ استفاده میشه

توصیه: برای پروژه‌های جدید Annotation، برای تنظیمات محیطی web.xml.

۳.۸-۱۴ درخواست GET و POST (با curl)

تفاوت POST و GET

ویژگی	GET	POST
داده	توی URL (Query String)	توی Body
امنیت	❌ داده توی URL دیده میشه	✅ نسبی بهتر
طول	محدود (حدود 2000 کاراکتر)	نامحدود
Cache	✅ Cache میشه	❌ Cache نمیشه
Idempotent	✅ بله	❌ خیر
کاربرد	دریافت داده	ارسال داده، تغییر وضعیت

نمونه درخواست با curl

1 bash

2 # GET ساده

3 curl http://localhost:8080/api/users

4

5 # GET با Query Parameter

6 curl "http://localhost:8080/api/users?id=5&name=ali"

1-Java Interview Question

```
7
8 # GET با Header
9 curl -H "Authorization: Bearer token123" http://localhost:8080/api/users
10
11 # POST با فرم
12 curl -X POST http://localhost:8080/login \
13     -d "username=ali&password=1234"
14
15 # POST با JSON
16 curl -X POST http://localhost:8080/api/users \
17     -H "Content-Type: application/json" \
18     -d '{"name":"Ali","age":30}'
19
20 # POST با Basic Auth
21 curl -X POST http://localhost:8080/login \
22     -u ali:1234
23
24 # ذخیره کوکی
25 curl -c cookies.txt http://localhost:8080/login
26
27 # ارسال کوکی
28 curl -b cookies.txt http://localhost:8080/dashboard
29
30 # های پاسخ Header نمایش
31 curl -i http://localhost:8080/api/users
```

خوندن داده توی Servlet

```
1 java
2 public class LoginServlet extends HttpServlet {
```

1-Java Interview Question

```
3
4  @Override
5  protected void doGet(HttpServletRequest req, HttpServletResponse resp)
6      throws IOException {
7      // خواندن Query Parameter
8      String id = req.getParameter("id");
9      String name = req.getParameter("name");
10
11     resp.setContentType("text/html");
12     resp.getWriter().println("ID: " + id + ", Name: " + name);
13 }
14
15 @Override
16 protected void doPost(HttpServletRequest req, HttpServletResponse resp)
17     throws IOException {
18     // خواندن فرم
19     String username = req.getParameter("username");
20     String password = req.getParameter("password");
21
22     // خواندن JSON از Body
23     BufferedReader reader = req.getReader();
24     StringBuilder json = new StringBuilder();
25     String line;
26     while ((line = reader.readLine()) != null) {
27         json.append(line);
28     }
29
30     // خواندن Header
31     String authHeader = req.getHeader("Authorization");
32
33     resp.getWriter().println("Login attempt: " + username);
34 }
35 }
```


۳.۹- ۱۴ Cookie و Token، Session

! سوال مصاحبه‌ای ۴: تفاوت Cookie، Session و Token چیه؟

جواب:

ویژگی	Cookie	Session	Token (JWT)
محل ذخیره	مرورگر کاربر	سرور	مرورگر (localStorage)
Stateless	-	Stateful ❌	Stateless ✅
امنیت	کم	متوسط	بالا (اگه درست پیاده بشه)
مقیاس‌پذیری	-	سخت (Sticky Session) ❌	راحت ✅
انقضا	قابل تنظیم	قابل تنظیم	داخل Token
کاربرد	ذخیره تنظیمات	لاگین سنتی	REST API مدرن

Cookie

کوکی یه فایل کوچیکه که سرور توی مرورگر کاربر ذخیره می‌کنه.

1 java

2 // ساخت Cookie

3 Cookie cookie = new Cookie("username", "ali");

4 cookie.setMaxAge(60 * 60 * 24); // روز ۱

5 cookie.setHttpOnly(true); // جلوگیری از دسترسی JS

6 cookie.setSecure(true); // فقط HTTPS

7 cookie.setPath("/");

8 resp.addCookie(cookie);

1-Java Interview Question

```
9
10 // خواندن Cookie
11 Cookie[] cookies = req.getCookies();
12 if (cookies != null) {
13     for (Cookie c : cookies) {
14         if (c.getName().equals("username")) {
15             String username = c.getValue();
16         }
17     }
18 }
19
20 // حذف Cookie
21 Cookie delete = new Cookie("username", "");
22 delete.setMaxAge(0);
23 resp.addCookie(delete);
```

Session

Session به شیء سمت سرور که اطلاعات کاربر رو نگه می‌داره. مرورگر فقط به شناسه (JSESSIONID) رو توی Cookie نگه می‌داره.

1 java

```
2 Session ساخت/گرفتن //
3 HttpSession session = req.getSession(true); // اگه نبود، بساز
4 session.setAttribute("user", userObject);
5 session.setMaxInactiveInterval(30 * 60); // دقیقه ۳۰
6
7 Session خواندن //
8 HttpSession session = req.getSession(false); // اگه نبود، null بده
9 if (session != null) {
10     User user = (User) session.getAttribute("user");
11 }
```

1-Java Interview Question

```
12
13 // حذف Session
14 session.invalidate();
15
16 // Timeout
17 session.setMaxInactiveInterval(60 * 30); // دقیقه ۳۰
```

Token (JWT)

JWT به رشته‌ست که سه بخش دارد:

1- Header: { "alg": "HS256" }

2- Payload: { "sub": "ali", "exp": ... }

3- Signature: a35e8d9c2b7e6...

```
1 java
2 // ساخت JWT (با کتابخانه jjwt)
3 String token = Jwts.builder()
4   .setSubject("ali")
5   .setIssuedAt(new Date())
6   .setExpiration(new Date(System.currentTimeMillis() + 3600000))
7   .signWith(SignatureAlgorithm.HS256, "secretKey")
8   .compact();
9
10 // خواندن JWT
11 String authHeader = req.getHeader("Authorization");
12 if (authHeader != null && authHeader.startsWith("Bearer ")) {
13   String token = authHeader.substring(7);
```

1-Java Interview Question

```
14 Claims claims = Jwts.parser()
15   .setSigningKey("secretKey")
16   .parseClaimsJws(token)
17   .getBody();
18 String username = claims.getSubject();
19 }
20
21 به کلاینت Token ارسال //
22 resp.setHeader("Authorization", "Bearer " + token);
```

! سوال مصاحبه‌ای ۵: Session-based یا Token-based – کدام بهتره؟

جواب:

:Session-based

- ساده‌تر ✓
- می‌تونی هر لحظه Logout کنی (invalidate) ✓
- Stateful – برای Cluster نیاز به Sticky Session یا Session Replication ✗
- برای Mobile App مناسب نیست ✗

:Token-based (JWT)

- Stateless – مقیاس‌پذیری عالی ✓
- برای REST API و Mobile عالی ✓
- Cross-domain راحت ✓
- نمی‌تونی قبل از Token، Expire، رو باطل کنی (مگر با Blacklist) ✗
- اگر Secret Leak بشه، همه چیز از دست میره ✗

توصیه: برای REST API مدرن، JWT. برای وب اپ سنتی، Session.

Servlet + Filter + Session با Login : نمونه کامل: ۱۴-۳.۱۰

LoginServlet.java

```

1 java
2 @WebServlet("/login")
3 public class LoginServlet extends HttpServlet {
4
5     @Override
6     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
7         throws ServletException, IOException {
8         // نمایش فرم لاگین
9         req.getRequestDispatcher("/login.jsp").forward(req, resp);
10    }
11
12    @Override
13    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
14        throws ServletException, IOException {
15
16        String username = req.getParameter("username");
17        String password = req.getParameter("password");
18
19        // اعتبارسنجی (ساده)
20        if ("ali".equals(username) && "1234".equals(password)) {
21            // لاگین موفق
22            HttpSession session = req.getSession(true);
23            session.setAttribute("user", username);
24            session.setMaxInactiveInterval(30 * 60);
25
26            resp.sendRedirect(req.getContextPath() + "/dashboard");
27        } else {
28            // لاگین ناموفق
29            req.setAttribute("error", "نام کاربری یا رمز عبور اشتباهه");
30            req.getRequestDispatcher("/login.jsp").forward(req, resp);
31        }

```

1-Java Interview Question

```
32 }  
33 }
```

LoginFilter.java

```
1 java  
2 @WebFilter(urlPatterns = {"/dashboard", "/admin/*"})  
3 public class LoginFilter implements Filter {  
4  
5     @Override  
6     public void doFilter(ServletRequest request, ServletResponse response,  
7         FilterChain chain) throws IOException, ServletException {  
8  
9         HttpServletRequest req = (HttpServletRequest) request;  
10        HttpServletResponse resp = (HttpServletResponse) response;  
11  
12        HttpSession session = req.getSession(false);  
13        boolean isLoggedIn = (session != null && session.getAttribute("user") != null);  
14  
15        if (isLoggedIn) {  
16            chain.doFilter(request, response);  
17        } else {  
18            resp.sendRedirect(req.getContextPath() + "/login");  
19        }  
20    }  
21 }
```

DashboardServlet.java

```
1 java  
2 @WebServlet("/dashboard")  
3 public class DashboardServlet extends HttpServlet {  
4  
5     @Override  
6     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
```

1-Java Interview Question

```
7     throws ServletException, IOException {
8
9     HttpSession session = req.getSession(false);
10    String username = (String) session.getAttribute("user");
11
12    resp.setContentType("text/html");
13    resp.getWriter().println("<h1>خوش اومدی " + username + "</h1>");
14    resp.getWriter().println("<a href=" + req.getContextPath() + "/logout'>خروج</a>");
15 }
16 }
```

LogoutServlet.java

```
1 java
2 @WebServlet("/logout")
3 public class LogoutServlet extends HttpServlet {
4
5     @Override
6     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
7         throws IOException {
8
9         HttpSession session = req.getSession(false);
10        if (session != null) {
11            session.invalidate();
12        }
13
14        resp.sendRedirect(req.getContextPath() + "/login");
15    }
16 }
```

۱۱-۳-۱۴ جمع‌بندی – نکات کلیدی مصاحبه

1. HTTP Stateless هست – برای نگه‌داشتن وضعیت از Session/Token استفاده می‌کنیم.
2. ServletContext برای کل اپ، ServletConfig برای به Servlet.
3. Filter قبل از Servlet اجرا میشه – برای احراز هویت، لاگ، فشرده‌سازی.
4. Listener به رویدادها گوش میده – برای راه‌اندازی منابع.
5. web.xml برای تنظیمات، Annotation برای مدرن بودن.
6. Session سمت سرور، Cookie سمت مرورگر.
7. JWT Stateless و مناسب REST API.
8. GET برای خواندن، POST برای تغییر.
9. curl ابزار خط فرمان برای تست HTTP.
10. load-on-startup باعث ساخت Servlet موقع استارت سرور میشه.

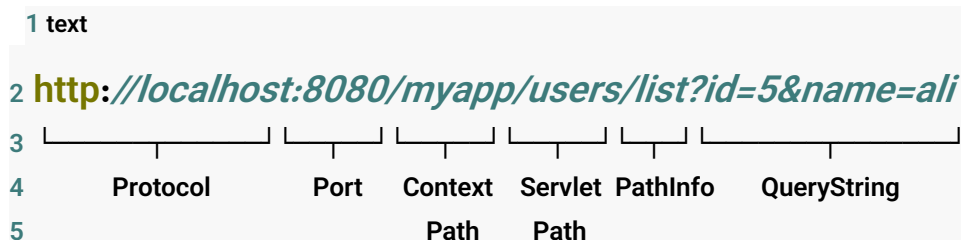
نکته طلایی: توی مصاحبه اگه پرسیدن "چرا از JWT استفاده می‌کنی؟" بگو: "چون Stateless هست و توی معماری Microservices و Cluster خیلی بهتر از Session عمل می‌کنه. Session توی Cluster نیاز به Sticky Session یا Session Replication داره که پیچیدگی رو زیاد می‌کنه."

getContextPath vs getServletPath vs getPathInfo vs getRequestURI

این چهار متد از پرکاربردترین‌های HttpServletRequest هستن و توی مصاحبه‌ها زیاد پرسیده میشن. اکثر برنامه‌نویس‌ها بینشون قاطی می‌کنن.

۱۴-۳.۱۲ تفکیک URL نوی Servlet

ساختار کامل یک URL



! سوال مصاحبه‌ای ۱۲: فرق getPathInfo، getServletPath، getContextPath و getRequestURI چیست؟

جواب:

متد	خروجی	توضیح
<code>()getContextPath</code>	myapp/	نام Context (نام WAR یا مسیر Deploy)
<code>()getServletPath</code>	users/	مسیری که به Servlet Map شده
<code>()getPathInfo</code>	list/	مسیر اضافه بعد از Servlet (اگره */ باشد)
<code>()getRequestURI</code>	myapp/users/list/	کل URI (بدون Port، Protocol، و QueryString)
<code>()getQueryString</code>	id=5&name=ali	پارامترهای Query
<code>()getRequestURL</code>	http://localhost:8080/myapp/users/list	کل URL (بدون QueryString)

مثال عملی کامل

فرض کن این تنظیمات رو داریم:

1-Java Interview Question

```
1 xml
2 <!-- web.xml -->
3 <servlet>
4   <servlet-name>userService</servlet-name>
5   <servlet-class>ir.nas.UserService</servlet-class>
6 </servlet>
7
8 <servlet-mapping>
9   <servlet-name>userService</servlet-name>
10  <url-pattern>/users/*</url-pattern>
11 </servlet-mapping>
```

و این درخواست:

```
1 text
2 http://localhost:8080/myapp/users/list?id=5&name=ali
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
```

3 **توی** **UserService:**

```
1 java
2
3
4
5 public class UserService extends HttpServlet {
6   @Override
7   protected void doGet(HttpServletRequest req, HttpServletResponse resp)
8     throws IOException {
9
10    System.out.println("ContextPath: " + req.getContextPath()); // /myapp
11    System.out.println("ServletPath: " + req.getServletPath()); // /users
12    System.out.println("PathInfo: " + req.getPathInfo()); // /list
13    System.out.println("RequestURI: " + req.getRequestURI()); // /myapp/users/list
14    System.out.println("RequestURL: " + req.getRequestURL()); // http://localhost:8080/myapp/users/list
15    System.out.println("QueryString: " + req.getQueryString()); // id=5&name=ali
16
17    // پارامترها
18    System.out.println("id: " + req.getParameter("id")); // 5
19    System.out.println("name: " + req.getParameter("name")); // ali
20
21    // اطلاعات اضافی
```

1-Java Interview Question

```
22 System.out.println("Method: " + req.getMethod());    // GET
23 System.out.println("Protocol: " + req.getProtocol());  // HTTP/1.1
24 System.out.println("ServerName: " + req.getServerName()); // localhost
25 System.out.println("ServerPort: " + req.getServerPort()); // 8080
26 System.out.println("Scheme: " + req.getScheme());      // http
27 }
28 }
```

حالت‌های مختلف Mapping

حالت ۱: users/* (Path Mapping)/

```
1 text
2 URL: http://localhost:8080/myapp/users/list/5
3
4 getContextPath() = /myapp
5 getServletPath() = /users
6 getPathInfo() = /list/5 ← بقیه مسیر اینجا میاد
7 getRequestURI() = /myapp/users/list/5
```

حالت ۲: users (Exact Mapping)/

```
1 text
2 URL: http://localhost:8080/myapp/users
3
4 getContextPath() = /myapp
5 getServletPath() = /users
6 getPathInfo() = null ← چون مسیر اضافه‌ای نیست
7 getRequestURI() = /myapp/users
```

حالت ۳: *.do (Extension Mapping)

```
1 text
2 URL: http://localhost:8080/myapp/users.do
3
4 getContextPath() = /myapp
5 getServletPath() = /users.do
6 getPathInfo() = null
7 getRequestURI() = /myapp/users.do
```

حالت ۴: / (Default Servlet)

```
1 text
2 URL: http://localhost:8080/myapp/users/list
3
4 getContextPath() = /myapp
5 getServletPath() = /users/list
6 getPathInfo() = null
7 getRequestURI() = /myapp/users/list
```

کاربردهای عملی

۱. ساخت لینک نسبی

```
1 java
2 // ✓ استفاده کن getContextPath درست - همیشه از ✓
3 String url = req.getContextPath() + "/users/list";
4
5 // ✗ عوض بشه، لینک میشکند Context غلط - اگر ✗
6 String url = "/myapp/users/list";
```

۲. Redirect

```

1 java
2 // ✓ درست
3 resp.sendRedirect(req.getContextPath() + "/login");
4
5 // ✗ غلط
6 resp.sendRedirect("/login"); // اشتباه بره Context ممکنه به

```

۳. تشخیص مسیر توی Filter

```

1 java
2 public class AuthFilter implements Filter {
3     @Override
4     public void doFilter(ServletRequest request, ServletResponse response,
5                         FilterChain chain) throws IOException, ServletException {
6
7         HttpServletRequest req = (HttpServletRequest) request;
8
9         String path = req.getRequestURI();
10        String contextPath = req.getContextPath();
11
12        // URI از ابتدای Context حذف
13        String relativePath = path.substring(contextPath.length());
14
15        System.out.println("Relative Path: " + relativePath);
16        // مثلاً: /users/list
17
18        // چک کردن مسیرهای عمومی
19        if (relativePath.startsWith("/public") ||
20            relativePath.startsWith("/css") ||
21            relativePath.startsWith("/js")) {
22            chain.doFilter(request, response);
23            return;
24        }
25

```

1-Java Interview Question

```
26 // چک کردن Login
27 HttpSession session = req.getSession(false);
28 if (session == null || session.getAttribute("user") == null) {
29     ((HttpServletResponse) response).sendRedirect(contextPath + "/login");
30     return;
31 }
32
33 chain.doFilter(request, response);
34 }
35 }
```

۴. استخراج Path Variable (قبل از Spring)

```
1 java
2 // URL: /myapp/users/123
3 // Mapping: /users/*
4
5 String pathInfo = req.getPathInfo(); // /123
6 if (pathInfo != null && pathInfo.length() > 1) {
7     String id = pathInfo.substring(1); // 123
8     Long userId = Long.parseLong(id);
9     // ...
10 }
```

! سوال مصاحبه‌ای ۱۳: چرا getCtxPath() مهمه؟

جواب: چون اسم WAR یا Context Path می‌تونه توی محیط‌های مختلف فرق کنه:

```
1 text
2 Development: http://localhost:8080/myapp/...
3 Production: https://example.com/shop/...
```

اگه Hard-code کنی /myapp، توی Production میشکنه. ولی getCtxPath() همیشه درست کار می‌کنه.

```

1 java
2 // ✓ درست
3 String loginUrl = req.getContextPath() + "/login";
4
5 // ✗ غلط
6 String loginUrl = "/myapp/login";

```

! سوال مصاحبه‌ای ۱۴: فرق ()getRequestURI و ()getRequestURL چیست؟

جواب:

متد	خروجی	شامل
()getRequestURI	myapp/users/list/	فقط مسیر (بدون Protocol/Port/Host/Query)
()getRequestURL	http://localhost:8080/myapp/users/list	مسیر کامل (بدون Query)

```

1 java
2 // URI: /myapp/users/list
3 // URL: http://localhost:8080/myapp/users/list
4
5 // Query کامل با URL ساخت
6 String fullUrl = req.getRequestURL().toString() +
7     (req.getQueryString() != null ? "?" + req.getQueryString() : "");
8 // http://localhost:8080/myapp/users/list?id=5&name=ali

```

! سوال مصاحبه‌ای ۱۵: getPathInfo() کی null برمی‌گردونه؟

جواب: وقتی:

1. Servlet با Exact Mapping تعریف شده باشه (/users/)
2. Servlet با Extension Mapping تعریف شده باشه (do.*)
3. Servlet با Default Servlet (/) باشه
4. مسیر اضافه‌ای بعد از Servlet نباشه

فقط با Path Mapping (/users/*) مقدار داره.

```

1 java
2 // Mapping: /users/*
3 // URL: /myapp/users/list/5
4 getPathInfo() = /list/5 // ✓
5
6 // Mapping: /users
7 // URL: /myapp/users
8 getPathInfo() = null // ✗
9
10 // Mapping: *.do
11 // URL: /myapp/users.do
12 getPathInfo() = null // ✗

```


جدول خلاصه نهایی

متد	مثال	Context شامل	Servlet شامل	PathInfo شامل	Query شامل
()getContextPath	myapp/	✓	✗	✗	✗
()getServletPath	users/	✗	✓	✗	✗
()getPathInfo	list/	✗	✗	✓	✗
()getRequestURI	myapp/users/list/	✓	✓	✓	✗
()getRequestURL	http://localhost:8080/myapp/users/list	✓	✓	✓	✗
()getQueryString	id=5&name=ali	✗	✗	✗	✓

نکته طلایی مصاحبه

«همیشه برای ساخت URL از ()getContextPath استفاده کن، نه Hard-code. چون Context Path توی محیط‌های مختلف (Dev, Test, Prod) فرق می‌کنه و Hard-code باعث شکستن لینک‌ها میشه.»

forward vs sendRedirect و getRequestDispatcher

این مفهوم یکی از پرتکرارترین سوالات مصاحبه‌های Servlet هست. خیلی‌ها بین forward و sendRedirect قاطی می‌کنن.

۱۳-۱۴ getRequestDispatcher چیست؟

getRequestDispatcher() به راه برای پاس دادن درخواست به یک منبع دیگر (JSP یا Servlet دیگر) داخل خود سرور، بدون اینکه مرورگر بفهمد.

مثال واقعی

فرض کن کاربر می‌ره: site.com/login

قدم ۱: به Servlet به اسم LoginServlet این درخواست رو می‌گیره.

قدم ۲: می‌خوایم به جای اینکه خودمون HTML بسازیم، بریم به JSP که آماده داریم (welcome.jsp) رو نشون بدیم:

```

1 java
2 // LoginServlet.java
3 protected void doGet(HttpServletRequest req, HttpServletResponse res)
4     throws ServletException, IOException {
5
6     req.setAttribute("name", "علی"); // یه داده می‌فرستیم
7
8     req.getRequestDispatcher("/welcome.jsp") // برو به این JSP
9     .forward(req, res); // کنترل رو بده بهش
10 }
11 jsp
12 <!-- welcome.jsp -->
13 <h1>سلام ${name}</h1>
```

چی اتفاق می‌افته؟

قبل	بعد
کاربر URL زده: site.com/login	✓ آدرس مرورگر همون site.com/login می‌مونه
	محتوای welcome.jsp نشون داده می‌شه
	name = "علی" قابل دسترسیه

خلاصه در یک خط

«سرور، این درخواست رو ببر به x/ و جوابش رو بده به کاربر، ولی آدرس مرورگر رو عوض نکن.»
`getRequestDispatcher("/x").forward(req, res)`

forward vs include

```

1 java
2 // forward: برنمی‌گرده، کنترل کامل می‌ره،
3 req.getRequestDispatcher("/page.jsp").forward(req, res);
4
5 // include: محتواش درج می‌شه، ولی ادامه می‌دی
6 req.getRequestDispatcher("/header.jsp").include(req, res);
7 res.getWriter().println("<p>متن اصلی</p>");

```

! سوال مصاحبه‌ای ۱۶: تفاوت forward و sendRedirect چیست؟ (خیلی مهم)

جواب:

```
1 java
2 // forward → داخل سرور ،عوض نمی‌شه URL ،حفظ می‌شه request
3 req.getRequestDispatcher("/welcome.jsp").forward(req, res);
4
5 // sendRedirect → جدید request ،مرورگر می‌ره آدرس جدید
6 res.sendRedirect("welcome.jsp");
```

ویژگی	forward	sendRedirect
محل اجرا	داخل سرور	مرورگر (Client)
URL مرورگر	عوض نمی‌شه ❌	عوض می‌شه ✅
Request	همون request حفظ میشه ✅	request جدید ❌
Attribute ها	قابل دسترسی ✅	از دست میرن ❌

ویژگی	forward	sendRedirect
تعداد Request	۱	۲
سرعت	سریعتر ⚡	کندتر (رفت و برگشت به مرورگر)
کاربرد	پاس دادن به JSP/Servlet	رفتن به سایت دیگه یا بعد از POST

قانون طلایی

می‌خوای URL عوض نشه → forward
می‌خوای URL عوض بشه → sendRedirect

مثال کاربردی: Login

1 java

```

2 // ✓ بعد از Login موفق، Redirect کن (عوض بشه URL)
3 if (isValidUser) {
4     HttpSession session = req.getSession(true);
5     session.setAttribute("user", username);
6     resp.sendRedirect(req.getContextPath() + "/dashboard"); // عوض میشه URL
7 }
8
9 // ✓ درست - نمایش فرم لاگین با خطا
10 if (!isValidUser) {
11     req.setAttribute("error", "نام کاربری اشتباه");
12     req.getRequestDispatcher("/login.jsp").forward(req, resp); // همونه URL
13 }
```

چرا بعد از Login موفق Redirect؟

- جلوگیری از Double Submit (کاربر F5 بزنه، دوباره Login نکنه)
- الگوی PRG (Post-Redirect-Get)

! سوال مصاحبه‌ای ۱۷: PRG Pattern چیه؟

جواب: Post-Redirect-Get یه الگو برای جلوگیری از ارسال دوباره فرم با Refresh هست.

1 text

2 ❌ بدون PRG:

3 POST /login → کاربر → F5 نمایش صفحه → POST دوباره Login)

4

5 ✅ با PRG:

6 POST /login → Redirect به /dashboard → GET /dashboard → کاربر F5 → فقط GET همیشه تکرار

7 java

8 // پیاده‌سازی PRG

9 **protected void doPost**(HttpServletRequest req, HttpServletResponse resp)

10 **throws** IOException {

11 *//۱. پردازش POST*

12 processLogin(req);

13

14 *//۲. Redirect (نه forward!)*

15 resp.sendRedirect(req.getContextPath() + "/dashboard");

16 }

17

18 **protected void doGet**(HttpServletRequest req, HttpServletResponse resp)

19 **throws** ServletException, IOException {

20 *//۳. نمایش صفحه*

21 req.getRequestDispatcher("/dashboard.jsp").forward(req, resp);

22 }

! سوال مصاحبه‌ای ۱۸: چرا بعد از forward نمی‌تونی sendRedirect کنی؟

جواب: چون forward پاسخ رو Commit می‌کنه. اگه بعدش sendRedirect بزنی، IllegalStateException می‌گیری.

```
1 java
2 // ❌ خطا
3 req.getRequestDispatcher("/page.jsp").forward(req, res);
4 res.sendRedirect("/other"); // IllegalStateException!
5
6 // ✅ درست - یکیش رو انتخاب کن
7 if (condition) {
8     req.getRequestDispatcher("/page.jsp").forward(req, res);
9 } else {
10     res.sendRedirect("/other");
11 }
```

نکات تکمیلی

۱. getRequestDispatcher نسبی vs مطلق

```
1 java
2 // نسبی (به مسیر فعلی)
```

1-Java Interview Question

```
3 req.getRequestDispatcher("welcome.jsp");
4
5 // Context از ریشه مطلق
6 req.getRequestDispatcher("/welcome.jsp");
7
8 // همیشه مطلق استفاده کن (با / شروع کن) ✓
```

۲. ServletContext از getDispatcher

```
1 java
2 // Request (نسبی یا مطلق) از
3 req.getRequestDispatcher("/page.jsp");
4
5 // ServletContext (همیشه مطلق) از
6 getServletContext().getDispatcher("/page.jsp");
```

۳. include برای Header و Footer

```
1 java
2 // layout.jsp
3 req.getRequestDispatcher("/header.jsp").include(req, res);
4 res.getWriter().println("<main>محتوا</main>");
5 req.getRequestDispatcher("/footer.jsp").include(req, res);
```

۴. forward بعد از getWriter()

```
1 1 java
2 2 // ❌ forward نمی‌تونی getWriter() خطا - بعد از
3 3 res.getWriter().println("text");
4 4 req.getRequestDispatcher("/page.jsp").forward(req, res);
5
6 6 // ✓ بگیر writer کن، بعد forward اول
7 7 req.getRequestDispatcher("/page.jsp").forward(req, res);
```


جمع بندی

موضوع	forward	sendRedirect
محل	Server-side	Client-side
URL	ثابت	عوض میشه
Request	حفظ میشه	جدید
Attribute	حفظ میشه	از دست میره
سرعت	سریع	کند
PRG	✗	✓
Double Submit	✗ داره	✓ نداره

فاز ۱۴: JDBC، JPA و Hibernate

۱۴.۱ JDBC (Java Database Connectivity)

چیست؟ پایه‌ای‌ترین راه ارتباط با دیتابیس در جاوا.

مراحل کار با JDBC:

```

1 // a. بارگذاری Driver
2 Class.forName("com.mysql.cj.jdbc.Driver");
3
4 // b. ایجاد Connection
5 Connection conn = DriverManager.getConnection(
6     "jdbc:mysql://localhost:3306/mydb",
7     "root",
8     "password"
9 );
10
11 // c. ساخت Statement
12 Statement stmt = conn.createStatement();
13
14 // d. اجرای Query
15 ResultSet rs = stmt.executeQuery("SELECT * FROM users");
16
17 // e. پردازش نتایج
18 while (rs.next()) {

```

```

19 String name = rs.getString("name");
20 int age = rs.getInt("age");
21 System.out.println(name + " - " + age);
22 }
23
24 // f. بستن منابع (مهم!)
25 rs.close();
26 stmt.close();
27 conn.close();

```

PreparedStatement و JDBC ۱۴.۱

! PreparedStatement (مهم برای جلوگیری از SQL Injection)

```

1 //  PreparedStatement
2 String query = "SELECT * FROM users WHERE username = ? AND password = ?";
3 PreparedStatement ps = conn.prepareStatement(query);
4 ps.setString(1, username);
5 ps.setString(2, password);
6 ResultSet rs = ps.executeQuery();
7
8 //  concatenation (خطر SQL Injection!)
9 String query = "SELECT * FROM users WHERE username = " + username + " ";
10 Statement stmt = conn.createStatement();
11 ResultSet rs = stmt.executeQuery(query);

```

JPA (Java Persistence API) ۱۴.۲

چیست؟ یه استاندارد (Specification) برای Mapping اشیاء جاوا به جداول دیتابیس.

مفاهیم کلیدی:

مفهوم	توضیح
<code>Entity@</code>	کلاس رو به یه جدول دیتابیس تبدیل می‌کنه
<code>Id@</code>	کلید اصلی رو مشخص می‌کنه
<code>GeneratedValue@</code>	تولید خودکار مقدار کلید اصلی
<code>Column@</code>	نگاشت فیلد به ستون دیتابیس
<code>OneToMany, @ManyToOne@</code>	روابط بین جداول

```

1 @Entity
2 @Table(name = "users")
3 public class User {
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     @Column(nullable = false, unique = true)
9     private String username;
10
11    @Column(nullable = false)
12    private String password;
13
14    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)

```

```
15 private List<Order> orders;
16
17 // getters and setters
18 }
```

۱۴.۳ Hibernate (پیاده‌سازی JPA)

چیست؟ محبوب‌ترین پیاده‌سازی JPA که کارهای زیر رو انجام میده:

• تبدیل خودکار Object به SQL (و برعکس)

• مدیریت Cache (کش)

• مدیریت Transaction

• Lazy Loading (بارگذاری دیر هنگام)

! سوال مصاحبه‌ای: N+1 Query Problem چیه و چطور حل میشه؟

مشکل: وقتی یه Entity رو می‌خونی و بعد به Property که Lazy هست دسترسی پیدا می‌کنی، Hibernate یه Query جداگانه می‌زنه.

```
1 // a. این کوئری ۱ بار اجرا میشه
2 List<User> users = userRepository.findAll();
3
4 // b. (N بار) اینجا برای هر کاربر یه کوئری جداگانه زده میشه
5 for (User user : users) {
6     System.out.println(user.getOrders().size()); // N+1 Query!
7 }
```

1-Java Interview Question

• راه حل‌ها:

۱. JOIN FETCH

```
1 @Query("SELECT u FROM User u JOIN FETCH u.orders")
```

```
2 List<User> findAllWithOrders();
```

۲. EntityGraph

```
1 @EntityGraph(attributePaths = {"orders"})
```

```
2 List<User> findAll();
```

۳. @NamedEntityGraph

```
1 @Entity
```

```
2 @NamedEntityGraph(name = "User.orders", attributeNodes = @NamedAttributeNode("orders"))
```

```
3 public class User { ... }
```

! سوال مصاحبه‌ای: تفاوت LAZY و FetchType.EAGER

FetchType	توضیح	مزیت	عیب
EAGER	داده‌های مرتبط رو همین الان بارگذاری کن	ساده‌تر	کندتر، مصرف حافظه بیشتر
LAZY (پیش‌فرض)	فقط وقتی نیاز شد بارگذاری کن	سریع‌تر، کم مصرف	ممکنه به N+1 مشکل بخوری

۱۴.۴ مقایسه JDBC vs JPA vs Hibernate

1 JDBC (پایین ترین سطح)

2 |

3 ▼

4 JPA (Interface استاندارد - فقط)

5 |

6 ▼

7 Hibernate (پیاده سازی - محبوب ترین)

8 |

9 ▼

10 Spring Data JPA (خام Hibernate راحت تر از)

ویژگی	JDBC	JPA	Hibernate
سطح	پایین	متوسط	بالا
کنترل	کامل	کمتر	کمترین
سرعت	سریع ترین	متوسط	کندتر (به خاطر Overhead)
راحتی	سخت	راحت	بسیار راحت
کد نویسی	زیاد	متوسط	کم

۱۴.۵ Spring Data JPA (لایه راحت تر از Hibernate)

Spring Data JPA به لایه بالاتر از Hibernate هست که نوشتن کدهای دیتابیس رو خیلی راحت تر می‌کند.

۱۴.۵.۱ چرا Spring Data JPA؟

```

1 // a. خام (با Hibernate Spring Data JPA بدون)
2 Session session = entityManager.unwrap(Session.class);
3 Query query = session.createQuery("FROM User WHERE username = :username");
4 query.setParameter("username", "ali");
5 User user = (User) query.uniqueResult();
6
7 // b. !خیلی ساده‌تر - Spring Data JPA با
8 User user = userRepository.findByUsername("ali");

```

۱۴.۵.۲ Repository ها در Spring Data

```

1 // a. کامل‌ترین - JpaRepository
2 public interface UserRepository extends JpaRepository<User, Long> {
3     // متدهای آماده: save(), findById(), findAll(), delete(), count()
4 }
5
6 // b. پایه‌ای‌تر - CrudRepository
7 public interface UserRepository extends CrudRepository<User, Long> {}
8
9 // c. برای صفحه‌بندی - PagingAndSortingRepository
10 public interface UserRepository extends PagingAndSortingRepository<User, Long> {}

```

۱۴.۵.۳ متدهای Query Method (نوشتن کوئری با اسم متد)

```

1 public interface UserRepository extends JpaRepository<User, Long> {
2

```


1-Java Interview Question

```
3  // پیدا کردن با username
4  User findByUsername(String username);
5
6  // پیدا کردن با username و password
7  User findByUsernameAndPassword(String username, String password);
8
9  // پیدا کردن همه کاربران با سن بیشتر از
10 List<User> findByAgeGreaterThan(int age);
11
12 // پیدا کردن کاربرانی که اسمشون با ... شروع میشه
13 List<User> findByUsernameStartingWith(String prefix);
14
15 // مرتب سازی
16 List<User> findAllOrderByUsernameAsc();
17
18 // صفحه بندی
19 Page<User> findAll(Pageable pageable);
20
21 // بررسی وجود داشتن
22 boolean existsByUsername(String username);
23
24 // شمارش
25 long countByAgeGreaterThan(int age);
26
27 // حذف
28 void deleteByUsername(String username);
```

29 }

۱۴.۵.۴ Query@ (نوشتن کوئری دستی)

```

1 public interface UserRepository extends JpaRepository<User, Long> {
2
3     // ۱. JPQL (مخصوص JPA)
4     @Query("SELECT u FROM User u WHERE u.username = :username")
5     User findUserByUsername(@Param("username") String username);
6
7     // ۲. Native Query (SQL خام)
8     @Query(value = "SELECT * FROM users WHERE age > :age", nativeQuery = true)
9     List<User> findUsersOlderThan(@Param("age") int age);
10
11     // ۳. JOIN کوئری با
12     @Query("SELECT u FROM User u JOIN u.orders o WHERE o.total > :amount")
13     List<User> findUsersWithOrdersAbove(@Param("amount") double amount);
14 }

```

۱۴.۵.۵ Modifying@ (برای UPDATE و DELETE)

```

1 public interface UserRepository extends JpaRepository<User, Long> {
2
3     @Modifying
4     @Query("UPDATE User u SET u.active = false WHERE u.lastLogin < :date")

```

1-Java Interview Question

```
5  int deactivateInactiveUsers(@Param("date") LocalDate date);
6
7  @Modifying
8  @Query("DELETE FROM User u WHERE u.active = false")
9  int deleteInactiveUsers();
10 }
```

۱۴.۵.۶ صفحه بندی (Pagination)

```
1  @Service
2  class UserService {
3      @Autowired
4      private UserRepository repository;
5
6      public Page<User> getUsers(int page, int size) {
7          // صفحه ۰، هر صفحه ۱۰ تا
8          Pageable pageable = PageRequest.of(page, size);
9          return repository.findAll(pageable);
10     }
11
12     public Page<User> getSortedUsers(int page, int size) {
13         Pageable pageable = PageRequest.of(page, size, Sort.by("username").ascending());
14         return repository.findAll(pageable);
15     }
16 }
```

۱۴.۵.۷ Specification (کوئری داینامیک)

```

1 // a. برای وقتی که فیلترها متغیر هستند
2 public class UserSpecification {
3     public static Specification<User> hasUsername(String username) {
4         return (root, query, criteriaBuilder) ->
5             criteriaBuilder.equal(root.get("username"), username);
6     }
7
8     public static Specification<User> isActive() {
9         return (root, query, criteriaBuilder) ->
10             criteriaBuilder.isTrue(root.get("active"));
11     }
12 }
13
14 // b. استفاده
15 @Service
16 class UserService {
17     public List<User> filterUsers(String username, Boolean active) {
18         Specification<User> spec = Specification.where(null);
19
20         if (username != null) {
21             spec = spec.and(UserSpecification.hasUsername(username));
22         }
23
24         if (active != null) {
25             spec = spec.and(UserSpecification.isActive());
26         }
27     }
28 }

```

1-Java Interview Question

```
27     return repository.findAll(spec);  
28 }  
29 }
```

📖 فاز ۱۵: DevOps، Docker، Kubernetes و

ابزارهای مرتبط

۱۵.۱ DevOps چیست؟

تعریف ساده: ترکیب "Development" و "Operations". به فرهنگ و مجموعه‌ای از روش‌ها برای:

- خودکارسازی فرآیندهای نرم‌افزاری
- کاهش فاصله بین تیم توسعه و تیم عملیات
- تحویل سریع‌تر و مطمئن‌تر نرم‌افزار

چرخه DevOps:

1 Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → (Plan بازگشت به)

۱۵.۲ Docker (ظروف مجازی)

چیست؟ به پلتفرم برای بسته‌بندی برنامه‌ها و وابستگی‌هاشون در به Container (ظرف) که می‌تونه روی هر سیستمی اجرا بشه.

تشبیه: کانتینر مثل به کانتینر حمل و نقل (کشتی) هست که برنامه و همه چیز مورد نیازش داخلش هست.

فرق Virtual Machine با Docker

ویژگی	Docker (Container)	Virtual Machine
سیستمعامل	مشارک با Host	مجزا (هر VM یه OS داره)
اندازه	سبک (MB)	سنگین (GB)
سرعت اجرا	سریع	کند
مصرف منابع	کم	زیاد

Dockerfile (دستور ساخت کانتینر)

```

1 # a. انتخاب /Base Image
2 FROM openjdk:17-jdk-alpine
3
4 # b. تنظیم متغیرهای محیطی
5 ENV APP_HOME=/app
6
7 # c. ایجاد دایرکتوری
8 WORKDIR $APP_HOME
9
10 # d. کپی کردن فایل ها
11 COPY target/myapp.jar app.jar
12
13 # e. پورت برنامه
14 EXPOSE 8080
15

```

16 # f. دستور اجرا

17 ENTRYPOINT ["java", "-jar", "app.jar"]

دستورات مهم Docker

1 # a. ساخت Image

2 docker build -t myapp:latest .

3

4 # b. اجرای Container

5 docker run -p 8080:8080 myapp:latest

6

7 # c. های در حال اجرا Container مشاهده

8 docker ps

9

10 # d. ها Container مشاهده همه

11 docker ps -a

12

13 # e. Container توقف

14 docker stop container_id

15

16 # f. Container حذف

17 docker rm container_id

18

19 # g. ها Image مشاهده

20 docker images

21

1-Java Interview Question

22 # h. ورود به Container

23 docker exec -it container_id /bin/sh

24

25 # i. مشاهده لاگ ها

26 docker logs container_id

Docker Compose (اجرای چند کانتینر با هم)

1 # docker-compose.yml

2 version: '3.8'

3

4 services:

5 # پلیکیشن / Spring Boot

6 app:

7 build: .

8 ports:

9 - "8080:8080"

10 depends_on:

11 - db

12 environment:

13 - SPRING_DATASOURCE_URL=jdbc:mysql://db:3306/mydb

14

15 # دیتابیس

16 db:

17 image: mysql:8.0

18 environment:

1-Java Interview Question

19 - MYSQL_ROOT_PASSWORD=root

20 - MYSQL_DATABASE=mydb

21 ports:

22 - "3306:3306"

23

24 # Redis برای کش

25 redis:

26 image: redis:alpine

27 ports:

28 - "6379:6379"

29 # اجرای همه سرویس ها

30 docker-compose up -d

31

32 # توقف همه سرویس ها

33 docker-compose down

Kubernetes (K8s) ۱۵.۳

چیست؟ سیستم مدیریت Container ها در مقیاس بزرگ. مثل یک Captin برای کشتی های (Container های) Docker.

مفاهیم کلیدی:

مفهوم	توضیح	تشبیه
Pod	کوچکترین واحد، شامل یک یا چند Container	یه سرباز
Deployment	مدیریت Pod ها (تعداد، بروزرسانی)	یه گروه سرباز
Service	دسترسی به Pod ها از بیرون	آدرس پادگان
Ingress	مدیریت ورودی (مسیریابی)	ورودی اصلی
ConfigMap	ذخیره تنظیمات	تابلوی اعلانات
Secret	ذخیره اطلاعات حساس (پسورد)	صندوق امانات

مثال ساده Kubernetes

```

1 # deployment.yaml
2 apiVersion: apps/v1
3 kind: Deployment
4 metadata:
5   name: myapp-deployment
6 spec:
7   replicas: 3 # اجرا کن Pod تا ۳
8   selector:
9     matchLabels:
10      app: myapp
11   template:
12     metadata:
13       labels:
14         app: myapp
15     spec:
16       containers:
17         - name: myapp

```

1-Java Interview Question

```
18   image: myapp:latest
19   ports:
20     - containerPort: 8080
21 ---
22 # service.yaml (برای دسترسی از بیرون)
23 apiVersion: v1
24 kind: Service
25 metadata:
26   name: myapp-service
27 spec:
28   selector:
29     app: myapp
30   ports:
31     - port: 80
32     targetPort: 8080
33   type: LoadBalancer
34 # a. اعمال به کلاستر
35 kubectl apply -f deployment.yaml
36 kubectl apply -f service.yaml
37
38 # b. Pod ها مشاهده
39 kubectl get pods
40
41 # c. Service مشاهده
42 kubectl get services
43
```

44 # d. ها Deployment مشاهده

45 kubectl get deployments

46

47 # e. (ها Pod تغییر تعداد) مقیاس دهی

48 kubectl scale deployment myapp-deployment --replicas=5

Apache Kafka ۱۵.۴

چیست؟ به سیستم پیام‌رسانی توزیع‌شده (Distributed Messaging System) برای پردازش جریان داده (Stream Processing).

تشبیه: مثل به سیستم حمل و نقل عمومی (مترو) که پیام‌ها رو بین Producerها (فرستنده) و Consumerها (گیرنده) جابه‌جا می‌کنه.

مفاهیم کلیدی:

مفهوم	توضیح
Producer	فرستنده پیام
Consumer	گیرنده پیام
Topic	دسته‌بندی پیام‌ها (مثل کانال)
Partition	تقسیم Topic به چند بخش برای مقیاس‌پذیری
Broker	سرور Kafka

چه موقع از Kafka استفاده کنیم؟

• Event Sourcing: ثبت همه رویدادها

• Log Aggregation: جمع‌آوری لاگ‌ها از چندین سرویس

•Stream Processing: پردازش لحظه‌ای داده (مثل تحلیل لحظه‌ای)

•Message Queue: ارتباط بین میکروسرویس‌ها

مثال ساده در جاوا (با Spring Boot)

```
1 // Producer - فرستنده
2 @Service
3 class KafkaProducerService {
4     @Autowired
5     private KafkaTemplate<String, String> kafkaTemplate;
6
7     public void sendMessage(String message) {
8         kafkaTemplate.send("my-topic", message);
9     }
10 }
11
12 // Consumer - گیرنده
13 @Service
14 class KafkaConsumerService {
15     @KafkaListener(topics = "my-topic", groupId = "my-group")
16     public void listen(String message) {
17         System.out.println("Received: " + message);
18     }
19 }
```

۱۵.۵ Linux (لینوکس) – دستورات مهم برای بک‌اندکار

دستورات حیاتی که باید بلد باشی:

۱. مدیریت فایل‌ها و دایرکتوری‌ها

```
1 ls -la      # نمایش همه فایل‌ها با جزئیات
2 cd /path    # تغییر دایرکتوری
3 mkdir newdir # ساخت دایرکتوری جدید
4 rm -rf dir   # حذف دایرکتوری (با احتیاط!)
5 cp file1 file2 # کپی فایل
6 mv file1 file2 # جابجایی یا تغییر نام
```

۲. مشاهده محتوای فایل

```
1 cat file.txt # نمایش کامل فایل
2 less file.txt # نمایش صفحه‌به‌صفحه
3 tail -f log.txt # (Live) دنبال کردن لاگ
4 head -n 10 file # خط اول ۱۰
```

۳. مدیریت فرآیندها (Process)

```
1 ps aux      # مشاهده همه فرآیندها
2 top          # مشاهده مصرف منابع (لایو)
3 kill -9 PID  # کشتن یک فرآیند
4 nohup java -jar app.jar & # اجرا در پس‌زمینه
```

۴. مدیریت شبکه

- 1 `ping google.com` # تست اتصال
- 2 `netstat -tulpn` # مشاهده پورت‌های باز
- 3 `curl http://api.test` # درخواست HTTP
- 4 `ss -tuln` # مشاهده سوکت‌ها

۵. مدیریت دیسک و حافظه

- 1 `df -h` # فضای دیسک
- 2 `du -sh *` # حجم پوشه‌ها
- 3 `free -h` # RAM حافظه

! سوال مصاحبه‌ای: فرق apt-get و yum چیه؟

Feature	apt-get	yum
توزیع	Debian/Ubuntu	RedHat/CentOS
مخزن	APT	RPM
دستور نصب	<code>apt-get install</code>	<code>yum install</code>

۱۵.۶ CI/CD (Continuous Integration / Continuous Deployment)

چیست؟ خودکارسازی فرآیند Build، Test و Deploy کد.

مراحل:

1. Commit: توسعه‌دهنده کد رو Push می‌کنه

2.Build: سرور خودکار کد رو Build می‌کنه

3.Test: تست‌ها اجرا میشن

4.Deploy: برنامه روی سرور Deploy میشه

ابزارهای معروف:

• Jenkins (محبوب‌ترین)

• GitHub Actions

• GitLab CI

• CircleCI

GitHub Actions مثال

```
1 name: Java CI/CD
2
3 on:
4   push:
5     branches: [ main ]
6
7 jobs:
8   build:
9     runs-on: ubuntu-latest
10
11     steps:
12     - uses: actions/checkout@v2
13
```

1-Java Interview Question

```
14 - name: Set up JDK 17
15   uses: actions/setup-java@v2
16   with:
17     java-version: '17'
18
19 - name: Build with Maven
20   run: mvn clean package
21
22 - name: Build Docker Image
23   run: docker build -t myapp .
24
25 - name: Deploy to Server
26   run: docker push myapp
```

۱۵.۷ خلاصه یک صفحه ای

ابزار	کاربرد	سطح یادگیری
Docker	بسته بندی برنامه	ضروری ★★★★★
Docker Compose	اجرای چند کانتینر	ضروری ☆★★★★
Kubernetes	مدیریت کانتینرها در مقیاس بزرگ	متوسط ☆☆☆★★
Kafka	پیام رسانی بین سرویس ها	متوسط ☆☆☆★★
Linux	سیستم عامل سرور	ضروری ★★★★★
Jenkins/GitHub Actions	CI/CD	مهم ☆★★★★

فاز ۱۶: تست‌نویسی (Testing) – کامل و عمیق

تست‌نویسی یکی از مهم‌ترین مهارت‌های یه توسعه‌دهنده حرفه‌ای هست. توی مصاحبه‌های شغلی، ازت انتظار می‌ره که حداقل با مفاهیم پایه تست و ابزارهای آشنا باشی.

۱۶.۱ چرا تست‌نویسی؟

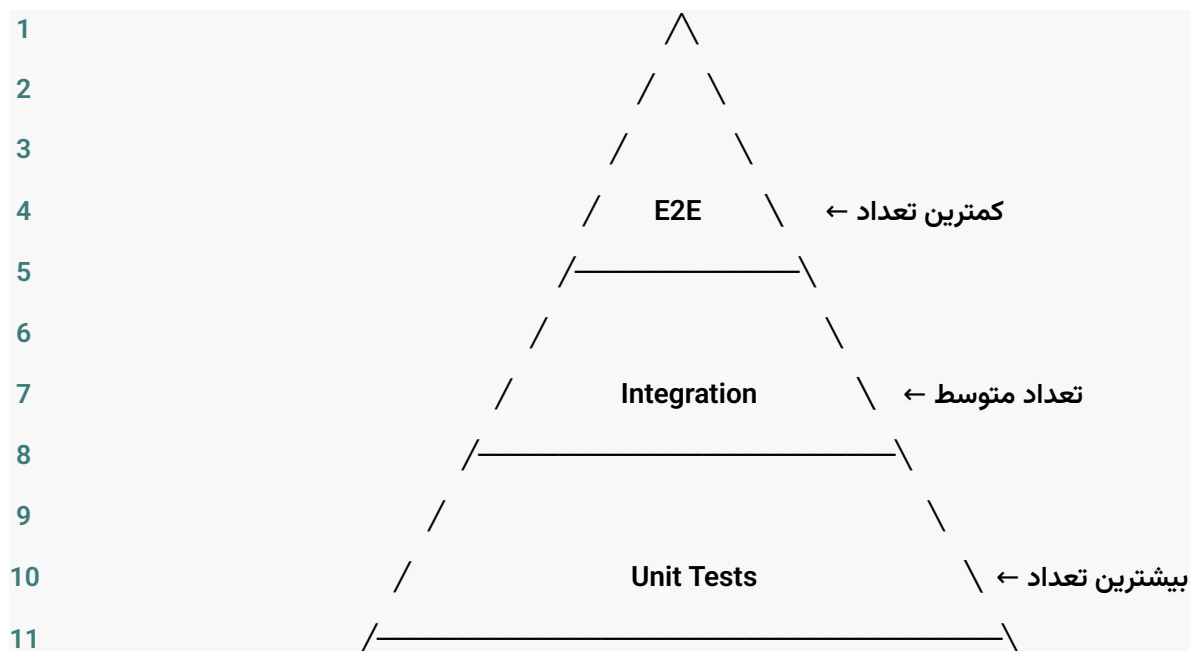
سوال مصاحبه‌ای: چرا باید تست بنویسیم؟

جواب:

۱. اطمینان از صحت کد: مطمئن میشیم کدمون درست کار می‌کنه
۲. جلوگیری از Regression Bug: تغییرات جدید، کد قدیمی رو نمی‌شکنه
۳. مستندسازی: تست‌ها به عنوان مستندات زنده عمل می‌کنن
۴. طراحی بهتر: کد تست‌پذیر، معمولاً طراحی بهتری داره
۵. اعتماد به نفس: با خیال راحت می‌تونی کد رو تغییر بدی

۱۶.۲ انواع تست از نظر سطح (Test Pyramid)

هرم تست (Test Pyramid) به ما می‌گه چه مقدار از هر نوع تست باید داشته باشیم:



جدول انواع تست

تعداد	مثال	توضیح	نوع تست
زیاد ✓	تست کردن <code>calculateSalary()</code>	تست یک واحد کوچک (متد یا کلاس) به تنهایی	Unit Test
متوسط	تست <code>UserService</code> با <code>UserRepository</code> و واقعی	تست ارتباط چند جزء با هم	Integration Test
کم	ثبت نام ← لاگین ← خرید ← خروج	تست کل سیستم از اول تا آخر	E2E Test

۱۶.۳ اصول طلایی تست (اصول FIRST)

این اصول پایه‌ای‌ترین چیزهایی هستند که باید در مورد تست بدونی:

مثال	توضیح کامل	اصل
تست‌های Unit نباید به دیتابیس یا شبکه وصل بشن	تست‌ها باید خیلی سریع اجرا بشن تا بتونی مرتب اجراشون کنی	Fast (سریع)
هر تست محیط خودش رو آماده کنه و بعد پاک کنه	هر تست نباید به ترتیب یا نتیجه تست دیگه وابسته باشه	Independent (مستقل)
از مقادیر ثابت استفاده کن، نه تاریخ و زمان جاری	هر بار که تست اجرا بشه، نتیجه یکسان باشه	Repeatable (تکرارپذیر)
استفاده از assert، نه اینکه خروجی رو چشمی چک کنی	تست باید خودش تشخیص بده قبول شده یا رد شده	Self-Validating (خودتأییدگر)
TDD: اول تست، بعد کد	تست رو قبل یا هم‌زمان با کد بنویس	Timely (به‌موقع)

۱۶.۴ ساختار Given-When-Then (AAA)

این ساختار استاندارد نوشتن تست‌هاست و باعث میشه تست‌ها خواناتر بشن:

```

1 @Test
2 void shouldReturnUserWhenValidUsernameProvided() {
3     // GIVEN (یا Arrange) - آماده‌سازی
4     UserService service = new UserService();
5     String username = "ali";
6
7     // WHEN (یا Act) - اجرای عملیات
8     User result = service.findUser(username);
9
10    // THEN (یا Assert) - تأیید نتیجه
11    assertNotNull(result);
12    assertEquals("ali", result.getUsername());
13 }
```

نام‌های دیگه این ساختار:

Arrange-Act-Assert (AAA)•

Given-When-Then (GWT)•

۱۶.۵ تکنیک‌های طراحی تست

۱. Boundary Value Analysis (تحلیل مرزها)

قانون: خطاها معمولاً در مرزهای ورودی رخ میدن. همیشه مرزها رو تست کن.

```

1 // فقط ۱۸ تا ۶۰ رو قبول می‌کنه acceptAge(int age) متد a.
2
3 @Test
4 void testAgeBoundaries() {
5     // مرزهای پایین
6     assertFalse validator.acceptAge(17); // دقیقاً زیر مرز
7     assertTrue validator.acceptAge(18); // درست روی مرز پایین
8
9     // مرزهای بالا
10    assertTrue validator.acceptAge(60); // درست روی مرز بالا
11    assertFalse validator.acceptAge(61); // دقیقاً بالای مرز
12
13    // داخل بازه
14    assertTrue validator.acceptAge(30);
15    assertTrue validator.acceptAge(45);
16 }
```

۲. Equivalence Partitioning (تقسیم به کلاس‌های هم‌ارز)

قانون: ورودی‌ها رو به دسته‌هایی تقسیم کن که رفتار مشابهی دارن. از هر دسته فقط یکی رو تست کن.

```

1 // b. بر اساس سن تخفیف می‌ده calculateDiscount(int age) متد
2
3 @Test
4 void testDiscountByAgeGroups() {
5     // دسته کودک (کمتر از ۱۲) - یک نمونه کافیست
6     assertEquals(0.5, calculator.calculateDiscount(5));
7
8     // دسته بزرگسال (۱۲ تا ۶۰)
9     assertEquals(0.0, calculator.calculateDiscount(25));
10
11    // دسته سالمند (بیشتر از ۶۰)
12    assertEquals(0.3, calculator.calculateDiscount(70));
13 }
```

۳. Negative Testing (تست منفی)

قانون: فقط مسیر خوش‌بخت رو تست نکن. ورودی‌های غلط رو هم تست کن.

```

1 @Test
2 void testNegativeScenarios() {
3     UserService service = new UserService();
4
5     // ورودی null
6     assertThrows(IllegalArgumentException.class,
```

1-Java Interview Question

```
7    () -> service.registerUser(null);
8
9    // ورودی خالی
10   User emptyUser = new User("", "");
11   assertThrows(ValidationException.class,
12       () -> service.registerUser(emptyUser));
13
14   // پسورد خیلی کوتاه
15   User weakUser = new User("ali", "123");
16   assertFalse(service.registerUser(weakUser));
17
18   // کاربر تکراری
19   service.registerUser(new User("ali", "pass123"));
20   assertThrows(DuplicateUserException.class,
21       () -> service.registerUser(new User("ali", "pass456")));
22 }
```

۴. اصل تک مسؤلیتی در تست

قانون: هر تست فقط یک رفتار رو بررسی کنه.

```
1 // a. ✗ چند تا - چند موضوعات مختلف assert بد
2 @Test
3 void testUserService() {
4     User user = service.register("ali", "pass");
5     assertNotNull(user);
6     assertEquals("ali", user.getUsername());
7 }
```


1-Java Interview Question

```
7  assertTrue(user.isActive());
8  assertNotNull(user.getCreatedAt());
9  // اینجا ۴ تا چیز مختلف رو تست کرده
10 }
11
12 // b. ☒ خوب - هر تست فقط به کار
13 @Test
14 void shouldCreateUserWithCorrectUsername() {
15     User user = service.register("ali", "pass");
16     assertEquals("ali", user.getUsername());
17 }
18
19 @Test
20 void shouldSetUserActiveByDefault() {
21     User user = service.register("ali", "pass");
22     assertTrue(user.isActive());
23 }
24
25 @Test
26 void shouldSetCreatedDateOnRegistration() {
27     User user = service.register("ali", "pass");
28     assertNotNull(user.getCreatedAt());
29 }
```

۵. اصل نام‌گذاری گویا

۵. اصل نام‌گذاری گویا اسم تست باید مشخص کنه چه چیزی رو تست میکنه، تحت چه شرایطی، و نتیجه‌ی مورد انتظار چیه.

```

1 // a. ✓ نام‌گذاری خوب
2 @Test
3 void shouldReturnFalseWhenPasswordIsTooShort() {}
4
5 @Test
6 void shouldThrowExceptionWhenUserNotFound() {}
7
8 @Test
9 void shouldCalculateDiscountCorrectlyForSeniorCitizens() {}
10
11 @Test
12 void shouldNotAllowDuplicateUsernames() {}
13
14 // b. ✗ نام‌گذاری بد
15 @Test
16 void test1() {}
17
18 @Test
19 void testLogin() {} // معلوم نیست چی رو تست میکنه
20
21 @Test
22 void test() {} // اصلاً چی؟

```

۶. اصل جداسازی دغدغه‌ها (Separation of Concerns)

قانون: تست‌های واحد باید فقط یک تابع یا کلاس رو تست کنن و بقیه وابستگی‌ها رو با Mock جایگزین کنی.

۱۶.۶ تست در جاوا با JUnit 5

JUnit 5 محبوب‌ترین فریم‌ورک تست در جاواست.

۱۶.۶.۱ مفاهیم پایه JUnit

```
1 import org.junit.jupiter.api.*;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 class CalculatorTest {
5
6     private Calculator calculator;
7
8     @BeforeAll
9     static void beforeAll() {
10         System.out.println("قبل از همه تست‌ها");
11     }
12
13     @AfterAll
14     static void afterAll() {
15         System.out.println("بعد از همه تست‌ها");
16     }
```

```

17
18 @BeforeEach
19 void setUp() {
20     calculator = new Calculator();
21 }
22
23 @AfterEach
24 void tearDown() {
25     System.out.println("بعد از هر تست");
26 }
27
28 @Test
29 @DisplayName("باید جمع دو عدد رو درست محاسبه کنه")
30 void shouldAddTwoNumbersCorrectly() {
31     int a = 5;
32     int b = 3;
33     int result = calculator.add(a, b);
34     assertEquals(8, result, "جمع ۵ و ۳ باید ۸ بشه");
35 }
36 @Test
37 void shouldThrowExceptionWhenDividingByZero() {
38     assertThrows(ArithmeticException.class,
39         () -> calculator.divide(10, 0));
40 }
41 }

```

۱۶.۶.۲ Assertion های مهم JUnit

بررسی برابری a. // 1

1-Java Interview Question

```
2 assertEquals(4, calculator.add(2, 2));
3 assertEquals(3.14, PI, 0.001);
4
5 // b. بررسی نابرابری
6 assertNotEquals(5, calculator.add(2, 2));
7
8 // c. بودن null بررسی
9 assertNull(user);
10 assertNotNull(user);
11
12 // d. بررسی شرط
13 assertTrue validator.isValid("ali");
14 assertFalse validator.isValid("");
15
16 // e. بررسی استثنا
17 assertThrows(ArithmeticException.class,
18     () -> calculator.divide(10, 0));
19
20 // f. بررسی همه شرطها
21 assertAll("user validation",
22     () -> assertNotNull(user),
23     () -> assertTrue(user.isActive()),
24     () -> assertEquals("ali", user.getUsername())
25 );
26
27 // g. بررسی آرایهها
```

```
28 assertEquals(expectedArray, actualArray);
```

```
29
```

```
30 // h. بررسی timeout
```

```
31 assertTimeout(Duration.ofSeconds(1),
```

```
32 () -> service.heavyOperation());
```

۱۶.۶.۳ تست‌های پارامتری (Parameterized Tests)

```
1 import org.junit.jupiter.params.ParameterizedTest;
```

```
2 import org.junit.jupiter.params.provider.*;
```

```
3
```

```
4 class ParameterizedTestExample {
```

```
5
```

```
6 @ParameterizedTest
```

```
7 @ValueSource(ints = {2, 4, 6, 8, 10})
```

```
8 void shouldReturnTrueForEvenNumbers(int number) {
```

```
9     assertTrue(calculator.isEven(number));
```

```
10 }
```

```
11
```

```
12 @ParameterizedTest
```

```
13 @CsvSource({
```

```
14     "5, 3, 8",
```

```
15     "10, 20, 30",
```

```
16     "-5, 3, -2"
```

```
17 })
```

```
18 void shouldAddNumbersCorrectly(int a, int b, int expected) {
```

```
19     assertEquals(expected, calculator.add(a, b));
```

```

20 }
21
22 @ParameterizedTest
23 @EnumSource(DayOfWeek.class)
24 void shouldHaveCorrectDayName(DayOfWeek day) {
25     assertNotNull(day.name());
26 }
27 }

```

۱۶.۷ Mock و Stub (شبیه سازی)

چرا؟ وقتی یه کلاس وابستگی به دیتابیس یا سرویس خارجی داره، برای تست باید اون وابستگی رو شبیه سازی کنیم.

۱۶.۷.۱ تفاوت Mock و Stub

مفهوم	توضیح	پیچیدگی	کاربرد
Stub	یه پیاده سازی ساده که همیشه یه مقدار ثابت برمی گردونه	ساده	وقتی فقط به یه مقدار خاص نیاز داری
Mock	یه شیء شبیه سازی شده که می تونی روش Expectation تعریف کنی و Verify کنی	پیچیده	وقتی می خوای رفتار رو کنترل و بررسی کنی

۱۶.۷.۲ مثال Stub (ساده)

```

1 class EmailServiceStub implements EmailService {
2     @Override
3     public boolean sendEmail(String to, String body) {
4         return true;

```

1-Java Interview Question

```
5  }
6  }
7
8  class UserServiceTest {
9      private UserService service;
10
11     @BeforeEach
12     void setUp() {
13         EmailService emailService = new EmailServiceStub();
14         service = new UserService(emailService);
15     }
16
17     @Test
18     void shouldRegisterUser() {
19         assertTrue(service.register("ali", "pass"));
20     }
21 }
```

۱۶.۷.۳ مثال Mock با Mockito

```
1 import org.mockito.*;
2 import static org.mockito.Mockito.*;
3
4 class UserServiceTest {
```


1-Java Interview Question

```
5
6  @Mock
7  private UserRepository repository;
8
9  @Mock
10 private EmailService emailService;
11
12 @InjectMocks
13 private UserService service;
14
15 @BeforeEach
16 void setUp() {
17     MockitoAnnotations.openMocks(this);
18 }
19
20 @Test
21 void shouldReturnUserWhenFound() {
22     User mockUser = new User("ali", "pass");
23     when(repository.findByUsername("ali"))
24         .thenReturn(Optional.of(mockUser));
25
26     User result = service.findUser("ali");
27
28     assertNotNull(result);
29     assertEquals("ali", result.getUsername());
30     verify(repository).findByUsername("ali");
```

1-Java Interview Question

```
31 }
32
33 @Test
34 void shouldThrowExceptionWhenUserNotFound() {
35     when(repository.findByUsername("unknown"))
36         .thenReturn(Optional.empty());
37
38     assertThrows(UserNotFoundException.class,
39         () -> service.findUser("unknown"));
40 }
41
42 @Test
43 void shouldSendEmailAfterRegistration() {
44     User user = new User("ali", "pass");
45     when(repository.save(any(User.class))).thenReturn(user);
46
47     service.register("ali", "pass");
48
49     verify(emailService).sendEmail(eq("ali"), anyString());
50     verify(emailService, times(1)).sendEmail(any(), any());
51 }
52 }
```

۱۶.۷.۴ متدهای پرکاربرد Mockito

1 // تنظیم رفتار

2 when(mock.method()).thenReturn(value);

```
3 when(mock.method()).thenThrow(exception);
4 when(mock.method()).thenReturn(value1).thenReturn(value2);
5 doThrow(exception).when(mock).voidMethod();
6
7 // b. Matcher
8 when(repository.findById(anyLong())).thenReturn(user);
9 when(repository.findById(eq(1L))).thenReturn(user);
10 when(repository.findByUsername(anyString())).thenReturn(user);
11
12 // c. Verification
13 verify(mock).method();
14 verify(mock, times(2)).method();
15 verify(mock, never()).method();
16 verify(mock, atLeast(1)).method();
17 verify(mock, atMost(3)).method();
18 verifyNoMoreInteractions(mock);
19
20 // d. Spying
21 @Spy
22 private UserService realService;
23 when(realService.expensiveMethod()).thenReturn("mock result");
```

Spring Boot در تست ۱۶.۸

Spring Boot ابزارهای قدرتمندی برای تست دارد.

۱۶.۸.۱ تست لایه Controller (با MockMvc)

```
1 import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
2 import org.springframework.test.web.servlet.MockMvc;
3 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
4 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
5
6 @WebMvcTest(UserController.class)
7 class UserControllerTest {
8
9     @Autowired
10     private MockMvc mockMvc;
11
12     @MockBean
13     private UserService userService;
14
15     @Test
16     void shouldReturnUserWhenExists() throws Exception {
17         User user = new User(1L, "ali");
18         when(userService.findUser(1L)).thenReturn(user);
19
20         mockMvc.perform(get("/api/users/1")
21             .contentType(MediaType.APPLICATION_JSON))
22             .andExpect(status().isOk())
23             .andExpect(jsonPath("$.username").value("ali"))
24             .andExpect(jsonPath("$.id").value(1));
25     }
```

1-Java Interview Question

```
26
27 @Test
28 void shouldReturn404WhenUserNotFound() throws Exception {
29     when(userService.findUser(999L))
30         .thenReturn(new UserNotFoundException());
31
32     mockMvc.perform(get("/api/users/999"))
33         .andExpect(status().isNotFound());
34 }
35 }
```

۱۶.۸.۲ تست لایه Service (با Mock)

```
1 import org.springframework.boot.test.context.SpringBootTest;
2 import org.springframework.boot.test.mock.mockito.MockBean;
3
4 @SpringBootTest
5 class UserServiceTest {
6
7     @MockBean
8     private UserRepository repository;
9
10    @Autowired
11    private UserService service;
12
13    @Test
14    void shouldRegisterNewUser() {
```

1-Java Interview Question

```
15 User newUser = new User("ali", "pass123");
16 when(repository.save(any(User.class)))
17     .thenReturn(newUser);
18
19 User result = service.register("ali", "pass123");
20
21 assertNotNull(result);
22 verify(repository).save(any(User.class));
23 }
24 }
```

۱۶.۸.۳ تست لایه Repository (با دیتابیس واقعی)

```
1 import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
2
3 @DataJpaTest
4 class UserRepositoryTest {
5
6     @Autowired
7     private UserRepository repository;
8
9     @Test
10    void shouldFindUserByUsername() {
11        User user = new User("ali", "pass");
12        repository.save(user);
13
14        Optional<User> found = repository.findByUsername("ali");
```

1-Java Interview Question

```
15
16     assertTrue(found.isPresent());
17     assertEquals("ali", found.get().getUsername());
18 }
19 }
```

١٦.٨.٤ تست Integration (کل سیستم)

```
1 import org.springframework.boot.test.context.SpringBootTest;
2 import org.springframework.boot.test.web.client.TestRestTemplate;
3 import org.springframework.boot.test.web.server.LocalServerPort;
4
5 @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
6 class IntegrationTest {
7
8     @LocalServerPort
9     private int port;
10
11     @Autowired
12     private TestRestTemplate restTemplate;
13
14     @Test
15     void shouldRegisterAndLoginUser() {
16         User user = new User("ali", "pass");
17         ResponseEntity<Void> registerResponse = restTemplate
18             .postForEntity("http://localhost:" + port + "/api/register",
19                 user, Void.class);
```

```

20 assertEquals(HttpStatus.CREATED, registerResponse.getStatusCode());
21
22 LoginRequest request = new LoginRequest("ali", "pass");
23 ResponseEntity<TokenResponse> loginResponse = restTemplate
24     .postForEntity("http://localhost:" + port + "/api/login",
25         request, TokenResponse.class);
26 assertEquals(HttpStatus.OK, loginResponse.getStatusCode());
27 assertNotNull(loginResponse.getBody().getToken());
28 }
29 }

```

1۶.۹ TDD (Test-Driven Development)

چرخه (Red-Green-Refactor):TDD



1-Java Interview Question

9	● REFACTOR (بازسازی)	
10	کد رو بهینه کن و تمیزش کن (تست‌ها باید سبز بمونن)	
11	↓	
12	(برگرد به مرحله ۱ برای ویژگی بعدی)	
13		
14		

مثال کامل TDD:

```
1 // a. ۱ مرحله: RED
2 @Test
3 void shouldReturnTrueForEvenNumber() {
4     assertTrue validator.isEven(4);
5 }
6
7 // b. ۲ مرحله: GREEN
8 public class NumberValidator {
9     public boolean isEven(int number) {
10         return number % 2 == 0;
11     }
12 }
13
14 // c. ۳ مرحله: REFACTOR
15 // (در صورت نیاز)
16
17 // ویژگی بعدی
18 @Test
```

```
19 void shouldReturnTrueForOddNumber() {  
20     assertTrue validator.isOdd(5);  
21 }
```

۱۶.۱۰ Coverage (پوشش تست)

سوال مصاحبه‌ای: Code Coverage چیست و چقدر باید باشد؟

جواب: درصد کدی که توسط تست‌ها پوشش داده شده. ابزارهایی مثل JaCoCo این رو اندازه می‌گیرن.

انواع Coverage:

• Line Coverage: چند درصد خطوط کد اجرا شدن

• Branch Coverage: چند درصد شرط‌ها (if/else) تست شدن

• Method Coverage: چند درصد متدها تست شدن

توصیه:

• هدف: ۷۰-۸۰٪ برای پروژه‌های معمولی

• ۱۰۰٪ همیشه ممکن نیست و گاهی هزینه‌بره

• کیفیت تست مهم‌تر از تعدادشه

```
1 <plugin>  
2   <groupId>org.jacoco</groupId>  
3   <artifactId>jacoco-maven-plugin</artifactId>  
4   <configuration>  
5     <rules>  
6       <rule>
```

```

7      <element>CLASS</element>
8      <limits>
9          <limit>
10         <counter>LINE</counter>
11         <value>COVEREDRATIO</value>
12         <minimum>0.80</minimum>
13     </limit>
14 </limits>
15 </rule>
16 </rules>
17 </configuration>
18 </plugin>

```

۱۶.۱۱ خلاصه یک صفحه ای برای مصاحبه

خلاصه مفاهیم کلیدی تست نرم افزار

مفهوم	خلاصه
Test Pyramid	Unit > Integration > E2E (هرچه بالاتر، تعداد تست ها کمتر)
FIRST	Fast, Independent, Repeatable, Self-Validating, Timely
Given-When-Then	ساختار استاندارد نوشتن تست (زمینه - اقدام - انتظار)
Boundary Values	همیشه مرزها رو تست کن (۱۸، ۶۰، ۰، ۱۲۰، ...)
Negative Testing	ورودی های غلط و غیرمنتظره رو هم تست کن
Mock vs Stub	Mock قابل برنامه ریزی با انتظار رفتاری، Stub پاسخ دهنده ساده
TDD	Red ← Green ← Refactor (چرخه توسعه تست محور)
JUnit	فریم ورک اصلی تست در جاوا

مفهوم	خلاصه
Mockito	کتابخانه اصلی Mock در جاوا
SpringBootTest@	تست کامل Spring Boot با بارگذاری کل Context
DataJpaTest@	تست لایه دیتابیس (تنها Repository ها و Hibernate)
WebMvcTest@	تست لایه Controller (تنها Web Layer)
Coverage	هدف: ۷۰-۸۰٪ (کیفیت مهم تر از درصد)

۱۶.۱۲ سوالات رایج مصاحبه‌ای تست

? سوال ۱: فرق Unit Test و Integration Test چیه؟

جواب:

- Unit Test: یک واحد کوچک رو به تنهایی تست میکنه. وابستگی‌ها رو Mock میکنه. سریع و ارزونه.
- Integration Test: ارتباط چند جزء رو با هم تست میکنه. از دیتابیس یا سرویس واقعی استفاده میکنه. کندتر و گرون‌تره.

? سوال ۲: چطور از تست‌های کند جلوگیری کنیم؟

جواب:

- تست‌های Unit رو از Integration جدا کن
- از @Profile("test") برای دیتابیس تستی استفاده کن
- از H2 (دیتابیس In-Memory) برای تست استفاده کن
- Mock کردن وابستگی‌های سنگین (مثل سرویس‌های خارجی)

? سوال ۳: فرق Mock@ و MockBean@ چیه؟

جواب:

•Mock@: از Mockito میاد و فقط به Mock ساده می‌سازه (بدون Spring)

•MockBean@: از Spring Boot میاد و به Mock رو توی Spring Context قرار میده (برای تست‌های Spring)

? سوال ۴: چطور به کلاس رو تست‌پذیر کنیم؟

جواب:

•وابستگی‌ها رو از طریق Constructor تزریق کن (نه new کردن داخل کلاس)

•از Interface‌ها استفاده کن تا بتونی Mock کنی

•متدها رو کوتاه و با یک مسئولیت بنویس

•از Static method‌ها تا حد امکان کمتر استفاده کن

1. JSP و Thymeleaf

یه فایل جدید درست کن به نام 17-Java-View-Technologies.md 

فاز ۱۷: تکنولوژی‌های View (JSP و Thymeleaf)

این بخش برای موقعی‌هایی که می‌خواهی یه برنامه وب با Frontend سمت سرور (Server-Side Rendering) بسازی.

۱۷.۱ Thymeleaf (موتور قالب‌سازی مدرن)

Thymeleaf یه موتور قالب‌سازی (Template Engine) برای برنامه‌های Spring Boot هست که به HTML شباهت داره.

۱۷.۱.۱ مزایای Thymeleaf

1. Natural Templating: قالب‌ها در مرورگر هم قابل مشاهده هستن

2. یکپارچگی عالی با Spring Boot

3. پشتیبانی از i18n (چندزبانه)

4. امنیت بالا (XSS Protection)

۱۷.۱.۲ تنظیمات

```
1 <!-- pom.xml -->
2 <dependency>
3   <groupId>org.springframework.boot</groupId>
4   <artifactId>spring-boot-starter-thymeleaf</artifactId>
5 </dependency>
6 # application.properties
7 spring.thymeleaf.prefix=classpath:/templates/
8 spring.thymeleaf.suffix=.html
9 spring.thymeleaf.cache=false # برای توسعه
10 spring.thymeleaf.encoding=UTF-8
```

Controller ۱۷.۱.۳

```
1 @Controller
2 @RequestMapping("/users")
3 class UserController {
4
5   @Autowired
6   private UserService userService;
7
8   @GetMapping("/list")
9   public String listUsers(Model model) {
10     List<User> users = userService.findAll();
11     model.addAttribute("users", users);
12     model.addAttribute("pageTitle", "لیست کاربران");
```

1-Java Interview Question

```
13     return "users/list"; // templates/users/list.html
14 }
15
16 @GetMapping("/{id}")
17 public String getUser(@PathVariable Long id, Model model) {
18     User user = userService.findById(id);
19     model.addAttribute("user", user);
20     return "users/profile";
21 }
22
23 @GetMapping("/new")
24 public String newUserForm(Model model) {
25     model.addAttribute("user", new User());
26     return "users/form";
27 }
28
29 @PostMapping("/save")
30 public String saveUser(@ModelAttribute User user) {
31     userService.save(user);
32     return "redirect:/users/list";
33 }
34 }
```


Thymeleaf قالب ۱۷.۱.۴

```

1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org">
3 <head>
4   <title th:text="${pageTitle}">عنوان پیش فرض</title>
5 </head>
6 <body>
7   <!-- نمایش متغیر ۱. -->
8   <h1 th:text="${pageTitle}">عنوان</h1>
9
10  <!-- (if) شرط ۲. -->
11  <div th:if="${user != null}">
12    <p>کاربر <span th:text="${user.username}"></span>، خوش آمدی</p>
13  </div>
14
15  <!-- (unless - if برعکس) شرط ۳. -->
16  <div th:unless="${user != null}">
17    <p>لطفاً وارد شوید</p>
18  </div>
19
20  <!-- حلقه (forEach) ۴. -->
21  <table>
22    <tr th:each="user : ${users}">

```

1-Java Interview Question

```
23     <td th:text="${user.id}">1</td>
24     <td th:text="${user.username}">علی</td>
25     <td th:text="${user.age}">25</td>
26     <td>
27         <!-- لینک ۵. -->
28         <a th:href="@{/users/{id}(id=${user.id})}">مشاهده</a>
29     </td>
30 </tr>
31 </table>
32
33 <!-- فرم ۶. -->
34 <form th:action="@{/users/save}" th:object="${user}" method="post">
35     <input type="text" th:field="*{username}" placeholder="نام کاربری"/>
36     <input type="password" th:field="*{password}" placeholder="رمز عبور"/>
37     <input type="number" th:field="*{age}" placeholder="سن"/>
38     <button type="submit">ذخیره</button>
39 </form>
40
41 <!-- نمایش خطاهای اعتبارسنجی ۷. -->
42 <div th:if="${#fields.hasErrors('username')}">
43     <span th:errors="*{username}">خطای اعتبارسنجی</span>
44 </div>
45
46 <!-- شامل کردن قالب دیگه ۸. -->
47 <div th:include="fragments/header :: header"></div>
48
```

1-Java Interview Question

```
49 <!-- مقادیر پیش فرض ۹. -->
50 <p th:text="\${user?.username} ?: 'کاربر مهمان'"></p>
51
52 <!-- عملیات ریاضی و منطقی ۱۰. -->
53 <p th:text="\${user.age + 5}">0</p>
54 <p th:text="\${user.age > 18} ? 'بزرگسال' : 'کودک'"></p>
55 </body>
56 </html>
```

۱۷.۱.۵ Fragment ها (اجزای قابل استفاده مجدد)

```
1 <!-- fragments/header.html -->
2 <div th:fragment="header">
3   <header>
4     <h1>سایت من</h1>
5     <nav>
6       <a th:href="@{/}">خانه</a>
7       <a th:href="@{/users/list}">کاربران</a>
8     </nav>
9   </header>
10 </div>
11
12 <!-- استفاده در صفحه -->
13 <div th:replace="fragments/header :: header"></div>
```

```

1 # messages_fa.properties
2 welcome=خوش آمدید
3 username=نام کاربری
4 password=رمز عبور
5
6 # messages_en.properties
7 welcome=Welcome
8 username=Username
9 password=Password
10 <!-- in html -->
11 <h1 th:text="#{welcome}">Welcome</h1>
12 <label th:text="#{username}">Username</label>

```

```

1 <dependency>
2   <groupId>org.thymeleaf.extras</groupId>
3   <artifactId>thymeleaf-extras-springsecurity6</artifactId>
4 </dependency>
5 <!-- Display logged in user information -->
6 <!-- نمایش اطلاعات کاربر لاگین شده است -->
7 <div sec:authorize="isAuthenticated()">
8   <p><span sec:authentication="name">کاربر</span>، خوش آمدی</p>
9   <a sec:authorize="hasRole('ADMIN')" href="/admin">پنل مدیریت</a>
10 </div>

```

11

12 <div sec:authorize="!isAuthenticated()">

13 ورود

14 </div>

JSP (JavaServer Pages) ۱۷.۲

JSP به تکنولوژی قدیمی‌تر ولی هنوز در برخی پروژه‌ها استفاده می‌شود.

۱۷.۲.۱ تنظیمات

```
1 <!-- pom.xml -->
2 <dependency>
3   <groupId>org.apache.tomcat.embed</groupId>
4   <artifactId>tomcat-embed-jasper</artifactId>
5   <scope>provided</scope>
6 </dependency>
7 <dependency>
8   <groupId>javax.servlet</groupId>
9   <artifactId>jstl</artifactId>
10 </dependency>
11 # application.properties
12 spring.mvc.view.prefix=/WEB-INF/views/
13 spring.mvc.view.suffix=.jsp
```

JSP Controller برای ۱۷.۲.۲

```

1 @Controller
2 @RequestMapping("/users")
3 class UserController {
4
5     @GetMapping("/list")
6     public String listUsers(Model model) {
7         List<User> users = userService.findAll();
8         model.addAttribute("users", users);
9         return "users/list"; // /WEB-INF/views/users/list.jsp
10    }
11 }

```

JSP قالب (با JSTL) ۱۷.۲.۳

```

1 <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
3 <%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
4 <!DOCTYPE html>
5 <html>
6 <head>
7     <title>لیست کاربران</title>
8 </head>
9 <body>
10     <h1>${pageTitle}</h1>
11

```

1-Java Interview Question

```
12 <!-- ۱. شرط -->
13 <c:if test="{not empty user}">
14     <p>خوش آمدی، ${user.username}</p>
15 </c:if>
16
17 <!-- ۲. حلقه --->
18 <table>
19     <c:forEach items="{users}" var="user">
20         <tr>
21             <td>${user.id}</td>
22             <td>${user.username}</td>
23             <td>${user.age}</td>
24             <td>
25                 <a href="/users/${user.id}">مشاهده</a>
26             </td>
27         </tr>
28     </c:forEach>
29 </table>
30
31 <!-- ۳. فرم --->
32 <form action="/users/save" method="post">
33     <input type="text" name="username" placeholder="نام کاربری"/>
34     <input type="password" name="password" placeholder="رمز عبور"/>
35     <input type="number" name="age" placeholder="سن"/>
36     <button type="submit">ذخیره</button>
37 </form>
```

38

39 <!-- بررسی خالی بودن ۴. -->

40 <c:if test="{empty users}">

41 <p>هیچ کاربری وجود ندارد</p>

42 </c:if>

43 </body>

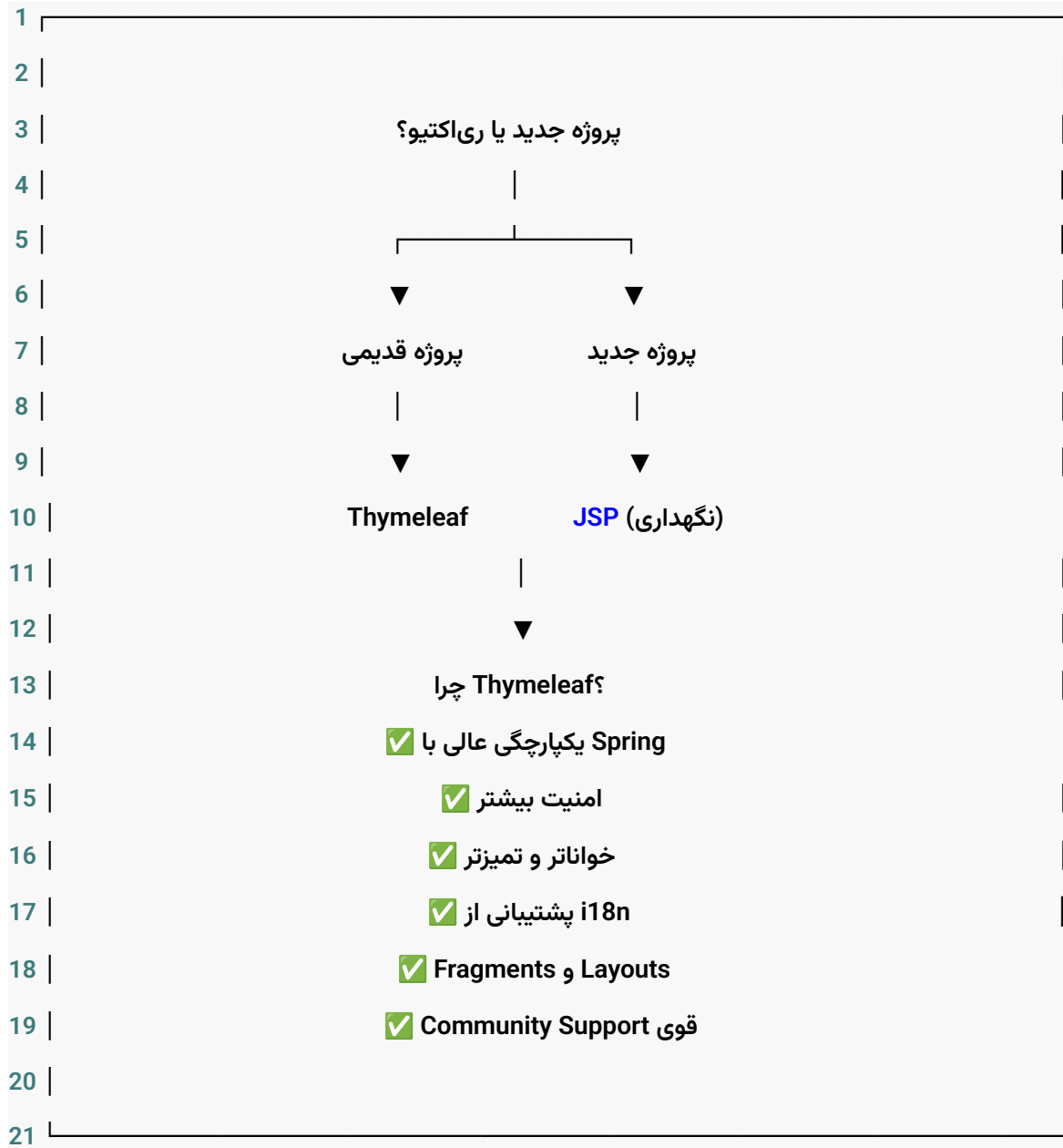
44 </html>

۱۷.۲.۴ مقایسه JSP و Thymeleaf

مقایسه JSP و Thymeleaf

ویژگی	Thymeleaf	JSP
تازه بودن	مدرن (+2011)	قدیمی (+1999)
یکپارچگی با Spring Boot	عالی ✓	متوسط
Natural Templating	بله ✓	خیر ✗
ایمنی در برابر XSS	خودکار ✓	دستی ✗
پشتیبانی از Layout	عالی ✓	ضعیف
سرعت	متوسط	سریع تر
یادگیری	آسان	سخت تر
مناسب برای	پروژه های جدید	پروژه های قدیمی

۱۷.۳ خلاصه انتخاب View Technology



📖 فاز ۱۸: Gradle و Maven – مدیریت وابستگی‌ها

و Build

Gradle و Maven دو ابزار اصلی برای مدیریت پروژه‌های جاوا هستند. در مصاحبه‌ها معمولاً ازت می‌پرسن با کدوم کار کردی و فرقشون چیه.

۱۸.۱ Maven (محبوب‌ترین)

ساختار پروژه Maven

```
1 my-app/
2 |— pom.xml
3 |— src/
4 |   |— main/
5 |   |   |— java/      ← کدهای جاوا
6 |   |   |— resources/ ← فایل‌های تنظیمات
7 |   |— test/
8 |   |   |— java/      ← تست‌ها
9 |   |   |— resources/ ← منابع تست
10 |— target/           ← build خروجی
```

فایل pom.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0">
3
4   <!-- ۱. اطلاعات پروژه -->
5   <modelVersion>4.0.0</modelVersion>
6   <groupId>com.example</groupId>
7   <artifactId>myapp</artifactId>
8   <version>1.0.0</version>
9   <packaging>jar</packaging>
10  <name>My Application</name>
11  <description>توضیحات پروژه</description>
12
13  <!-- ۲. والد (Spring Boot برای) تنظیمات والد -->
14  <parent>
15    <groupId>org.springframework.boot</groupId>
16    <artifactId>spring-boot-starter-parent</artifactId>
17    <version>3.1.0</version>
18    <relativePath/>
19  </parent>
20
21  <!-- ۳. Properties (متغیرهای سراسری) -->
22  <properties>
```

1-Java Interview Question

```
23     <java.version>17</java.version>
24     <maven.compiler.source>17</maven.compiler.source>
25     <maven.compiler.target>17</maven.compiler.target>
26     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
27 </properties>
28
29 <!-- وابستگی ها ۴. -->
30 <dependencies>
31     <dependency>
32         <groupId>org.springframework.boot</groupId>
33         <artifactId>spring-boot-starter-web</artifactId>
34     </dependency>
35     <dependency>
36         <groupId>org.springframework.boot</groupId>
37         <artifactId>spring-boot-starter-data-jpa</artifactId>
38     </dependency>
39     <dependency>
40         <groupId>mysql</groupId>
41         <artifactId>mysql-connector-java</artifactId>
42         <scope>runtime</scope>
43     </dependency>
44     <dependency>
45         <groupId>org.projectlombok</groupId>
46         <artifactId>lombok</artifactId>
47         <optional>true</optional>
48     </dependency>
```

```

49     <dependency>
50         <groupId>org.springframework.boot</groupId>
51         <artifactId>spring-boot-starter-test</artifactId>
52         <scope>test</scope>
53     </dependency>
54 </dependencies>
55
56 <!-- ۵. Build settings -->
57 <build>
58     <plugins>
59         <plugin>
60             <groupId>org.springframework.boot</groupId>
61             <artifactId>spring-boot-maven-plugin</artifactId>
62             <configuration>
63                 <excludes>
64                     <exclude>
65                         <groupId>org.projectlombok</groupId>
66                         <artifactId>lombok</artifactId>
67                     </exclude>
68                 </excludes>
69             </configuration>
70         </plugin>
71
72         <!-- جاوا کامپایلر -->
73         <plugin>
74             <groupId>org.apache.maven.plugins</groupId>

```

1-Java Interview Question

```
75     <artifactId>maven-compiler-plugin</artifactId>
76     <configuration>
77         <source>17</source>
78         <target>17</target>
79     </configuration>
80 </plugin>
81
82 <!-- JaCoCo (كأورج تست) -->
83 <plugin>
84     <groupId>org.jacoco</groupId>
85     <artifactId>jacoco-maven-plugin</artifactId>
86     <version>0.8.10</version>
87 </plugin>
88 </plugins>
89 </build>
90
91 <!-- مخازن (Repository) -->
92 <repositories>
93     <repository>
94         <id>central</id>
95         <url>https://repo.maven.apache.org/maven2</url>
96     </repository>
97 </repositories>
98 </project>
```

1 پاک کردن خروجی‌های قبلی # a.

2 mvn clean

3

4 کامپایل کردن کد # b.

5 mvn compile

6

7 اجرای تست‌ها # c.

8 mvn test

9

10 ساختن JAR/WAR # d.

11 mvn package

12

13 نصب در مخزن محلی # e.

14 mvn install

15

16 اجرای برنامه /Spring Boot # f.

17 mvn spring-boot:run

18

19 اجرا با پروفایل خاص # g.

20 mvn spring-boot:run -Dspring-boot.run.profiles=dev

21

22 ساخت با اسکیپ تست # h.

23 mvn clean install -DskipTests

24

25 مشاهده درخت وابستگی‌ها # i.

1-Java Interview Question

26 mvn dependency:tree

27

28 # j. بروزرسانی وابستگی ها

29 mvn versions:display-dependency-updates

30

31 # k. همه کارها با هم

32 mvn clean install -DskipTests

Maven Scope های

```
1 <!-- scope مشخص می‌کند وابستگی کی در دسترس باشه -->
2 <dependency>
3   <groupId>...</groupId>
4   <artifactId>...</artifactId>
5   <scope>provided</scope> <!-- فراهم میشه Container یا JDK توسط -->
6 </dependency>
7
8 <!-- scope ها -->
9 <!-- compile: پیش فرض، در همه جا در دسترس -->
10 <!-- provided: مثل servlet-api (نمیداد JAR ولی توی compile مثل -->
11 <!-- runtime: در دسترس runtime فقط در -->
12 <!-- test: فقط برای تست -->
13 <!-- system: (کمتر استفاده میشه) سیستم از فایل -->
```


۱۸.۲ Gradle (جایگزین مدرن Maven)

ساختار پروژه Gradle

```

1 my-app/
2 |— build.gradle
3 |— settings.gradle
4 |— src/
5 |   |— main/
6 |   |   |— java/
7 |   |   |   |— resources/
8 |   |   |— test/
9 |   |   |   |— java/
10 |   |   |   |— resources/
11 |— build/      ← خروجی build

```

فایل build.gradle

```

1 // a. پلاگین‌ها
2 plugins {
3     id 'java'
4     id 'org.springframework.boot' version '3.1.0'
5     id 'io.spring.dependency-management' version '1.1.0'
6 }
7
8 // b. اطلاعات پروژه
9 group = 'com.example'

```

1-Java Interview Question

```
10 version = '1.0.0'
11 sourceCompatibility = '17'
12
13 // c. مخازن
14 repositories {
15     mavenCentral()
16 }
17
18 // d. وابستگی‌ها
19 dependencies {
20     // Spring Boot Starters
21     implementation 'org.springframework.boot:spring-boot-starter-web'
22     implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
23
24     // Database
25     runtimeOnly 'mysql:mysql-connector-java'
26
27     // Lombok
28     compileOnly 'org.projectlombok:lombok'
29     annotationProcessor 'org.projectlombok:lombok'
30
31     // Test
32     testImplementation 'org.springframework.boot:spring-boot-starter-test'
33     testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
34 }
35
```

```
36 // e. Test configuration
37 test {
38     useJUnitPlatform()
39 }
40
41 // f. Custom tasks
42 task hello {
43     doLast {
44         println 'Hello from Gradle!'
45     }
46 }
47
48 rootProject.name = 'myapp'
```

دستورات مهم Gradle

```
1 # a. مشاهده همه tasks
2 ./gradlew tasks
3
4 # b. پاک کردن
5 ./gradlew clean
6
7 # c. ساختن
8 ./gradlew build
9
```

- 10 # d. *اجرا (با اسکپ تست)*
- 11 `./gradlew build -x test`
- 12
- 13 # e. *اجرای برنامه*
- 14 `./gradlew bootRun`
- 15
- 16 # f. *اجرا با پروفایل خاص*
- 17 `./gradlew bootRun --args='--spring.profiles.active=dev'`
- 18
- 19 # i. *مشاهده درخت وابستگی ها*
- 20 `./gradlew dependencies`
- 21
- 22 # j. *بروزرسانی وابستگی ها*
- 23 `./gradlew dependencyUpdates`

۱۸.۳ مقایسه Gradle و Maven

مقایسه Gradle و Maven

ویژگی	Maven	Gradle
زبان	XML	Groovy/Kotlin (کوتاه تر)
سرعت	کندتر (به خاطر XML)	سریع تر (به خاطر Incremental Build)
یادگیری	آسان تر	سخت تر
وابستگی ها	خوب	عالی
قابلیت گسترش	محدود	بسیار زیاد
محبوبیت	بیشتر	در حال رشد

ویژگی
مناسب برای

Maven
پروژه‌های ساده

Gradle
پروژه‌های بزرگ

مثال مقایسه (همان کار با دو ابزار)

:Maven (pom.xml)

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-web</artifactId>
4 </dependency>
```

5 Gradle (build.gradle):

```
6 implementation 'org.springframework.boot:spring-boot-starter-web'
```

۱۸.۴ ! سوالات مصاحبه‌ای

? سوال ۱: Maven چیه و چه کارایی داره؟

جواب: Maven به ابزار مدیریت و ساخت (Build Tool) برای پروژه‌های جاواست که:

• وابستگی‌ها رو مدیریت می‌کنه

• پروژه رو Build می‌کنه

• چرخه حیات (Lifecycle) استاندارد داره

• با pom.xml پیکربندی میشه

? سوال ۲: فرق compile و provided و runtime و test چیه؟

جواب:

اسکوپ‌های (Scope) وابستگی در Maven

Scope	توضیح
compile	پیش فرض، در همه جا در دسترس (هم در کامپایل، هم در اجرا و هم در تست)
provided	مثل compile ولی در JAR نهایی نمیاد (مثل Servlet API که توسط سرور تأمین می‌شود)
runtime	فقط در زمان اجرا نیاز است (مثل JDBC Driver) - در کامپایل نیاز نیست
test	فقط برای تست‌ها در دسترس است (مثل JUnit، Mockito)
system	از فایل سیستم محلی بارگذاری می‌شود (کمتر استفاده میشه و وابستگی به مسیر سیستمی دارد)

? سوال ۳: چرخه حیات Maven رو توضیح بده

جواب: سه تا چرخه حیات اصلی داره:

default: validate → compile → test → package → verify → install → deploy•

clean: pre-clean → clean → post-clean•

site: pre-site → site → post-site → site-deploy•

? سوال ۴: فرق Gradle و Maven چیه؟

جواب:

• Maven با XML پیکربندی میشه، Gradle با Groovy/Kotlin

• Gradle سریع‌تره (Incremental Build)

• Gradle انعطاف پذیرتره

• Maven ساده تر برای شروع

• هر دو برای مدیریت وابستگی ها و Build هستند

فایل جدید: 19-Java-Logging.md 

فاز ۱۹: لاگ‌گیری (Logging) در جاوا

لاگ‌گیری یکی از مهم‌ترین کارها برای Debugging، Monitoring و عیب‌یابی در برنامه‌های واقعی. توی مصاحبه ازت می‌پرسن با چه ابزاری کار کردی و چرا.

۱۹.۱ چرا لاگ؟

1. Debugging: پیدا کردن باگ‌ها

2. Monitoring: بررسی وضعیت برنامه

3. Audit: ثبت فعالیت‌های کاربران

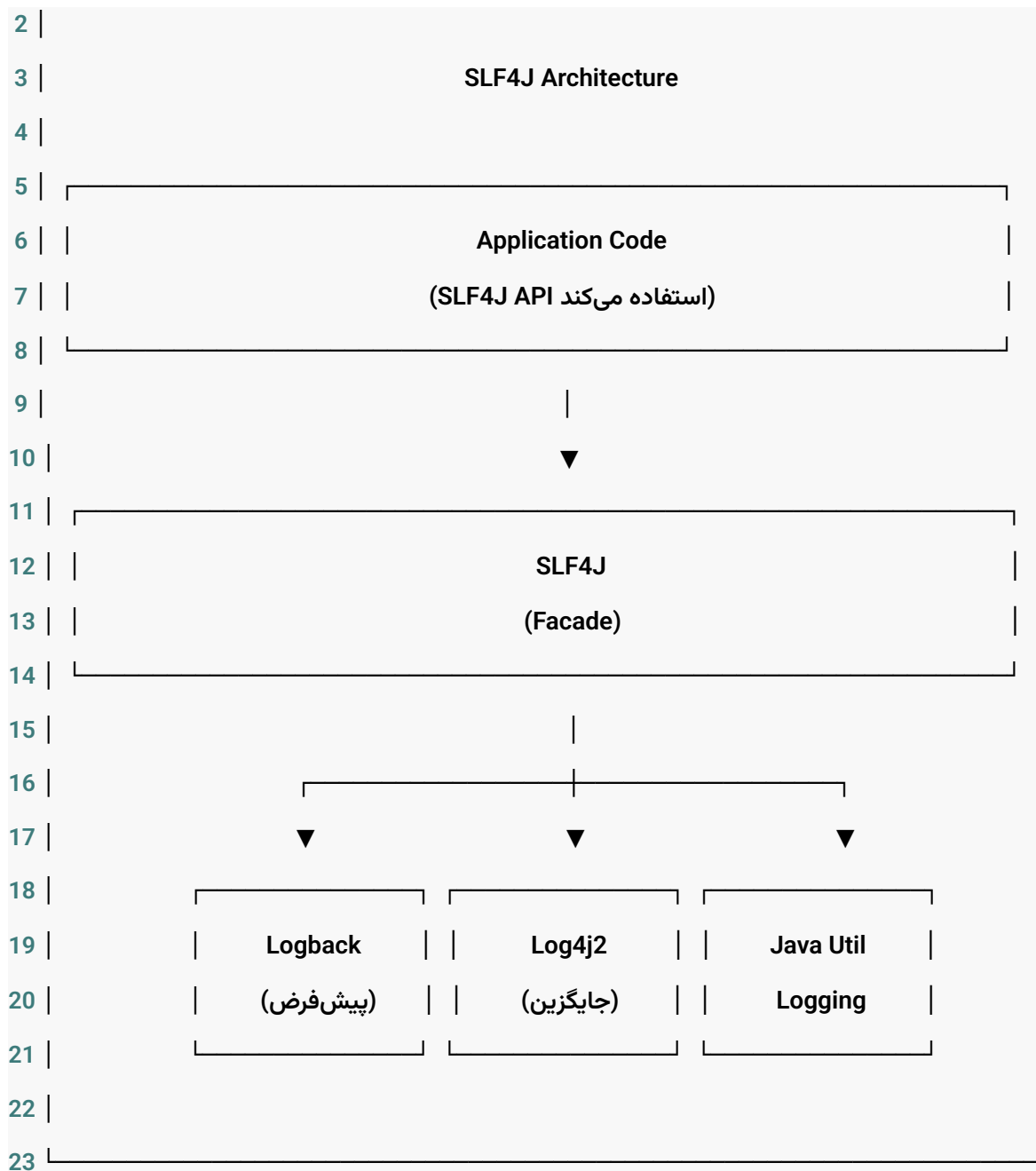
4. Troubleshooting: عیب‌یابی مشکلات

5. Performance: بررسی عملکرد

۱۹.۲ SLF4J (Simple Logging Facade for Java)

SLF4J یه Facade (رابط) برای لاگ‌گیری هست. یعنی خودش لاگ نمی‌نویسه، بلکه به کتابخانه‌های دیگه وصل میشه.

1-Java Interview Question



مزایای SLF4J

• وابستگی کم: می‌تونی هر کتابخانه‌ای رو استفاده کنی

• ساختار یکسان: فرقی نمی‌کنه از Logback یا Log4j استفاده کنی، کد یکسانه

• پارامتریشدن: با {} راحت تر

۱۹.۳ Logback (پیش فرض Spring Boot)

تنظیمات در application.properties

```

1 # a. سطح لاگ برای پکیج ها
2 logging.level.root=INFO
3 logging.level.com.example=DEBUG
4 logging.level.org.springframework=INFO
5 logging.level.org.hibernate=WARN
6 logging.level.org.hibernate.SQL=DEBUG
7 logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
8
9 # b. فایل لاگ
10 logging.file.name=logs/myapp.log
11 logging.file.path=logs/
12
13 # c. فرمت لاگ
14 logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n
15 logging.pattern.file=%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n
16
17 # d. گروه های لاگ (برای گروه بندی)
18 logging.group.db=org.hibernate,org.springframework.jdbc
19 logging.level.db=DEBUG

```

فایل logback-spring.xml (تنظیمات پیشرفته)

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <configuration>
3
4 <!-- 1. Console Appender -->
5 <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
6   <encoder>
7     <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
8   </encoder>
9 </appender>
10
11 <!-- ۲. File Appender -->
12 <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
13   <file>logs/myapp.log</file>
14
15   <!-- چرخش فایل بر اساس زمان -->
16   <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
17     <fileNamePattern>logs/myapp.%d{yyyy-MM-dd}.log</fileNamePattern>
18     <maxHistory>30</maxHistory> <!-- روز نگهداری ۳۰ -->
19   </rollingPolicy>
20
21   <encoder>
22     <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
23   </encoder>
24 </appender>
25

```

```

26 <!-- ۳. File Appender با اندازه محدود -->
27 <appender name="FILE_SIZE" class="ch.qos.logback.core.rolling.RollingFileAppender">
28     <file>logs/myapp.log</file>
29     <rollingPolicy class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
30         <fileNamePattern>logs/myapp.%d{yyyy-MM-dd}.%i.log</fileNamePattern>
31         <maxFileSize>10MB</maxFileSize>
32         <maxHistory>30</maxHistory>
33         <totalSizeCap>1GB</totalSizeCap>
34     </rollingPolicy>
35     <encoder>
36         <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
37     </encoder>
38 </appender>
39
40 <!-- ۴. JSON Formatter ( برای ELK ) -->
41 <appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
42     <encoder class="net.logstash.logback.encoder.LogstashEncoder">
43         <includeMdc>true</includeMdc>
44     </encoder>
45 </appender>
46
47 <!-- ۵. تنظیم سطح لاگ برای پکیج های مختلف -->
48 <logger name="com.example" level="DEBUG"/>
49 <logger name="org.springframework" level="INFO"/>
50 <logger name="org.hibernate" level="WARN"/>
51 <logger name="com.example.security" level="DEBUG"/>

```

```

52
53 <!-- ۶. Root Logger -->
54 <root level="INFO">
55     <appender-ref ref="CONSOLE"/>
56     <appender-ref ref="FILE"/>
57 </root>
58
59 <!-- ۷. در محیط خاص JSON فعال کردن -->
60 <springProfile name="prod">
61     <root level="INFO">
62         <appender-ref ref="JSON"/>
63     </root>
64 </springProfile>
65
66 </configuration>

```

۱۹.۴ استفاده از SLF4J در کد

روش ۱: با Lombok (توصیه شده)

```

1 import lombok.extern.slf4j.Slf4j;
2
3 @Service
4 @Slf4j
5 public class UserService {
6

```

1-Java Interview Question

```
7 public void processUser(Long userId) {
8     // ۱. اطلاعات
9     log.info("Processing user: {}", userId);
10
11     // ۲. خطا
12     try {
13         // کد...
14     } catch (Exception e) {
15         log.error("Error processing user: {}", userId, e);
16     }
17
18     // ۳. هشدار
19     if (userId < 0) {
20         log.warn("Invalid user ID: {}", userId);
21     }
22
23     // ۴. دیباگ
24     log.debug("User processing started: {}", userId);
25
26     // ۵. trace (جزئی ترین)
27     log.trace("User processing step 1");
28
29     // ۶. بدون پارامتر
30     log.info("Method called");
31 }
32 }
```

روش ۲: بدون Lombok

```

1 import org.slf4j.Logger;
2 import org.slf4j.LoggerFactory;
3
4 @Service
5 public class UserService {
6     private static final Logger log = LoggerFactory.getLogger(UserService.class);
7
8     public void processUser(Long userId) {
9         log.info("Processing user: {}", userId);
10    }
11}

```

روش ۳: Dynamic Logging (با پارامترهای شرطی)

```

1 @Service
2 @Slf4j
3 public class UserService {
4
5     public void processUser(Long userId) {
6         // چک کردن سطح لاگ (برای بهینه سازی)
7         if (log.isDebugEnabled()) {
8             log.debug("Processing user: {}", userId);
9         }
10
11     // با چند پارامتر

```

1-Java Interview Question

```
12 String username = "ali";
13 int age = 25;
14 log.info("User: {}, Age: {}", username, age);
15
16 // ↳ exception
17 try {
18     // ...
19 } catch (Exception e) {
20     log.error("Error occurred", e);
21 }
22 }
23 }
```

۱۹.۵ سطوح لاگ (Log Levels)

1	
2	
3	از کمترین به بیشترین Log Levels
4	
5	
6	خطاهای جدی که برنامه رو متوقف می‌کنن ← ERROR
7	
8	
9	وضعیت‌های غیرعادی ولی قابل بازیابی ← WARN
10	
11	
12	اطلاعات عمومی (کاربر ثبت شد، سفارش ایجاد شد) ← INFO

1-Java Interview Question

13			
14			
15		اطلاعات برای دیباگ (ورودی متدها، مقادیر) ← DEBUG	
16			
17			
18		جزئی ترین اطلاعات (هر قدم کوچک) ← TRACE	
19			
20			
21		هر سطح، سطوح بالاتر از خودش رو هم شامل میشه	
22		DEBUG و TRACE (نه هست INFO, WARN, ERROR شامل INFO مثلاً)	
23			
24			

مثال: سطوح در عمل

```
1 @Slf4j
2 public class LogExample {
3
4     public void divide(int a, int b) {
5         log.trace("divide called with a={}, b={}", a, b);
6
7         if (b == 0) {
8             log.error("Cannot divide by zero! a={}, b={}", a, b);
9             throw new ArithmeticException("Division by zero");
10        }
11
12        if (a < 0) {
```

1-Java Interview Question

```
13     log.warn("Dividing negative number: {}", a);
14 }
15
16 int result = a / b;
17 log.debug("Result: {}", result);
18 log.info("Division completed: {} / {} = {}", a, b, result);
19 }
20 }
```

MDC (Mapped Diagnostic Context) ۱۹.۶

MDC برای ذخیره اطلاعات مرتبط با هر درخواست (مثل User ID, Request ID) مفیده.

```
1 import org.slf4j.MDC;
2
3 @Service
4 @Slf4j
5 public class UserService {
6
7     public void processUser(Long userId) {
8         // ۱. ذخیره در MDC
9         MDC.put("userId", userId.toString());
10        MDC.put("requestId", UUID.randomUUID().toString());
11
12        try {
13            log.info("Processing user");
14            // کد پردازش...
```

1-Java Interview Question

```
15     log.info("User processed successfully");
16 } finally {
17     // ۲. پاک کردن MDC (مهم!)
18     MDC.clear();
19 }
20 }
21 }
```

پیگر بندی MDC در logback

```
1 <encoder>
2   <pattern>%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - [%X{userId}] [%X{requestId}] -
   %msg%n</pattern>
3 </encoder>
```

Filter برای پر کردن MDC خودکار

```
1 @Component
2 @Order(Ordered.HIGHEST_PRECEDENCE)
3 public class MDCFilter implements Filter {
4
5     @Override
6     public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) {
7         try {
8             MDC.put("requestId", UUID.randomUUID().toString());
9             // اگر کاربر لاگین هست
10            Authentication auth = SecurityContextHolder.getContext().getAuthentication();
11            if (auth != null && auth.isAuthenticated()) {
```

```

12     MDC.put("username", auth.getName());
13 }
14     chain.doFilter(request, response);
15 } finally {
16     MDC.clear();
17 }
18 }
19 }

```

۱۹.۷ ! سوالات مصاحبه‌ای

? سوال ۱: SLF4J چیه و چرا ازش استفاده می‌کنیم؟

جواب: SLF4J یه Facade برای کتابخانه‌های لاگ‌گیری هست. مزایا:

- وابستگی به کتابخانه خاصی نداریم
- می‌تونیم کتابخانه لاگ رو عوض کنیم بدون تغییر کد
- API یکسان داره
- پشتیبانی از پارامتریشدن ({})

? سوال ۲: فرق Logback و Log4j2 چیه؟

جواب:

مقایسه Logback و Log4j2

ویژگی	Logback	Log4j2
وضعیت در Spring Boot	پیش فرض Spring Boot	جایگزین Logback
سرعت	سریع	سریع تر
سادگی	ساده تر	پیچیده تر
قابلیت ها	خوب	قابلیت های بیشتر (Async, Lambda) نکات کلیدی:

• Logback به عنوان جانشین Log4j نسخه 1 طراحی شده و به صورت پیش فرض در Spring Boot استفاده می شود.

• Log4j2 نسخه بازطراحی شده Log4j است که از نظر عملکرد (به ویژه با قابلیت Async) از Logback سریع تر است.

• Log4j2 از Lambda Expressions برای Lazy Logging پشتیبانی می کند که می تواند بهبود عملکرد داشته باشد.

• انتخاب بین این دو معمولاً به نیاز پروژه و ترجیح تیم بستگی دارد؛ هر دو کتابخانه قدرتمندی هستند.

? سوال ۳: سطوح لاگ رو به ترتیب از کمترین به بیشترین بگو

جواب:

1 TRACE < DEBUG < INFO < WARN < ERROR

هر سطح، سطوح بالاتر رو هم شامل میشه.

? سوال ۴: چطور لاگ رو در فایل ذخیره کنیم؟

جواب: با تنظیمات logback-spring.xml یا application.properties:

1 `logging.file.name=logs/myapp.log`

? سوال ۵: MDC چیه و چه کاربردی داره؟

جواب: MDC (Mapped Diagnostic Context) یه مکانیزم برای ذخیره اطلاعات مرتبط با هر درخواست (مثل User ID, Request ID) در Context لاگ‌گیری هست. این اطلاعات می‌تونن توی همه لاگ‌های اون درخواست ظاهر بشن.

فاز ۲۰: مفاهیم پیشرفته Spring – Caching

Async, Scheduling, Validation

این بخش شامل مباحث پیشرفته‌تر Spring هست که توی پروژه‌های واقعی خیلی استفاده میشن.

۲۰.۱ Caching (کش کردن)

۲۰.۱.۱ فعال‌سازی Caching

```
1 @Configuration
2 @EnableCaching
3 public class CacheConfig {
4
5     @Bean
6     public CacheManager cacheManager() {
7         return new ConcurrentMapCacheManager("users", "orders", "products");
8     }
9 }
```

۲۰.۱.۲ Cacheable@ (ذخیره در کش)

1-Java Interview Question

```
1 @Service
2 @Slf4j
3 public class UserService {
4
5     @Autowired
6     private UserRepository repository;
7
8     // ۱. ساده
9     @Cacheable("users")
10    public User findUser(Long id) {
11        log.info("Fetching user from database: {}", id);
12        return repository.findById(id).orElseThrow();
13    }
14
15    // ۲. با کلید سفارشی
16    @Cacheable(value = "users", key = "#username")
17    public User findByUsername(String username) {
18        return repository.findByUsername(username);
19    }
20
21    // ۳. با کلید ترکیبی
22    @Cacheable(value = "users", key = "#username + ':' + #password")
23    public User findByUsernameAndPassword(String username, String password) {
24        return repository.findByUsernameAndPassword(username, password);
25    }
26
```


1-Java Interview Question

```
27  // ۴. باشد  $id > 0$  فقط اگر (با شرط ۴)
28  @Cacheable(value = "users", condition = "#id > 0")
29  public User findUserWithCondition(Long id) {
30      return repository.findById(id).orElseThrow();
31  }
32
33  // ۵. (نداشتن) با شرط
34  @Cacheable(value = "users", unless = "#result == null")
35  public User findUserUnlessNull(Long id) {
36      return repository.findById(id).orElse(null);
37  }
38 }
```

۲۰.۱.۳ CacheEvict@ پاک کردن کش)

```
1  @Service
2  public class UserService {
3
4      // ۱. پاک کردن یک کلید خاص
5      @CacheEvict(value = "users", key = "#id")
6      public void deleteUser(Long id) {
7          repository.deleteById(id);
8      }
9
10     // ۲. پاک کردن همه کش‌های مربوط
11     @CacheEvict(value = "users", allEntries = true)
12     public void clearAllCache() {
```

1-Java Interview Question

```
13    // پاک میشن users همه کش های
14 }
15
16    // پاک کردن قبل از اجرای متد ۳.
17    @CacheEvict(value = "users", key = "#id", beforeInvocation = true)
18    public void updateUser(User user) {
19        // قبل از اجرا کش پاک میشه
20        repository.save(user);
21    }
22 }
```

۲۰.۱.۴ CachePut@ (به روزرسانی کش)

```
1 @Service
2 public class UserService {
3
4    // همیشه کش رو به روز می کنه (حتی اگه متد خطا بده)
5    @CachePut(value = "users", key = "#user.id")
6    public User updateUser(User user) {
7        return repository.save(user);
8    }
9 }
```

۲۰.۱.۵ Redis با Caching

1-Java Interview Question

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-data-redis</artifactId>
4 </dependency>
5 spring.redis.host=localhost
6 spring.redis.port=6379
7 spring.redis.password=
8 spring.redis.timeout=2000
9
10 spring.cache.type=redis
11 spring.cache.redis.time-to-live=600000 # ١٠ دقيقه
12 spring.cache.redis.cache-null-values=false
13 @Configuration
14 @EnableCaching
15 public class RedisCacheConfig {
16
17     @Bean
18     public RedisCacheConfiguration cacheConfiguration() {
19         return RedisCacheConfiguration.defaultCacheConfig()
20             .entryTtl(Duration.ofMinutes(10))
21             .disableCachingNullValues()
22             .serializeValuesWith(RedisSerializationContext.SerializationPair
23                 .fromSerializer(new GenericJackson2JsonRedisSerializer()));
24     }
25 }
```

۲۰.۲ Async (پردازش ناهمگام)

۲۰.۲.۱ فعال سازی Async

```

1 @Configuration
2 @EnableAsync
3 public class AsyncConfig {
4
5     @Bean(name = "taskExecutor")
6     public Executor taskExecutor() {
7         ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
8         executor.setCorePoolSize(5);    // های اولیه Thread
9         executor.setMaxPoolSize(10);    // ها حداکثر Thread
10        executor.setQueueCapacity(100); // صف انتظار
11        executor.setThreadNamePrefix("Async-");
12        executor.initialize();
13        return executor;
14    }
15}

```

Async@ ۲۰.۲.۲

```

1 @Service
2 @Slf4j
3 public class EmailService {
4

```

1-Java Interview Question

```
5  // ۱. ساده
6  @Async
7  public void sendEmail(String to, String body) {
8      log.info("Sending email to: {} on thread: {}", to, Thread.currentThread().getName());
9      // کد ارسال ایمیل
10     try {
11         Thread.sleep(3000);
12     } catch (InterruptedException e) {
13         // ignore
14     }
15     log.info("Email sent to: {}", to);
16 }
17
18 // ۲. خاص Executor با
19 @Async("taskExecutor")
20 public void sendEmailWithCustomExecutor(String to, String body) {
21     // ...
22 }
23
24 // ۳. برگرداندن Future
25 @Async
26 public CompletableFuture<String> processData(String data) {
27     String result = data.toUpperCase();
28     return CompletableFuture.completedFuture(result);
29 }
30 }
```

۲۰.۲.۳ استفاده از Async

```
1 @RestController
2 @RequestMapping("/api/email")
3 public class EmailController {
4
5     @Autowired
6     private EmailService emailService;
7
8     @GetMapping("/send")
9     public ResponseEntity<String> sendEmail() {
10         emailService.sendEmail("test@example.com", "Hello!");
11         return ResponseEntity.ok("Email sending started");
12     }
13
14     @GetMapping("/process")
15     public CompletableFuture<ResponseEntity<String>> process() throws ExecutionException,
16     InterruptedException {
17         CompletableFuture<String> future = emailService.processData("hello");
18         String result = future.get();
19         return CompletableFuture.completedFuture(ResponseEntity.ok(result));
20 }
```

۲۰.۲.۴ مدیریت خطا در Async

1-Java Interview Question

```
1 @Configuration
2 @EnableAsync
3 public class AsyncConfig implements AsyncConfigurer {
4
5     @Override
6     public Executor getAsyncExecutor() {
7         ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
8         executor.setCorePoolSize(5);
9         executor.setMaxPoolSize(10);
10        executor.setThreadNamePrefix("Async-");
11        executor.initialize();
12        return executor;
13    }
14
15    @Override
16    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
17        return (throwable, method, params) -> {
18            log.error("Async error in method: {}", method.getName(), throwable);
19        };
20    }
21 }
```

۲۰.۳ Scheduling (برنامه‌ریزی)

۲۰.۳.۱ فعال‌سازی Scheduling

```

1 @Configuration
2 @EnableScheduling
3 public class SchedulingConfig { ... }

```

۲۰.۳.۲ Scheduled@

```

1 @Component
2 @Slf4j
3 public class ScheduledTasks {
4
5     // ۱. با فاصله ثابت (بعد از اتمام کار قبلی) ۱.
6     @Scheduled(fixedDelay = 5000)
7     public void fixedDelayTask() {
8         log.info("Fixed delay task executed");
9     }
10
11     // ۲. با فاصله ثابت (بدون توجه به اتمام کار قبلی) ۲.
12     @Scheduled(fixedRate = 10000)
13     public void fixedRateTask() {
14         log.info("Fixed rate task executed");
15     }
16

```


1-Java Interview Question

```
17  // ۳. با تأخیر اولیه
18  @Scheduled(initialDelay = 10000, fixedRate = 30000)
19  public void delayedTask() {
20      log.info("Delayed task executed");
21  }
22
23  // ۴. با Cron Expression
24  @Scheduled(cron = "0 0 2 * *") // هر روز ساعت ۲ بامداد
25  public void cronTask() {
26      log.info("Cron task executed - daily backup");
27  }
28
29  // ۵. با Cron و TimeZone
30  @Scheduled(cron = "0 0 0 * * SAT", zone = "Asia/Tehran")
31  public void weeklyTask() {
32      log.info("Weekly task - every Saturday midnight Tehran time");
33  }
34 }
```

کامل Cron Expression ۲۰.۳.۳

- 1 _____ ثانیه (59-0)
- 2 | _____ دقیقه (59-0)
- 3 | | _____ ساعت (23-0)
- 4 | | | _____ روز ماه (31-1)
- 5 | | | | _____ ماه (یا 12-1 JAN-DEC)
- 6 | | | | | _____ روز هفته (یا 7-0 SUN-SAT)
- 7 | | | | |
- 8 * * * * *
- 9
- 10 مثال‌ها:
- 11 "0 0 2 * * *" → هر روز ساعت ۲:۰۰ بامداد
- 12 "0 30 9 * * MON-FRI" → روزهای کاری ساعت ۹:۳۰ صبح
- 13 "0 0 0 1 * *" → اولین روز هر ماه ساعت ۱۲:۰۰
- 14 "0 0 12 * * 6" → هر جمعه ساعت ۱۲:۰۰
- 15 "0 0 0 * * 6" → هر جمعه نیمه شب
- 16 "0 0 0 1 1 *" → اول ژانویه نیمه شب
- 17 "0 0 0 * * 0" → هر یکشنبه نیمه شب
- 18 "0 0 0 15 * *" → پانزدهم هر ماه نیمه شب

۲۰.۳.۴ Scheduling با پروفایل

```
1 @Component
2 @Slf4j
3 @Profile("prod")
4 public class ProdScheduledTasks {
5
6     @Scheduled(cron = "0 0 2 * * *")
7     public void backupDatabase() {
8         log.info("Backing up database...");
9     }
10 }
```

۲۰.۴ Validation (اعتبارسنجی)

۲۰.۴.۱ فعال سازی Validation

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-validation</artifactId>
4 </dependency>
```

۲۰.۴.۲ Annotation های پرکاربرد

```
1 @Data
2 public class UserRequest {
3
4     @NotNull(message = "ID نباید null باشد")
5     private Long id;
```

```
6
7  @NotBlank(message = "نام کاربری اجباری است")
8  @Size(min = 3, max = 20, message = "نام کاربری باید بین ۳ تا ۲۰ کاراکتر باشد")
9  private String username;
10
11 @NotBlank(message = "پسورد اجباری است")
12 @Size(min = 8, message = "پسورد باید حداقل ۸ کاراکتر باشد")
13 @Pattern(
14     regexp = "^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z]).+$",
15     message = "پسورد باید شامل عدد و حروف بزرگ و کوچک باشد"
16 )
17 private String password;
18
19 @Email(message = "ایمیل نامعتبر است")
20 @NotBlank(message = "ایمیل اجباری است")
21 private String email;
22
23 @Min(value = 18, message = "سن باید حداقل ۱۸ باشد")
24 @Max(value = 100, message = "سن باید حداکثر ۱۰۰ باشد")
25 private Integer age;
26
27 @Pattern(
28     regexp = "^\\d{10}$",
29     message = "شماره تلفن باید ۱۰ رقم باشد"
30 )
31 private String phoneNumber;
```

```

32
33 @Size(max = 500, message = "توضیحات حداکثر ۵۰۰ کاراکتر")
34 private String description;
35 }

```

۲۰.۴.۳ استفاده در Controller

```

1 @RestController
2 @RequestMapping("/api/users")
3 public class UserController {
4
5     // ۱. @Valid اعتبارسنجی
6     @PostMapping
7     public ResponseEntity<User> createUser(@Valid @RequestBody UserRequest request) {
8         return ResponseEntity.ok(userService.save(request));
9     }
10
11     // ۲. @Validated با گروه‌ها
12     @PostMapping("/register")
13     public ResponseEntity<User> registerUser(@Validated @RequestBody UserRequest request) {
14         return ResponseEntity.ok(userService.register(request));
15     }
16
17     // ۳. BindingResult برای خطاهای اعتبارسنجی
18     @PostMapping("/validate")
19     public ResponseEntity<?> validateUser(@Valid @RequestBody UserRequest request,
20                                           BindingResult bindingResult) {

```

1-Java Interview Question

```
21  if (bindingResult.hasErrors()) {
22      List<String> errors = bindingResult.getAllErrors()
23          .stream()
24          .map(DefaultMessageSourceResolvable::getDefaultMessage)
25          .collect(Collectors.toList());
26      return ResponseEntity.badRequest().body(errors);
27  }
28  return ResponseEntity.ok("Valid");
29  }
30 }
```

۲۰.۴.۴ مدیریت خطاهای Validation (Global)

```
1  @ControllerAdvice
2  public class GlobalExceptionHandler {
3
4      @ExceptionHandler(MethodArgumentNotValidException.class)
5      public ResponseEntity<Map<String, List<String>>> handleValidationErrors(
6          MethodArgumentNotValidException ex) {
7          List<String> errors = ex.getBindingResult()
8              .getAllErrors()
9              .stream()
10             .map(DefaultMessageSourceResolvable::getDefaultMessage)
11             .collect(Collectors.toList());
12
13         return ResponseEntity.badRequest()
14             .body(Map.of("errors", errors));
15     }
```

1-Java Interview Question

```
15 }
16
17 @ExceptionHandler(ConstraintViolationException.class)
18 public ResponseEntity<Map<String, String>> handleConstraintViolation(
19     ConstraintViolationException ex) {
20     String message = ex.getConstraintViolations()
21         .stream()
22         .map(violation -> violation.getMessage())
23         .collect(Collectors.joining(", "));
24
25     return ResponseEntity.badRequest()
26         .body(Map.of("error", message));
27 }
28 }
```

۲۰.۴.۵ Group Validation (اعتبارسنجی گروهی)

```
1 // a. تعریف گروه ها
2 public interface Create {}
3 public interface Update {}
4
5 // b. استفاده در کلاس
6 @Data
7 public class UserRequest {
8     @NotNull(groups = Update.class)
9     private Long id;
10 }
```

1-Java Interview Question

```
11 @NotBlank(groups = {Create.class, Update.class})
12 private String username;
13
14 @NotBlank(groups = {Create.class})
15 private String password;
16 }
17
18 // استفاده در Controller
19 @PostMapping
20 public ResponseEntity<User> create(@Validated(Create.class) @RequestBody UserRequest request) {
21     // گروه Create فقط اعتبارسنجی
22 }
23
24 @PutMapping
25 public ResponseEntity<User> update(@Validated(Update.class) @RequestBody UserRequest request) {
26     // گروه Update فقط اعتبارسنجی
27 }
```

۲۰.۴.۶ Custom Validation (سفارشی)

```
1 //۱. Annotation
2 @Target({ElementType.FIELD, ElementType.PARAMETER})
3 @Retention(RetentionPolicy.RUNTIME)
4 @Constraint(validatedBy = IranNationalCodeValidator.class)
5 public @interface IranNationalCode {
6     String message() default "کد ملی نامعتبر است";
7     Class<?>[] groups() default {};
```


1-Java Interview Question

```
8  Class<? extends Payload>[] payload() default {};  
9  }  
10  
11 // ۲. Validator  
12 public class IranNationalCodeValidator implements  
ConstraintValidator<IranNationalCode, String> {  
13  
14     @Override  
15     public boolean isValid(String code, ConstraintValidatorContext context) {  
16         if (code == null || code.length() != 10) {  
17             return false;  
18         }  
19         // الگوریتم اعتبارسنجی کد ملی ایران  
20         try {  
21             // کد الگوریتم...  
22             return true;  
23         } catch (Exception e) {  
24             return false;  
25         }  
26     }  
27 }  
28 // استفاده ۳.  
29 @Data  
30 public class PersonRequest {  
31     @IranNationalCode  
32     private String nationalCode;  
33 }
```

۲۰.۵ ! سوالات مصاحبه‌ای

? سوال ۱: فرق @Cacheable و @CacheEvict چیست؟

جواب:

• @Cacheable: نتیجه متد رو در کش ذخیره می‌کنه

• @CacheEvict: کش رو پاک می‌کنه

? سوال ۲: @Async چه کاربردی داره؟

جواب: @Async برای اجرای ناهمگام (Asynchronous) متدهاست. یعنی متد در یک Thread جداگانه اجرا میشه و Thread اصلی منتظر نمی‌مونه.

? سوال ۳: Cron Expression چیست؟

جواب: یه فرمت برای مشخص کردن زمان اجرای کارهای زمان‌بندی شده. شامل ۶ بخش: ثانیه، دقیقه، ساعت، روز ماه، ماه، روز هفته.

? سوال ۴: Validation با @Valid و @Validated چه فرقی دارن؟

جواب:

• @Valid: از JPA میاد، برای اعتبارسنجی استاندارد

• @Validated: از Spring میاد، پشتیبانی از Group Validation داره

? سوال ۵: چطور کش رو در Spring فعال می‌کنیم؟

جواب: با اضافه کردن @EnableCaching به یک @Configuration کلاس.

فاز ۲۱: میکروسرویس‌ها (Microservices) با

Spring Cloud

میکروسرویس‌ها یکی از مهم‌ترین مباحث برای بک‌اندکاران حرفه‌ای هستند. در مصاحبه‌های سطح بالا حتماً ازت می‌پرسن.

۲۱.۱ Microservices چیست؟

تعریف

میکروسرویس‌ها به سبک معماری هستند که در اون یک برنامه بزرگ به چندین سرویس کوچک و مستقل تقسیم میشه. هر سرویس:

- یک Business Capability رو پیاده‌سازی می‌کنه

- مستقل از بقیه Deploy میشه

- با HTTP یا Message Queue ارتباط برقرار می‌کنه

- می‌تونه با زبان‌های مختلف نوشته بشه

مقایسه Monolith vs Microservices

1		
2		
3	Monolith (تکنولوژی)	Microservices
4		
5	چندین سرویس کوچک	یک برنامه یکپارچه
6	JAR همه چیز توی یک	جداگانه JAR هر سرویس
7	هر سرویس دیتابیس خودش رو داره	یک دیتابیس مشترک
8	هر سرویس مستقل Deploy = تغییر	میشه Deploy کل برنامه
9	خطا فقط اون سرویس رو می‌گیره	خطا = کل برنامه کرش می‌کنه
10	مقیاس‌پذیری: فقط سرویس مورد نیاز	مقیاس‌پذیری: کل برنامه
11		
12		

مزایا و معایب معماری Microservices

مزایا	معایب
مقیاس‌پذیری بالا	پیچیدگی زیاد
جدا بودن تیم‌ها (استقلال تیم‌ها)	ارتباطات شبکه (Latency و مسائل شبکه)
Deploy مستقل (هر سرویس جداگانه)	مدیریت داده (تراکنش‌های توزیع‌شده)
خطا فقط یک سرویس را تحت تأثیر قرار می‌دهد	دییاباگ سخت‌تر (ردیابی در چند سرویس)
انتخاب تکنولوژی آزاد برای هر سرویس	هزینه بیشتر (زیرساخت، عملیات)

نکات تکمیلی:

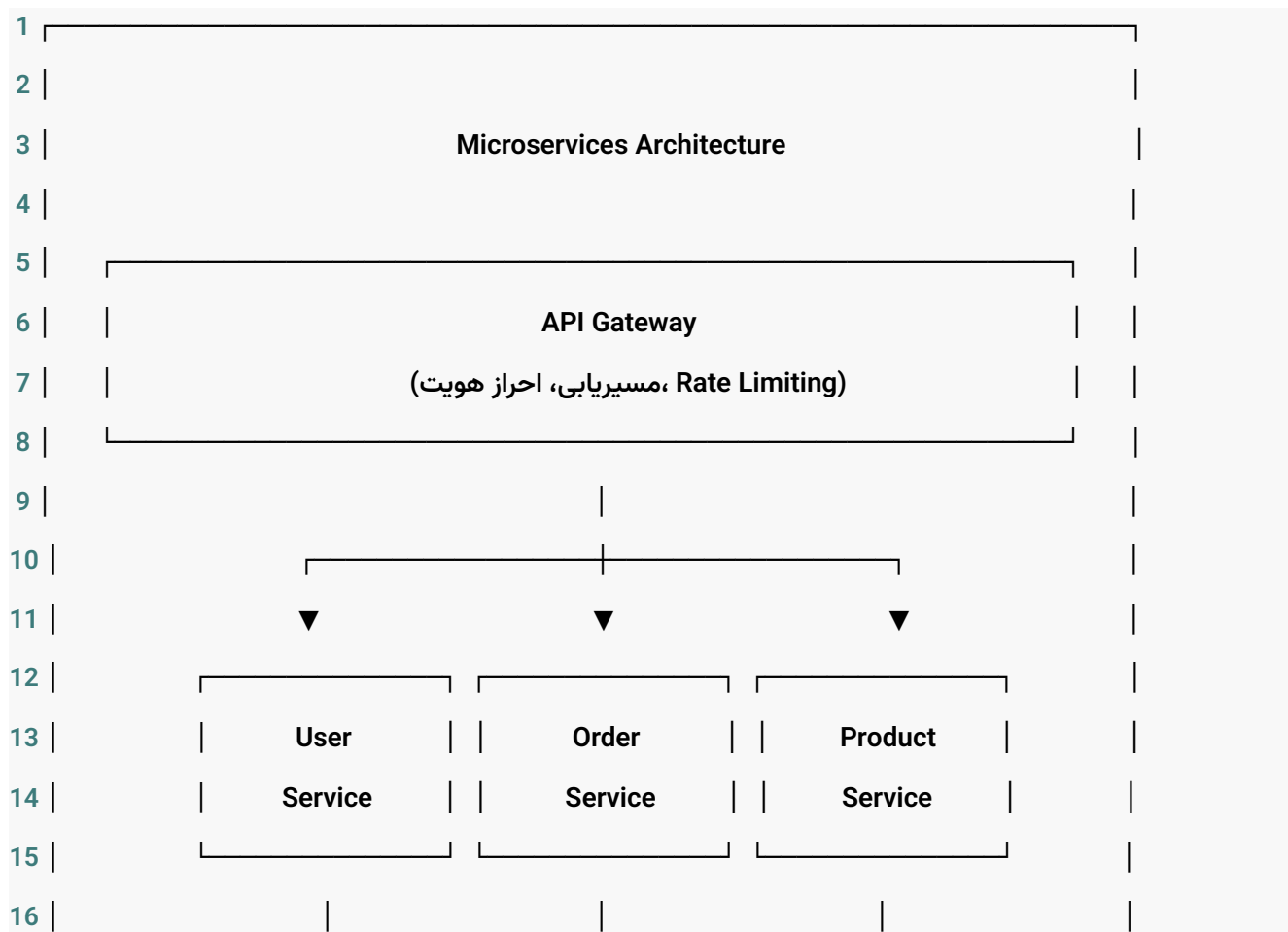
• مقیاس پذیری: هر سرویس به صورت مستقل مقیاس پذیر است و می توان فقط سرویس های پرتراфик را مقیاس کرد.

• استقلال تیم ها: هر تیم می تواند روی سرویس خود کار کند بدون وابستگی به تیم های دیگر.

• مدیریت داده: چالش اصلی در Microservices، مدیریت تراکنش ها و یکپارچگی داده بین سرویس ها است.

• هزینه بیشتر: نیاز به مدیریت چندین سرویس، لاگ گیری متمرکز، کشف سرویس و زیرساخت پیچیده تر دارد.

۲۱.۲ معماری میکروسرویس ها



1-Java Interview Question



Spring Cloud Netflix ۲۱.۳

وابستگی‌ها

```
1 <dependencyManagement>
2   <dependencies>
3     <dependency>
4       <groupId>org.springframework.cloud</groupId>
5       <artifactId>spring-cloud-dependencies</artifactId>
6       <version>2022.0.0</version>
7       <type>pom</type>
8       <scope>import</scope>
9     </dependency>
10  </dependencies>
11 </dependencyManagement>
12
13 <dependencies>
14   <!-- Eureka Server -->
15   <dependency>
```

```
16     <groupId>org.springframework.cloud</groupId>
17     <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
18 </dependency>
19
20 <!-- Eureka Client -->
21 <dependency>
22     <groupId>org.springframework.cloud</groupId>
23     <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
24 </dependency>
25
26 <!-- API Gateway -->
27 <dependency>
28     <groupId>org.springframework.cloud</groupId>
29     <artifactId>spring-cloud-starter-gateway</artifactId>
30 </dependency>
31
32 <!-- Circuit Breaker -->
33 <dependency>
34     <groupId>org.springframework.cloud</groupId>
35     <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
36 </dependency>
37
38 <!-- Load Balancer -->
39 <dependency>
40     <groupId>org.springframework.cloud</groupId>
41     <artifactId>spring-cloud-starter-loadbalancer</artifactId>
```



```
42 </dependency>
```

```
43 </dependencies>
```

Service Discovery (Eureka) ۲۱.۴

Eureka Server

```
1 @SpringBootApplication
2 @EnableEurekaServer
3 public class DiscoveryApplication {
4     public static void main(String[] args) {
5         SpringApplication.run(DiscoveryApplication.class, args);
6     }
7 }
8 # application.properties (Eureka Server)
9 server.port=8761
10 eureka.client.register-with-eureka=false
11 eureka.client.fetch-registry=false
12 eureka.server.wait-time-in-ms-when-sync-empty=0
```

Eureka Client

```
1 @SpringBootApplication
2 @EnableDiscoveryClient
3 public class UserServiceApplication {
4     public static void main(String[] args) {
5         SpringApplication.run(UserServiceApplication.class, args);
```

1-Java Interview Question

```
6  }
7  }
8  # application.properties (Eureka Client)
9  spring.application.name=user-service
10 server.port=8081
11
12 eureka.client.serviceUrl.defaultZone=http://localhost:8761/eureka/
13 eureka.instance.prefer-ip-address=true
14 eureka.instance.instance-id=${spring.application.name}:${server.port}
```

استفاده از Service Discovery

```
1 @Service
2 @Slf4j
3 public class OrderService {
4
5     @Autowired
6     private LoadBalancerClient loadBalancer;
7
8     @Autowired
9     private RestTemplate restTemplate;
10
11     @Autowired
12     private DiscoveryClient discoveryClient;
13
14     // ۱. استفاده از LoadBalancerClient
15     public User getUserByUsername(String username) {
```

1-Java Interview Question

```
16     ServiceInstance instance = loadBalancer.choose("user-service");
17     String url = "http://" + instance.getHost() + ":" + instance.getPort() + "/api/users/" + username;
18     return restTemplate.getForObject(url, User.class);
19 }
20
21 // ۲. استفاده از P. DiscoveryClient
22 public List<User> getAllUsers() {
23     List<ServiceInstance> instances = discoveryClient.getInstances("user-service");
24     // ...
25 }
26
27 // ۳. با RestTemplate + @LoadBalanced
28 @Bean
29 @LoadBalanced
30 public RestTemplate restTemplate() {
31     return new RestTemplate();
32 }
33 // ۴. استفاده از RestTemplate (با LoadBalanced)
34 public User getUserByName(String username) {
35     return restTemplate.getForObject(
36         "http://user-service/api/users/" + username,
37         User.class
38     );
39 }
40 }
```

API Gateway 21.5

Gateway Service

```
1 @SpringBootApplication
2 @EnableDiscoveryClient
3 public class GatewayApplication {
4     public static void main(String[] args) {
5         SpringApplication.run(GatewayApplication.class, args);
6     }
7 }
8 # application.yml
9 server:
10  port: 8080
11
12 spring:
13  application:
14    name: api-gateway
15
16  cloud:
17    gateway:
18      routes:
19        - id: user-service
20          uri: lb://user-service
21          predicates:
22            - Path=/api/users/**
```

1-Java Interview Question

```
23
24     - id: order-service
25     uri: lb://order-service
26     predicates:
27         - Path=/api/orders/**
28
29     - id: product-service
30     uri: lb://product-service
31     predicates:
32         - Path=/api/products/**
33
34     # Global filters
35     default-filters:
36         - name: CircuitBreaker
37         args:
38             name: default
39             fallbackUri: forward:/fallback
40         - name: Retry
41         args:
42             retries: 3
43             statuses: SERVER_ERROR
44
45     management:
46     endpoints:
47     web:
48     exposure:
```

49 include: health,info,gateway

Gateway Filters

```
1 @Configuration
2 public class GatewayConfig {
3
4     // 1. Custom Filter
5     @Bean
6     public GlobalFilter customFilter() {
7         return (exchange, chain) -> {
8             // route قبل از
9             ServerHttpRequest request = exchange.getRequest();
10            log.info("Request: {}", request.getMethod(), request.getURI());
11
12            // ادامه
13            return chain.filter(exchange).then(Mono.fromRunnable(() -> {
14                // route بعد از
15                ServerHttpResponse response = exchange.getResponse();
16                log.info("Response status: {}", response.getStatusCode());
17            }));
18        };
19    }
20
21    // ۲. Rate Limiter Filter
22    @Bean
23    public RedisRateLimiter redisRateLimiter() {
```

1-Java Interview Question

```
24     return new RedisRateLimiter(10, 20, 1); // 10 requests per second
25 }
26 }
```

Circuit Breaker (Resilience4j) ۲۱.۶

فعال سازی

```
1 @Configuration
2 @EnableCircuitBreaker
3 public class CircuitBreakerConfig { ... }
```

استفاده

```
1 @Service
2 @Slf4j
3 public class PaymentService {
4
5     @Autowired
6     private PaymentClient paymentClient;
7
8     // ۱. Circuit Breaker ب Fallback
9     @CircuitBreaker(name = "paymentService", fallbackMethod = "fallbackPayment")
10    public Payment processPayment(PaymentRequest request) {
11        log.info("Processing payment...");
12        return paymentClient.process(request);
13    }
```

```
14
15 // ٢. Fallback Method
16 public Payment fallbackPayment(PaymentRequest request, Throwable t) {
17     log.error("Payment fallback: {}", t.getMessage());
18     return Payment.builder()
19         .status("FAILED")
20         .message("Payment service unavailable")
21         .build();
22 }
23
24 // ٣. Retry
25 @Retry(name = "paymentService", fallbackMethod = "fallbackPayment")
26 public Payment processPaymentWithRetry(PaymentRequest request) {
27     return paymentClient.process(request);
28 }
29
30 // ٤. Rate Limiter
31 @RateLimiter(name = "paymentService")
32 public Payment processPaymentWithRateLimit(PaymentRequest request) {
33     return paymentClient.process(request);
34 }
35
36 // ٥. Bulkhead (محدودیت همزمان)
37 @Bulkhead(name = "paymentService", type = Bulkhead.Type.THREADPOOL)
38 public Payment processPaymentWithBulkhead(PaymentRequest request) {
39     return paymentClient.process(request);
```


1-Java Interview Question

40 }

41 }

پیکربندی Resilience4j

1 **resilience4j:**

2 **circuitbreaker:**

3 **instances:**

4 **paymentService:**

5 **sliding-window-size: 10**

6 **failure-rate-threshold: 50**

7 **wait-duration-in-open-state: 10s**

8 **minimum-number-of-calls: 5**

9

10 **retry:**

11 **instances:**

12 **paymentService:**

13 **max-attempts: 3**

14 **wait-duration: 2s**

15

16 **ratelimiter:**

17 **instances:**

18 **paymentService:**

19 **limit-for-period: 10**

20 **limit-refresh-period: 1s**

21

22 **bulkhead:**

1-Java Interview Question

23 instances:

24 paymentService:

25 max-concurrent-calls: 5

Distributed Tracing (Sleuth + Zipkin) ۲۱.۷

```
1 <dependency>
2   <groupId>org.springframework.cloud</groupId>
3   <artifactId>spring-cloud-starter-sleuth</artifactId>
4 </dependency>
5 <dependency>
6   <groupId>org.springframework.cloud</groupId>
7   <artifactId>spring-cloud-starter-zipkin</artifactId>
8 </dependency>
9 spring.sleuth.sampler.probability=1.0
10 spring.zipkin.base-url=http://localhost:9411
```

استفاده از Trace ID

```
1 @Service
2 @Slf4j
3 public class OrderService {
4
5   @Autowired
6   private Tracer tracer;
7
8   public void processOrder(Order order) {
```

```

9    // Trace ID خودکار در لاگ ها ظاهر میشه
10   log.info("Processing order: {}", order.getId());
11
12   Span span = tracer.nextSpan().name("processOrder");
13   try (Tracer.SpanInScope ws = tracer.withSpan(span.start())) {
14       // پردازش
15       log.info("Order processed");
16   } finally {
17       span.end();
18   }
19 }
20 }

```

۲۱.۸ ! سوالات مصاحبه‌ای

? سوال ۱: میکروسرویس چیه و چرا استفاده میشه؟

جواب: Microservices به معماری هست که برنامه رو به چندین سرویس کوچک و مستقل تقسیم می‌کنه. مزایا: مقیاس‌پذیری بالا، Deploy مستقل، انتخاب تکنولوژی آزاد، خطا فقط یک سرویس.

? سوال ۲: Eureka چیه؟

جواب: Eureka به سرویس ثبت و کشف (Service Discovery) هست. سرویس‌ها خودشون رو در Eureka ثبت می‌کنن و سرویس‌های دیگه می‌تونن همونجا پیداشون کنن.

? سوال ۳: API Gateway چیه و چرا استفاده میشه؟

جواب: API Gateway یه نقطه ورودی واحد برای همه سرویس‌هاست. کارها: مسیریابی درخواست‌ها، احراز هویت، Rate Limiting، Caching، Logging.

? سوال ۴: Circuit Breaker چیه؟

جواب: Circuit Breaker یه الگو برای مدیریت خطا در سرویس‌های دیگه هست. وقتی یه سرویس خطا بده، Circuit Breaker باز میشه و درخواست‌ها دیگه به اون سرویس نمی‌رن (تا بازیابی).

? سوال ۵: فرق Monolith و Microservices چیه؟

جواب: Monolith یک برنامه یکپارچه است، Microservices چندین سرویس کوچک. Monolith ساده‌تر ولی کمتر مقیاس‌پذیر، Microservices پیچیده‌تر ولی مقیاس‌پذیرتر.

♦ بخش ۱: مبانی جاوا

۱.۱ انواع داده

- 1 **byte**: نوع داده عددی ۸ بیتی با بازه -۱۲۸ تا ۱۲۷.
- 2 **short**: عدد صحیح ۱۶ بیتی.
- 3 **int**: عدد صحیح ۳۲ بیتی، پرکاربردترین نوع عددی.
- 4 **long**: عدد صحیح ۶۴ بیتی (با پسوند L).
- 5 **float**: عدد اعشاری ۳۲ بیتی (با پسوند f).
- 6 **double**: عدد اعشاری ۶۴ بیتی، پیش فرض اعداد اعشاری.
- 7 **char**: یک کاراکتر یونیکد ۱۶ بیتی.
- 8 **boolean**: مقدار **true** یا **false**.
- 9 **String**: کلاس برای نگهداری رشته متن، **immutable**.
- 10 **Array**: ساختار با اندازه ثابت برای نگهداری چند مقدار هم نوع.
- 11 **Enum**: نوع داده برای مجموعه‌ای ثابت از مقادیر نام دار.
- 12 **Boxing**: تبدیل **primitive** به **class wrapper**.
- 13 **Unboxing**: تبدیل **class wrapper** به **primitive**.
- 14 **Autoboxing**: boxing: خودکار توسط کامپایلر.
- 15 **Wrapper Classes**: کلاس‌های پوششی مثل **Integer** برای **primitive** ها.
- 16 **Integer**: wrapper کلاس **int**.
- 17 **IEEE 754**: استاندارد نمایش اعداد اعشاری در حافظه.
- 18 **BigDecimal**: کلاس برای محاسبات اعشاری **دقیق** (بدون خطای شناوری).
- 19 **String Pool**: ناحیه‌ای در **heap** برای ذخیره **String** های **literal** و **reuse** آن‌ها.
- 20 **Immutable**: تغییرناپذیر؛ مثل **String**.
- 21 **StringBuilder**: ساخت رشته **mutable** و **سریع** (غیر **thread-safe**).
- 22 **StringBuffer**: ساخت رشته **mutable** و **thread-safe** (کندتر).
- 23 **Thread-Safe**: امن برای استفاده هم‌زمان چند **thread**.
- 24 **Mutable**: قابل تغییر پس از ساخت.
- 25 **Stack**: ناحیه حافظه برای متغیرهای محلی و فراخوانی متدها.

- 26 Heap: ناحیه حافظه برای آجکت‌ها.
- 27 NullPointerException: خطای دسترسی به reference با مقدار null.
- 28 Strongly Typed: زبان با قواعد سختگیرانه نوع داده.
- 29 Statically Typed: بررسی نوع در زمان کامپایل، نه اجرا.

۱.۲ عملگرها

- 1 <<: شیفت راست با حفظ علامت.
- 2 <<<: شیفت راست بدون حفظ علامت (پُر کردن با صفر).
- 3 & AND: بیتی / منطقی (هر دو طرف ارزیابی می‌شوند).
- 4 && AND: منطقی با short-circuit.
- 5 |: OR: بیتی / منطقی.
- 6 || OR: منطقی با short-circuit.
- 7 Short-circuit Evaluation: ارزیابی شرط دوم فقط در صورت نیاز.

۱.۳ ساختارهای کنترلی

- 1 switch-case: انتخاب چندشاخه‌ای بر اساس مقدار.
- 2 Enhanced switch: نسخه جدید switch با arrow و بدون fall-through.
- 3 fall-through: اجرای ادامه case وقتی break نباشد.
- 4 break: خروج از حلقه یا switch.
- 5 default: شاخه پیش‌فرض switch.
- 6 for: حلقه با شمارنده.
- 7 for-each: پیمایش ساده عناصر آرایه/کالکشن.
- 8 while: حلقه با شرط قبل از اجرا.
- 9 do-while: حلقه با شرط بعد از اجرا (حداقل یک بار اجرا).
- 10 if / else: شرط ساده.

۱.۴ آرایه‌ها

- 1 Array: مجموعه با اندازه ثابت و هم‌نوع.

- 2 Fixed Size: اندازه آرایه پس از ساخت تغییر نمی‌کند.
- 3 **ArrayIndexOutOfBoundsException**: خطای دسترسی به ایندکس خارج از محدوده.
- 4 **Unchecked Exception**: خطای زمان اجرا که کامپایلر اجبار به **catch** نمی‌کند.
- 5 **int[]**: آرایه یک‌بعدی **int**.
- 6 **int[][]**: آرایه دوبعدی **int**.
- 7 **matrix**: آرایه دوبعدی.
- 8 **Arrays.sort()**: مرتب‌سازی آرایه.
- 9 **Arrays.stream()**: تبدیل آرایه به **Stream**.
- 10 **Arrays.asList()**: تبدیل آرایه به **List** با اندازه ثابت.

۱.۵ آرگومان‌های خط فرمان

- 1 **String[] args**: آرایه آرگومان‌های ورودی برنامه.
- 2 **main method**: نقطه ورود اجرای برنامه.

♦ بخش ۲: ورودی/خروجی (I/O) در جاوا

۲.۱ معماری I/O

- 1 **I/O Byte-Oriented**: مبتنی بر بایت برای داده‌های باینری.
- 2 **I/O Character-Oriented**: مبتنی بر کاراکتر برای متن.
- 3 **InputStream**: کلاس پایه برای خواندن بایت.
- 4 **OutputStream**: کلاس پایه برای نوشتن بایت.
- 5 **Reader**: کلاس پایه برای خواندن کاراکتر.
- 6 **Writer**: کلاس پایه برای نوشتن کاراکتر.
- 7 **Stream**: جریان داده از مبدأ به مقصد.
- 8 **Buffer**: حافظه موقت برای افزایش سرعت I/O.

۲.۲ Byte Streams

- **1 FileInputStream**: خواندن بایت از فایل.
- **2 FileOutputStream**: نوشتن بایت در فایل.
- **3 ByteArrayInputStream**: خواندن از آرایه بایت در حافظه.
- **4 ByteArrayOutputStream**: نوشتن در آرایه بایت در حافظه.
- **5 DataInputStream**: خواندن داده‌های primitive به صورت قابل حمل.
- **6 DataOutputStream**: نوشتن داده‌های primitive به صورت قابل حمل.
- **7 BufferedInputStream**: بافر کردن خواندن بایت برای سرعت بیشتر.
- **8 BufferedOutputStream**: بافر کردن نوشتن بایت.
- **9 ObjectInputStream**: خواندن آبجکت سریالایز شده.
- **10 ObjectOutputStream**: نوشتن آبجکت به صورت سریالایز.
- **11 FilterInputStream**: کلاس پایه برای streamهای فیلترکننده ورودی.
- **12 FilterOutputStream**: کلاس پایه برای streamهای فیلترکننده خروجی.
- **13 buffer**: حافظه موقت داده.
- **14 read()**: خواندن یک بایت/کاراکتر.
- **15 write()**: نوشتن یک بایت/کاراکتر.
- **16 IOException**: خطای عمومی I/O.

۲.۳ Reader/Writer

- **1 FileReader**: خواندن کاراکتر از فایل.
- **2 FileWriter**: نوشتن کاراکتر در فایل.
- **3 StringReader**: خواندن از رشته.
- **4 StringWriter**: نوشتن در رشته.
- **5 InputStreamReader**: پل بین **byte stream** و **character stream**.
- **6 OutputStreamWriter**: پل بین **byte stream** و **character stream** خروجی.
- **7 BufferedReader**: بافر کردن خواندن کاراکتر + **readLine**.
- **8 BufferedWriter**: بافر کردن نوشتن کاراکتر + **newLine**.
- **9 PrintWriter**: نوشتن متن با متدهای **print/println**.
- **10 readLine()**: خواندن یک خط کامل.

- 11• `newLine()`: درج خط جدید.
- 12• `flush()`: تخلیه بافر و نوشتن فوری.
- 13• UTF-8: کدگذاری یونیکد پرکاربرد.
- 14• `StandardCharsets`: کلاس ثابت‌های Charset مثل UTF_8.

Java NIO ۲.۴

- 1 • `java.nio`: پکیج I/O غیرمسدودکننده و مبتنی بر بافر.
- 2 • `Files`: کلاس ابزاری برای کار با فایل‌ها.
- 3 • `Paths`: کلاس ابزاری برای ساخت `Path`.
- 4 • `Path`: نماینده مسیر فایل/دایرکتوری.
- 5 • `Files.readAllLines()`: خواندن همه خطوط فایل در `List`.
- 6 • `Files.write()`: نوشتن داده در فایل.
- 7 • `Files.copy()`: کپی فایل.
- 8 • `Files.deleteIfExists()`: حذف فایل اگر وجود داشته باشد.
- 9 • `Files.exists()`: بررسی وجود فایل.
- 10• `Files.readString()`: خواندن کل محتوای فایل به صورت `String`.
- 11• `Files.writeString()`: نوشتن `String` در فایل.
- 12• `Files.list()`: لیست کردن محتوای دایرکتوری به صورت `Stream`.
- 13• `Files.createDirectories()`: ساخت دایرکتوری‌ها به صورت بازگشتی.
- 14• `Files.size()`: اندازه فایل.
- 15• `Files.getLastModifiedTime()`: زمان آخرین تغییر فایل.
- 16• `Files.probeContentType()`: تشخیص نوع MIME فایل.
- 17• `StandardCopyOption`: گزینه‌های کپی مثل `REPLACE_EXISTING`.
- 18• `Stream<Path>`: جریان مسیرها برای پردازش دایرکتوری.
- 19• `Blocking`: I/O مسدودکننده که `thread` منتظر می‌ماند.
- 20• `Non-blocking`: I/O غیرمسدودکننده با `Selector`.
- 21• `Selector`: انتخابگر کانال‌های آماده در NIO.
- 22• `Buffer-oriented`: مدل NIO بر پایه بافر.

23• Stream-oriented: مدل I/O کلاسیک بر پایه stream.

۲.۵ Serialization

- 1 **Serializable**: اینترفیس نشانه‌گذاری برای قابلیت سریالایز شدن.
- 2 **serialVersionUID**: شناسه نسخه کلاس برای سازگاری سریالایز.
- 3 **transient**: فیلدی که سریالایز نمی‌شود.
- 4 **static**: فیلد **static** سریالایز نمی‌شود.
- 5 **ObjectOutputStream**: نوشتن آبجکت سریالایز شده.
- 6 **ObjectInputStream**: خواندن آبجکت سریالایز شده.
- 7 **writeObject()**: متد نوشتن آبجکت.
- 8 **readObject()**: متد خواندن آبجکت.
- 9 **defaultWriteObject()**: نوشتن پیش‌فرض فیلدهای **non-transient**.
- 10 **defaultReadObject()**: خواندن پیش‌فرض فیلدها.
- 11 **ClassNotFoundException**: خطای پیدا نشدن کلاس هنگام **deserialize**.
- 12 **Deserialize**: بازسازی آبجکت از بایت.
- 13 **Serialize**: تبدیل آبجکت به بایت.

۲.۶ Console

- 1 **Scanner**: کلاس ساده برای خواندن ورودی.
- 2 **stream** : **System.in** ورودی استاندارد.
- 3 **nextLine()**: خواندن یک خط **String**.
- 4 **nextInt()**: خواندن **int**.
- 5 **nextDouble()**: خواندن **double**.
- 6 **BufferedReader**: خواندن سریع‌تر ورودی متنی.
- 7 **InputStreamReader**: پل بین **byte** و **character** برای ورودی.
- 8 **Integer.parseInt()**: تبدیل **String** به **int**.
- 9 **Console**: کلاس خواندن ورودی از کنسول واقعی.
- 10 **System.console()**: دریافت **Console** (ممکن است **null** باشد).

- 11• `readLine()`: خواندن خط از کنسول.
- 12• `readPassword()`: خواندن رمز بدون نمایش.

۲.۷ سوالات مصاحبه‌ای I/O

- 1• `Try-with-Resources`: بستن خودکار منابع در `try`.
- 2• `AutoCloseable`: اینترفیس برای منابع قابل بستن خودکار.
- 3• `Suppressed Exceptions`: استثناهای ثانویه که در `try-with-resources` سرکوب می‌شوند.
- 4• `Memory Leak`: نشت حافظه به‌خاطر بسته نشدن منابع.

♦ بخش ۳: برنامه‌نویسی شی‌گرا (OOP)

۳.۱ مفاهیم پایه

- 1• `Class`: قالب/نقشه برای ساخت آبجکت.
- 2• `Object`: نمونه ساخته‌شده از کلاس.
- 3• `Constructor`: متد ویژه برای مقداردهی اولیه آبجکت.
- 4• `Default Constructor`: سازنده بدون آرگومان که کامپایلر می‌سازد.
- 5• `this`: ارجاع به آبجکت جاری.
- 6• `new`: عملگر ساخت آبجکت.
- 7• `Heap`: ناحیه حافظه برای آبجکت‌ها.
- 8• `Stack`: ناحیه حافظه برای متغیرهای محلی و فراخوانی متد.
- 9• `Reference`: اشاره‌گر به آبجکت در `heap`.
- 10• `Instance`: نمونه‌ای از کلاس.
- 11• `Field`: متغیر عضو کلاس.
- 12• `Method`: رفتار/تابع عضو کلاس.

۳.۲ کپسوله‌سازی (Encapsulation)

- 1• `Encapsulation`: پنهان‌سازی داده‌ها و دسترسی از طریق متد.

- 2 Access Modifiers: سطوح دسترسی (private, default, protected, public).
- 3 private: فقط داخل همان کلاس.
- 4 default: فقط داخل همان پکیج.
- 5 protected: همان پکیج + زیرکلاس‌ها.
- 6 public: از همه جا قابل دسترسی.
- 7 Getter: متد خواندن فیلد.
- 8 Setter: متد تغییر فیلد.
- 9 POJO: کلاس ساده جاوا بدون وابستگی خاص.
- 10 POJO: Java Bean با constructor بدون آرگومان، getter/setter و Serializable.

۳.۳ وراثت (Inheritance)

- 1 Inheritance: ارث‌بری ویژگی‌ها از کلاس والد.
- 2 extends: کلمه کلیدی وراثت کلاس.
- 3 IS-A: رابطه وراثت (Car IS-A Vehicle).
- 4 Superclass: کلاس والد.
- 5 Subclass: کلاس فرزند.
- 6 Parent Class: کلاس والد.
- 7 Child Class: کلاس فرزند.
- 8 Code Reusability: استفاده مجدد از کد.
- 9 Tight Coupling: وابستگی زیاد بین کلاس‌ها.
- 10 Fragile Base Class: مشکل شکستن زیرکلاس‌ها با تغییر والد.
- 11 Association: رابطه عمومی بین دو کلاس.
- 12 Aggregation: رابطه HAS-A ضعیف (جزء مستقل).
- 13 Composition: رابطه HAS-A قوی (جزء وابسته به کل).
- 14 HAS-A: رابطه مالکیت.
- 15 Uses a: رابطه استفاده موقت.
- 16 Whole-Part: رابطه کل-جزء.
- 17 Lifecycle Dependency: وابستگی چرخه حیات جزء به کل.

18• **UML**: زبان مدل‌سازی نموداری.

19• **Diamond Problem**: ابهام وراثت چندگانه.

20• **Multiple Inheritance**: ارث‌بری از چند کلاس (در جاوا فقط با **interface**).

۳.۴ چندریختی (Polymorphism)

1• **Polymorphism**: چندشکلی؛ یک نام، رفتارهای متفاوت.

2• **Compile-Time Polymorphism**: چندریختی زمان کامپایل (overloading).

3• **Runtime Polymorphism**: چندریختی زمان اجرا (overriding).

4• **Method Overloading**: چند متد هم‌نام با امضای متفاوت.

5• **Method Overriding**: بازنویسی متد والد در فرزند.

6• **@Override**: انوتیشن بررسی صحت overriding.

7• **Dynamic Method Dispatch**: انتخاب متد در زمان اجرا.

8• **Signature**: نام + پارامترهای متد.

۳.۵ انتزاع (Abstraction)

1• **Abstraction**: پنهان‌سازی جزئیات پیاده‌سازی.

2• **Abstract Class**: کلاس ناقص که نمی‌توان از آن نمونه ساخت.

3• **Interface**: قرارداد رفتاری بدون پیاده‌سازی.

4• **abstract**: کلمه کلیدی برای کلاس/متد انتزاعی.

5• **implements**: کلمه کلیدی پیاده‌سازی **interface**.

6• **default method**: متد پیش‌فرض در **interface** (جاوا ۸+).

7• **static method**: متد **static** در **interface**.

8• **Functional Interface**: **interface** با یک متد **abstract**.

9• **@FunctionalInterface**: انوتیشن تضمین تک‌متدی بودن.

10• **Contract**: قرارداد رفتاری که **interface** تعریف می‌کند.

۳.۶ مفاهیم تکمیلی

1• **static**: متعلق به کلاس نه نمونه.

- 2• **final**: غیرقابل **تغییر** (متغیر/متد/کلاس).
- 3• **static field**: فیلد مشترک بین همه نمونه‌ها.
- 4• **static method**: متد قابل فراخوانی بدون نمونه.
- 5• **final variable**: ثابت.
- 6• **final method**: متد غیرقابل **override**.
- 7• **final class**: **کلاس** غیرقابل ارث‌بری.
- 8• **main method**: نقطه ورود برنامه.
- 9• **JVM**: ماشین مجازی اجرای بایت‌کد جاوا.

♦ بخش ۴: مباحث پیشرفته پایه

۴.۱ مدیریت استثنایا (Exception Handling)

- 1• **Throwable**: کلاس پایه همه خطاها و استثنایا.
- 2• **Error**: خطای جدی سیستمی که معمولاً قابل بازیابی نیست.
- 3• **Exception**: خطای قابل مدیریت در برنامه.
- 4• **Checked Exception**: استثنای اجبار به **catch/throws** در کامپایل.
- 5• **Unchecked Exception**: استثنای زمان اجرا بدون اجبار کامپایلر.
- 6• **RuntimeException**: کلاس پایه استثنایا **unchecked**.
- 7• **OutOfMemoryError**: خطای تمام شدن حافظه **heap**.
- 8• **StackOverflowError**: خطای پر شدن **stack** (مثلاً بازگشت بی‌پایان).
- 9• **NullPointerException**: خطای دسترسی به **null**.
- 10• **FileNotFoundException**: خطای پیدا نشدن فایل.
- 11• **SQLException**: خطای مربوط به دیتابیس.
- 12• **try**: بلوک کد مشکوک به خطا.
- 13• **catch**: بلوک مدیریت استثنا.
- 14• **finally**: بلوکی که همیشه اجرا می‌شود.
- 15• **throw**: پرتاب استثنا.

- **16 throws**: اعلام استثنا در امضای متد.
- **17 System.exit()**: خروج فوری از **JVM** (**finally** اجرا نمی‌شود).
- **18 Try-with-Resources**: بستن خودکار منابع.
- **19 AutoCloseable**: اینترفیس منابع قابل بستن خودکار.
- **20 Suppressed Exceptions**: استثناهای ثانویه سرکوب‌شده.

۴.۲ جنریک‌ها (Generics)

- **1 Generics**: نوع‌دهی پارامتری برای type safety.
- **2 Type Safety**: جلوگیری از خطای نوع در زمان کامپایل.
- **3 Type Erasure**: حذف اطلاعات جنریک در زمان کامپایل.
- **4 Wildcard**: علامت ? برای نوع نامشخص.
- **5 wildcard ?** upper-bounded **T extends** (خواندن).
- **6 wildcard ?** lower-bounded **T super** (نوشتن).
- **7 Upper-Bounded Wildcard**: محدود به T و زیرکلاس‌هایش.
- **8 Lower-Bounded Wildcard**: محدود به T و والد‌هایش.
- **9 PECS**: Producer Extends, Consumer Super.
- **10 Producer Extends**: برای تولیدکننده از **extends**.
- **11 Consumer Super**: برای مصرف‌کننده از **super**.
- **12 ClassCastException**: خطای cast نامعتبر.
- **13 instanceof**: بررسی نوع آبجکت.
- **14 List<String>**: لیست نوع‌دار String.
- **15 List<Integer>**: لیست نوع‌دار Integer.

۴.۳ کالکشن‌ها (Collections Framework)

- **1 Collection**: اینترفیس پایه مجموعه‌ها.
- **2 List**: مجموعه مرتب با ایندکس و تکراری مجاز.
- **3 Set**: مجموعه بدون تکرار.
- **4 Queue**: صف برای پردازش ترتیبی.

- **5 Map**: ساختار کلید-مقدار.
- **6 ArrayList**: پیاده‌سازی List با آرایه پویا.
- **7 LinkedList**: پیاده‌سازی List/Deque با لیست پیوندی.
- **8 HashSet: Set** بدون ترتیب با hash.
- **9 LinkedHashSet: Set** با حفظ ترتیب درج.
- **10 TreeSet: Set** مرتب بر اساس درخت سرخ-سیاه.
- **11 HashMap: Map** بدون ترتیب با hash.
- **12 LinkedHashMap: Map** با حفظ ترتیب درج.
- **13 TreeMap: Map** مرتب بر اساس درخت سرخ-سیاه.
- **14 Hashtable: Map** قدیمی و thread-safe.
- **15 ConcurrentHashMap: Map** thread-safe با قفل سطح bucket.
- **16 Bucket**: سبد نگهداری عناصر در hash table.
- **17 Collision**: برخورد دو کلید در یک bucket.
- **18 Chaining**: حل collision با لیست پیوندی.
- **19 Red-Black Tree**: درخت متوازن خودتنظیم.
- **20 hashCode()**: تولید کد هش آبجکت.
- **21 equals()**: بررسی برابری منطقی.
- **22 Contract**: قرارداد hashCode/equals.
- **23 O(1)**: پیچیدگی ثابت.
- **24 O(n)**: پیچیدگی خطی.
- **25 O(log n)**: پیچیدگی لگاریتمی.
- **26 synchronized**: قفل هم‌زمانی.
- **27 bucket-level locking**: قفل در سطح bucket در ConcurrentHashMap.
- **28 Node**: گره داخلی ساختار داده.
- **29 put()**: افزودن به Map.
- **30 get()**: خواندن از Map.
- **31 contains()**: بررسی وجود عنصر.
- **32 add()**: افزودن به کالکشن.

- 33• **remove()**: حذف از کالکشن.
- 34• **Comparator**: مقایسه‌گر خارجی.
- 35• **Comparable**: مقایسه‌گر داخلی (compareTo).
- 36• **Natural Ordering**: ترتیب طبیعی.
- 37• **Insertion Order**: ترتیب درج.
- 38• **Deque**: صف دوطرفه.
- 39• **ArrayDeque**: پیاده‌سازی Deque با آرایه.
- 40• **PriorityQueue**: صف اولویت‌دار.
- 41• **Stack**: پشته LIFO.
- 42• **push()**: افزودن به پشته.
- 43• **pop()**: حذف از پشته.
- 44• **peek()**: مشاهده بدون حذف.
- 45• **offer()**: افزودن به صف.
- 46• **poll()**: حذف از صف.
- 47• **enqueue()**: افزودن به صف.
- 48• **dequeue()**: حذف از صف.

♦ بخش ۵: جاوا پیشرفته (Advanced Java)

۵.۱ برنامه‌نویسی تابعی (Functional Programming)

- 1 • **Lambda Expression**: تابع بی‌نام و مختصر $x \rightarrow x + 1$.
- 2 • **Anonymous Function**: تابع بدون نام.
- 3 • **Anonymous Inner Class**: کلاس داخلی بدون نام برای پیاده‌سازی سریع.
- 4 • **Runnable**: اینترفیس تابعی برای اجرای thread.
- 5 • **Functional Interface**: اینترفیس با یک متد **abstract**.
- 6 • **@FunctionalInterface**: انوتیشن تضمین تک‌متدی بودن.
- 7 • **java.util.function**: پکیج اینترفیس‌های تابعی.

- 8 <T>Predicate: تابع ورودی T و خروجی **boolean**.
- 9 <T, R>Function: تابع ورودی T و خروجی R.
- 10 <T>Consumer: تابع مصرف‌کننده بدون خروجی.
- 11 <T>Supplier: تابع تولیدکننده بدون ورودی.
- 12 Stream API: API پردازش اعلانی مجموعه داده‌ها.
- 13 **Stream**: جریان داده برای عملیات زنجیره‌ای.
- 14 Intermediate Operation: عملیات میانی و lazy مثل filter/map.
- 15 Terminal Operation: عملیات پایانی که Stream را اجرا می‌کند.
- 16 Lazy Evaluation: اجرای عملیات فقط هنگام نیاز.
- 17 **filter**(): فیلتر عناصر.
- 18 **map**(): تبدیل هر عنصر.
- 19 **sorted**(): مرتب‌سازی.
- 20 **distinct**(): حذف تکراری‌ها.
- 21 **limit**(): محدود کردن تعداد.
- 22 **collect**(): جمع‌آوری نتیجه.
- 23 **forEach**(): اجرای عمل روی هر عنصر.
- 24 **reduce**(): کاهش به یک مقدار.
- 25 **Collectors.toList**(): جمع‌آوری در List.
- 26 **Declarative**: سبک **اعلانی** (چه چیزی، نه چگونه).
- 27 **BinaryOperator**: تابع دو ورودی هم‌نوع با خروجی هم‌نوع.
- 28 **Memoization**: کش کردن نتیجه محاسبات تکراری.

۵.۲ هم‌روندی (Concurrency)

- 1 **Thread**: ریسه اجرای سبک.
- 2 **Runnable**: اینترفیس بدون خروجی برای thread.
- 3 <V>Callable: اینترفیس با خروجی و امکان پرتاب استثنا.
- 4 **run**(): بدنه اجرای Runnable.
- 5 **start**(): شروع thread جدید.

- 6 **call**(): بدنه اجرای Callable.
- 7 **Race Condition**: شرایط رقابتی روی منابع مشترک.
- 8 **Shared Resource**: منبع مشترک بین threadها.
- 9 **synchronized**: قفل انحصاری برای همزمانی.
- 10 **Lock**: اینترفیس قفل پیشرفته.
- 11 **Deadlock**: بن بست بین threadها.
- 12 **tryLock**(): تلاش غیرمسدودکننده برای گرفتن قفل.
- 13 **Thread Pool**: مجموعه threadهای آماده استفاده مجدد.
- 14 **ExecutorService**: مدیریت thread pool.
- 15 **Executors.newFixedThreadPool**(): ساخت pool با اندازه ثابت.
- 16 **submit**(): ارسال task به executor.
- 17 **shutdown**(): خاموشی تدریجی executor.
- 18 **CompletableFuture: API** برنامه نویسی async در جاوا 8+.
- 19 **supplyAsync**(): اجرای async با خروجی.
- 20 **thenApply**(): تبدیل نتیجه async.
- 21 **thenAccept**(): مصرف نتیجه async.
- 22 **thenCombine**(): ترکیب دو CompletableFuture.
- 23 **Asynchronous**: ناهمزمان.
- 24 **Optional**: ظرف امن برای مقدار ممکن **null**.
- 25 **Optional.ofNullable**(): ساخت Optional از مقدار ممکن **null**.
- 26 **ifPresent**(): اجرا در صورت وجود مقدار.
- 27 **orElse**(): مقدار پیش فرض در صورت خالی بودن.
- 28 **orElseThrow**(): پرتاب استثنا در صورت خالی بودن.
- 29 **map**(): تبدیل مقدار داخل Optional.
- 30 **NullPointerException**: خطای **null**.
- 31 **Default Method**: متد پیش فرض **interface**.
- 32 **Static Method**: متد **static interface**.
- 33 **Interface**: قرارداد رفتاری.

- 34• **Vehicle**: مثال کلاس انتزاعی.
- 35• **Car**: مثال کلاس فرزند.
- 36• **start**(): متد شروع.
- 37• **stop**(): متد توقف.
- 38• **service**(): متد سرویس در Servlet.

◆ بخش ۶: ساختمان داده و الگوریتم

۶.۱ تحلیل پیچیدگی (Big O)

- 1 • Big O Notation: نماد تحلیل رشد زمانی/فضایی الگوریتم.
- 2 • $O(1)$: زمان ثابت.
- 3 • $O(\log n)$: زمان لگاریتمی.
- 4 • $O(n)$: زمان خطی.
- 5 • $O(n \log n)$: زمان خطی-لگاریتمی.
- 6 • $O(n^2)$: زمان درجه دوم.
- 7 • $O(2^n)$: زمان نمایی.
- 8 • **Constant**: پیچیدگی ثابت.
- 9 • **Logarithmic**: پیچیدگی لگاریتمی.
- 10• **Linear**: پیچیدگی خطی.
- 11• **Linearithmic**: پیچیدگی $n \log n$.
- 12• **Quadratic**: پیچیدگی درجه دوم.
- 13• **Exponential**: پیچیدگی نمایی.
- 14• **Scalability**: مقیاس پذیری.
- 15• **Time Complexity**: پیچیدگی زمانی.
- 16• **Space Complexity**: پیچیدگی فضایی.
- 17• **Amortized**: تحلیل میانگین هزینه در طول زمان.
- 18• **binarySearch**: جستجوی دودویی.

- 19• **QuickSort**: مرتب‌سازی سریع.
- 20• **MergeSort**: مرتب‌سازی ادغامی.
- 21• **BubbleSort**: مرتب‌سازی حبابی.

۶.۲ ساختمان داده‌ها

- 1 • **Array**: آرایه با دسترسی $O(1)$.
- 2 • **Linked List**: لیست پیوندی با درج/حذف سریع.
- 3 • **Stack**: پشته LIFO.
- 4 • **Queue**: صف FIFO.
- 5 • **Hash Map**: نگاشت کلید-مقدار با جستجوی $O(1)$.
- 6 • **Hash Table**: جدول هش.
- 7 • **LIFO**: آخرین ورود، اولین خروج.
- 8 • **FIFO**: اولین ورود، اولین خروج.
- 9 • **Push**: افزودن به پشته.
- 10 • **Pop**: حذف از پشته.
- 11 • **Peek**: مشاهده بدون حذف.
- 12 • **Enqueue**: افزودن به صف.
- 13 • **Dequeue**: حذف از صف.
- 14 • **Bucket**: سبد در hash table.
- 15 • **Collision**: برخورد کلیدها.
- 16 • **Chaining**: حل برخورد با زنجیره.
- 17 • **Red-Black Tree**: درخت متوازن خودتنظیم.

۶.۳ الگوریتم‌ها

- 1 • **Linear Search**: جستجوی خطی $O(n)$.
- 2 • **Binary Search**: جستجوی دودویی $O(\log n)$ روی داده مرتب.
- 3 • **Bubble Sort**: مرتب‌سازی حبابی $O(n^2)$.
- 4 • **Quick Sort**: مرتب‌سازی سریع با pivot.

- 5 Merge Sort: مرتب‌سازی ادغامی پایدار.
- 6 Divide and Conquer: تقسیم و غلبه.
- 7 Dynamic Programming: برنامه‌نویسی پویا برای مسائل بهینه‌سازی.
- 8 Memoization: کش نتایج برای جلوگیری از محاسبه تکراری.
- 9 Greedy: الگوریتم حریصانه.
- 10 Backtracking: عقب‌گرد برای جستجوی کامل.
- 11 Dijkstra: کوتاه‌ترین مسیر در گراف وزن‌دار.
- 12 Eight Queens: مسئله هشت وزیر.
- 13 Sudoku: حل سودوکو.
- 14 Fibonacci: دنباله فیبوناچی.
- 15 Pivot: عنصر محوری در QuickSort.
- 16 Stable Sort: مرتب‌سازی پایدار (حفظ ترتیب عناصر برابر).

♦ بخش ۷: شبکه‌های کامپیوتری (Computer Networks)

۷.۱ مدل OSI و TCP/IP

- 1 OSI Model: مدل هفت‌لایه مرجع شبکه.
- 2 TCP/IP Model: مدل چهارلایه عملی اینترنت.
- 3 Application Layer: لایه برنامه‌های کاربر (HTTP, FTP, DNS).
- 4 Presentation Layer: لایه قالب‌بندی و رمزنگاری داده.
- 5 Session Layer: لایه مدیریت نشست.
- 6 Transport Layer: لایه انتقال (TCP/UDP) end-to-end.
- 7 Network Layer: لایه مسیریابی (IP).
- 8 Data Link Layer: لایه انتقال فریم (MAC, Ethernet).
- 9 Physical Layer: لایه سیگنال‌های فیزیکی.
- 10 HTTP: پروتکل انتقال ابرمتن.
- 11 HTTPS: HTTP امن با TLS.

- 12• **FTP**: پروتکل انتقال فایل.
- 13• **SMTP**: پروتکل ارسال ایمیل.
- 14• **DNS**: سیستم نام دامنه.
- 15• **SSL/TLS**: پروتکل‌های رمزنگاری ارتباط.
- 16• **ASCII**: کدگذاری کاراکتر متنی.
- 17• **JPEG**: فرمت فشرده تصویر.
- 18• **NetBIOS: API**: ارتباطی قدیمی شبکه‌های میکروسافتی.
- 19• **RPC**: فراخوانی رویه از راه دور.
- 20• **TCP**: پروتکل انتقال مطمئن و اتصال‌گرا.
- 21• **UDP**: پروتکل انتقال سریع و بدون اتصال.
- 22• **IP**: پروتکل آدرس‌دهی و مسیریابی.
- 23• **ICMP**: پروتکل پیام‌های **کنترلی** (ping).
- 24• **Router**: مسیریاب بین شبکه‌ها.
- 25• **Ethernet**: فناوری شبکه محلی سیمی.
- 26• **Switch**: سوئیچ لایه دو.
- 27• **MAC**: آدرس فیزیکی کارت شبکه.
- 28• **Hub**: هاب ساده لایه یک.

۷.۲ پروتکل‌های TCP و UDP

- 1 • **TCP**: اتصال‌گرا، مطمئن، با ترتیب.
- 2 • **UDP**: بدون اتصال، سریع، بدون تضمین.
- 3 • **Connection-Oriented**: نیازمند برقراری اتصال.
- 4 • **Connectionless**: بدون برقراری اتصال.
- 5 • **Acknowledgement**: تأیید دریافت داده.
- 6 • **Flow Control**: کنترل جریان داده.
- 7 • **Congestion Control**: کنترل ازدحام شبکه.
- 8 • **Overhead**: سرباره اضافی پروتکل.
- 9 • **Three-way Handshake**: برقراری اتصال TCP در سه مرحله.

10• **SYN**: بسته درخواست اتصال.

11• **SYN-ACK**: پاسخ تأیید اتصال.

12• **ACK**: تأیید دریافت.

HTTP ۷.۳

1• **HTTP**: پروتکل درخواست-پاسخ بدون حالت.

2• **HTTPS**: HTTP روی TLS.

3• **GET**: دریافت منبع.

4• **POST**: ایجاد منبع.

5• **PUT**: جایگزینی کامل منبع.

6• **PATCH**: به روزرسانی جزئی منبع.

7• **DELETE**: حذف منبع.

8• **HEAD**: دریافت فقط هدر.

9• **OPTIONS**: دریافت متدهای مجاز.

10• **Idempotent**: متدی که تکرارش نتیجه یکسان دارد.

11• **Safe**: متدی بدون تغییر سرور.

12• **Status Code**: کد وضعیت پاسخ.

13• **OK 200**: موفقیت.

14• **Created 201**: منبع ساخته شد.

15• **No Content 204**: موفقیت بدون بدنه.

16• **Moved Permanently 301**: انتقال دائمی.

17• **Found 302**: انتقال موقت.

18• **Bad Request 400**: درخواست نامعتبر.

19• **Unauthorized 401**: نیاز به احراز هویت.

20• **Forbidden 403**: دسترسی ممنوع.

21• **Not Found 404**: منبع پیدا نشد.

22• **Method Not Allowed 405**: متد غیرمجاز.

23• **Internal Server Error 500**: خطای داخلی سرور.

- 24• Bad Gateway 502: پاسخ نامعتبر از سرور بالادست.
- 25• Service Unavailable 503: سرویس در دسترس نیست.
- 26• Header: بخش متادیتای درخواست/پاسخ.
- 27• Body: بخش داده اصلی.
- 28• Content-Type: نوع محتوای بدنه.
- 29• Authorization: هدر احراز هویت.
- 30• Bearer Token: توکن احراز هویت.
- 31• Accept: نوع محتوای قابل قبول.
- 32• Host: نام میزبان مقصد.

۷.۴ مفاهیم پیشرفته شبکه

- 1• REST API: مبتنی بر منابع و متدهای HTTP.
- 2• Stateless: بدون نگهداری وضعیت بین درخواست‌ها.
- 3• Resource-Based: مبتنی بر منابع.
- 4• HTTP Verbs: متدهای HTTP.
- 5• JSON: فرمت داده سبک.
- 6• XML: فرمت داده نشانه‌گذاری.
- 7• HTTP/1.1: نسخه کلاسیک HTTP با keep-alive.
- 8• HTTP/2: نسخه چندگانه با multiplexing.
- 9• Head-of-Line Blocking: مسدود شدن صف به‌خاطر یک درخواست.
- 10• Multiplexing: ارسال چند درخواست روی یک اتصال.
- 11• Header Compression: فشرده‌سازی هدرها.
- 12• HPACK: الگوریتم فشرده‌سازی هدر در HTTP/2.
- 13• Server Push: ارسال پیش‌دستانه منابع توسط سرور.
- 14• Binary: پروتکل دودویی (HTTP/2).
- 15• Text-based: پروتکل متنی (HTTP/1.1).
- 16• CORS: اشتراک منابع بین دامنه‌ای.
- 17• Same-Origin Policy: سیاست امنیتی مرورگر.

- 18• **Preflight**: درخواست OPTIONS پیش از درخواست اصلی.
- 19• **Access-Control-Allow-Origin**: هدر مجوز CORS.
- 20• **SSL/TLS**: لایه امنیت ارتباط.
- 21• **Certificate**: گواهی دیجیتال سرور.
- 22• **DNS**: سیستم نام دامنه.
- 23• **IP Address**: آدرس عددی دستگاه.
- 24• **Domain Name**: نام خوانا مثل `google.com`.
- 25• **DNS Resolver**: حل کننده نام دامنه.
- 26• **DNS Lookup**: فرآیند تبدیل نام به IP.
- 27• **TCP Connection**: اتصال TCP.
- 28• **TLS Handshake**: دست دادن برای برقراری امنیت.

♦ بخش ۸: سیستم عامل (Operating System)

۸.۱ پردازش و ریس

- 1• **Process**: برنامه در حال اجرا با فضای حافظه مستقل.
- 2• **Thread**: ریس سبک داخل process با حافظه مشترک.
- 3• **Heap**: ناحیه حافظه پویا برای آبجکت ها.
- 4• **Stack**: ناحیه حافظه برای متغیرهای محلی و فراخوانی ها.
- 5• **Code Segment**: بخش کد برنامه در حافظه.
- 6• **Data Segment**: بخش داده های سراسری و **static**.
- 7• **IPC (Inter-Process Communication)**: ارتباط بین process ها.
- 8• **Socket**: نقطه پایانی ارتباط شبکه ای/بین پردازش های.
- 9• **Pipe**: کانال ارتباطی یک طرفه بین process ها.
- 10• **Race Condition**: شرایط رقابتی روی منابع مشترک.
- 11• **Lightweight Process**: نام دیگر thread.
- 12• **Context Switch**: تعویض context بین process/thread ها.

13• Page Tables: جدول نگاشت صفحات حافظه مجازی.

14• TLB: کش ترجمه آدرس‌های مجازی به فیزیکی.

15• Program Counter: اشاره‌گر به دستور بعدی.

16• Stack Pointer: اشاره‌گر به بالای stack.

۸.۲ مدیریت حافظه

1 • Virtual Memory: حافظه مجازی بزرگ‌تر از RAM فیزیکی.

2 • RAM: حافظه اصلی فرار.

3 • Disk: حافظه جانبی غیرفرار.

4 • Memory Layout: چیدمان حافظه برنامه.

5 • Text Segment: بخش کد اجرایی.

6 • Data Segment: بخش داده‌های سراسری.

7 • Stack: ناحیه متغیرهای محلی.

8 • Heap: ناحیه تخصیص پویا.

9 • Dynamic Memory Allocation: تخصیص حافظه در زمان اجرا.

10• Garbage Collector: جمع‌آورنده خودکار حافظه بی‌استفاده.

11• StackOverflowError: خطای پر شدن stack.

۸.۳ هم‌روندی و بن‌بست

1 • Deadlock: بن‌بست؛ انتظار چرخشی threadها.

2 • Coffman Conditions: چهار شرط لازم برای deadlock.

3 • Mutual Exclusion: انحصار منابع.

4 • Hold and Wait: نگه‌داشتن منبع و انتظار برای دیگری.

5 • No Preemption: عدم امکان گرفتن منبع از thread.

6 • Circular Wait: انتظار چرخشی.

7 • tryLock(): تلاش غیرمسدودکننده برای گرفتن قفل.

8 • ReentrantLock: قفل قابل ورود مجدد.

9 • Semaphore: شمارنده کنترل دسترسی به منابع.

10• Lock: قفل هم‌زمانی.

۸.۴ زمان‌بندی CPU

- 1• CPU Scheduling: انتخاب process بعدی برای اجرا.
- 2• Context Switch: تعویض process/thread روی CPU.
- 3• Dispatcher: ماژول تعویض context.
- 4• Process Context Switch: تعویض کامل process.
- 5• Thread Context Switch: تعویض سبک thread.
- 6• TLB Flush: پاک کردن TLB هنگام تعویض.

۸.۵ مفاهیم سیستمی

- 1• Virtual Memory: حافظه مجازی.
- 2• Cache: حافظه موقت سریع.
- 3• Redis: دیتابیس/کش درون حافظه‌ای.
- 4• Ehcache: کتابخانه کش جاوا.
- 5• Lock: قفل.
- 6• Semaphore: سمافور.

♦ بخش ۹: امنیت (Security)

۹.۱ مفاهیم پایه

- 1• Authentication: احراز هویت (چه کسی هستی؟).
- 2• Authorization: مجوزدهی (چه کاری می‌توانی بکنی؟).
- 3• Unauthorized 401: عدم احراز هویت.
- 4• Forbidden 403: احراز هویت شده اما بدون مجوز.
- 5• Plain Text: متن بدون رمزنگاری.
- 6• Hashing: تبدیل یک‌طرفه داده به هش.

- 7 Salt: مقدار تصادفی افزوده به رمز قبل از hash.
- 8 Rainbow Table Attack: حمله با جدول هش‌های آماده.
- 9 BCrypt: الگوریتم هش رمز کند و امن.
- 10 SCrypt: الگوریتم هش مقاوم به حافظه.
- 11 Argon2: برنده مسابقه هش رمز، مقاوم به GPU.
- 12 BCryptPasswordEncoder: پیاده‌سازی BCrypt در Spring Security.
- 13 Brute-Force: حمله آزمون همه ترکیب‌ها.
- 14 One-way: تابع غیرقابل برگشت.

JWT ۹.۲

- 1 JWT (JSON Web Token): توکن استاندارد برای احراز هویت بدون حالت.
- 2 RFC 7519: استاندارد رسمی JWT.
- 3 Claims: ادعاهای داخل payload.
- 4 Stateless: بدون نگهداری session در سرور.
- 5 Session: نگهداری وضعیت کاربر در سرور.
- 6 Header: بخش الگوریتم و نوع توکن.
- 7 Payload: بخش داده‌های JWT.
- 8 Signature: امضای دیجیتال JWT.
- 9 Base64-encoded: کدگذاری Base64 بخش‌های JWT.
- 10 HMACSHA256: الگوریتم امضای متقارن.
- 11 SECRET_KEY: کلید مخفی امضا.
- 12 Authorization: Bearer: هدر ارسال توکن.
- 13 Tamper: دستکاری توکن.
- 14 Local Storage: ذخیره‌سازی سمت مرورگر.
- 15 Cookie: ذخیره‌سازی کوچک سمت کلاینت.
- 16 Expiration: زمان انقضای توکن.
- 17 JwtService: سرویس ساخت/اعتبارسنجی JWT.
- 18 Jwts.builder(): سازنده JWT در JJWT.

- 19 • `Jwts.parser()`: تجزیه‌کننده JWT.
- 20 • `JwtException`: خطای مربوط به JWT.
- 21 • `JwtAuthenticationFilter`: فیلتر احراز هویت با JWT.
- 22 • `OncePerRequestFilter`: فیلتر اجرا شده یک بار در هر درخواست.
- 23 • `SecurityContextHolder`: نگهدارنده `SecurityContext`.
- 24 • `UsernamePasswordAuthenticationToken`: توکن احراز هویت Spring.

۹.۳ آسیب‌پذیری‌ها (OWASP)

- 1 • SQL Injection: تزریق SQL از ورودی کاربر.
- 2 • `PreparedStatement`: جلوگیری از SQL Injection با پارامتر.
- 3 • `Parameterized Queries`: کوئری پارامتری.
- 4 • `XSS` (Cross-Site Scripting): تزریق اسکریپت در مرورگر.
- 5 • `CSRF` (Cross-Site Request Forgery): جعل درخواست از سایت دیگر.
- 6 • `CSRF Token`: توکن ضد CSRF.
- 7 • `Escape`: خنثی‌سازی کاراکترهای خاص.
- 8 • `Encode`: کدگذاری خروجی.
- 9 • `Thymeleaf`: قالب‌ساز امن Spring.
- 10 • OWASP Top 10: فهرست ده آسیب‌پذیری مهم وب.

۹.۴ Spring Security

- 1 • Spring Security: فریم‌ورک امنیت Spring.
- 2 • `UsernamePasswordAuthenticationFilter`: فیلتر ورود با نام کاربری/رمز.
- 3 • `AuthenticationManager`: مدیر احراز هویت.
- 4 • `PasswordEncoder`: اینترفیس هش رمز.
- 5 • `BCryptPasswordEncoder`: پیاده‌سازی `BCrypt`.
- 6 • `SCryptPasswordEncoder`: پیاده‌سازی `SCrypt`.
- 7 • `Argon2PasswordEncoder`: پیاده‌سازی `Argon2`.
- 8 • `UserDetailsService`: سرویس بارگذاری کاربر.

- 9 **UserDetails**: اینترفیس اطلاعات کاربر.
- 10 **UsernameNotFoundException**: خطای پیدا نشدن کاربر.
- 11 **SimpleGrantedAuthority**: پیاده‌سازی ساده **Authority**.
- 12 **SecurityContext**: نگهدارنده اطلاعات احراز هویت.
- 13 **SecurityContextHolder**: دسترسی به **SecurityContext**.
- 14 **JwtService**: سرویس **JWT**.
- 15 **JwtAuthenticationFilter**: فیلتر **JWT**.
- 16 **SecurityFilterChain**: زنجیره فیلترهای امنیتی.
- 17 **EnableWebSecurity@**: فعال‌سازی امنیت وب.
- 18 **EnableMethodSecurity@**: فعال‌سازی امنیت سطح متد.
- 19 **authorizeHttpRequests**: پیکربندی مجوز درخواست‌ها.
- 20 **requestMatchers**: تطبیق مسیرها.
- 21 **permitAll**: اجازه به همه.
- 22 **hasRole**: بررسی نقش.
- 23 **hasAnyRole**: بررسی چند نقش.
- 24 **authenticated**: فقط کاربران احراز هویت شده.
- 25 **csrf**: پیکربندی **CSRF**.
- 26 **sessionManagement**: مدیریت **session**.
- 27 **SessionCreationPolicy.STATELESS**: بدون ساخت **session**.
- 28 **addFilterBefore**: افزودن فیلتر قبل از فیلتر دیگر.
- 29 **PreAuthorize@**: بررسی مجوز قبل از اجرای متد.
- 30 **Secured@**: بررسی نقش ساده.
- 31 **hasRole('ADMIN')**: نیاز به نقش **ADMIN**.
- 32 **hasAnyRole('USER', 'ADMIN')**: نیاز به یکی از نقش‌ها.
- 33 **authentication.principal.id**: دسترسی به **id** کاربر جاری.
- 34 **AuthenticationEntryPoint**: نقطه ورود برای احراز هویت.
- 35 **commence()**: متد شروع چالش احراز هویت.
- 36 **SC_UNAUTHORIZED**: کد وضعیت 401.

37. APPLICATION_JSON_VALUE: MediaType: نوع محتوای JSON.

♦ بخش ۱۰: دیزاین پترن‌ها (Design Patterns)

۱۰.۱ Creational (ایجاد)

- 1 Singleton: تنها یک نمونه از کلاس در کل برنامه.
- 2 Factory Method: ساخت آبجکت با متد کارخانه‌ای و بدون افشای منطق ساخت.
- 3 Abstract Factory: ساخت خانواده‌ای از آبجکت‌های مرتبط.
- 4 Builder: ساخت گام‌به‌گام آبجکت پیچیده.
- 5 Prototype: ساخت آبجکت جدید با clone از نمونه موجود.
- 6 GameManager: مثال Singleton برای مدیریت بازی.
- 7 ConfigManager: مثال Singleton برای تنظیمات.
- 8 InputManager: مثال Singleton برای ورودی.
- 9 GameConfig: مثال Singleton برای پیکربندی بازی.
- 10 Logger: مثال Singleton برای لاگ.
- 11 EnemyFactory: مثال Factory برای ساخت دشمن.
- 12 Orc: مثال محصول Factory.
- 13 Troll: مثال محصول Factory.
- 14 Transport: مثال Abstract Factory برای حمل‌ونقل.
- 15 Truck: مثال محصول Abstract Factory.
- 16 Ship: مثال محصول Abstract Factory.
- 17 Airplane: مثال محصول Abstract Factory.
- 18 Logistics: مثال Abstract Factory.
- 19 RoadLogistics: کارخانه جاده‌ای.
- 20 SeaLogistics: کارخانه دریایی.
- 21 SkyLogistics: کارخانه هوایی.
- 22 GUIFactory: کارخانه Abstract برای UI.
- 23 Button: مثال محصول UI.

- 24• **Checkbox**: مثال محصول UI.
- 25• **WindowsButton**: دکمه ویندوزی.
- 26• **MacButton**: دکمه مک.
- 27• **WindowsFactory**: کارخانه UI ویندوز.
- 28• **MacFactory**: کارخانه UI مک.
- 29• **Character**: مثال Builder برای شخصیت بازی.
- 30• **Builder**: سازنده گام به گام.
- 31• **House**: مثال Builder برای خانه.
- 32• **HousePlan**: نقشه خانه.
- 33• **HouseBuilder**: سازنده خانه.
- 34• **IglooHouseBuilder**: سازنده ایگلو.
- 35• **TipiHouseBuilder**: سازنده تپپی.
- 36• **CivilEngineer**: مهندس ناظر ساخت.
- 37• **clone()**: متد کلون کردن آبجکت.
- 38• **Cloneable**: اینترفیس نشانه گذاری برای clone.
- 39• **CloneNotSupportedException**: خطای عدم پشتیبانی clone.

۱۰.۲ Structural (ساختاری)

- 1 • **Adapter**: تبدیل **interface** یک کلاس به **interface** مورد انتظار.
- 2 • **Bridge**: جدا کردن abstraction از implementation.
- 3 • **Composite**: ساختار درختی از آبجکت ها.
- 4 • **Decorator**: افزودن رفتار به آبجکت به صورت پویا.
- 5 • **Facade: interface** ساده برای زیرسیستم پیچیده.
- 6 • **Flyweight**: اشتراک گذاری آبجکت های مشابه برای صرفه جویی حافظه.
- 7 • **Proxy**: نماینده/واسط برای کنترل دسترسی.
- 8 • **GameInput**: مثال **interface** ورودی بازی.
- 9 • **ExternalInputSystem**: سیستم ورودی خارجی.
- 10• **InputAdapter**: آداپتور ورودی.

- 11• **Renderer**: مثال Bridge برای رندر.
- 12• **VectorRenderer**: رندر برداری.
- 13• **RasterRenderer**: رندر رستری.
- 14• **Circle**: مثال Composite/Bridge.
- 15• **GameObject**: مثال Composite.
- 16• **Sprite**: مثال Composite.
- 17• **Group**: گروه آبجکت‌ها در Composite.
- 18• **Weapon**: مثال Decorator.
- 19• **BasicSword**: شمشیر پایه.
- 20• **FireDecorator**: دکوراتور آتش.
- 21• **AudioSystem**: مثال Facade.
- 22• **PhysicsSystem**: مثال Facade.
- 23• **GameEngineFacade**: فاساد موتور بازی.
- 24• **TreeType**: مثال Flyweight.
- 25• **Tree**: مثال Flyweight.
- 26• **Image**: مثال Proxy.
- 27• **RealImage**: تصویر واقعی.
- 28• **ImageProxy**: پروکسی تصویر.

۱۰.۳ Behavioral (رفتاری)

- 1 • **Strategy**: انتخاب الگوریتم در زمان اجرا.
- 2 • **Observer**: اطلاع‌رسانی تغییرات به observerها.
- 3 • **Command**: کپسوله‌سازی درخواست به صورت آبجکت.
- 4 • **Iterator**: پیمایش عناصر بدون افشای ساختار.
- 5 • **Mediator**: واسطه ارتباط بین آبجکت‌ها.
- 6 • **Memento**: ذخیره و بازیابی وضعیت.
- 7 • **State**: تغییر رفتار بر اساس وضعیت.
- 8 • **Template Method**: اسکلت الگوریتم با گام‌های قابل بازنویسی.

- 9 Visitor: عملیات جدید بدون تغییر کلاس‌ها.
- 10 Chain of Responsibility: زنجیره handlerها.
- 11 PaymentStrategy: مثال Strategy پرداخت.
- 12 CreditCardPayment: پرداخت با کارت.
- 13 CryptoPayment: پرداخت با کریپتو.
- 14 PaymentProcessor: پردازشگر پرداخت.
- 15 RouteStrategy: استراتژی مسیریابی.
- 16 Navigator: ناوبر.
- 17 CarRouteStrategy: مسیر ماشین.
- 18 WalkingRouteStrategy: مسیر پیاده.
- 19 Observer: ناظر.
- 20 Player: مثال Observer.
- 21 Command: فرمان.
- 22 JumpCommand: فرمان پرش.
- 23 InputHandler: مدیریت ورودی.
- 24 Television: مثال Command.
- 25 TurnOnCommand: فرمان روشن.
- 26 TurnOffCommand: فرمان خاموش.
- 27 RemoteControllerInvoker: کنترل از راه دور.
- 28 Iterator: پیمایشگر.
- 29 Mediator: واسطه.
- 30 Component: جزء در Mediator.
- 31 GameState: وضعیت بازی در State.
- 32 Game: بازی.
- 33 PlayerState: وضعیت بازیکن.
- 34 StandingState: ایستاده.
- 35 RunningState: در حال دویدن.
- 36 DocumentState: وضعیت سند.

- 37 Document: سند.
- 38 DraftState: پیش نویس.
- 39 ModerationState: در حال بررسی.
- 40 PublishState: منتشرشده.
- 41 GameLoop: حلقه بازی.
- 42 Visitor: بازدیدکننده.
- 43 Element: عنصر در Visitor.
- 44 Handler: هندلر در Chain of Responsibility.
- 45 AuthHandler: هندلر احراز هویت.

۱۰.۴ Concurrency (همروندی)

- 1 Active Object: جدا کردن فراخوانی متد از اجرا با صف پیام.
- 2 Monitor Object: قفل و همزمانی داخلی در آبجکت.

♦ بخش ۱۱: Spring Boot

۱۱.۱ چرخه حیات Bean

- 1 Bean: آبجکت مدیریت شده توسط Spring Container.
- 2 Instantiation: ساخت نمونه Bean.
- 3 Dependency Injection: تزریق وابستگی ها به Bean.
- 4 Initialization: مرحله مقداردهی اولیه Bean.
- 5 @PostConstruct: انوتیشن متد اجراشده بعد از مقداردهی اولیه.
- 6 afterPropertiesSet: متد InitializingBean اجراشده بعد از پر شدن property ها.
- 7 @PreDestroy: انوتیشن متد اجراشده قبل از نابودی Bean.
- 8 DisposableBean: اینترفیس با متد destroy.
- 9 InitializingBean: اینترفیس با متد afterPropertiesSet.
- 10 BeanNameAware: اینترفیس دریافت نام Bean.
- 11 BeanClassLoaderAware: اینترفیس دریافت ClassLoader.

12• **BeanFactoryAware**: اینترفیس دریافت BeanFactory.

13• **setBeanName**: متد دریافت نام Bean.

14• **setBeanClassLoader**: متد دریافت ClassLoader.

15• **setBeanFactory**: متد دریافت BeanFactory.

۱۱.۲ تزریق وابستگی (DI)

1 • Constructor Injection: تزریق از طریق سازنده (توصیه شده).

2 • Setter Injection: تزریق از طریق setter.

3 • Field Injection: تزریق مستقیم روی فیلد (توصیه نشده).

4 • **@Autowired**: انوتیشن تزریق خودکار.

5 • **@Qualifier**: انتخاب Bean خاص در تزریق.

6 • **@Primary**: Bean پیش فرض در تزریق.

7 • **@Lazy**: ساخت Bean با تأخیر.

8 • **@Bean**: تعریف Bean در کلاس Configuration.

9 • **@Configuration**: کلاس تعریف Bean ها.

10 • **@Import**: وارد کردن کلاس های Configuration.

11 • **@Component**: نشانه گذاری کلاس به عنوان Bean.

12 • **@Service**: Component لایه سرویس.

13 • **@Repository**: Component لایه دسترسی داده.

14 • **@Controller**: Component لایه کنترلر MVC.

15 • **@RestController**: کنترلر REST با خروجی JSON.

16 • **@ResponseBody**: بدنه پاسخ مستقیماً برگردانده می شود.

17 • **@RequestMapping**: نگاشت مسیر درخواست.

18 • **@GetMapping**: نگاشت GET.

19 • **@PostMapping**: نگاشت POST.

20 • **@PutMapping**: نگاشت PUT.

21 • **@DeleteMapping**: نگاشت DELETE.

22 • **@PathVariable**: استخراج متغیر از مسیر.

- 23• **RequestBody@**: استخراج بدنه درخواست.
- 24• **ModelAttribute@**: اتصال پارامترها به مدل.

Transactional@ ۱۱.۳

- 1• **Transactional@**: مدیریت تراکنش به صورت اعلانی.
- 2• **Proxy**: واسط ایجادشده برای اعمال تراکنش.
- 3• **Transaction**: واحد کاری اتمیک.
- 4• **Commit**: تأیید نهایی تراکنش.
- 5• **Rollback**: بازگردانی تراکنش.
- 6• **RuntimeException**: باعث rollback پیش فرض.
- 7• **Checked Exception**: باعث rollback پیش فرض نمی شود.
- 8• **rollbackFor**: تعیین استثنای باعث rollback.

Scope ۱۱.۴ها

- 1• **singleton**: یک نمونه در کل **ApplicationContext** (پیش فرض).
- 2• **prototype**: نمونه جدید در هر درخواست.
- 3• **request**: یک نمونه در هر HTTP request.
- 4• **session**: یک نمونه در هر HTTP session.
- 5• **application**: یک نمونه در کل **ServletContext**.
- 6• **Scope@**: تعیین scope برای Bean.

Conditional Beans ۱۱.۵

- 1• **ConditionalOnClass@**: ساخت Bean در صورت وجود کلاس.
- 2• **ConditionalOnMissingClass@**: ساخت Bean در صورت نبود کلاس.
- 3• **ConditionalOnProperty@**: ساخت Bean بر اساس property.
- 4• **ConditionalOnBean@**: ساخت Bean در صورت وجود Bean دیگر.
- 5• **havingValue**: مقدار مورد انتظار property.

۱۱.۶ تنظیمات

- 1 application.properties: فایل تنظیمات Spring Boot.
- 2 server.port: پورت سرور.
- 3 server.servlet.context-path: مسیر پایه اپلیکیشن.
- 4 spring.datasource.url: آدرس دیتابیس.
- 5 spring.datasource.username: نام کاربری دیتابیس.
- 6 spring.datasource.password: رمز دیتابیس.
- 7 spring.jpa.hibernate.ddl-auto: حالت ساخت/به‌روزرسانی schema.
- 8 spring.jpa.show-sql: نمایش کوئری‌های SQL.
- 9 spring.jpa.properties.hibernate.dialect: دیالکت دیتابیس.
- 10 logging.level: سطح لاگ‌گیری.

بخش ۱۲: JPA، JDBC و Hibernate

۱۲.۱ JDBC

- 1 API: JDBC (Java Database Connectivity) سطح‌پایین اتصال جاوا به دیتابیس.
- 2 Driver: درایور اختصاصی هر دیتابیس.
- 3 Class.forName(): بارگذاری کلاس درایور.
- 4 DriverManager.getConnection(): برقراری اتصال به دیتابیس.
- 5 Connection: نماینده اتصال به دیتابیس.
- 6 Statement: اجرای کوئری ساده.
- 7 PreparedStatement: اجرای کوئری پارامتری و امن.
- 8 ResultSet: نتیجه کوئری SELECT.
- 9 executeQuery(): اجرای کوئری SELECT.
- 10 executeUpdate(): اجرای INSERT/UPDATE/DELETE.
- 11 rs.next(): رفتن به ردیف بعدی ResultSet.
- 12 rs.getString(): خواندن ستون String.

- 13• `rs.getInt()`: خواندن ستون `int`.
- 14• `close()`: بستن منبع.
- 15• SQL Injection: تزریق SQL از ورودی.
- 16• `concatenation`: چسباندن رشته‌ها (خطر SQL Injection).

JPA ۱۲.۲

- 1 • JPA (Java Persistence API): استاندارد ORM در جاوا.
- 2 • `ORM`: نگاشت آبجکت به جدول دیتابیس.
- 3 • `Entity`: کلاس نگاشت شده به جدول.
- 4 • `@Entity`: انوتیشن تعریف `Entity`.
- 5 • `@Table`: تعیین نام جدول.
- 6 • `@Id`: تعیین کلید اصلی.
- 7 • `@GeneratedValue`: تولید خودکار کلید.
- 8 • `GenerationType.IDENTITY`: تولید با `auto-increment` دیتابیس.
- 9 • `@Column`: تنظیم ستون.
- 10 • `@OneToMany`: رابطه یک به چند.
- 11 • `@ManyToOne`: رابطه چند به یک.
- 12 • `@JoinColumn`: تعیین ستون کلید خارجی.
- 13 • `@mappedBy`: سمت معکوس رابطه.
- 14 • `cascade`: انتشار عملیات به موجودیت‌های مرتبط.
- 15 • `CascadeType.ALL`: همه عملیات `cascade`.
- 16 • `FetchType`: نحوه بارگذاری رابطه.
- 17 • `EAGER`: بارگذاری فوری.
- 18 • `LAZY`: بارگذاری با تأخیر.
- 19 • `N+1 Query Problem`: اجرای N کوئری اضافی برای روابط.
- 20 • `JOIN FETCH`: بارگذاری رابطه با یک کوئری.
- 21 • `EntityGraph`: گراف بارگذاری موجودیت.
- 22 • `@NamedEntityGraph`: تعریف `EntityGraph` نام‌دار.

23• @NamedAttributeNode: تعیین attribute در EntityGraph.

۱۲.۳ Hibernate

1• Hibernate: پیاده‌سازی محبوب JPA.

2• Session: جلسه کار با دیتابیس در Hibernate.

3• EntityManager: اینترفیس JPA برای مدیریت موجودیت‌ها.

4• Query: کوئری JPQL.

5• uniqueResult(): دریافت نتیجه یکتا.

6• createQuery(): ساخت کوئری.

7• setParameter(): مقداردهی پارامتر کوئری.

۱۲.۴ Spring Data JPA

1• Spring Data JPA: لایه انتزاعی روی JPA.

2• JpaRepository: اینترفیس CRUD + امکانات پیشرفته.

3• CrudRepository: اینترفیس CRUD ساده.

4• PagingAndSortingRepository: اینترفیس صفحه‌بندی و مرتب‌سازی.

5• findByUsername(): جستجو بر اساس نام کاربری.

6• findByUsernameAndPassword(): جستجو با دو شرط.

7• findByAgeGreaterThan(): جستجو با شرط بزرگ‌تر.

8• findByUsernameStartingWith(): جستجو با شروع رشته.

9• findAllOrderByUsernameAsc(): همه با ترتیب صعودی.

10• findAll(Pageable): صفحه‌بندی نتایج.

11• existsByUsername(): بررسی وجود.

12• countByAgeGreaterThan(): شمارش با شرط.

13• deleteByUsername(): حذف با شرط.

14• Query@: تعریف کوئری دلخواه.

15• Param@: نام‌گذاری پارامتر کوئری.

16• nativeQuery: استفاده از SQL خام.

- 17• **Modifying@**: برای کوئری‌های UPDATE/DELETE.
- 18• **Pageable**: اینترفیس صفحه‌بندی.
- 19• **PageRequest**: پیاده‌سازی Pageable.
- 20• **Sort**: مرتب‌سازی.
- 21• **Page**: نتیجه صفحه‌بندی‌شده.
- 22• **Specification**: ساخت شرط‌های پویا.
- 23• **Specification.where()**: شروع ساخت Specification.

بخش ۱۳: DevOps، Docker، Kubernetes

DevOps ۱۳.۱

- 1• **DevOps**: فرهنگ همکاری توسعه و عملیات.
- 2• **CI/CD**: یکپارچه‌سازی و تحویل/استقرار مداوم.
- 3• **Continuous Integration**: ادغام مکرر کد در شاخه اصلی.
- 4• **Continuous Deployment**: استقرار خودکار تا تولید.
- 5• **Jenkins**: سرور CI/CD متن‌باز.
- 6• **GitHub Actions: CI/CD**: داخلی گیت‌هاب.
- 7• **GitLab CI: CI/CD**: داخلی گیت‌لب.
- 8• **CircleCI**: سرویس CI/CD ابری.
- 9• **Plan**: مرحله برنامه‌ریزی.
- 10• **Code**: مرحله کدنویسی.
- 11• **Build**: مرحله ساخت.
- 12• **Test**: مرحله تست.
- 13• **Release**: مرحله انتشار.
- 14• **Deploy**: مرحله استقرار.
- 15• **Operate**: مرحله بهره‌برداری.
- 16• **Monitor**: مرحله پایش.

Docker ۱۳.۲

- **Docker 1**: پلتفرم کانتینری سازی.
- **Container 2**: نمونه اجرایی سبک از ایمج.
- **Image 3**: قالب فقط خواندنی برای ساخت کانتینر.
- **Dockerfile 4**: فایل دستورات ساخت ایمج.
- **FROM 5**: تعیین ایمج پایه.
- **ENV 6**: تعیین متغیر محیطی.
- **WORKDIR 7**: تعیین دایرکتوری کاری.
- **COPY 8**: کپی فایل به ایمج.
- **EXPOSE 9**: اعلام پورت.
- **ENTRYPOINT 10**: دستور اجرای اصلی کانتینر.
- **docker build 11**: ساخت ایمج.
- **docker run 12**: اجرای کانتینر.
- **docker ps 13**: لیست کانتینرهای در حال اجرا.
- **docker stop 14**: توقف کانتینر.
- **docker rm 15**: حذف کانتینر.
- **docker images 16**: لیست ایمج‌ها.
- **docker exec 17**: اجرای دستور در کانتینر در حال اجرا.
- **docker logs 18**: مشاهده لاگ کانتینر.
- **Docker Compose 19**: ابزار مدیریت چند کانتینر.
- **docker-compose.yml 20**: فایل تعریف سرویس‌ها.
- **services 21**: بخش تعریف سرویس‌ها.
- **depends_on 22**: وابستگی ترتیبی سرویس‌ها.
- **environment 23**: متغیرهای محیطی.
- **ports 24**: نگاشت پورت‌ها.
- **docker-compose up 25**: اجرای سرویس‌ها.
- **docker-compose down 26**: توقف و حذف سرویس‌ها.
- **Virtual Machine 27**: ماشین مجازی با OS کامل.
- **Host OS 28**: سیستم عامل میزبان.

29• Base Image: ایمج پایه.

Kubernetes ۱۳.۳

- 1 **Kubernetes**: ارکستراتور کانتینر.
- 2 **K8s**: مخفف Kubernetes.
- 3 **Pod**: کوچک‌ترین واحد اجرایی شامل یک یا چند کانتینر.
- 4 **Deployment**: مدیریت نسخه‌ها و replica های Pod.
- 5 **Service**: نقطه دسترسی پایدار به Pod ها.
- 6 **Ingress**: مدیریت دسترسی HTTP/HTTPS خارجی.
- 7 **ConfigMap**: نگهداری تنظیمات غیرحساس.
- 8 **Secret**: نگهداری داده‌های حساس.
- 9 **kubectl**: ابزار خط فرمان Kubernetes.
- 10 **kubectl apply**: اعمال manifest.
- 11 **kubectl get pods**: لیست Pod ها.
- 12 **kubectl get services**: لیست سرویس‌ها.
- 13 **kubectl get deployments**: لیست Deployment ها.
- 14 **kubectl scale**: تغییر تعداد replica.
- 15 **replicas**: تعداد نمونه‌های Pod.
- 16 **LoadBalancer**: نوع سرویس با IP خارجی.
- 17 **containerPort**: پورت داخلی کانتینر.
- 18 **targetPort**: پورت مقصد در Pod.

Kafka ۱۳.۴

- 1 **Apache Kafka**: پلتفرم streaming توزیع شده.
- 2 **Producer**: تولیدکننده پیام.
- 3 **Consumer**: مصرف‌کننده پیام.
- 4 **Topic**: کانال پیام‌ها.
- 5 **Partition**: بخش‌های موازی Topic.

- **6 Broker**: سرور Kafka.
- **7 KafkaTemplate**: کلاس ارسال پیام در Spring Kafka.
- **8 KafkaListener@**: انوتیشن مصرف پیام.
- **9 groupId**: شناسه گروه مصرف کننده.
- **10 Event Sourcing**: ذخیره رویدادها به جای وضعیت.
- **11 Log Aggregation**: جمع آوری لاگ ها.
- **12 Stream Processing**: پردازش جریانی داده.
- **13 Message Queue**: صف پیام.

۱۳.۵ Linux

- **1 ls**: لیست فایل ها.
- **2 cd**: تغییر دایرکتوری.
- **3 mkdir**: ساخت دایرکتوری.
- **4 rm**: حذف فایل / دایرکتوری.
- **5 cp**: کپی.
- **6 mv**: انتقال / تغییر نام.
- **7 cat**: نمایش محتوای فایل.
- **8 less**: نمایش صفحه به صفحه فایل.
- **9 tail**: نمایش انتهای فایل.
- **10 head**: نمایش ابتدای فایل.
- **11 ps**: لیست process ها.
- **12 top**: نمایش زنده process ها.
- **13 kill**: ارسال سیگنال به process.
- **14 nohup**: اجرای process مستقل از terminal.
- **15 ping**: بررسی اتصال شبکه.
- **16 netstat**: نمایش اتصالات شبکه.
- **17 curl**: ارسال درخواست HTTP.
- **18 ss**: نمایش سوکت ها.

- 19 df: فضای دیسک.
- 20 du: حجم دایرکتوری.
- 21 free: وضعیت حافظه.
- 22 apt-get: مدیریت بسته Debian/Ubuntu.
- 23 yum: مدیریت بسته RedHat/CentOS.
- 24 Debian: توزیع پایه اوبونتو.
- 25 Ubuntu: توزیع محبوب Debian-based.
- 26 RedHat: توزیع تجاری لینوکس.
- 27 CentOS: توزیع متن‌باز RedHat-based.
- 28 APT: سیستم مدیریت بسته Debian.
- 29 RPM: سیستم مدیریت بسته RedHat.

◆ بخش ۱۴: تست‌نویسی (Testing)

۱۴.۱ مفاهیم پایه

- 1 Unit Test: تست یک واحد کوچک کد.
- 2 Integration Test: تست تعامل چند ماژول.
- 3 E2E Test: تست کامل جریان از ابتدا تا انتها.
- 4 Test Pyramid: هرم **تست** (unit زیاد، integration متوسط، E2E کم).
- 5 FIRST: اصول تست خوب.
- 6 Fast: سریع.
- 7 Independent: مستقل از سایر تست‌ها.
- 8 Repeatable: تکرارپذیر.
- 9 Self-Validating: خوداعتبارسنج.
- 10 Timely: به‌موقع.
- 11 Given-When-Then: ساختار **تست** (شرط-عمل-نتیجه).
- 12 Arrange-Act-Assert: چیدمان-عمل-بررسی.
- 13 AAA: مخفف Arrange-Act-Assert.

- 14• **GWT**: مخفف Given-When-Then.
- 15• **Boundary Value Analysis**: تست مقادیر مرزی.
- 16• **Equivalence Partitioning**: تقسیم ورودی‌ها به کلاس‌های هم‌ارز.
- 17• **Negative Testing**: تست ورودی‌های نامعتبر.
- 18• **Regression Bug**: باگ بازگشتی پس از تغییر.
- 19• **TDD**: توسعه مبتنی بر تست.
- 20• **Red-Green-Refactor**: چرخه TDD.
- 21• **Code Coverage**: درصد کد پوشش داده‌شده.
- 22• **Line Coverage**: پوشش خطوط.
- 23• **Branch Coverage**: پوشش شاخه‌ها.
- 24• **Method Coverage**: پوشش متدها.
- 25• **JaCoCo**: ابزار سنجش پوشش کد.

JUnit 5 ۱۴.۲

- 1 • **JUnit 5**: فریمورک تست جاوا.
- 2 • **@Test**: انوتیشن متد تست.
- 3 • **@BeforeAll**: اجرا یک بار قبل از همه تست‌ها.
- 4 • **@AfterAll**: اجرا یک بار بعد از همه تست‌ها.
- 5 • **@BeforeEach**: اجرا قبل از هر تست.
- 6 • **@AfterEach**: اجرا بعد از هر تست.
- 7 • **@DisplayName**: نام نمایشی تست.
- 8 • **@ParameterizedTest**: تست پارامتری.
- 9 • **@ValueSource**: منبع مقادیر ساده.
- 10• **@CsvSource**: منبع مقادیر CSV.
- 11• **@EnumSource**: منبع مقادیر enum.
- 12• **assertEquals**: بررسی برابری.
- 13• **assertNotEquals**: بررسی عدم برابری.
- 14• **assertNull**: بررسی null بودن.

- 15• **assertNotNull**: بررسی **null** نبودن.
- 16• **assertTrue**: بررسی **true** بودن.
- 17• **assertFalse**: بررسی **false** بودن.
- 18• **assertThrows**: بررسی پرتاب استثنا.
- 19• **assertAll**: گروه‌بندی **assertion** ها.
- 20• **assertArrayEquals**: بررسی برابری آرایه‌ها.
- 21• **assertTimeout**: بررسی محدودیت زمانی.
- 22• **Duration**: کلاس نمایش مدت زمان.

Mockito ۱۴.۳

- 1 • **Mockito**: فریمورک **mock** سازی.
- 2 • **Mock**: آجکت جعلی.
- 3 • **Stub**: پاسخ از پیش تعیین شده.
- 4 • **Mock@**: انوتیشن ساخت **mock**.
- 5 • **InjectMocks@**: تزریق **mock** ها به کلاس هدف.
- 6 • **Spy@**: جاسوس روی آجکت واقعی.
- 7 • **when()**: تعریف رفتار **mock**.
- 8 • **thenReturn()**: تعیین مقدار بازگشتی.
- 9 • **thenThrow()**: تعیین استثنای پرتابی.
- 10 • **doThrow()**: پرتاب استثنا در متد **void**.
- 11 • **verify()**: بررسی فراخوانی متد.
- 12 • **times()**: تعداد فراخوانی.
- 13 • **never()**: هرگز فراخوانی نشده.
- 14 • **atLeast()**: حداقل تعداد فراخوانی.
- 15 • **atMost()**: حداکثر تعداد فراخوانی.
- 16 • **verifyNoMoreInteractions()**: بررسی عدم فراخوانی اضافه.
- 17 • **any()**: هر آرگومان.
- 18 • **eq()**: آرگومان مشخص.

19• `anyString()`: هر `String`.

20• `anyLong()`: هر `long`.

21• `MockitoAnnotations.openMocks()`: فعال سازی انوتیشن ها.

Spring Boot Testing ۱۴.۴

1 • `SpringBootTest@`: تست کامل `Spring context`.

2 • `WebMvcTest@`: تست لایه کنترلر.

3 • `DataJpaTest@`: تست لایه JPA.

4 • `MockBean@`: جایگزینی Bean با `mock`.

5 • `MockMvc`: شبیه ساز درخواست `HTTP`.

6 • `TestRestTemplate`: کلاینت `REST` برای تست.

7 • `LocalServerPort@`: دریافت پورت تصادفی.

8 • `webEnvironment`: تنظیم محیط وب تست.

9 • `RANDOM_PORT`: پورت تصادفی.

10 • `MockMvcRequestBuilders`: ساخت درخواست تست.

11 • `MockMvcResultMatchers`: بررسی پاسخ تست.

12 • `jsonPath()`: بررسی مقدار `JSON`.

13 • `status()`: بررسی کد وضعیت.

14 • `contentType()`: بررسی نوع محتوا.

15 • `APPLICATION_JSON`: `Media Type`: `JSON`.

16 • `HttpStatus`: کدهای وضعیت `HTTP`.

17 • `ResponseEntity`: پاسخ `HTTP`.

بخش ۱۵: JSP و Thymeleaf

Thymeleaf ۱۵.۱

1 • `Thymeleaf`: قالب ساز سمت سرور `Spring`.

2 • `Template Engine`: موتور پردازش قالب.

- 3 Natural Templating: قالب قابل نمایش در مرورگر بدون پردازش.
- 4 spring-boot-starter-thymeleaf: استارتر Thymeleaf در Spring Boot.
- 5 spring.thymeleaf.prefix: پیشوند مسیر قالب‌ها.
- 6 spring.thymeleaf.suffix: پسوند فایل قالب.
- 7 spring.thymeleaf.cache: فعال/غیرفعال کردن کش قالب.
- 8 spring.thymeleaf.encoding: کدگذاری قالب.
- 9 Model: ظرف داده‌های ارسالی به view.
- 10 model.addAttribute(): افزودن داده به Model.
- 11 Controller@: کنترلر MVC.
- 12 GetMapping@: نگاشت GET.
- 13 PostMapping@: نگاشت POST.
- 14 ModelAttribute@: اتصال داده به مدل.
- 15 redirect::: ریدایرکت به مسیر دیگر.
- 16 th:text: درج متن.
- 17 th:if: شرط نمایش.
- 18 th:unless: شرط معکوس نمایش.
- 19 th:each: حلقه روی مجموعه.
- 20 th:href: لینک.
- 21 th:action: آدرس فرم.
- 22 th:object: شیء فرم.
- 23 th:field: اتصال فیلد فرم.
- 24 th:errors: نمایش خطاهای اعتبارسنجی.
- 25 th:include: درج fragment.
- 26 th:replace: جایگزینی fragment.
- 27 th:fragment: تعریف fragment.
- 28 #fields.hasErrors(): بررسی خطای فیلد.
- 29 i18n: بین‌المللی‌سازی.
- 30 messages_fa.properties: فایل پیام فارسی.

- 31• `messages_en.properties`: فایل پیام انگلیسی.
- 32• `sec:authorize`: کنترل دسترسی در قالب.
- 33• `sec:authentication`: دسترسی به اطلاعات احراز هویت.
- 34• `isAuthenticated()`: بررسی ورود کاربر.
- 35• `hasRole('ADMIN')`: بررسی نقش.
- 36• `thymeleaf-extras-springsecurity6`: افزونه امنیت Thymeleaf.

JSP ۱۵.۲

- 1 • JSP (JavaServer Pages): فناوری قدیمی قالب ساز جاوا.
- 2 • `tomcat-embed-jasper`: موتور JSP برای Tomcat داخلی.
- 3 • `jstl`: کتابخانه تگ استاندارد JSP.
- 4 • `spring.mvc.view.prefix`: پیشوند مسیر `view`.
- 5 • `spring.mvc.view.suffix`: پسوند فایل `view`.
- 6 • **JSTL**: JavaServer Pages Standard Tag Library
- 7 • `c:if`: شرط در JSTL.
- 8 • `c:forEach`: حلقه در JSTL.
- 9 • `c:out`: چاپ مقدار.
- 10• `fmt`: تگ های قالب بندی.
- 11• Expression Language: `{ }$`
- 12• `<% page @%>`: دایرکتیو صفحه.
- 13• `<% taglib @%>`: دایرکتیو کتابخانه تگ.
- 14• `/WEB-INF/views`: مسیر معمول فایل های JSP.

بخش ۱۶: Gradle و Maven

Maven ۱۶.۱

- 1 • **Maven**: ابزار مدیریت build و وابستگی جاوا.
- 2 • `pom.xml`: فایل اصلی پیکربندی Maven.

- **3 groupId**: شناسه گروه/سازمان.
- **4 artifactId**: شناسه پروژه.
- **5 version**: نسخه پروژه.
- **6 packaging**: نوع بسته‌بندی (jar/war).
- **7 parent**: پروژه والد ارث‌بری.
- **8 properties**: تعریف property‌های قابل استفاده.
- **9 dependencies**: بخش تعریف وابستگی‌ها.
- **10 dependency**: یک وابستگی مشخص.
- **11 scope**: دامنه وابستگی.
- **12 compile: scope**: پیش‌فرض.
- **13 provided**: وابستگی فراهم‌شده در زمان اجرا توسط container.
- **14 runtime**: وابستگی موردنیاز فقط زمان اجرا.
- **15 test**: وابستگی فقط برای تست.
- **16 system**: وابستگی **سیستمی** (منسوخ).
- **17 build**: بخش پیکربندی build.
- **18 plugins**: پلاگین‌های build.
- **19 plugin**: یک پلاگین مشخص.
- **20 repositories**: مخازن دانلود وابستگی.
- **21 mvn clean**: پاک کردن خروجی build.
- **22 mvn compile**: کامپایل کد.
- **23 mvn test**: اجرای تست‌ها.
- **24 mvn package**: ساخت **بسته** (jar/war).
- **25 mvn install**: نصب در مخزن محلی.
- **26 mvn spring-boot:run**: اجرای اپلیکیشن Spring Boot.
- **27 mvn dependency:tree**: نمایش درخت وابستگی‌ها.
- **28 spring-boot-maven-plugin**: پلاگین بسته‌بندی Spring Boot.
- **29 maven-compiler-plugin**: پلاگین کامپایلر.
- **30 jacoco-maven-plugin**: پلاگین سنجش پوشش کد.

31• **Lifecycle**: چرخه حیات build.

32• **validate**: اعتبارسنجی پروژه.

33• **deploy**: انتشار در مخزن راه دور.

34• **site**: تولید مستندات سایت.

Gradle ۱۶.۲

1 • **Gradle**: ابزار build مدرن و انعطاف‌پذیر.

2 • **build.gradle**: فایل build پروژه.

3 • **settings.gradle**: فایل تنظیمات پروژه.

4 • **plugins**: افزونه‌های build.

5 • **group**: شناسه گروه پروژه.

6 • **version**: نسخه پروژه.

7 • **sourceCompatibility**: نسخه سازگاری سورس.

8 • **repositories**: مخازن دانلود وابستگی.

9 • **mavenCentral()**: مخزن مرکزی Maven.

10 • **dependencies**: بخش وابستگی‌ها.

11 • **implementation**: وابستگی اصلی.

12 • **runtimeOnly**: وابستگی فقط زمان اجرا.

13 • **compileOnly**: وابستگی فقط زمان کامپایل.

14 • **annotationProcessor**: پردازنده انوتیشن.

15 • **testImplementation**: وابستگی تست.

16 • **testRuntimeOnly**: وابستگی تست زمان اجرا.

17 • **useJUnitPlatform()**: استفاده از JUnit Platform.

18 • **task**: تعریف task سفارشی.

19 • **doLast**: اجرای عمل در پایان task.

20 • **gradlew tasks/**: لیست task‌ها.

21 • **gradlew clean/**: پاک کردن build.

22 • **gradlew build/**: ساخت پروژه.

- 23• ./gradlew bootRun: اجرای Spring Boot.
- 24• ./gradlew dependencies: نمایش وابستگی‌ها.
- 25• Incremental Build: ساخت افزایشی برای سرعت بیشتر.

♦ بخش ۱۷: میکروسرویس‌ها (Microservices)

۱۷.۱ مفاهیم پایه

- 1 • **Microservices**: معماری تقسیم برنامه به سرویس‌های کوچک مستقل.
- 2 • **Monolith**: برنامه یکپارچه با یک واحد استقرار.
- 3 • **Business Capability**: تقسیم سرویس‌ها بر اساس قابلیت کسب‌وکار.
- 4 • **Independent Deploy**: استقرار مستقل هر سرویس.
- 5 • **HTTP**: پروتکل ارتباط بین سرویس‌ها.
- 6 • **Message Queue**: صف پیام برای ارتباط غیرهم‌زمان.
- 7 • **Scalability**: مقیاس‌پذیری.
- 8 • **Latency**: تأخیر.
- 9 • **Distributed Transactions**: تراکنش‌های توزیع‌شده.
- 10 • **Service Discovery**: کشف خودکار سرویس‌ها.
- 11 • **API Gateway**: دروازه ورودی به میکروسرویس‌ها.
- 12 • **Rate Limiting**: محدودسازی نرخ درخواست.
- 13 • **Circuit Breaker**: قطع‌کننده مدار برای جلوگیری از خرابی آبخاری.
- 14 • **Load Balancer**: توزیع‌کننده بار.
- 15 • **Distributed Tracing**: ردیابی درخواست‌ها در سرویس‌های مختلف.

Spring Cloud ۱۷.۲

- 1 • **Spring Cloud**: مجموعه ابزار میکروسرویس در Spring.
- 2 • **Spring Cloud Netflix**: مجموعه ابزار Netflix.
- 3 • **Eureka**: سرور کشف سرویس.

- 4 Eureka Server: سرور ثبت سرویس‌ها.
- 5 Eureka Client: کلاینت ثبت/کشف سرویس.
- 6 EnableEurekaServer@: فعال‌سازی سرور Eureka.
- 7 EnableDiscoveryClient@: فعال‌سازی کشف سرویس.
- 8 EnableCircuitBreaker@: فعال‌سازی Circuit Breaker.
- 9 spring-cloud-starter-netflix-eureka-server: استارتر Eureka Server.
- 10 spring-cloud-starter-netflix-eureka-client: استارتر Eureka Client.
- 11 spring-cloud-starter-gateway: استارتر API Gateway.
- 12 spring-cloud-starter-circuitbreaker-resilience4j: استارتر Resilience4j.
- 13 spring-cloud-starter-loadbalancer: استارتر LoadBalancer.
- 14 spring-cloud-dependencies: مدیریت نسخه Spring Cloud.
- 15 eureka.client.register-with-eureka: ثبت خود در Eureka.
- 16 eureka.client.fetch-registry: دریافت رجیستری.
- 17 eureka.client.serviceUrl.defaultZone: آدرس Eureka.
- 18 eureka.instance.prefer-ip-address: ترجیح IP به hostname.
- 19 eureka.instance.instance-id: شناسه نمونه.
- 20 LoadBalancerClient: کلاینت توزیع بار.
- 21 RestTemplate: کلاینت HTTP ساده.
- 22 LoadBalanced@: فعال‌سازی LoadBalancer روی RestTemplate.
- 23 DiscoveryClient: کلاینت کشف سرویس.
- 24 ServiceInstance: نمونه سرویس کشف‌شده.
- 25 lb://user-service: آدرس منطقی سرویس با LoadBalancer.

۱۷.۳ API Gateway

- 1 API Gateway: دروازه ورودی سرویس‌ها.
- 2 spring.cloud.gateway.routes: تعریف route‌ها.
- 3 id: شناسه route.
- 4 uri: مقصد route.

- **5 predicates**: شرطهای تطبیق route.
- **6 Path: predicate**: تطبیق مسیر.
- **7 filters**: فیلترهای route.
- **8 CircuitBreaker**: فیلتر Circuit Breaker.
- **9 fallbackUri**: مسیر جایگزین در خطا.
- **10 Retry**: فیلتر تلاش مجدد.
- **11 GlobalFilter**: فیلتر سراسری Gateway.
- **12 ServerHttpRequest**: درخواست HTTP در WebFlux.
- **13 ServerHttpResponse**: پاسخ HTTP در WebFlux.
- **14 RedisRateLimiter**: محدودکننده نرخ مبتنی بر Redis.

Circuit Breaker ۱۷.۴

- **1 Resilience4j**: کتابخانه تحمل خطا.
- **2 @CircuitBreaker**: انوتیشن Circuit Breaker.
- **3 fallbackMethod**: متد جایگزین در خطا.
- **4 @Retry**: تلاش مجدد خودکار.
- **5 @RateLimiter**: محدودسازی نرخ.
- **6 @Bulkhead**: محدودسازی همزمانی.
- **7 sliding-window-size**: اندازه پنجره لغزان.
- **8 failure-rate-threshold**: آستانه نرخ خطا.
- **9 wait-duration-in-open-state**: مدت باز ماندن مدار.
- **10 minimum-number-of-calls**: حداقل تعداد فراخوانی.
- **11 max-attempts**: حداکثر تلاش.
- **12 wait-duration**: مدت انتظار بین تلاشها.
- **13 limit-for-period**: محدودیت در بازه.
- **14 limit-refresh-period**: بازه بازنشانی محدودیت.
- **15 max-concurrent-calls**: حداکثر فراخوانی همزمان.

Distributed Tracing ۱۷.۵

- 1• **Sleuth**: ابزار ردیابی Spring Cloud.
- 2• **Zipkin**: سرور نمایش trace.
- 3• **spring-cloud-starter-sleuth**: استارتر Sleuth.
- 4• **spring-cloud-starter-zipkin**: استارتر Zipkin.
- 5• **spring.sleuth.sampler.probability**: احتمال نمونه‌گیری.
- 6• **spring.zipkin.base-url**: آدرس Zipkin.
- 7• **Tracer**: مؤلفه ساخت span.
- 8• **Span**: واحد کار در trace.
- 9• **Trace ID**: شناسه یکتای trace.

♦ بخش ۱۸: اصطلاحات عمومی جاوا

۱۸.۱ محیط و ابزارها

- 1• **JDK**: کیت توسعه جاوا شامل کامپایلر و JRE.
- 2• **JRE**: محیط اجرای جاوا شامل JVM و کتابخانه‌ها.
- 3• **JVM**: ماشین مجازی اجرای بایت‌کد جاوا.
- 4• **Java SE**: نسخه استاندارد جاوا.
- 5• **Java EE**: نسخه سازمانی **جاوا** (اکنون Jakarta EE).
- 6• **Java ME**: نسخه سبک جاوا برای دستگاه‌های محدود.
- 7• **javac**: کامپایلر جاوا.
- 8• **java**: دستور اجرای برنامه جاوا.
- 9• **jar**: ابزار ساخت/مدیریت فایل JAR.
- 10• **javadoc**: ابزار تولید مستندات.
- 11• **JAR**: آرشیو جاوا.
- 12• **WAR**: آرشیو وب اپلیکیشن جاوا.
- 13• **Classpath**: مسیر جستجوی کلاس‌ها.
- 14• **ClassLoader**: بارگذار کلاس‌ها.
- 15• **Bytecode**: کد میانی اجراشده توسط JVM.

- 16• Garbage Collector: جمع‌آورنده خودکار حافظه.
- 17• JIT (Just-In-Time): کامپایلر بلادرنگ بایت‌کد به ماشین.
- 18• JVM Memory Model: مدل حافظه JVM.

۱۸.۲ اصطلاحات عمومی

- 1• API: رابط برنامه‌نویسی.
- 2• Servlet: کامپوننت وب جاوا.
- 3• ServletContext: بستر مشترک اپلیکیشن وب.
- 4• Tomcat: کاننتینر Servlet.
- 5• PostgreSQL: دیتابیس رابطه‌ای متن‌باز.
- 6• MySQL: دیتابیس رابطه‌ای محبوب.
- 7• H2: دیتابیس درون‌حافظه‌ای جاوا.
- 8• Spring Framework: فریمورک جامع جاوا.
- 9• Spring Core: هسته (IoC/DI) Spring.
- 10• Spring Boot: فریمورک ساده‌سازی Spring.
- 11• Spring MVC: ماژول وب Spring.
- 12• Spring Data: ماژول دسترسی داده.
- 13• Spring Security: ماژول امنیت.
- 14• Spring Cloud: ماژول میکروسرویس.
- 15• IoC (Inversion of Control): وارونگی کنترل.
- 16• DI (Dependency Injection): تزریق وابستگی.
- 17• AOP (Aspect-Oriented Programming): برنامه‌نویسی جنبه‌گرا.
- 18• Bean: آبجکت مدیریت‌شده Spring.
- 19• ApplicationContext: context اصلی Spring.
- 20• BeanFactory: کارخانه پایه Bean.
- 21• DispatcherServlet: کنترلر جلویی Spring MVC.
- 22• Front Controller: الگوی کنترلر مرکزی.
- 23• ViewResolver: تشخیص view.

24• **HandlerMapping**: نگاشت درخواست به handler.

25• **ModelAndView**: مدل + view در Spring MVC.

۱۸.۳ اصطلاحات شبکه و وب

- 1 **HTTP**: پروتکل انتقال ابرمتن.
- 2 **HTTPS**: HTTP امن.
- 3 **TCP**: پروتکل انتقال مطمئن.
- 4 **UDP**: پروتکل انتقال سریع.
- 5 **IP**: پروتکل اینترنت.
- 6 **DNS**: سیستم نام دامنه.
- 7 **REST**: سبک معماری مبتنی بر منابع.
- 8 **RESTful**: سازگار با REST.
- 9 **JSON**: فرمت داده سبک.
- 10 **XML**: فرمت داده نشانه‌گذاری.
- 11 **SOAP**: پروتکل پیام‌رسانی مبتنی بر XML.
- 12 **WebSocket**: ارتباط دوطرفه پایدار.
- 13 **CORS**: اشتراک منابع بین دامنه‌ای.
- 14 **CSRF**: جعل درخواست از سایت دیگر.
- 15 **XSS**: تزریق اسکریپت.
- 16 **SQL Injection**: تزریق SQL.
- 17 **JWT**: توکن وب JSON.
- 18 **OAuth**: پروتکل مجوزدهی.
- 19 **OAuth2**: نسخه دوم OAuth.
- 20 **Basic Auth**: احراز هویت پایه.
- 21 **Bearer Token**: توکن حامل.
- 22 **Session**: نشست کاربر.
- 23 **Cookie**: داده ذخیره‌شده در مرورگر.
- 24 **Local Storage**: ذخیره‌سازی محلی مرورگر.

- 25• SSL/TLS: پروتکل امنیت ارتباط.
- 26• Certificate: گواهی دیجیتال.
- 27• Load Balancer: توزیع کننده بار.
- 28• Reverse Proxy: پروکسی معکوس.
- 29• Forward Proxy: پروکسی رو به جلو.
- 30• Nginx: وب سرور/پروکسی محبوب.
- 31• Apache: وب سرور محبوب.

۱۸.۴ اصطلاحات دیتابیس

- 1 • JDBC: API اتصال جاوا به دیتابیس.
- 2 • JPA: استاندارد ORM جاوا.
- 3 • Hibernate: پیاده سازی JPA.
- 4 • ORM: نگاشت شیء-رابطه ای.
- 5 • Entity: کلاس نگاشت شده به جدول.
- 6 • Repository: لایه دسترسی داده.
- 7 • DAO: شیء دسترسی داده.
- 8 • DTO: شیء انتقال داده.
- 9 • Transaction: واحد کاری اتمیک.
- 10 • ACID: اتمی بودن، سازگاری، انزوا، پایداری.
- 11 • Isolation Level: سطح انزوا تراکنش.
- 12 • Lock: قفل.
- 13 • Optimistic Lock: قفل خوش بینانه.
- 14 • Pessimistic Lock: قفل بدبینانه.
- 15 • Connection Pool: استخر اتصال.
- 16 • HikariCP: استخر اتصال سریع.
- 17 • Flyway: ابزار مهاجرت دیتابیس.
- 18 • Liquibase: ابزار مهاجرت دیتابیس.
- 19 • Migration: مهاجرت schema.

- 20 Schema: ساختار دیتابیس.
- 21 Index: فهرست برای سرعت جستجو.
- 22 Primary Key: کلید اصلی.
- 23 Foreign Key: کلید خارجی.
- 24 Join: اتصال جدول‌ها.
- 25 Inner Join: اتصال داخلی.
- 26 Left Join: اتصال چپ.
- 27 Right Join: اتصال راست.
- 28 Subquery: پرس‌وجوی تودرتو.
- 29 View: نمای مجازی.
- 30 Stored Procedure: رویه ذخیره‌شده.
- 31 Trigger: تریگر دیتابیس.

۱۸.۵ اصطلاحات هم‌روندی

- 1 Thread: ریسه اجرا.
- 2 Runnable: اینترفیس task.
- 3 Callable: اینترفیس task با خروجی.
- 4 ExecutorService: مدیریت thread pool.
- 5 Thread Pool: استخر thread.
- 6 Fork/Join: چارچوب موازی‌سازی.
- 7 CompletableFuture: API async جاوا ۸.
- 8 Future: نتیجه آینده async.
- 9 AtomicInteger: int اتمیک.
- 10 AtomicLong: long اتمیک.
- 11 Volatile: تضمین visibility متغیر.
- 12 Synchronized: قفل هم‌زمانی.
- 13 Lock: قفل پیشرفته.
- 14 ReentrantLock: قفل قابل ورود مجدد.

- 15• **ReadWriteLock**: قفل خواندن/نوشتن.
- 16• **Semaphore**: سمافور.
- 17• **CountDownLatch**: هماهنگ کننده انتظار.
- 18• **CyclicBarrier**: مانع چرخه ای.
- 19• **Phaser**: هماهنگ کننده چندفازی.
- 20• **ConcurrentHashMap**: Map هم زمان امن.
- 21• **CopyOnWriteArrayList**: لیست امن با کپی.
- 22• **BlockingQueue**: صف مسدودکننده.
- 23• **ArrayBlockingQueue**: صف مسدودکننده آرایه ای.
- 24• **LinkedBlockingQueue**: صف مسدودکننده پیوندی.
- 25• **PriorityBlockingQueue**: صف مسدودکننده اولویت دار.
- 26• **SynchronousQueue**: صف هم زمان.
- 27• **Deadlock**: بن بست.
- 28• **Livelock**: قفل زنده (تغییر بدون پیشرفت).
- 29• **Starvation**: گرسنگی thread.
- 30• **Race Condition**: شرایط رقابتی.
- 31• **Thread Safety**: امنیت thread.
- 32• **Happens-Before**: رابطه ترتیب حافظه.
- 33• **Memory Visibility**: دیدپذیری حافظه.

۱۸.۶ اصطلاحات معماری

- 1 • **Monolith**: یکپارچه.
- 2 • **Microservices**: ریزسرویس ها.
- 3 • **SOA**: معماری سرویس گرا.
- 4 • **Event-Driven**: رویدادمحور.
- 5 • **Serverless**: بدون سرور.
- 6 • **Layered Architecture**: معماری لایه ای.
- 7 • **Hexagonal Architecture**: معماری شش ضلعی.

- 8 Clean Architecture: معماری تمیز.
- 9 MVC: مدل-نما-کنترلر.
- 10 MVP: مدل-نما-ارائه‌دهنده.
- 11 MVVM: مدل-نما-مدل‌نما.
- 12 CQRS: جداسازی خواندن/نوشتن.
- 13 Event Sourcing: ذخیره رویدادمحور.
- 14 Saga Pattern: الگوی Saga برای تراکنش توزیع‌شده.
- 15 API Gateway: دروازه API.
- 16 Service Mesh: مش سرویس.
- 17 Sidecar: کانتینر کمکی.
- 18 Circuit Breaker: قطع‌کننده مدار.
- 19 Bulkhead: جداسازی منابع.
- 20 Retry: تلاش مجدد.
- 21 Timeout: مهلت.
- 22 Rate Limiting: محدودسازی نرخ.
- 23 Load Balancing: توزیع بار.
- 24 Service Discovery: کشف سرویس.
- 25 Config Server: سرور تنظیمات.
- 26 Distributed Tracing: ردیابی توزیع‌شده.
- 27 Centralized Logging: لاگ متمرکز.
- 28 Health Check: بررسی سلامت.
- 29 Graceful Shutdown: خاموشی نرم.

۱۸.۷ دیزاین پترن‌های تکمیلی

- 1 Repository Pattern: لایه دسترسی داده.
- 2 DTO Pattern: انتقال داده بین لایه‌ها.
- 3 DAO Pattern: دسترسی داده.
- 4 Service Layer: لایه منطق کسب‌وکار.

- **5 Controller**: کنترلر.
- **6 Facade**: فاساد.
- **7 Adapter**: آداپتور.
- **8 Decorator**: دکوراتور.
- **9 Proxy**: پروکسی.
- **10 Factory**: کارخانه.
- **11 Singleton**: تک نمونه.
- **12 Builder**: سازنده.
- **13 Observer**: ناظر.
- **14 Strategy**: استراتژی.
- **15 Template Method**: متد قالب.
- **16 Chain of Responsibility**: زنجیره مسئولیت.
- **17 Command**: فرمان.
- **18 State**: وضعیت.
- **19 Visitor**: بازدیدکننده.
- **20 Mediator**: واسطه.
- **21 Memento**: یادگار.
- **22 Iterator**: پیمایشگر.
- **23 Composite**: ترکیبی.
- **24 Bridge**: پل.
- **25 Flyweight**: وزن سبک.
- **26 Abstract Factory**: کارخانه انتزاعی.
- **27 Prototype**: نمونه اولیه.

◆ بخش ۱۹: Annotation های مهم جاوا و Spring

۱۹.۱ Annotation های Core Java

- **1 @Override**: بازنویسی متد والد.

- 2 @Deprecated: منسوخ شده.
- 3 @SuppressWarnings: سرکوب هشدارهای کامپایلر.
- 4 @SafeVarargs: تضمین امنیت متد varargs جنریک.
- 5 @FunctionalInterface: اینترفیس تک‌متدی.
- 6 @Retention: تعیین مدت نگهداری انوتیشن.
- 7 @Target: تعیین محل مجاز انوتیشن.
- 8 @Documented: درج در مستندات.
- 9 @Inherited: ارث‌بری انوتیشن توسط زیرکلاس.
- 10 @Repeatable: امکان تکرار انوتیشن.
- 11 @Native: نشانه‌گذاری فیلد بومی.

۱۹.۲ Annotation های Spring

- 1 @Component: Bean عمومی Spring.
- 2 @Service: Bean لایه سرویس.
- 3 @Repository: Bean لایه دسترسی داده.
- 4 @Controller: Bean کنترلر MVC.
- 5 @RestController: Bean کنترلر REST.
- 6 @Configuration: کلاس پیکربندی Bean ها.
- 7 @Bean: تعریف Bean در متد.
- 8 @Autowired: تزریق خودکار وابستگی.
- 9 @Qualifier: انتخاب Bean مشخص.
- 10 @Primary: پیش‌فرض.
- 11 @Lazy: ساخت با تأخیر.
- 12 @Value: تزریق مقدار از property.
- 13 @PropertySource: تعیین فایل property.
- 14 @Profile: فعال‌سازی شرطی بر اساس profile.
- 15 @Scope: تعیین scope.
- 16 @PostConstruct: بعد از ساخت Bean.

- 17• **PreDestroy@**: قبل از نابودی Bean.
- 18• **Import@**: وارد کردن Configuration.
- 19• **ImportResource@**: وارد کردن XML configuration.
- 20• **ComponentScan@**: اسکن پکیج برای Component ها.
- 21• **EnableAutoConfiguration@**: فعال سازی پیکربندی خودکار.
- 22• **SpringBootApplication@**: ترکیب سه انوتیشن اصلی Spring Boot.
- 23• **EnableWebSecurity@**: فعال سازی امنیت وب.
- 24• **EnableMethodSecurity@**: فعال سازی امنیت سطح متد.
- 25• **EnableEurekaServer@**: فعال سازی سرور Eureka.
- 26• **EnableDiscoveryClient@**: فعال سازی کشف سرویس.
- 27• **EnableCircuitBreaker@**: فعال سازی Circuit Breaker.
- 28• **EnableJpaRepositories@**: فعال سازی ریپازیتوری های JPA.
- 29• **EnableTransactionManagement@**: فعال سازی مدیریت تراکنش.
- 30• **EnableScheduling@**: فعال سازی زمان بندی.
- 31• **EnableAsync@**: فعال سازی async.
- 32• **EnableCaching@**: فعال سازی کش.
- 33• **ConditionalOnClass@**: شرط وجود کلاس.
- 34• **ConditionalOnMissingClass@**: شرط نبود کلاس.
- 35• **ConditionalOnProperty@**: شرط property.
- 36• **ConditionalOnBean@**: شرط وجود Bean.
- 37• **ConditionalOnMissingBean@**: شرط نبود Bean.

۱۹.۳ Annotation های Spring MVC

- 1 • **RequestMapping@**: نگاشت مسیر.
- 2 • **GetMapping@**: نگاشت GET.
- 3 • **PostMapping@**: نگاشت POST.
- 4 • **PutMapping@**: نگاشت PUT.
- 5 • **DeleteMapping@**: نگاشت DELETE.

- 6 @PatchMapping: نگاشت PATCH.
- 7 @PathVariable: متغیر مسیر.
- 8 @RequestParam: پارامتر درخواست.
- 9 @RequestBody: بدنه درخواست.
- 10 @ResponseBody: بدنه پاسخ.
- 11 @ModelAttribute: اتصال مدل.
- 12 @RequestHeader: هدر درخواست.
- 13 @CookieValue: مقدار کوکی.
- 14 @SessionAttribute: نشست.
- 15 @RequestAttribute: درخواست.
- 16 @CrossOrigin: فعال سازی CORS.
- 17 @ExceptionHandler: مدیریت استثنا.
- 18 @ControllerAdvice: مشاور سراسری کنترلرها.
- 19 @RestControllerAdvice: مشاور سراسری REST.
- 20 @ResponseStatus: تعیین کد وضعیت.

۱۹.۴ JPA Annotation های

- 1 @Entity: کلاس موجودیت.
- 2 @Table: نام جدول.
- 3 @Id: کلید اصلی.
- 4 @GeneratedValue: تولید خودکار کلید.
- 5 @Column: تنظیم ستون.
- 6 @JoinColumn: ستون کلید خارجی.
- 7 @OneToOne: رابطه یک به یک.
- 8 @OneToMany: رابطه یک به چند.
- 9 @ManyToOne: رابطه چند به یک.
- 10 @ManyToMany: رابطه چند به چند.
- 11 @Embedded: جاسازی کلاس.

- 12• @Embeddable: کلاس قابل جاسازی.
- 13• @MappedSuperclass: کلاس والد نگاشت شده.
- 14• @Inheritance: استراتژی وراثت.
- 15• @DiscriminatorColumn: ستون تشخیص نوع.
- 16• @DiscriminatorValue: مقدار تشخیص نوع.
- 17• @NamedQuery: کوئری نام دار.
- 18• @NamedQueries: چند کوئری نام دار.
- 19• @NamedEntityGraph: گراف موجودیت نام دار.
- 20• @NamedAttributeNode: گره attribute در گراف.
- 21• @Version: فیلد نسخه برای قفل خوش بینانه.
- 22• @Transient: فیلد غیرنگاشت شده.
- 23• @Temporal: تعیین نوع تاریخ.
- 24• @Enumerated: نگاشت enum.
- 25• @Lob: فیلد داده بزرگ.
- 26• @ElementCollection: مجموعه عناصر ساده.
- 27• @CollectionTable: جدول مجموعه.
- 28• @OrderColumn: ستون ترتیب.
- 29• @OrderBy: ترتیب پیش فرض.
- 30• @Where: شرط پیش فرض.
- 31• @Filter: فیلتر پویا.
- 32• @SQLDelete: سفارشی حذف.
- 33• @SQLInsert: سفارشی درج.
- 34• @SQLUpdate: سفارشی به روزرسانی.

Spring Data JPA های Annotation ۱۹.۵

- 1• @Query: کوئری سفارشی.
- 2• @Param: نام گذاری پارامتر.
- 3• @Modifying: کوئری تغییردهنده.

- 4• **Transactional@**: مدیریت تراکنش.
- 5• **NoRepositoryBean@**: عدم ساخت Bean برای ریپازیتوری پایه.

۱۹.۶ Annotation های Spring Security

- 1• **PreAuthorize@**: بررسی مجوز قبل از متد.
- 2• **PostAuthorize@**: بررسی مجوز بعد از متد.
- 3• **PreFilter@**: فیلتر ورودی قبل از متد.
- 4• **PostFilter@**: فیلتر خروجی بعد از متد.
- 5• **Secured@**: بررسی نقش ساده.
- 6• **RolesAllowed@**: بررسی نقش (JSR-250).
- 7• **DenyAll@**: رد همه.
- 8• **PermitAll@**: اجازه به همه.
- 9• **AuthenticationPrincipal@**: تزریق کاربر جاری.

۱۹.۷ Annotation های تست

- 1• **Test@**: متد تست.
- 2• **BeforeEach@**: قبل از هر تست.
- 3• **AfterEach@**: بعد از هر تست.
- 4• **BeforeAll@**: قبل از همه تست ها.
- 5• **AfterAll@**: بعد از همه تست ها.
- 6• **DisplayName@**: نام نمایشی.
- 7• **Disabled@**: غیرفعال سازی تست.
- 8• **ParameterizedTest@**: تست پارامتری.
- 9• **ValueSource@**: منبع مقادیر ساده.
- 10• **CsvSource@**: منبع CSV.
- 11• **EnumSource@**: منبع enum.
- 12• **MethodSource@**: منبع متد.
- 13• **Mock@**: ساخت mock.

- 14• **InjectMocks@**: تزریق mock ها.
- 15• **Spy@**: جاسوس.
- 16• **MockBean: Bean@** جعلی Spring.
- 17• **SpyBean: Bean@** جاسوس Spring.
- 18• **SpringBootTest@**: تست کامل Spring Boot.
- 19• **WebMvcTest@**: تست لایه وب.
- 20• **DataJpaTest@**: تست لایه JPA.
- 21• **WebFluxTest@**: تست WebFlux.
- 22• **RestClientTest@**: تست کلاینت REST.
- 23• **JsonTest@**: تست JSON.
- 24• **TestConfiguration@**: پیکربندی مخصوص تست.
- 25• **ActiveProfiles@**: فعال سازی profile.
- 26• **Sql@**: اجرای SQL قبل از تست.
- 27• **Commit@**: ثبت تراکنش تست.
- 28• **Rollback@**: بازگردانی تراکنش تست.
- 29• **DirtyContext@**: آلوده کردن context.

۱۹.۸ Annotation های Lombok

- 1 • **Data@**: تولید getter/setter/toString>equals/hashCode.
- 2 • **Getter@**: تولید getter.
- 3 • **Setter@**: تولید setter.
- 4 • **Builder@**: تولید Builder.
- 5 • **NoArgsConstructor@**: سازنده بدون آرگومان.
- 6 • **AllArgsConstructor@**: سازنده همه آرگومان ها.
- 7 • **RequiredArgsConstructor@**: سازنده فیلدهای **final**.
- 8 • **ToString@**: تولید toString.
- 9 • **EqualsAndHashCode@**: تولید equals/hashCode.
- 10• **Slf4j@**: تزریق Logger.

- 11• **Value@**: کلاس immutable.
- 12• **NonNull@**: بررسی null.
- 13• **Cleanup@**: بستن خودکار منبع.
- 14• **SneakyThrows@**: پرتاب پنهان استثنا.

۱۹.۹ Jackson های Annotation

- 1 • **JsonProperty@**: نام JSON فیلد.
- 2 • **JsonIgnore@**: نادیده گرفتن فیلد.
- 3 • **JsonFormat@**: قالب بندی مقدار.
- 4 • **JsonInclude@**: شرط درج فیلد.
- 5 • **JsonCreator@**: سازنده deserialization.
- 6 • **JsonValue@**: مقدار سریالایز.
- 7 • **getter@**: **JsonAnyGetter** فیلدهای پویا.
- 8 • **setter@**: **JsonAnySetter** فیلدهای پویا.
- 9 • **JsonNaming@**: استراتژی نام گذاری.
- 10• **JsonSerialize@**: سریالایزر سفارشی.
- 11• **JsonDeserialize@**: دی سریالایزر سفارشی.

۱۹.۱۰ Validation های Annotation

- 1 • **Valid@**: فعال سازی اعتبارسنجی.
- 2 • **NotNull@**: نباید null باشد.
- 3 • **NotEmpty@**: نباید خالی باشد.
- 4 • **NotBlank@**: نباید فقط فضای خالی باشد.
- 5 • **Size@**: محدودیت اندازه.
- 6 • **Min@**: حداقل مقدار.
- 7 • **Max@**: حداکثر مقدار.
- 8 • **Email@**: قالب ایمیل.
- 9 • **Pattern@**: الگوی regex.

- 10• **Positive@**: عدد مثبت.
- 11• **Negative@**: عدد منفی.
- 12• **Past@**: تاریخ گذشته.
- 13• **Future@**: تاریخ آینده.
- 14• **AssertTrue@**: باید **true** باشد.
- 15• **AssertFalse@**: باید **false** باشد.

♦ بخش ۲۰: کلمات کلیدی جاوا (Java Keywords)

۲۰.۱ کلمات کلیدی پایه

- 1 • **abstract**: تعریف کلاس /متد انتزاعی.
- 2 • **assert**: بررسی شرط در تست/دیباگ.
- 3 • **boolean**: نوع داده منطقی.
- 4 • **break**: خروج از حلقه/**switch**.
- 5 • **byte**: نوع داده عددی ۸ بیتی.
- 6 • **case**: شاخه **switch**.
- 7 • **catch**: مدیریت استثنا.
- 8 • **char**: نوع داده کاراکتر.
- 9 • **class**: تعریف کلاس.
- 10• **const**: رزرو شده و استفاده نمی‌شود.
- 11• **continue**: پرش به تکرار بعدی حلقه.
- 12• **default**: شاخه پیش فرض **switch** / متد پیش فرض **interface**.
- 13• **do**: شروع حلقه **do-while**.
- 14• **double**: نوع داده اعشاری ۶۴ بیتی.
- 15• **else**: شاخه جایگزین **if**.
- 16• **enum**: تعریف نوع شمارشی.
- 17• **extends**: وراثت کلاس.
- 18• **final**: غیرقابل تغییر.

- 19• **finally**: بلوک همیشه اجرا.
- 20• **float**: نوع داده اعشاری ۳۲ بیتی.
- 21• **for**: حلقه شمارنده.
- 22• **goto**: رزرو شده و استفاده نمی‌شود.
- 23• **if**: شرط.
- 24• **implements**: پیاده‌سازی **interface**.
- 25• **import**: وارد کردن کلاس/پکیج.
- 26• **instanceof**: بررسی نوع.
- 27• **int**: نوع داده عددی ۳۲ بیتی.
- 28• **interface**: تعریف اینترفیس.
- 29• **long**: نوع داده عددی ۶۴ بیتی.
- 30• **native**: متد پیاده‌سازی شده به زبان بومی.
- 31• **new**: ساخت آبجکت.
- 32• **package**: تعریف پکیج.
- 33• **private**: دسترسی فقط داخل کلاس.
- 34• **protected**: دسترسی پکیج + زیرکلاس.
- 35• **public**: دسترسی عمومی.
- 36• **return**: بازگشت از متد.
- 37• **short**: نوع داده عددی ۱۶ بیتی.
- 38• **static**: متعلق به کلاس.
- 39• **strictfp**: محدودسازی محاسبات اعشاری به استاندارد.
- 40• **super**: ارجاع به کلاس والد.
- 41• **switch**: انتخاب چندشاخه‌ای.
- 42• **synchronized**: قفل هم‌زمانی.
- 43• **this**: ارجاع به آبجکت جاری.
- 44• **throw**: پرتاب استثنا.
- 45• **throws**: اعلام استثنا در امضای متد.
- 46• **transient**: فیلد غیرقابل سریالایز.

- 47• **try**: بلوک کد مشکوک به خطا.
- 48• **void**: بدون مقدار بازگشتی.
- 49• **volatile**: تضمین visibility متغیر بین threadها.
- 50• **while**: حلقه شرطی.

۲۰.۲ کلمات کلیدی غیرمستقیم

- 1• **true**: مقدار منطقی درست.
- 2• **false**: مقدار منطقی نادرست.
- 3• **null**: مقدار پوچ reference.
- 4• **var**: استنتاج نوع متغیر محلی (جاوا ۱۰+).
- 5• **record**: تعریف رکورد (جاوا ۱۶+).
- 6• **sealed**: کلاس/اینترفیس محدود (جاوا ۱۷+).
- 7• **permits**: تعیین زیرکلاس‌های مجاز.
- 8• **yield**: بازگشت مقدار از switch expression.
- 9• **non-sealed**: باز کردن کلاس sealed برای وراثت.

♦ بخش ۲۱: کلاس‌ها و اینترفیس‌های مهم جاوا

۲۱.۱ java.lang

- 1• **Object**: ریشه همه کلاس‌های جاوا.
- 2• **String**: رشته متن immutable.
- 3• **StringBuilder**: ساخت رشته mutable و سریع.
- 4• **StringBuffer**: ساخت رشته mutable و thread-safe.
- 5• **Math**: توابع ریاضی.
- 6• **System**: دسترسی به سیستم (in/out/exit).
- 7• **Runtime**: دسترسی به runtime JVM.
- 8• **Thread**: کلاس ریسه.
- 9• **Runnable**: اینترفیس task.

- 10• **Throwable**: ریشه خطاها و استثناها.
- 11• **Exception**: کلاس پایه استثناها.
- 12• **Error**: کلاس پایه خطاهای جدی.
- 13• **RuntimeException**: کلاس پایه استثنای unchecked.
- 14• **Class**: نماینده کلاس در runtime.
- 15• **ClassLoader**: بارگذار کلاس‌ها.
- 16• **Integer**: wrapper کلاس **int**.
- 17• **Long**: wrapper کلاس **long**.
- 18• **Double**: wrapper کلاس **double**.
- 19• **Float**: wrapper کلاس **float**.
- 20• **Boolean**: wrapper کلاس **boolean**.
- 21• **Character**: wrapper کلاس **char**.
- 22• **Byte**: wrapper کلاس **byte**.
- 23• **Short**: wrapper کلاس **short**.
- 24• **Void**: نماینده نوع **void**.
- 25• **Process**: فرآیند سیستمی.
- 26• **ProcessBuilder**: سازنده فرآیند.
- 27• **SecurityManager**: مدیر امنیت (منسوخ).
- 28• **Package**: نماینده پکیج.
- 29• **Module**: نماینده ماژول (جاوا ۹+).

۲۱.۲ java.util

- 1• **List**: اینترفیس لیست.
- 2• **ArrayList**: پیاده‌سازی آرایه‌ای List.
- 3• **LinkedList**: پیاده‌سازی پیوندی List.
- 4• **Vector**: هم‌زمان (قدیمی).
- 5• **Stack**: پشته LIFO.
- 6• **Set**: اینترفیس مجموعه بدون تکرار.

- **7 Set: HashSet** بر پایه hash.
- **8 Set: LinkedHashSet** با حفظ ترتیب درج.
- **9 Set: TreeSet** مرتب.
- **10 Map:** اینترفیس کلید-مقدار.
- **11 Map: HashMap** بر پایه hash.
- **12 Map: LinkedHashMap** با حفظ ترتیب درج.
- **13 Map: TreeMap** مرتب.
- **14 Map: Hashtable** همزمان (قدیمی).
- **15 Queue:** اینترفیس صف.
- **16 PriorityQueue** صف اولویت‌دار.
- **17 Deque** صف دوطرفه.
- **18 Deque: ArrayDeque** آرایه‌ای.
- **19 Iterator:** پیمایشگر.
- **20 ListIterator** پیمایشگر لیست.
- **21 Collections** ابزارهای کالکشن.
- **22 Arrays** ابزارهای آرایه.
- **23 Comparator** مقایسه‌گر خارجی.
- **24 Comparable** مقایسه‌گر داخلی.
- **25 Optional** ظرف امن مقدار.
- **26 Scanner** خواندن ورودی.
- **27 Random** تولید اعداد تصادفی.
- **28 UUID** شناسه یکتا.
- **29 Date** تاریخ (قدیمی).
- **30 Calendar** تقویم (قدیمی).
- **31 Timer** زمان‌بند.
- **32 task: TimerTask** زمان‌بند.
- **33 Properties** نگهداری کلید-مقدار تنظیمات.
- **34 Locale** نماینده locale.

- 35• **Currency**: نماینده ارز.
- 36• **Formatter**: قالب‌بندی خروجی.
- 37• **StringTokenizer**: تجزیه رشته (قدیمی).
- 38• **BitSet**: مجموعه بیتی.
- 39• **EnumMap**: Map برای **enum**.
- 40• **EnumSet**: Set برای **enum**.
- 41• **WeakHashMap**: Map با کلید **weak**.
- 42• **IdentityHashMap**: Map بر اساس **==**.
- 43• **LinkedBlockingQueue**: صف مسدودکننده پیوندی.
- 44• **ArrayBlockingQueue**: صف مسدودکننده آرایه‌ای.
- 45• **PriorityBlockingQueue**: صف مسدودکننده اولویت‌دار.
- 46• **SynchronousQueue**: صف هم‌زمان.
- 47• **ConcurrentHashMap**: Map هم‌زمان امن.
- 48• **CopyOnWriteArrayList**: لیست امن با کپی.
- 49• **CopyOnWriteArraySet**: Set امن با کپی.
- 50• **AtomicInteger**: **int** اتمیک.
- 51• **AtomicLong**: **long** اتمیک.
- 52• **AtomicBoolean**: **boolean** اتمیک.
- 53• **AtomicReference**: **reference** اتمیک.
- 54• **CountDownLatch**: هماهنگ‌کننده انتظار.
- 55• **CyclicBarrier**: مانع چرخه‌ای.
- 56• **Semaphore**: سمافور.
- 57• **Exchanger**: تبادل داده بین **thread**ها.
- 58• **Phaser**: هماهنگ‌کننده چندفازی.

۲۱.۳ **java.util.function**

- 1 • **<Predicate<T**: ورودی **boolean** $T \rightarrow$
- 2 • **<Function<T,R**: ورودی $T \rightarrow$ خروجی **R**

- 3 Consumer<T>: ورودی T → بدون خروجی.
- 4 Supplier<T>: بدون ورودی → خروجی T.
- 5 BiFunction<T,U,R>: دو ورودی → خروجی R.
- 6 BiPredicate<T,U>: دو ورودی → **boolean**.
- 7 BiConsumer<T,U>: دو ورودی → بدون خروجی.
- 8 UnaryOperator<T>: ورودی T → خروجی T.
- 9 BinaryOperator<T>: دو T → T.
- 10 IntFunction<R>: **int** → R
- 11 IntPredicate: **int** → **boolean**
- 12 IntConsumer: **int** → بدون خروجی.
- 13 IntSupplier: بدون ورودی → **int**.
- 14 LongFunction<R>: **long** → R
- 15 DoubleFunction<R>: **double** → R
- 16 ToIntFunction<T>: T → **int**
- 17 ToLongFunction<T>: T → **long**
- 18 ToDoubleFunction<T>: T → **double**

۲۱.۴ java.util.stream

- 1 Stream: جریان داده.
- 2 IntStream: Stream **int**
- 3 LongStream: Stream **long**
- 4 DoubleStream: Stream **double**
- 5 Collectors: جمع‌آورنده‌های Stream
- 6 StreamSupport: ابزار ساخت Stream
- 7 BaseStream: اینترفیس پایه Stream

۲۱.۵ java.util.concurrent

- 1 Executor: اینترفیس اجرای task.

- **2 ExecutorService**: مدیریت thread pool.
- **3 Executors**: کارخانه ساخت executor.
- **4 ThreadPoolExecutor**: پیاده‌سازی thread pool.
- **5 ScheduledExecutorService**: executor زمان‌بندی‌شده.
- **6 ScheduledThreadPoolExecutor**: پیاده‌سازی آن.
- **7 ForkJoinPool**: استخر Fork/Join.
- **8 Future**: نتیجه آینده.
- **9 CompletableFuture**: Future کامل‌تر و async.
- **10 Callable**: task با خروجی.
- **11 TimeUnit**: واحد زمان.
- **12 Lock**: اینترفیس قفل.
- **13 ReentrantLock**: قفل قابل ورود مجدد.
- **14 ReentrantReadWriteLock**: قفل خواندن/نوشتن.
- **15 ReadWriteLock**: اینترفیس قفل خواندن/نوشتن.
- **16 Condition**: شرط انتظار.
- **17 StampedLock**: قفل نسخه‌دار.
- **18 AbstractQueuedSynchronizer**: پایه ساخت synchronizer.
- **19 ConcurrentLinkedQueue**: صف پیوندی هم‌زمان.
- **20 ConcurrentLinkedDeque**: Deque پیوندی هم‌زمان.
- **21 ConcurrentSkipListMap**: Map هم‌زمان مبتنی بر skip list.
- **22 ConcurrentSkipListSet**: Set هم‌زمان مبتنی بر skip list.
- **23 ThreadLocalRandom**: Random مخصوص thread.

۲۱.۶ java.io

- **1 File**: نماینده فایل/دایرکتوری.
- **2 InputStream**: پایه stream ورودی بایت.
- **3 OutputStream**: پایه stream خروجی بایت.
- **4 Reader**: پایه stream ورودی کاراکتر.

- **5 Writer**: پایه stream خروجی کاراکتر.
- **6 FileInputStream**: خواندن بایت از فایل.
- **7 FileOutputStream**: نوشتن بایت در فایل.
- **8 FileReader**: خواندن کاراکتر از فایل.
- **9 FileWriter**: نوشتن کاراکتر در فایل.
- **10 BufferedInputStream**: بافر ورودی بایت.
- **11 BufferedOutputStream**: بافر خروجی بایت.
- **12 BufferedReader**: بافر ورودی کاراکتر.
- **13 BufferedWriter**: بافر خروجی کاراکتر.
- **14 DataInputStream**: خواندن primitive.
- **15 DataOutputStream**: نوشتن primitive.
- **16 ObjectInputStream**: خواندن آبجکت.
- **17 ObjectOutputStream**: نوشتن آبجکت.
- **18 PrintStream**: چاپ داده.
- **19 PrintWriter**: چاپ متن.
- **20 PipedInputStream: stream**: ورودی لوله‌ای.
- **21 PipedOutputStream: stream**: خروجی لوله‌ای.
- **22 SequenceInputStream**: ادغام streamهای ورودی.
- **23 LineNumberReader**: خواندن خط با شماره.
- **24 PushbackReader**: امکان برگرداندن کاراکتر.
- **25 RandomAccessFile**: دسترسی تصادفی به فایل.
- **26 Serializable**: اینترفیس سریالایز.
- **27 Externalizable**: سریالایز سفارشی.
- **28 IOException**: خطای I/O.
- **29 FileNotFoundException**: خطای پیدا نشدن فایل.
- **30 EOFException**: خطای پایان فایل.
- **31 InterruptedIOException**: خطای وقفه I/O.
- **32 UncheckedIOException**: خطای unchecked I/O.

- 1 Buffer: ظرف داده NIO.
- 2 Buffer: **ByteBuffer**: بایت.
- 3 Buffer: **CharBuffer**: کاراکتر.
- 4 Buffer: **ShortBuffer**: short.
- 5 Buffer: **IntBuffer**: int.
- 6 Buffer: **LongBuffer**: long.
- 7 Buffer: **FloatBuffer**: float.
- 8 Buffer: **DoubleBuffer**: double.
- 9 Channel: کانال I/O.
- 10 FileChannel: کانال فایل.
- 11 SocketChannel: کانال سوکت.
- 12 ServerSocketChannel: کانال سرور سوکت.
- 13 DatagramChannel: کانال UDP.
- 14 Selector: انتخابگر کانال‌ها.
- 15 SelectionKey: کلید انتخاب.
- 16 Paths: ابزار ساخت Path.
- 17 Path: نماینده مسیر.
- 18 Files: ابزار کار با فایل.
- 19 FileSystems: ابزار ساخت FileSystem.
- 20 FileSystem: نماینده فایل سیستم.
- 21 FileStore: نماینده انبار فایل.
- 22 WatchService: سرویس پایش تغییرات.
- 23 WatchKey: کلید پایش.
- 24 StandardOpenOption: گزینه‌های باز کردن فایل.
- 25 StandardCopyOption: گزینه‌های کپی.
- 26 Charset: **StandardCharsets**: های استاندارد.
- 27 Charset: کدگذاری کاراکتر.

28• **CharsetDecoder**: رمزگشای کاراکتر.

29• **CharsetEncoder**: رمزگذار کاراکتر.

۲۱.۸ java.net

1 • **Socket**: سوکت کلاینت.

2 • **ServerSocket**: سوکت سرور.

3 • **DatagramSocket**: سوکت UDP.

4 • **DatagramPacket**: بسته UDP.

5 • **InetAddress**: آدرس IP.

6 • **URL**: آدرس منبع.

7 • **URI**: شناسه منبع.

8 • **URLConnection**: اتصال URL.

9 • **HttpURLConnection**: اتصال HTTP.

10• **InetSocketAddress**: آدرس سوکت.

11• **Proxy**: پروکسی.

12• **NetworkInterface**: واسط شبکه.

13• **Cookie**: کوکی HTTP.

14• **CookieManager**: مدیر کوکی.

15• **Authenticator**: احراز هویت کننده.

16• **PasswordAuthentication**: احراز هویت نام/رمز.

۲۱.۹ java.sql

1 • **Driver**: درایور دیتابیس.

2 • **DriverManager**: مدیر درایورها.

3 • **Connection**: اتصال دیتابیس.

4 • **Statement**: اجرای کوئری.

5 • **PreparedStatement**: کوئری پارامتری.

6 • **CallableStatement**: فراخوانی stored procedure.

- **ResultSet 7**: نتیجه کوئری.
- **ResultSetMetaData 8**: متادیتای نتیجه.
- **DatabaseMetaData 9**: متادیتای دیتابیس.
- **SQLException 10**: خطای SQL.
- **BatchUpdateException 11**: خطای batch.
- **SQLWarning 12**: هشدار SQL.
- **Savepoint 13**: نقطه ذخیره تراکنش.
- **Types 14**: ثابت‌های نوع SQL.
- **Date 15**: تاریخ SQL.
- **Time 16**: زمان SQL.
- **Timestamp 17**: زمان SQL estamp.
- **Blob 18**: داده باینری بزرگ.
- **Clob 19**: داده متنی بزرگ.
- **Array 20**: آرایه SQL.
- **Ref 21**: ارجاع SQL.
- **RowId 22**: شناسه ردیف.
- **SQLData 23**: نگاشت نوع سفارشی.
- **SQLInput: stream 24**: ورودی SQL.
- **SQLOutput: stream 25**: خروجی SQL.

۲۱.۱۰ java.math

- **BigInteger 1**: عدد صحیح دلخواه بزرگ.
- **BigDecimal 2**: عدد اعشاری دقیق.
- **MathContext 3**: تنظیم دقت.
- **RoundingMode 4**: حالت گرد کردن.

۲۱.۱۱ java.text

- **DateFormat 1**: قالب‌بندی تاریخ.

- **SimpleDateFormat**: پیاده‌سازی **DateFormat**.
- **NumberFormat**: قالب‌بندی عدد.
- **DecimalFormat**: پیاده‌سازی **NumberFormat**.
- **MessageFormat**: قالب‌بندی پیام.
- **ChoiceFormat**: قالب‌بندی شرطی.
- **Collator**: مقایسه رشته‌ها.
- **Collator**: **RuleBasedCollator** قاعده‌محور.
- **DateFormatSymbols**: نمادهای تاریخ.
- **DecimalFormatSymbols**: نمادهای عدد.
- **ParsePosition**: موقعیت تجزیه.
- **FieldPosition**: موقعیت فیلد.
- **Format**: پایه قالب‌بندی.
- **AttributedCharacterIterator**: پیمایشگر کاراکتر با **attribute**.
- **Normalizer**: نرمال‌سازی یونیکد.

۲۱.۱۲ java.time

- **LocalDate**: تاریخ بدون زمان.
- **LocalTime**: زمان بدون تاریخ.
- **LocalDateTime**: تاریخ و زمان.
- **ZonedDateTime**: تاریخ/زمان با منطقه.
- **OffsetDateTime**: تاریخ/زمان با **offset**.
- **OffsetTime**: زمان با **offset**.
- **Instant**: لحظه دقیق **UTC**.
- **Duration**: مدت زمان.
- **Period**: دوره تاریخ.
- **ZoneId**: شناسه منطقه زمانی.
- **ZoneOffset**: **offset** منطقه.
- **Clock**: ساعت.

- 13• **DayOfWeek**: روز هفته.
- 14• **Month**: ماه.
- 15• **Year**: سال.
- 16• **YearMonth**: سال-ماه.
- 17• **MonthDay**: ماه-روز.
- 18• **DateTimeFormatter**: قالب‌بندی تاریخ/زمان.
- 19• **DateTimeFormatterBuilder**: سازنده formatter.
- 20• **Temporal**: اینترفیس موجودیت زمانی.
- 21• **TemporalAccessor**: دسترسی به فیلدهای زمانی.
- 22• **TemporalAdjuster**: تنظیم‌کننده زمانی.
- 23• **TemporalAdjusters**: تنظیم‌کننده‌های آماده.
- 24• **TemporalAmount**: مقدار زمانی.
- 25• **TemporalField**: فیلد زمانی.
- 26• **TemporalUnit**: واحد زمانی.
- 27• **ChronoField**: فیلد استاندارد.
- 28• **ChronoUnit**: واحد استاندارد.
- 29• **ChronoLocalDate**: تاریخ گاه‌شماری.
- 30• **ChronoLocalDateTime**: تاریخ/زمان گاه‌شماری.
- 31• **ChronoZonedDateTime**: تاریخ/زمان منطقه‌ای گاه‌شماری.
- 32• **Chronology**: گاه‌شماری.
- 33• **IsoChronology**: گاه‌شماری ISO.
- 34• **HijrahChronology**: گاه‌شماری هجری.
- 35• **JapaneseChronology**: گاه‌شماری ژاپنی.
- 36• **MinguoChronology**: گاه‌شماری مینگو.
- 37• **ThaiBuddhistChronology**: گاه‌شماری بودایی تایلندی.

۲۱.۱۳ `java.lang.reflect`

• **Class 1**: نماینده کلاس.

- **Field 2**: فیلد بازتابی.
- **Method 3**: متد بازتابی.
- **Constructor 4**: سازنده بازتابی.
- **Modifier 5**: تغییردهنده‌ها.
- **Array 6**: ابزار آرایه بازتابی.
- **Proxy 7**: پروکسی پویا.
- **InvocationHandler 8**: هندلر فراخوانی پروکسی.
- **MethodHandles 9**: ابزار method handle.
- **MethodHandle 10**: ارجاع اجرایی متد.
- **MethodType 11**: نوع متد.
- **VarHandle 12**: ارجاع به متغیر.
- **Parameter 13**: پارامتر متد.
- **AnnotatedElement 14**: عنصر دارای انوتیشن.
- **Annotation 15**: انوتیشن.
- **AccessibleObject 16**: پایه فیلد/متد/سازنده.
- **ReflectPermission 17**: مجوز بازتاب.
- **Type 18**: نوع جنریک.
- **ParameterizedType 19**: نوع پارامتری.
- **GenericArrayType 20**: آرایه جنریک.
- **WildcardType 21**: نوع wildcard.
- **TypeVariable 22**: متغیر نوع.

۲۱.۱۴ java.lang.annotation

- **Annotation 1**: اینترفیس پایه انوتیشن.
- **Retention 2**: سیاست نگهداری.
- **RetentionPolicy 3**: مقادیر سیاست نگهداری.
- **Target 4**: محل مجاز.
- **ElementType 5**: انواع عناصر هدف.

- **6 Documented**: درج در مستندات.
- **7 Inherited**: ارث‌بری.
- **8 Repeatable**: تکرارپذیری.
- **9 Native**: نشانه بومی.
- **10 AnnotationFormatError**: خطای قالب انوتیشن.
- **11 AnnotationTypeMismatchException**: عدم تطابق نوع.
- **12 IncompleteAnnotationException**: انوتیشن ناقص.

۲۱.۱۵ java.util.regex

- **1 Pattern**: الگوی regex.
- **2 Matcher**: تطبیق‌دهنده.
- **3 MatchResult**: نتیجه تطبیق.
- **4 PatternSyntaxException**: خطای نحو regex.

۲۱.۱۶ java.util.zip

- **1 ZipFile**: فایل ZIP.
- **2 ZipEntry**: ورودی ZIP.
- **3 ZipInputStream: stream**: ورودی ZIP.
- **4 ZipOutputStream: stream**: خروجی ZIP.
- **5 GZIPInputStream: stream**: ورودی GZIP.
- **6 GZIPOutputStream: stream**: خروجی GZIP.
- **7 Inflater**: بازکننده فشرده‌سازی.
- **8 Deflater**: فشرده‌ساز.
- **9 InflaterInputStream: stream**: بازکننده.
- **10 DeflaterOutputStream: stream**: فشرده‌ساز.
- **11 CRC32**: چک‌سام CRC32.
- **12 Adler32**: چک‌سام Adler32.
- **13 CheckedInputStream: stream**: با چک‌سام.

• 14 stream: **CheckedOutputStream** با چک‌سام.

۲۱.۱۷ **java.util.logging**

- 1 **Logger**: لاگر.
- 2 **Level**: سطح لاگ.
- 3 **Handler**: هندلر لاگ.
- 4 **ConsoleHandler**: هندلر کنسول.
- 5 **FileHandler**: هندلر فایل.
- 6 **SocketHandler**: هندلر سوکت.
- 7 **StreamHandler**: هندلر stream.
- 8 **MemoryHandler**: هندلر حافظه.
- 9 **Formatter**: قالب‌بندی لاگ.
- 10 **SimpleFormatter**: قالب ساده.
- 11 **XMLFormatter**: قالب XML.
- 12 **LogRecord**: رکورد لاگ.
- 13 **LogManager**: مدیر لاگ.
- 14 **Filter**: فیلتر لاگ.
- 15 **ErrorManager**: مدیر خطا.

♦ ادامه بخش ۲۱: کلاس‌ها و اینترفیس‌های مهم جاوا (فقط مفاهیم)

۲۱.۱۸ **Swing / AWT**

- 1 **Swing**: کتابخانه UI قدیمی جاوا برای ساخت پنجره و دکمه.
- 2 **AWT**: کتابخانه گرافیکی قدیمی‌تر و وابسته به سیستم‌عامل.
- 3 **Layout Manager**: مدیر چیدمان اجزای UI.
- 4 **Event Listener**: شنونده رویدادهای UI.
- 5 **Adapter Class**: کلاس میانی برای پیاده‌سازی ساده listener.

۲۱.۱۹ Applet (منسوخ)

- **Applet 1**: برنامه کوچک جاوا که در مرورگر اجرا می‌شد (منسوخ).
- **AppletContext 2**: محیط اجرای Applet.
- **AudioClip 3**: پخش صدا در Applet.

۲۱.۲۰ Servlet

- **Servlet 1**: کامپوننت سمت سرور برای پردازش درخواست HTTP.
- **GenericServlet: Servlet 2**: مستقل از پروتکل.
- **HttpServlet: Servlet 3**: مخصوص HTTP.
- **ServletRequest / HttpServletRequest 4**: نماینده درخواست کلاینت.
- **ServletResponse / HttpServletResponse 5**: نماینده پاسخ سرور.
- **ServletConfig 6**: تنظیمات اختصاصی یک Servlet.
- **ServletContext 7**: بستر مشترک کل وب اپلیکیشن.
- **RequestDispatcher 8**: ارسال درخواست به منبع دیگر.
- **Filter 9**: فیلتر پردازش درخواست/پاسخ.
- **FilterChain 10**: زنجیره فیلترها.
- **HttpSession 11**: نشست کاربر.
- **Cookie 12**: داده ذخیره‌شده در مرورگر.
- **AsyncContext 13**: پردازش غیرهمزمان درخواست.
- **DispatcherType 14**: نوع (REQUEST/FORWARD/INCLUDE) **dispatch**.
- **MultipartConfig 15**: پیکربندی آپلود فایل.
- **Listener 16**: شنونده رویدادهای چرخه حیات Servlet.

۲۱.۲۱ JSP

- **JSP 1**: فناوری قالب‌ساز سمت **سرور** (منسوخ‌تر از Thymeleaf).
- **JspWriter 2**: نویسنده خروجی JSP.
- **PageContext: context 3**: صفحه JSP.
- **Tag 4**: تگ سفارشی JSP.

- **TagSupport 5**: پایه ساخت تگ سفارشی.
- **SimpleTag 6**: تگ ساده JSP.
- **TagExtraInfo 7**: متادیتای تگ.
- **TagLibraryInfo 8**: اطلاعات کتابخانه تگ.
- **JspFragment 9**: قطعه قابل استفاده مجدد JSP.
- **DynamicAttributes 10**: پشتیبانی attribute پویا.

JPA ۲۱.۲۲

- **EntityManagerFactory 1**: کارخانه ساخت EntityManager.
- **EntityManager 2**: مدیر موجودیت‌ها و تراکنش‌ها.
- **EntityTransaction 3**: مدیریت تراکنش.
- **Persistence 4**: ابزار ساخت EntityManagerFactory.
- **Query / TypedQuery 5**: اجرای کوئری JPQL.
- **StoredProcedureQuery 6**: فراخوانی stored procedure.
- **Parameter 7**: پارامتر کوئری.
- **Tuple / TupleElement 8**: نتیجه چندستونی.
- **EntityGraph 9**: گراف بارگذاری موجودیت.
- **Metamodel 10**: مدل متادیتای موجودیت‌ها.
- **EntityType / EmbeddableType / MappedSuperclassType 11**: انواع موجودیت.
- **ManagedType 12**: نوع مدیریت‌شده JPA.
- **SingularAttribute / PluralAttribute: attribute 13**: تک/چندمقداری.
- **Join / Fetch 14**: اتصال و بارگذاری رابطه.
- **FetchType 15**: نحوه بارگذاری (EAGER/LAZY).
- **CascadeType 16**: نوع انتشار عملیات.
- **LockModeType 17**: حالت قفل.
- **FlushModeType 18**: حالت flush.
- **GenerationType 19**: استراتژی تولید کلید.
- **TemporalType 20**: نوع تاریخ.

- 21• **EnumType**: نحوه ذخیره **enum**.
- 22• **PersistenceContextType**: نوع **context**.
- 23• **AccessType**: نوع دسترسی (FIELD/PROPERTY).
- 24• **CallbackType**: نوع **callback** چرخه حیات.
- 25• **SchemaGeneration**: تولید **schema**.
- 26• **Index / UniqueConstraint / ForeignKey**: قیدهای دیتابیس.
- 27• **NamedQuery / NamedNativeQuery**: کوئری نام‌دار.
- 28• **SqlResultSetMapping**: نگاشت نتیجه **SQL**.
- 29• **EntityListeners**: شنونده‌های چرخه حیات موجودیت.
- 30• **PrePersist / PostPersist / PreUpdate / PostUpdate / PreRemove / PostRemove / PostLoad**: **callback** های چرخه حیات.

بخش ۲۲: ابزارها و تکنولوژی‌های جانبی

۲۲.۱ ابزارهای Build

- 1• **Maven**: ابزار **build** و مدیریت وابستگی مبتنی بر **XML**.
- 2• **Gradle**: ابزار **build** مدرن مبتنی بر **DSL**.
- 3• **Ant**: ابزار **build** قدیمی و دستی.
- 4• **Bazel**: ابزار **build** مقیاس‌پذیر گوگل.
- 5• **SBT**: ابزار **build** مخصوص **Scala**.

۲۲.۲ سرورهای اپلیکیشن

- 1• **Tomcat**: کانتنر **Servlet** سبک.
- 2• **Jetty**: کانتنر **Servlet** سبک و **embeddable**.
- 3• **Undertow**: وب‌سرور سبک **JBoss**.
- 4• **WildFly**: سرور اپلیکیشن **JBoss**.
- 5• **WebLogic**: سرور اپلیکیشن **Oracle**.
- 6• **WebSphere**: سرور اپلیکیشن **IBM**.

- 7• **GlassFish**: سرور اپلیکیشن مرجع Jakarta EE.
- 8• **JBoss**: سرور اپلیکیشن قدیمی RedHat.
- 9• **Netty**: فریمورک شبکه غیرهمزمان.

۲۲.۳ دیتابیس‌ها

- 1 • **PostgreSQL**: دیتابیس رابطه‌ای پیشرفته متن‌باز.
- 2 • **MySQL**: دیتابیس رابطه‌ای محبوب.
- 3 • **MariaDB**: انشعاب متن‌باز MySQL.
- 4 • **Oracle**: دیتابیس تجاری Oracle.
- 5 • **SQL Server**: دیتابیس مایکروسافت.
- 6 • **DB2**: دیتابیس IBM.
- 7 • **SQLite**: دیتابیس سبک فایلی.
- 8 • **H2**: دیتابیس درون‌حافظه‌ای جاوا.
- 9 • **HSQLDB**: دیتابیس درون‌حافظه‌ای جاوا.
- 10 • **Derby**: دیتابیس درون‌حافظه‌ای جاوا.
- 11 • **MongoDB**: دیتابیس NoSQL سندی.
- 12 • **Cassandra**: دیتابیس توزیع‌شده ستونی.
- 13 • **Redis**: دیتابیس/کش کلید-مقدار درون‌حافظه‌ای.
- 14 • **Neo4j**: دیتابیس گرافی.
- 15 • **Elasticsearch**: موتور جستجو و تحلیل.
- 16 • **InfluxDB**: دیتابیس سری‌زمانی.

۲۲.۴ ابزارهای تست

- 1 • **JUnit**: فریمورک تست اصلی جاوا.
- 2 • **TestNG**: فریمورک تست جایگزین JUnit.
- 3 • **Mockito**: فریمورک mock سازی.
- 4 • **EasyMock**: فریمورک mock قدیمی.
- 5 • **PowerMock**: mock کردن **static/final**.

- **6 assertion J: AssertJ**: assertion های روان.
- **7 Hamcrest: matcher**: های تست.
- **8 Selenium**: تست خودکار مرورگر.
- **9 Cucumber**: تست BDD.
- **10 REST Assured**: تست API های REST.
- **11 WireMock: mock**: سرور HTTP.
- **12 Testcontainers**: اجرای Docker در تست.
- **13 JaCoCo**: سنجش پوشش کد.
- **14 Cobertura**: سنجش پوشش کد قدیمی.
- **15 SonarQube**: تحلیل کیفیت کد.

۲۲.۵ ابزارهای CI/CD

- **1 Jenkins**: سرور CI/CD متن باز.
- **2 GitHub Actions: CI/CD**: داخلی GitHub.
- **3 GitLab CI: CI/CD**: داخلی GitLab.
- **4 CircleCI: CI/CD**: ابری.
- **5 Travis CI: CI/CD**: ابری قدیمی.
- **6 TeamCity: CI/CD**: JetBrains.
- **7 Bamboo: CI/CD**: Atlassian.
- **8 Azure DevOps: CI/CD**: مایکروسافت.
- **9 ArgoCD**: استقرار GitOps در Kubernetes.
- **10 Spinnaker**: استقرار مداوم چندابری.

۲۲.۶ ابزارهای Container

- **1 Docker**: پلتفرم کانتینری سازی.
- **2 Podman**: جایگزین بدون daemon برای Docker.
- **3 containerd: runtime**: کانتینر.
- **4 CRI-O: runtime**: کانتینر Kubernetes.

- 5• **Kubernetes**: ارکستراتور کانتینر.
- 6• **OpenShift**: پلتفرم Kubernetes ردهت.
- 7• **Docker Swarm**: ارکستراتور بومی Docker.
- 8• **Nomad**: ارکستراتور HashiCorp.
- 9• **Rancher**: مدیریت Kubernetes.

۲۲.۷ ابزارهای Monitoring

- 1• **Prometheus**: سیستم مانیتورینگ و هشدار.
- 2• **Grafana**: داشبورد بصری مانیتورینگ.
- 3• **ELK Stack: Elasticsearch + Logstash + Kibana**
- 4• **Elasticsearch**: ذخیره و جستجوی لاگ.
- 5• **Logstash**: جمع‌آوری و پردازش لاگ.
- 6• **Kibana**: نمایش لاگ.
- 7• **Zipkin**: ردیابی توزیع‌شده.
- 8• **Jaeger**: ردیابی توزیع‌شده Uber.
- 9• **Sleuth**: ردیابی Spring Cloud.
- 10• **Micrometer**: متریک‌های اپلیکیشن.
- 11• **New Relic**: مانیتورینگ APM.
- 12• **Datadog**: مانیتورینگ ابری.
- 13• **AppDynamics**: مانیتورینگ APM.
- 14• **Dynatrace**: مانیتورینگ APM.

۲۲.۸ Message Broker ابزارهای

- 1• **Apache Kafka**: پلتفرم streaming توزیع‌شده.
- 2• **RabbitMQ**: پیام broker AMQP.
- 3• **ActiveMQ**: پیام broker جاوا.
- 4• **Redis Pub/Sub**: پیام‌رسانی سبک Redis.
- 5• **NATS**: پیام‌رسانی سبک و سریع.

6• **Pulsar**: پیام‌رسانی توزیع‌شده.

۲۲.۹ ابزارهای Caching

1• **Redis**: کش درون‌حافظه‌ای.

2• **Ehcache**: کش جاوا.

3• **Caffeine**: کش سریع جاوا.

4• **Hazelcast**: کش توزیع‌شده.

5• **Infinispan**: کش توزیع‌شده JBoss.

6• **Memcached**: کش توزیع‌شده ساده.

۲۲.۱۰ ابزارهای API

1• **Swagger**: مستندسازی API.

2• **OpenAPI**: استاندارد توصیف API.

3• **Postman**: تست API.

4• **Insomnia**: تست API.

5• **SoapUI**: تست سرویس‌های SOAP/REST.

6• **GraphQL**: زبان کوئری API.

۲۲.۱۱ Version Control

1• **Git**: سیستم کنترل نسخه توزیع‌شده.

2• **SVN**: سیستم کنترل نسخه متمرکز.

3• **Mercurial**: سیستم کنترل نسخه توزیع‌شده.

4• **Bitbucket**: میزبانی Git Atlassian.

5• **GitHub**: میزبانی Git مایکروسافت.

6• **GitLab**: میزبانی Git متن‌باز.

۲۲.۱۲ IDE

1• **IntelliJ IDEA**: محبوب JetBrains.

2• **Eclipse**: IDE متن باز.

3• **NetBeans**: IDE متن باز Oracle.

4• **VS Code**: ویرایشگر سبک میکروسافت.

5• **JDeveloper**: IDE Oracle.

6• **BlueJ**: IDE آموزشی.

۲۲.۱۳ ابزارهای Static Analysis

1• **Checkstyle**: بررسی سبک کد.

2• **PMD**: تحلیل کد.

3• **SpotBugs**: یاب باگ.

4• **FindBugs**: نسخه قدیمی SpotBugs.

5• **SonarLint**: تحلیل کیفیت کد.

6• **Error Prone**: بررسی خطاهای رایج.

۲۲.۱۴ ابزارهای Profiling

1• **VisualVM**: پروفایلر بصری JVM.

2• **JProfiler**: پروفایلر تجاری.

3• **YourKit**: پروفایلر تجاری.

4• **Java Mission Control**: پروفایلر Oracle.

5• **JConsole**: مانیتور JVM.

6• **MAT** (Memory Analyzer Tool): تحلیل heap dump.

7• **Async Profiler**: پروفایلر کم هزینه.

۲۲.۱۵ ابزارهای Dependency Management

1• **Maven**: مدیریت وابستگی.

2• **Gradle**: مدیریت وابستگی.

3• **Ivy**: مدیریت وابستگی Ant.

4• **Nexus**: مخزن وابستگی.

5• **Artifactory**: مخزن وابستگی JFrog.

۲۲.۱۶ ابزارهای Configuration Management

1• **Ansible**: مدیریت پیکربندی agentless.

2• **Puppet**: مدیریت پیکربندی.

3• **Chef**: مدیریت پیکربندی.

4• **SaltStack**: مدیریت پیکربندی.

5• **Terraform**: زیرساخت به صورت کد.

۲۲.۱۷ ابزارهای Virtualization

1• **VMware**: مجازی سازی تجاری.

2• **VirtualBox**: مجازی سازی متن باز Oracle.

3• **Hyper-V**: مجازی سازی مایکروسافت.

4• **KVM**: مجازی سازی متن باز Linux.

5• **Xen**: هایپروایزر متن باز.

۲۲.۱۸ ابزارهای Cloud

1• **AWS**: ابر آمازون.

2• **Azure**: ابر مایکروسافت.

3• **Google Cloud**: ابر گوگل.

4• **Heroku**: PaaS ساده.

5• **DigitalOcean**: ابر ساده و ارزان.

6• **Linode**: ابر ساده.

7• **Oracle Cloud**: ابر Oracle.

8• **IBM Cloud**: ابر IBM.

بخش ۲۳: مفاهیم پیشرفته Spring

Spring Core ۲۳.۱

- 1 IoC Container: ظرف مدیریت چرخه حیات و وابستگی Bean ها.
- 2 context: **ApplicationContext** اصلی Spring با امکانات کامل تر.
- 3 **BeanFactory**: ظرف پایه و سبک Spring.
- 4 **BeanDefinition**: تعریف متادیتای یک Bean.
- 5 **BeanPostProcessor**: پردازشگر قبل/بعد از مقداردهی Bean.
- 6 **BeanFactoryPostProcessor**: پردازشگر متادیتای BeanFactory.
- 7 **ApplicationListener**: شنونده رویدادهای اپلیکیشن.
- 8 **ApplicationEvent**: رویداد اپلیکیشن.
- 9 **MessageSource**: منبع پیام برای i18n.
- 10 **ResourceLoader**: بارگذار منابع.
- 11 **Environment**: محیط اجرا (property ها و profile ها).
- 12 **PropertyResolver**: حل کننده property.
- 13 **Profiles**: پروفایل های محیطی.
- 14 **SpEL**: زبان عبارت Spring برای دسترسی پویا به مقادیر.

Spring AOP ۲۳.۲

- 1 **Aspect**: جنبه؛ ماژول سازی concern عرضی.
- 2 **JoinPoint**: نقطه قابل اتصال در اجرای برنامه.
- 3 **Pointcut**: تعریف شرط انتخاب JoinPoint.
- 4 **Advice**: عملیات اجراشده در JoinPoint.
- 5 **Before**: قبل از متد.
- 6 **After**: بعد از متد.
- 7 **AfterReturning**: بعد از بازگشت موفق.
- 8 **AfterThrowing**: بعد از پرتاب استثنا.

- **9 Around**: دور **متد** (قبل و بعد).
- **10 Introduction**: افزودن متد/فیلد جدید به کلاس.
- **11 Weaving**: اتصال aspect به کد هدف.
- **12 Proxy**: واسط ایجادشده برای AOP.
- **13 Aspect@**: تعریف Aspect.
- **14 Pointcut@**: تعریف Pointcut.
- **15 @Around@ / @AfterThrowing / @AfterReturning / @After / Before**: انواع Advice.
- **16 @EnableAspectJAutoProxy**: فعال‌سازی AOP خودکار.

Spring MVC ۲۳.۳

- **1 DispatcherServlet**: کنترلر جلویی و مرکز MVC.
- **2 HandlerMapping**: نگاشت درخواست به handler.
- **3 HandlerAdapter**: اجرای handler.
- **4 ViewResolver**: تشخیص view از نام.
- **5 View**: نمای خروجی.
- **6 ModelAndView**: مدل + view.
- **7 Model / ModelMap**: ظرف داده مدل.
- **8 Controller / RestController**: کنترلر MVC/REST.
- **9 @RequestMapping** و مشتقاتش: نگاشت مسیر.
- **10 @RequestParam / PathVariable**: استخراج پارامتر.
- **11 @ResponseBody /RequestBody**: بدنه درخواست/پاسخ.
- **12 @ModelAttribute**: اتصال داده به مدل.
- **13 @CookieValue@ / RequestHeader**: هدر و کوکی.
- **14 @RequestAttribute@ / SessionAttribute**: نشست/درخواست.
- **15 @CrossOrigin**: فعال‌سازی CORS.
- **16 @ExceptionHandler**: مدیریت استثنا.
- **17 @RestControllerAdvice / ControllerAdvice**: مشاور سراسری.
- **18 @ResponseStatus**: تعیین کد وضعیت.

- 19• **ResponseEntity**: پاسخ کامل HTTP.
- 20• **MultipartFile**: فایل آپلودی.
- 21• **WebDataBinder**: اتصال و اعتبارسنجی داده.
- 22• **ConversionService**: تبدیل نوع.
- 23• **Formatter / Converter**: قالب‌بندی و تبدیل.
- 24• **Validator / Errors / BindingResult**: اعتبارسنجی و خطاها.
- 25• **LocaleResolver / ThemeResolver**: مدیریت theme و locale.
- 26• **HandlerInterceptor**: میان‌گیر درخواست.
- 27• **WebMvcConfigurer**: پیکربندی MVC.

Spring Boot ۲۳.۴

- 1 • **SpringBootApplication**: ترکیب Configuration + ComponentScan + EnableAutoConfiguration.
- 2 • **EnableAutoConfiguration**: فعال‌سازی پیکربندی خودکار.
- 3 • **ComponentScan**: اسکن Component ها.
- 4 • **Configuration**: کلاس پیکربندی.
- 5 • **SpringApplication**: کلاس اجرای اپلیکیشن Boot.
- 6 • **SpringApplicationBuilder**: سازنده اپلیکیشن Boot.
- 7 • **ApplicationRunner / CommandLineRunner**: اجرای کد در شروع.
- 8 • **ApplicationContextInitializer**: مقداردهی اولیه context.
- 9 • **SpringApplicationRunListener**: شنونده چرخه اجرای Boot.
- 10 • **Banner**: بنر شروع اپلیکیشن.
- 11 • **spring.factories**: فایل ثبت auto-configuration.
- 12 • **AutoConfiguration**: پیکربندی خودکار.
- 13 • **Condition**: شرط فعال‌سازی Bean.
- 14 • **Conditional** و مشتقاتش: شرط‌های مختلف برای فعال‌سازی.
- 15 • **ImportSelector / DeferredImportSelector**: انتخاب import پویا.
- 16 • **ImportBeanDefinitionRegistrar**: ثبت BeanDefinition سفارشی.

- 17• **SpringFactoriesLoader**: بارگذار فایل **spring.factories**.
- 18• **FailureAnalyzer / FailureAnalysis**: تحلیل خطاهای شروع.
- 19• **ExitCodeGenerator / ExitCodeExceptionMapper**: تولید کد خروج.
- 20• **SpringBootExceptionReporter**: گزارشگر خطاهای Boot.
- 21• **ApplicationContextFactory**: کارخانه ساخت context.
- 22• **ConfigurableApplicationContext**: context قابل پیکربندی.
- 23• **WebApplicationType**: نوع اپلیکیشن (Servlet/Reactive/None).

Spring Boot Actuator ۲۳.۵

- 1• **Actuator**: ماژول پایش و مدیریت اپلیکیشن.
- 2• **Endpoint**: نقطه پایانی مدیریتی.
- 3• **Endpoint / @ReadOperation / @WriteOperation / @DeleteOperation**: تعریف endpoint سفارشی.
- 4• **Selector@**: انتخابگر در endpoint.
- 5• **EndpointWebExtension / @EndpointJmxExtension**: افزونه‌های endpoint.
- 6• **RestControllerEndpoint / @ServletEndpoint / @ControllerEndpoint / @JmxEndpoint / @**
- @WebEndpoint**: انواع endpoint.
- 7• **EndpointAutoConfiguration@**: پیکربندی خودکار endpoint ها.
- 8• **HealthIndicator**: نشانگر سلامت.
- 9• **InfoContributor**: تأمین‌کننده اطلاعات.
- 10• **Metrics / MeterRegistry**: متریک‌ها و ثبت‌کننده آن‌ها.
- 11• **Counter / Gauge / Timer / DistributionSummary**: انواع متریک.
- 12• **AuditEventRepository**: مخزن رویدادهای حسابرسی.
- 13• **HttpTraceRepository**: مخزن ردگیری HTTP.